

Information Systems Infrastructure and Cloud Services

Module 1 asked what an information system is. Module 2 asked which system a company should build and why. This module asks the question underneath both of those: **what does all of it actually run on**, and who pays for it. Every app you have ever used is a program on a machine, in a room, reachable over a wire. A manager who cannot name those parts cannot judge a proposal about them — and cannot tell the difference between a bill that is large because the work is large and a bill that is large because nobody was watching.

1

OBJECTIVE 3.2

Describe the essential components of an organization's information systems infrastructure: hardware, system software, storage, networks, and data centers

2

OBJECTIVE 3.3

Discuss the managerial issues that come with owning an infrastructure — obsolescence, storage growth, demand spikes, energy, and the need to move fast

3

OBJECTIVE 3.4

Describe cloud computing and the other current trends that help an organization answer those challenges

4

APPLICATION 3.5

Follow the data itself from a file to a stored, queryable database, and see where the course's own database work fits

5

APPLICATION 3.6

Use the same internet technology three ways: selling to customers, communicating with employees, and working with partners

How to use this page

1. **Assume no background.** Every part is explained in ordinary language before anything uses it. If you have never opened a computer or thought about where a website lives, this page was written for you and nothing here assumes otherwise.

2. **Answer as you go.** The activities sit inside the reading, at the point where the reading has just earned them. When you answer a question, **every option explains itself** — including the three you did not pick, because knowing why a wrong answer is wrong is the part that transfers.
3. **Watch for the two supplements.** The last two sections are labeled application supplements. They are not textbook objectives; they connect this chapter's material to the database work the course does elsewhere and to the way businesses actually use the internet.
4. **Retry anything.** Restart one activity, one section, or the whole page. Your best result stays in this browser until you clear it; answers reset on reload so every attempt is fresh.
5. **Finish with the final challenge.** Situations rather than definitions, scored by objective, so a weak area shows up instead of being averaged away.

Where this module sits in the course objectives

The three modules are organized around the textbook's own learning objectives, but the course is graded against a different list. This table says plainly which of the eleven course objectives this module serves, and which ones it does not, so you always know why you are reading something.

COURSE OBJECTIVE	WHAT THIS MODULE DOES WITH IT
4. Apply a working knowledge of computer hardware, software, database technology and computer networks	This module's main job. Hardware, system software and storage, networks, and the internet and the web are four full sections; the database half is picked up in the supplement on databases and SQLite.
5. Delineate how businesses use internet technology for electronic commerce, communicating with employees and working with business partners	Also central. The web section explains how the internet carries anything at all; the section on intranets, extranets and data centers covers employees and partners directly; the last supplement puts all three uses side by side.
1. Describe what a company's information systems department does	Supporting. Obtaining, running, upgrading and eventually renting the infrastructure is a large part of that department's work, and the sections on managerial issues and on choosing a provider are about those decisions.
2. Discuss how companies use information systems to manage internal operations and make decisions	Supporting. Transaction processing, operational and archival data, and the extranet that gives management real-time information all appear here as the plumbing under Module 2's decisions.

COURSE OBJECTIVE

WHAT THIS MODULE DOES WITH IT

3. Discuss how companies use information technology for competitive advantage by changing their marketplace

Supporting. Renting the commodity so you can build the thing that makes you different is the same argument Module 2 made about strategy, applied to plumbing — and several firms in this chapter sell the infrastructure they had to build anyway.

8. Describe the individual and organizational consequences of the use of information technology

Supporting. Energy and cooling, electronic waste and equipment disposal, personal devices arriving at work, and data protection rules deciding where a company's data may physically sit.

9. Recognize security breaches and computer crimes

Supporting. A stolen credential that produced a ruinous bill, encrypted versus unencrypted web traffic, firewalls and private tunnels, vehicles as computers on wheels, and who can audit your data once it is not in your building.

6. Outline the life cycle of an information system · **7.** Recognize a business problem solvable with technology, then work with the information systems team on a solution

Not taught here. Objective 7's first half — spotting the problem and saying what kind of system it needs — is practised in Modules 1 and 2.

10. Solve a business problem by creating a database application using SQLite · **11.** Use advanced spreadsheet functions to create a decision support system

Served by the course's separate database and spreadsheet work; the supplement on databases here is the bridge to it, not a substitute for it.

PARTS

What it is made of

Machines, the software that drives them, somewhere to keep the data, and wires between them. Five components, and every system you use is some arrangement of them.

REACH

How it connects

A network is three things: something to say, a path, and an agreed way of saying it. The internet and the web are that idea at world scale.

DECAY

Why it never stays finished

Hardware ages, software stops being supported, data piles up, demand spikes in December, and the electricity bill grows. Owning is a running cost, not a purchase.

RENT

Why so much of it is rented

Somebody else owns the building, the power and the cooling, and you pay for what you use. That changes the economics, the risks and the questions you have to ask.

ONE IDEA CONNECTS THE WHOLE MODULE

Infrastructure is the part of technology a business would rather not think about, and that is exactly why it decides so much. What you can build, how fast you can try it, what it costs you when nobody uses it and what it costs you when everybody does — all of that is settled by the plumbing long before anyone writes a strategy about it.

SECTION 3-2A

Why infrastructure is a management question

A city works because of things almost nobody looks at: pipes under the road, cables under the pavement, a power station somewhere outside town. You notice them on the day one of them stops. Organizations have the same layer underneath them, made of machines and programs and wire instead of concrete, and this module is about that layer — what it is made of, why it never stays finished, and who ends up paying for it.

Infrastructure is the part you only see when it breaks

Start with the ordinary meaning of the word. A city's infrastructure is the shared foundation everything else is built on. You do not lay your own roads when you open a shop; you rely on the ones already there, and your shop is only ever as good as they are.

An organization's technology works the same way. The **information systems infrastructure** is the shared foundation of machines, programs, storage and connections that every application in the company runs on. No single department owns it, everyone depends on it, and most of the time nobody thinks about it at all.

That last part is exactly why it is a management question and not a purely technical one. Decisions about a foundation are made once and lived with for years, they are expensive to reverse, and they quietly settle what the business is able to attempt later. Somebody who cannot name the parts cannot judge a proposal about them.

The companies you know for something else

The chapter opens with a handful of companies nearly everybody in the developed world can name, and it names each one for the product you would think of first: Apple for its laptops, phones and tablets, Microsoft for its operating system and productivity software, Facebook for its social network, Google for its search engine.

Then it points out how little that captures. Google also works on driverless cars and other technologies, on life sciences, on investment capital, and on space exploration through its investments in SpaceX. Many of those ventures, in the chapter's own phrase, did not fall neatly under a search engine company, and billions of dollars were going into research projects that investors did not always view favourably.

The answer was structural. A holding company called **Alphabet** was created, placing around a dozen companies under one umbrella — the robotics business Intrinsic, the life sciences company Verily, the autonomous driving company Waymo, and the others the chapter groups as the "Other Bets" — so that financial reporting now shows where revenue is generated and where investments in new ideas are being made.

Then comes the sentence this whole module hangs from. Beyond the holding company itself, what unites those very different businesses is **the need for a reliable and high-performing information systems infrastructure**. A robotics company and a search engine have almost nothing in common except that neither of them works if the machines underneath it do not.

The chapter reports the scale of what Google built for itself, and two things in that report should be read differently.

The count is a snapshot — the chapter gives a number of data center locations across a number of countries in the opening case and a different, smaller figure later on, so read it as a few dozen major sites spread over many countries rather than as an exact tally that will still be right next year.

The reason is not a snapshot — those sites exist so that people all over the world get a fast and reliable answer, and speed and reliability at that scale are bought with buildings, electricity and network capacity rather than with clever programming alone.

Selling the thing you had to build anyway

Having built that foundation for its own products, Google now rents it to other organizations as **cloud computing** services, ranging from basic computing, storage and networking through to analytics, artificial intelligence and security. Module 2 gave this move a name — a capability built for internal use turned into something customers pay for — and here it is again, down in the plumbing.

The chapter is careful to say that Google is not alone in doing this, and equally careful to say that not everybody does it. That contrast is the most useful part of the whole case.

Rented out — Google, Microsoft with Azure and Amazon with Amazon Web Services each built a high-performing, reliable infrastructure to support their own operating model, and now capitalize on that investment by selling capacity on it to everybody else.

Kept in-house — Apple uses its data centers only to host services for users of the Apple ecosystem, and the company that owns the Facebook app, whose corporate parent renamed itself Meta in 2021, uses its own only to power that social network and its generative artificial intelligence models.

The judgment underneath — either choice can be the right one, because renting your foundation out turns a cost into revenue, but it also turns you into a supplier with customers, outages and support obligations that you did not have the week before.

The chapter closes the case by asking the student three questions. They are worth writing down, because they turn out to be the map of this entire module.

- 1 **How does a company benefit from a well-functioning infrastructure?** That is objective 3.2 — the five components and what each one contributes — which is the rest of this section and the five sections that follow it.
- 2 **Why would companies use cloud computing?** That is objective 3.3 and the first half of objective 3.4: what goes wrong when you own the foundation yourself, and what renting somebody else's actually fixes.
- 3 **What should an organization consider when it builds on infrastructure provided by somebody else?** That is the second half of objective 3.4, where the module stops describing the cloud and starts asking what to establish before signing anything.

CHECK YOURSELF

The case underneath the case

Four options, one best answer; read every explanation, including the ones for the options you did not choose, since that is where the confusions get named.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. The chapter describes a group of businesses that runs from search to robotics to life sciences to autonomous driving. Beyond the holding company itself, what does the chapter say unites them?

- **A.** A single shared product line, since each of the companies is ultimately a way of selling advertising against search results — Advertising revenue is real, but the chapter's own list of driverless cars, life sciences, investment capital and space exploration is precisely the point that these ventures did not fall neatly under a search engine company.
- **B.** The need for a reliable and high-performing information systems infrastructure, which every one of them depends on — **correct.** The chapter states that beyond the holding company, what unites the different companies under the umbrella is the need for a reliable and high-performing information systems infrastructure.
- **C.** A common set of customers, because the same people encounter each of the group's businesses in the course of a week — The customers barely overlap at all. A buyer of industrial robots, a health researcher and a person running a search are not one market, which is part of why a holding company was needed in the first place.
- **D.** Nothing beyond common ownership, since the chapter presents the restructuring as a purely financial arrangement — The restructuring did change financial reporting, showing where revenue is generated and where investments in new ideas are made, but the chapter names the shared infrastructure specifically as the thing lying under all of them.

2. Every company named in the opening case operates large data centers. The chapter still draws a contrast between two groups of them. What is the contrast?

- **A.** Some run facilities of their own while the others rent all of their capacity from a third party — Every company in the case operates infrastructure of its own. The case is about what each of them then does with that infrastructure, not about which of them bothered to build one.
- **B.** Some build facilities in many countries while the others keep all of their equipment inside one country — The chapter does describe one company's sites as spread across many countries, but it never sorts the companies by geography, and a single-country footprint is not the distinction being drawn.
- **C.** Some rent capacity on their infrastructure to other organizations, while the others use theirs only to run their own services — **correct.** Google, Microsoft with Azure and Amazon with Amazon Web Services capitalize on the investment by selling capacity, while Apple hosts services only for users of its own ecosystem and the owner of the Facebook app powers only that social network and its generative artificial intelligence models.
- **D.** Some use their facilities for artificial intelligence work while the others use theirs only to store records — Artificial intelligence appears on both sides of the contrast, among the services sold to other organizations and among the workloads a company runs on its own machines, so it cannot be what separates the two groups.

3. A manager at a hypothetical distribution company argues that because infrastructure produces nothing a customer ever buys, the right goal is always to spend as little on it as possible. Which reply does the chapter best support?

- **A.** Agree, because the chapter treats infrastructure spending as overhead to be squeezed wherever a business can get away with it — The chapter opens this objective by saying that computing, storage and networking technologies create value, and it spends its entire opening case on companies for which the infrastructure is central rather than incidental.
- **B.** Agree, but only for companies selling physical goods, since the chapter limits its argument to firms whose product is itself digital — The argument is not limited by industry. The chapter's later examples run from a parcel carrier's facilities to an electronics factory in which sensors and controllers automate three-quarters of the production process, and it says that almost any business today is an electronic business.
- **C.** Disagree, because the chapter argues that an organization should aim to own the largest infrastructure it can afford, on the grounds that owned capacity is the only capacity that can be relied on — Nothing in the chapter recommends owning as much as possible. The objectives that follow are largely about why owning is difficult and expensive, and about when renting somebody else's foundation is the better answer.
- **D.** Disagree, because the infrastructure creates value through efficiency, effectiveness and increasingly agility, and some firms sell the foundation they had to build anyway — **correct.** The lead-in says these technologies create value by enabling efficiency and effectiveness and increasingly by enabling agility, and the case shows several firms turning the infrastructure they built for themselves into a service that other organizations pay to use.

The five parts, named once

This objective takes the infrastructure apart into five components and then walks them in order. You are not expected to understand them yet; the point of naming them here is that from now on every unfamiliar thing in this module is one of these five, or a decision about one of these five.

Hardware — the physical machines that run the applications and databases an organization needs, from the phone in a pocket to the racks of servers that answer everybody at once.

System software — the programs that control the basic operations of those machines, so that an application can ask for something to be saved without knowing what kind of disk is fitted.

Storage — the place data stays when the day ends and the power goes off, which is a different question from where it is being used while somebody is working on it.

Networking — the sharing of data or services between machines, which is what turns five separate pieces of equipment into something an organization can actually use.

Data centers — the dedicated space the equipment lives in, with the power, the cooling and the physical security that keep it running, because all of that has to be bought by somebody.

Those five are easier to hold onto if you follow one ordinary request through them. The diagram takes each component in turn and shows what it is, what its single job is, where it appears when you tap a button to place an order, and what you notice on the day it is missing.

INTERACTIVE DIAGRAM

The five components, and one ordinary request passing through them

Pick a component to see what it is, what its single job is, where it appears when somebody taps a button to place an order, and what you notice when it is missing.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Hardware — The physical machines, from the phone in a pocket to the racks in a locked room

What it is (The machines that run the applications and databases) → Its one job (Doing the computing work itself) → In an ordinary order (Your phone, and the machine that answers it) → Missing (There is nothing for anything else to happen on)

- Organizations carry out hundreds or thousands of different activities, so they need several different **types** of computer to support them. A machine sized for one designer is not the machine that serves ten thousand customers at the same moment.
- Hardware is where the spending is most visible, which is also why it is where the arguments start. It wears out, it is bought in whole units, and it has to be paid for whether or not it is busy.
- A good deal of the hardware in a modern organization is not a computer in any form an ordinary person would recognise — the small controllers that start and stop a production line, or the sensors reporting a temperature every few seconds.
- Section 3–2b takes this apart properly, from the classes of computer up through the specialised processors that explain why an artificial intelligence project carries a different budget line from an ordinary one.

System software — The layer sitting between an application and the machine it happens to be running on

What it is (Programs controlling the basic operations of the hardware) → Its one job (Letting software be written once, not per machine) → In an ordinary order (The operating system handing the app the screen) → Missing (Every program needs rewriting for every model of machine)

- Application software does not talk to hardware directly. It asks the system software, and the system software talks to the machine, which is why one version of a program can run on equipment built by different manufacturers.

- The **operating system** is the most prominent kind of system software, coordinating the hardware components, the attached devices, the application software and the users all at once.
- This is the first appearance of an idea the whole module keeps reusing: put a common layer in the middle, and the things above it stop having to know anything about the things below it.
- Section 3–2c covers it, together with the reason a company's software eventually stops working on hardware that still switches on perfectly well.

Storage — Where data stays once the power is off and the working day is over

What it is (The place records are kept rather than used) → Its one job (Keeping data available for as long as it is needed) → In an ordinary order (The record of the order, still there tomorrow) → Missing (The organization forgets everything overnight)

- Memory inside a running machine holds whatever is in use at this instant; storage is what survives the machine being switched off, and an organization needs far more of the second than of the first.
- Not all stored data has the same job. Today's live records, a nightly copy taken in case something fails, and records kept for years differ in how fast they must be reachable, how searchable they need to be, and how long they must last.
- Storage is the component that grows without anybody deciding to grow it. Nobody chooses to have more data; it accumulates as a by-product of operating, and the cost follows it.
- Section 3–2c covers storage alongside system software, because the choice of medium follows directly from what a given pile of data is actually for.

Networking — The path from one machine to another, whether across a room or across the world

What it is (The sharing of data or services between machines) → Its one job (Making the other four components reachable) → In an ordinary order (Everything between your tap and the answer) → Missing (Each machine becomes an island with a screen)

- Taken alone, each individual piece of technology has little value; it is through connecting the different pieces that business value is realized. The best-performing database ever built is useless if the people who need it cannot reach it.
- A network needs three things and no fewer: two parties with something to share, a pathway between them, and rules that both sides follow. Remove any one of the three and nothing has been communicated.
- This is the component the internet and the World Wide Web belong to, which is why it needs two full sections rather than one, and why more of this module's vocabulary lives here than anywhere else.
- Sections 3–2d and 3–2e cover it, from what a network is at all up to how an address typed into a browser turns into a page in front of you.

Data centers — The room or the building around all of the above, with its power and its cooling

What it is (Dedicated space set aside for the infrastructure) → Its one job (Keeping the equipment running, cool and secure) → In an ordinary order (The building the answering machine stands in) → Missing (Equipment that overheats, loses power or walks away)

- Equipment gathered in one place is easier to manage, repair, upgrade, power, cool and physically secure than the same equipment scattered through the cupboards of a dozen offices.
- The range of sizes is enormous, from a single room with one server in it to a purpose-built facility the size of a warehouse with its own generators and its own cooling plant.
- This is the component people forget, and it is the one that decides whether the other four are actually available at eight in the morning on the busiest day of the year.
- Section 3–2f covers it, along with the two arrangements — a private area for employees and a shared one for business partners — that put the finished foundation to work.

What a well-run infrastructure is actually worth

The objective opens with a claim about value, and it names three different kinds. Two of them you already met in Module 2, so they get one clause each here rather than a re-teaching.

Efficiency — the extent to which goals are accomplished faster, at lower cost, or with relatively little time and effort, which is the value most people assume technology is for and the easiest of the three to measure.

Effectiveness — the extent to which goals and tasks are accomplished well, which is a different question and often a more valuable one, since a process can be fast and still produce the wrong result.

Agility — the ability to change what the organization does and to change it quickly, which the chapter says computing, storage and networking increasingly deliver, and which is the newest of the three arguments.

Agility is the one worth slowing down on, because it is the reason so much of this module is about renting rather than owning. Efficiency and effectiveness both ask how well you are doing the work you already do. Agility asks a different question: how fast could you do something else instead.

The chapter attributes the rising importance of agility to increasing **digital density** — more of the world, and more of every business process in it, now existing as data that something can act on. The denser that gets, the more often the answer to a business problem turns out to be a technology decision, and the more it costs to be slow at making them.

Efficiency is doing today's work faster, effectiveness is doing it well, and agility is being able to do tomorrow's work at all. An infrastructure decision usually trades one against another, which is why it belongs to a manager rather than only to a technician.

Why this is your problem even if you never touch a server

The chapter puts its own answer to that in a box aimed squarely at students who do not intend to work in technology, and its argument runs in four beats. They are worth taking seriously rather than reading past, because each one describes a specific thing that goes wrong when a manager does not have this vocabulary.

EXPLORE

Four reasons this is not somebody else's subject

Open each card and read all four panels; the third card is the one that connects this chapter back to the strategy material in Module 2.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

The foundation · It sits under nearly everything the organization does

- **The claim, in one line:** The information systems infrastructure forms the foundation of almost all organizational activities, from managing customer relationships and supply chains through to supporting decision-making and innovation.
- **What it looks like in an ordinary week:** Every ordinary thing you do at work rests on it: a customer record that loads, a report that runs, a file a colleague in another building can open, a payment that clears while somebody is still standing at the counter.
- **What skipping it costs you:** Without a grasp of hardware, software, networks and data management, the chapter says, a future business leader risks missing critical opportunities or making uninformed decisions — and that sentence is aimed at business students rather than at technologists.
- **What it lets you do:** It lets you treat an infrastructure proposal as a business proposal: what does this let us do, what does it stop us doing, and what happens to the business on the day it is not working.

The shared language · You will have to talk to technical people, and they to you

- **The claim, in one line:** Whether you are working on a marketing campaign, developing a new product or improving an internal process, you will have to collaborate with the people who run the technology.
- **What it looks like in an ordinary week:** Somebody tells you the delay is a network issue, or that the request needs another server, or that the data is in the cloud. Each of those sentences means something specific, and each one has a cost and a timescale attached to it.

- **What skipping it costs you:** Not being able to speak the language of infrastructure produces misunderstandings in both directions. You cannot state what you actually need, and you cannot tell a genuine constraint from a convenient one.
- **What it lets you do:** The chapter's claim is that understanding terms such as servers, cloud computing, cybersecurity and databases enables clear communication, reduces misunderstandings and achieves better outcomes. It is also the cheapest of the four to act on.

The opportunities · Where this chapter reaches back into strategy

- **The claim, in one line:** Businesses able to make use of cloud services, the Internet of Things and artificial intelligence infrastructure are better positioned to innovate and to respond to changes in their market.
- **What it looks like in an ordinary week:** A competitor tries four ideas in the time it takes you to buy the equipment for one, because it rented its foundation instead of buying it and simply stopped paying for the three experiments that did not work.
- **What skipping it costs you:** Without foundational knowledge of information systems, the chapter warns, you might overlook how technology can be used strategically to drive efficiency, customer satisfaction and growth. Nobody recommends a move they cannot picture.
- **What it lets you do:** This is the hinge back to Module 2. Competitive advantage was the question there; here you learn what the answers are physically made of, and which parts of them a company would have to own rather than rent.

The personal skill · The same knowledge, turned on your own life

- **The claim, in one line:** Individuals manage personal data across several devices, rely on storage they do not own, and face security threats such as phishing and identity theft.
- **What it looks like in an ordinary week:** Your photographs sit on somebody else's machines under an agreement you did not read, and your bank sends you a message that may or may not have come from your bank.
- **What skipping it costs you:** Not knowing how data is stored, transmitted and protected leaves you unable to tell which of those situations is safe, which is merely irritating, and which is the one that will take a year to undo.
- **What it lets you do:** The chapter's conclusion is that this is not just a technical skill but a core life and business competency, one that supports smarter leadership, innovation and resilience in an increasingly digital world.

Notice which of the four does the most work. The first says the subject is unavoidable, the second says it makes you easier to work with, and the fourth says it protects you personally. The third is the one that reaches back into Module 2.

Businesses able to make use of cloud services, connected devices and artificial intelligence infrastructure are better placed to innovate and to respond when their market moves. The chapter's

warning is that without this material you might simply overlook how technology could be used strategically at all, and you cannot recommend a move you are unable to picture.

CHECK YOURSELF

Five components, three kinds of value

Four options, one best answer; every situation naming no real company is a hypothetical written for practice, and every explanation says what the option would have described if it were right.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical accounting firm lists its technology as the laptops and servers it owns, the operating systems running on them, the disks its records sit on, and the wiring and wireless equipment connecting them all. Which of the chapter's five components has been left off the list?

- **A.** Nothing has been left off, because those four descriptions cover all five components between them — Four of the five are indeed present — hardware, system software, storage and networking — but the chapter names a fifth component, and it is the one that is a place rather than a thing.
- **B.** Data centers, meaning the dedicated space that houses the equipment and supplies its power, cooling and physical security — **correct**. The five components are hardware, system software, storage, networking and data centers, and the space itself counts because power, cooling and physical security are decisions somebody has to make and pay for.
- **C.** Storage, because disks belong to hardware and the chapter means something different by storage — The disks are on the list and storage is exactly what they are for. Storage is a separate component because what the data is for decides which medium it belongs on, not because disks somehow stop being physical.
- **D.** Networking, because cables and wireless equipment are hardware rather than a network — Cables and wireless equipment are physical, but the chapter counts networking as its own component because the value lies in the connection rather than in the parts, and this inventory does mention them.

2. The chapter says computing, storage and networking create value by enabling efficiency and effectiveness, and increasingly by enabling agility. What does the third of those add that the first two do not?

- **A.** The ability to carry out the current work at a lower cost for each unit than competitors manage — Accomplishing goals faster, at lower cost or with relatively little time and effort is efficiency, which is the first of the two the chapter already had and which Module 2 defined.
- **B.** The ability to carry out the current work to a higher standard, so that fewer results have to be corrected — Accomplishing goals and tasks well is effectiveness, the second of the two the chapter already had. Agility is not a measure of how good the output is.

- **C.** The ability to change what the organization does, and to change it quickly — **correct.** Efficiency is about doing the current work faster and more cheaply and effectiveness about doing it well, whereas agility is about being able to change course quickly, which is why the chapter ties its rising importance to increasing digital density.
 - **D.** The ability to keep the current work running without interruption through an unusually busy period — Staying available under load is a genuine requirement and the module returns to it more than once, but that describes reliability rather than the capacity to do something different.
3. The chapter gives four reasons a business student should understand the infrastructure. Which one is the reason that connects this chapter back to competitive strategy?
- **A.** That people who can make use of cloud services, connected devices and AI infrastructure are better placed to innovate — **correct.** The chapter says businesses able to leverage cloud services, the Internet of Things and artificial intelligence infrastructure are better positioned to innovate and respond to market changes, and that without this knowledge you might overlook how technology can be used strategically.
 - **B.** That the infrastructure is the foundation of almost all organizational activity — This is the chapter's first reason and it is perfectly true, but it argues that the subject is unavoidable rather than that it is a source of advantage over anybody else.
 - **C.** That understanding the vocabulary makes conversations with technical colleagues clearer, so that a manager is not obliged to accept a recommendation simply because it was explained in words nobody else in the room understood — This is the second reason, about communicating with technical teams and reducing misunderstandings. It makes you better at working with a plan somebody else has proposed rather than better at seeing one.
 - **D.** That the same knowledge protects your own privacy, devices and personal data — This is the fourth reason, the personal one, about managing your own data and recognising threats such as phishing. Valuable, but it concerns you rather than the organization's position in its market.

Where this section hands you on

The rest of objective 3.2 walks the five components in the order they were named. Hardware comes first, because everything else is a program running on a machine somewhere; then system software, then storage, then two full sections on networking, because the internet and the World Wide Web are large enough to need them; then the buildings, and the two arrangements that put the whole foundation to work for employees and for business partners.

After that the module changes its question twice. Objective 3.3 asks why an infrastructure you own never stays finished, and objective 3.4 asks what changes when you rent one instead. Keep the case in this section in view while you read them: a company whose foundation was so good that other organizations started paying to use it.

THE HABIT WORTH FORMING NOW

Every time this module introduces something new, ask which of the five components it belongs to and which of the three values it is supposed to deliver. Those two questions turn a list of unfamiliar words into a small number of recurring decisions, and they are the same two questions a manager asks when somebody arrives with a proposal and a price.

Hardware: the machines and the things that are not computers

Everything a business does with technology finally happens on a physical object that somebody bought, plugged in, put somewhere, and will one day have to throw away. That object is the first component of an infrastructure, and it is the one people underestimate, because half of it does not look like a computer at all: no screen, no keyboard, nobody sitting at it.

What hardware is, and why one kind is never enough

Start with what the word covers. **Hardware** is the physical machinery of an information system — the computers that run the applications and databases an organization needs in order to process its transactions and analyze its business data. If you can trip over it, unplug it, or drop it, it is hardware.

Organizations carry out hundreds or thousands of activities belonging to many different business processes, and those activities make very different demands. So a company does not buy one kind of computer. It buys several kinds, deliberately, and the differences between them are what this section is about.

You already know this from your own life. The phone in your pocket and the machine that answers when you tap the button to buy something are both computers, and swapping them would be absurd in either direction. The phone could not serve a hundred thousand shoppers at once, and nobody wants to carry a shop's order system around in a coat.

The six classes of computer

The chapter sorts the computers used inside organizations into six general classes. One warning before the list: the chapter's own sentence announces six classes and then names five, adding mobile devices afterwards as though they were an afterthought rather than a class. Its comparison table has six rows. Count six.

Personal computer — the machine used for personal computing and small business computing, serving one person at a time, and the reference point against which every other class ends up being described.

Workstation — a single-user machine built for medical, engineering, architectural, animation or graphics work, optimized for visualizing and rendering three-dimensional models with a fast processor, large memory and advanced video hardware.

Server — any computer on a network that makes access to files, printing, communications and other services available to the other users of that network, optimized for many people using it at the same moment.

Mainframe — a large computer used as the main, central computing system of a major organization, optimized for high availability, heavy shared use and security, and used for mission-critical work such as transaction processing.

Supercomputer — the most expensive and most powerful class of computer, used mainly to help solve massive scientific problems and, the chapter is explicit, typically not used by business organizations at all.

Mobile device — tablets and smartphones, which have stopped being a personal convenience sitting beside the infrastructure and become part of it, because a great deal of real work is now done on them.

The chapter compares the six on five measures at once, and the comparison rewards reading slowly. Treat the memory and cost columns as the chapter's snapshot of a moving market rather than as current figures; they drift every year, and the low end of the mobile row is already a museum piece.

CLASS	SIMULTANEOUS USERS	PHYSICAL SIZE	TYPICAL USE	MEMORY
Supercomputer	One to many	From the size of a car to several rooms	Scientific research	1,000+ TB
Mainframe	10,000+	Like a refrigerator	Transaction processing and enterprise-wide applications	Up to 40 TB
Server	1,000+	From a slab mounted in a rack to something that fits on a desk	Providing websites, or access to databases, applications and files	Up to 512 GB
Workstation	Typically one	From a desktop machine to the size of a filing cabinet	Engineering, medical and graphic design work	Up to 512 GB

CLASS	SIMULTANEOUS USERS	PHYSICAL SIZE	TYPICAL USE	MEMORY
Personal computer	One	Fits on a desk	Personal productivity	8 GB to 128 GB
Mobile device	One	Handheld	Personal productivity	512 MB to 16 GB

What survives the drift is the ordering and the ratios, and those carry most of what a manager actually needs from the table.

Simultaneous users is the axis that matters — a personal computer serves one person and a mainframe serves ten thousand at once, a gap of four orders of magnitude that explains nearly every other difference along the row.

Cost follows difficulty, not size or popularity — a supercomputer can cost roughly twenty times a mainframe while often serving fewer people, because it is bought for problems nothing else can finish rather than for volume of routine work.

Two classes serve one user each and cost very differently — a workstation can cost twice a personal computer and look identical from across the room, because what the extra money buys is rendering power for one demanding person.

Pair each class with what it was optimized for, because the optimization is the part still worth knowing next year, long after the memory figures have gone stale.

MATCH

Six classes, six things they are optimized for

Pair each class of computer with what the chapter says it is built to be good at; the explanations say what each pairing rules out as well as what it rules in.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Personal computer

Personal computing and small business computing, for one person at a time This is the chapter's reference machine, and the whole comparison table is really a description of how the other five differ from it. Its defining limit is the single user, not its speed.

Workstation

Visualizing and rendering three-dimensional models for a single demanding user Medical, engineering, architectural, animation and graphics work all need a fast processor, large

memory and advanced video hardware. The user count stays at one, which is why it is not a server however expensive it gets.

Server

Access by many concurrent users, with high reliability and fast network connectivity The chapter defines a server by what it does for a network rather than by its size, and by the fact that many people reach it at once. That is what buys it more processing power, memory and storage than a single-user machine.

Mainframe

High availability, heavy shared use and security for mission-critical transaction processing This is the main, central computing system of a major organization, the machine the chapter reaches for when ten thousand or more users depend on the same records and the work cannot stop.

Supercomputer

Massive scientific problems that no ordinary machine could finish in a useful time It is the most expensive and most powerful class, and the chapter says plainly that business organizations typically do not use one. It is bought for a problem, not for a workload.

Mobile device

One person's work, done on a handheld machine that travels with them Tablets and smartphones are in the table because they became part of the infrastructure rather than an accessory beside it, supporting today's working arrangements as well as personal use.

Servers, because almost everything you use is one

One of the six classes deserves its own paragraph, because it is the one you interact with constantly and never see. A **server** is any computer on a network that makes access to files, printing, communications and other services available to the users of that network. Every website, every shared drive and every company email system is one.

What makes a server a server is not its shape, since some are no larger than a desktop machine. It is what the machine is optimized for. Because it is reached by many concurrent users, it carries more advanced processors, more memory and more disk storage than a single-user computer, and it is built for high reliability and fast network connectivity, because the machine a thousand people depend on cannot be the machine that reboots.

Organizations run many servers in many configurations, and the chapter draws one distinction among them that explains a surprising amount about cost.

Some hand the same page to everyone — informational websites give every visitor identical content, so the machine is mostly handing out something it prepared once and can hand out again unchanged.

Others build a page for each person — a social network assembles a different page for every visitor out of that person's own network of friends, so the machine does fresh work for each request instead of repeating itself.

And plenty never serve pages at all — email servers, print servers and file servers do entirely different jobs, with correspondingly different requirements in processing power, network connectivity and software.

A request answered from something prepared earlier and a request answered by building something new are two different amounts of machine, which is why two sites that look identical to a visitor can cost wildly different sums to run.

CHECK YOURSELF

Classes of computer, and what a server is for

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. The chapter's sentence introducing the classes of computer says there are six and then names five. Which class does that sentence leave out, and how can you tell it belongs?

- **A.** The mobile device, which the sentence adds afterwards and which the chapter's own comparison table carries as a sixth row — **correct**. The sentence names personal computer, workstation, server, mainframe and supercomputer, then adds mobile devices as an afterthought; the table settles it by listing six rows, mobile devices among them.
- **B.** The embedded system, which is a computer built into another product and so ought to be counted among the general classes — Embedded systems are real and important, but the chapter deliberately places them outside the six general classes, under hardware that is not a general-purpose computer at all.
- **C.** The workstation, which is easily mistaken for a personal computer and is the class most often dropped from such lists — The workstation is named explicitly in that sentence and has its own row in the table, with its own definition about rendering three-dimensional models for a single user.
- **D.** Nothing is left out, because the chapter counts the personal computer and the notebook computer as two separate classes — The chapter treats personal computers and notebook computers together in one class, both used for personal and small business computing, so splitting them does not produce the missing sixth.

2. A hypothetical national parcel carrier keeps its billing and consignment records on one central system that ten thousand staff use simultaneously and that is not permitted to stop. A consultant suggests buying a supercomputer instead. What does the chapter's comparison say?

- **A.** A supercomputer fits, because it is the most powerful class available and this is the carrier's most demanding system — Power is not the axis here. The chapter says supercomputers are used primarily for massive scientific problems and typically not by business organizations; buying one would spend a fortune on the wrong strength.
- **B.** A mainframe fits, because it is optimized for high availability, heavy shared use and security, and is the chapter's machine for mission-critical transaction processing — **correct**. Ten thousand simultaneous users on records that cannot go down is precisely the mainframe description: the main central computing system of a major organization, optimized for availability, shared use and security.
- **C.** Several servers fit better than either, since the chapter describes servers as the class that handles more than a thousand simultaneous users — The figure is right and the conclusion is not. Servers do serve a thousand or more users, but the chapter reserves enterprise-wide transaction processing at this scale and this availability requirement for the mainframe.
- **D.** The two classes are interchangeable at this scale, since the table describes both as serving anywhere from one user to many — The table gives supercomputers one to many users and mainframes ten thousand and more, and it gives them entirely different typical uses, so they are not interchangeable in either direction.

3. Two hypothetical sites look the same to a visitor. One is a public library's opening-hours page, identical for everyone. The other is a member portal that assembles a different page for each person out of that person's own records. Why does the chapter treat these as different requirements?

- **A.** Because the second must be a mainframe, since building a page for each person counts as mission-critical transaction processing — Nothing in the situation says the records cannot go down or that ten thousand people use them at once. The chapter's dynamic-page example is an ordinary web server, not a mainframe.
- **B.** Because the first is an output device and the second is a general-purpose computer, which is the distinction the chapter draws whenever a machine hands something out rather than working something out — Output devices are monitors, printers and speakers. Neither of these machines is one, and the difference the chapter is drawing is between two kinds of server rather than between hardware families.
- **C.** Because the first hands out content prepared once, while the second does fresh work for every request, so the two differ in processing power, network connectivity and software — **correct**. The chapter contrasts servers that display the same static pages for every visitor with servers that create pages dynamically from each user's request, and says these have different requirements in processing power, network connectivity and software.

- **D.** Because the second is a server and the first is not, since a machine handing out an unchanged file provides no service to the network — Both are servers. The chapter's definition is any computer on a network making access to files, printing, communications or other services available to users, and handing out a page is exactly that.

The computers nobody calls computers

Everything so far has a screen and a user. Most of the computing in a modern economy has neither. The second half of the chapter's treatment of hardware is the machinery that does one job, invisibly, for years, and it is where the word "computer" stops being a useful guide.

Embedded system — a computer designed to perform one specific set of tasks well and built into something nobody thinks of as a computer, from a media player to the unit managing engine performance or a traffic light.

Programmable logic controller — a specialized computing device used to automate machines and processes, in settings as different as a manufacturing plant and a ski lift, running one control task reliably in conditions no office machine would survive.

Input device — hardware whose job is getting data into a computer, which is what a mouse, a touch screen, a camera or a scanner at a receiving dock is for.

Output device — hardware that presents results to a person, such as a monitor, a printer or a speaker, and the reason a system that decides something nobody can read has not finished its work.

Sensor — an input technology that measures the physical world, reporting temperature, atmospheric pressure, vibration or proximity in real time, which is how a system observes instead of waiting to be told.

Sensors are why this category has exploded. The **Internet of Things** is a vast network of interconnected physical devices that collect and share data, often producing enormous volumes of it that can be gathered centrally and analyzed to give insight and control over real-world situations. Objects that were connected to nothing now report continuously.

When that reporting is wired tightly enough to control something, the arrangement has its own name. A **cyber-physical system** integrates computing with physical processes, coordinating software, sensors and mechanical components closely enough to monitor and control physical activity with a high degree of autonomy and efficiency.

The chapter's own example is one of Siemens' electronics factories, where a combination of sensors, controllers and manufacturing machines automates about three-quarters of the production process and generates something like fifty million readings a day. Industrial devices such as intelligent product codes let products communicate directly with machines, so production processes reorganize themselves without a person intervening.

Why this kind of system is harder to protect. The chapter is careful here, and its caution transfers to everything else in this module. Cyber-physical systems present hard problems for three reasons: the systems are densely interconnected, that interconnection creates security vulnerabilities, and their actions land in the physical world.

A failed web page is an inconvenience. A failed controller opens a valve, stops a line, or moves something heavy at the wrong moment. That difference is why a later section returns to vehicles, which are now computers on wheels.

Sort a handful of devices into the three families now, before the vocabulary settles into a blur.

SORT

General-purpose computer, special-purpose device, or input and output

Place each piece of hardware in the family the chapter puts it in. Every situation described here is a hypothetical practice situation, and each one states the conditions you need.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

General-purpose computer — one of the six classes: it runs whatever software is installed on it, and the classes differ mainly in scale and in how many people it serves

- A machine in the back room of a hypothetical online shop that holds the product catalogue and answers a thousand shoppers at once — This is the chapter's server: a computer on a network making access to files and services available to many concurrent users, which is what buys it the extra processing power, memory and reliability.
- The central system of a hypothetical insurer, used simultaneously by ten thousand staff for transaction processing and never permitted to stop — Ten thousand simultaneous users on mission-critical transaction processing is the chapter's description of a mainframe, optimized for high availability, heavy shared use and security.
- The machine a hypothetical animation studio buys so one artist can render three-dimensional scenes — A workstation is still a general-purpose computer serving one user; what separates it from an ordinary desktop is the processor, memory and video hardware needed for visualization and rendering.

- The handheld a hypothetical delivery driver uses to receive the day's jobs, record what was delivered and take payment — Mobile devices are the sixth class in the chapter's table, and this one is doing organizational work rather than personal work, which is exactly why the chapter counts them as infrastructure.

Special-purpose device — built into something else to do one job for years, usually with no screen, no keyboard and nobody sitting at it

- The computer inside a media player that will only ever play discs and that nobody will ever install other software on — This is the chapter's definition of an embedded system: designed to perform one specific set of tasks effectively, inside a product nobody thinks of as a computer.
- A small unit at a hypothetical resort that starts, stops and speeds up a ski lift according to a fixed control program — The chapter names ski lifts alongside manufacturing plants when it defines a programmable logic controller, a specialized device used to automate machines and processes.
- The unit built into a car that manages engine performance and is never used for anything else — Controlling engine performance is one of the chapter's own examples of an embedded system. The giveaway is that the task set is fixed at manufacture rather than chosen later by a user.

Input or output device — it does not do the deciding; its job is getting data into a system, or getting a result out to a person

- A touch screen on a self-service ordering kiosk in a hypothetical cafe — The chapter lists touch screens with mice and cameras as input devices. It gathers what a person wants and passes it on; the machine behind it does the processing and the deciding.
- A printer in a hypothetical warehouse producing the list of items a picker must collect — Printers are one of the chapter's output devices, and this is the moment a system's decision becomes something a person can act on. Nothing is being computed by the printer itself.
- A small unit clamped to a motor in a hypothetical plant that measures vibration and reports it continuously to a maintenance system — This is a sensor, which the chapter treats as an input technology: it measures vibration in the physical environment and supplies that information in real time to something else that decides.
- A reader mounted in a doorway at a hypothetical depot that picks up identifiers from tags on passing cartons and hands them to the stock system — The reader's entire job is getting data into a system without anyone typing or aiming a scanner, which is what an input device does. The Internet of Things widened this category rather than creating a new one.

Tags that answer when asked

One special-purpose technology gets its own treatment in the chapter, because so much of modern logistics rests on it. **Radio frequency identification**, usually shortened to RFID, uses electromagnetic fields to move data between a reader and a small **tag** attached to an object.

Tags run from a fraction of an inch across, small enough to sit beneath an animal's skin, up to several inches, fixed to a product or a shipping container. What they carry ranges from the name of a pet's owner to instructions for how a product is to be manufactured on a shop floor. The comparison the chapter draws is with the ordinary bar code, and it lists six advantages.

- 1 **No line of sight is needed** — the reader does not have to see the tag, so nothing has to be turned, unpacked, or held at exactly the right angle first.
- 2 **No hand scanning** — the time-consuming business of a person pointing a scanner at each item disappears, and that is where most of the labour saving comes from.
- 3 **Position stops mattering** — the data is readable regardless of how the item happens to be sitting, or whether the tag is plainly visible at all.
- 4 **More data fits** — a tag can hold considerably more than the single identifier a bar code carries, which changes what the scan can tell you.
- 5 **The data can be chosen and changed** — a company can program whatever it wants onto a tag, such as version, origin, location or maintenance history, and update that record later.
- 6 **Range is greater** — scanning can happen at a distance rather than at arm's length, which is what lets a doorway count a whole pallet as it passes through.

Tags come in two kinds, and choosing between them is a straight cost decision rather than a technical one.

Passive tags — small and inexpensive, starting from a few cents each, carrying no battery, and typically read from up to several feet away, which suits anything shipped in bulk.

Active tags — several dollars each, containing a battery of their own, readable from hundreds of feet, which suits containers and equipment valuable enough to justify the cost.

The chapter says RFID is rapidly replacing traditional bar codes. Treat that as directional rather than settled: bar codes remain entirely ordinary at a supermarket checkout, and tags win where an item is valuable enough to carry the cost of one. What is not in doubt is the capability — real-time tracking of goods and materials through a supply chain, which genuinely bridges digital records and physical operations.

Inside the box: why an artificial-intelligence project has a different budget line

The last piece of hardware worth a manager's attention is the processor, because this is where one technology proposal stops being interchangeable with another. For decades a server was essentially a large personal computer: storage holding the data, a **central processing unit** doing the calculating, and temporary memory holding instructions and intermediate results.

A central processing unit is built to be versatile and to handle instructions in sequence, very quickly, each result feeding the next. That is the right design for ordinary software, where step two genuinely needs the answer from step one. Recalculating a spreadsheet is the everyday case: a formula may have to wait for the cell above it.

Training a machine-learning model is not that shape of problem, and the chapter explains the difference using pictures. Rendering an image means working out the colour value and intensity of every pixel, and each pixel can be worked out without knowing anything about its neighbours. A **graphics processing unit** was designed for exactly that: thousands of small cores performing the same operation on many pieces of data at the same time.

Training turns out to have the same shape, because the matrix and vector calculations underneath deep learning can largely run side by side rather than in a queue. That is the whole reason a kind of chip built for rendering images became standard equipment for machine learning, and the specialization has continued past it.

Tensor processing units — custom-designed by large providers to optimize the matrix calculations that dominate deep learning, and faster than general graphics hardware on the particular work they were shaped around.

Field-programmable gate arrays — processors whose circuitry can be reconfigured after manufacture, so a team with one unusual calculation can reshape the hardware around it instead of waiting for a chip nobody will ever make.

Neuromorphic chips — designs inspired by the architecture of the human brain, aimed at improving energy efficiency and processing speed for artificial-intelligence work rather than at raw general throughput.

Wafer-scale engines — the chapter names Cerebras as an example of integrating vast numbers of cores on a single piece of silicon in order to handle very large models efficiently.

The business consequence is simple, and it is worth carrying out of this section. A proposal to train models is not a proposal to buy more of the machines you already own. It is a proposal to buy a different kind of machine, at a different unit price, and the later section on what drives infrastructure change comes back to what that does to a budget.

EXPLORE

Four kinds of processor, and what choosing one commits you to

Open each card and read all four panels; the last card sets the families against each other, which is where the reason for all this specialization becomes visible.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Central processing unit · The general-purpose processor

- **What it is:** The processor that performs a computer's calculations, built for versatility across a very wide range of operations, which is why it sat at the heart of server architecture for decades.
- **The kind of work it suits:** Sequential work, where instructions and results depend on one another, which the chapter says describes most traditional software.
- **The example that makes it click:** Recalculating a spreadsheet, where a formula may need the answer from the cell above it before it can run at all, so nothing is gained by having a thousand helpers standing idle.
- **What it means for a budget:** It is the default line in almost every hardware budget, and for transaction processing, reporting and ordinary business applications it is also the correct one. Nothing here needs a special case.

Graphics processing unit · Thousands of small cores working at once

- **What it is:** A processor optimized for parallel processing, performing the same operation on very many pieces of data simultaneously using thousands of small cores.
- **The kind of work it suits:** Work that splits into many identical, independent pieces, where no piece has to wait for another piece's answer before it can start.
- **The example that makes it click:** Rendering an image, where determining the colour value and intensity of one pixel needs nothing from the surrounding pixels, so every pixel can be handled at the same moment.
- **What it means for a budget:** Faster training times made these a staple of machine-learning infrastructure, which means a modelling project buys different hardware than a database project and should be costed separately from the start.

Tensor processing unit · Custom-built for one shape of calculation

- **What it is:** A processor developed by large providers and custom-designed to optimize the tensor computations that dominate deep learning, using a matrix-based architecture.
- **The kind of work it suits:** The narrow band of work it was designed around, where the chapter reports it outperforming general graphics hardware on specific tasks.
- **The example that makes it click:** The provider's own language models, processed in batches on hardware the same provider designed, and then rented out to customers doing work of a similar shape.
- **What it means for a budget:** You are unlikely to buy one; you rent time on it. That is the first place in this module where owning and renting stop being equivalent choices, and the cloud sections pick the thread up.

Reconfigurable and experimental silicon · Gate arrays, brain-inspired chips, whole wafers

- **What it is:** Three further steps along the same road: field-programmable gate arrays offering customizable hardware acceleration, neuromorphic chips inspired by the brain's architecture, and wafer-scale engines integrating vast numbers of cores.
- **The kind of work it suits:** Work specialized enough that no general processor fits it well, or constrained enough by electricity that ordinary hardware is uneconomic to run.
- **The example that makes it click:** A team with one unusual calculation reshaping a gate array around that calculation, rather than waiting for a chip that no manufacturer has any reason to build.
- **What it means for a budget:** These are research and niche lines rather than standing budget items, but they are the reason to ask what a proposal actually runs on instead of assuming every machine is priced alike.

The four compared · Why specialization keeps increasing

- **What it is:** A progression from general-purpose processors toward increasingly specialized hardware, which the chapter presents as the industry's response to the escalating demands of machine learning and generative artificial intelligence.
- **The kind of work it suits:** Every step trades flexibility for performance on a narrower band of work, which is a bargain worth making only when the work is both demanding and predictable.
- **The example that makes it click:** The same chip family that renders video games became standard for training models, because the underlying calculation had the same independent, repeated shape in both cases.
- **What it means for a budget:** The manager's version of this whole box is one question: does this project run on the machines we already buy, or on a different kind of machine at a different price? The answer changes the number before anyone negotiates.

Where hardware stops

One sentence in the chapter marks the edge of this section and is the hinge into the next. The application software used for business processes cannot interact with all this hardware directly. It interacts with the system software, and the system software interacts with the machine.

That is not a technicality. It is the reason a business can buy a program without knowing what is inside its computers, and the reason the same program runs on a laptop, on a rack-mounted server and on a handheld device. The next section takes that middle layer apart.

Name the machine and you have named the constraint: how many people it can serve at once, whether it does one job or any job, and whether it works in sequence or in parallel. Almost every hardware argument a manager will ever sit through is one of those three questions wearing a different coat.

CHECK YOURSELF**Devices, tags, and processors**

Three questions on the hardware that is not a general-purpose computer; each explanation says what the wrong option genuinely describes.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical dairy installs small units that start and stop its bottling conveyors according to a fixed control program. None of them has a screen or a keyboard, and no one will install anything else on them. What does the chapter call these, and what separates them from the computers in the office?

- **A.** Servers, because they make a service available to the other machines on the plant network and are reached by many devices at once — A server provides files, printing, communications and other services to users of a network. These units are controlling machinery, and nobody is requesting anything from them.
- **B.** Input devices, because they take signals from the line and act on them, which is what the chapter says an input device does — An input device supplies data to a computer and stops there. These units decide and act, which is the difference between reading a sensor and running a conveyor.
- **C.** Embedded systems, because any computer without a screen is by definition embedded inside the product it controls — The absence of a screen is not the test. Embedded systems sit inside a consumer product such as a media player or a traffic light; the chapter gives control of machines and processes, including ski lifts, its own name.
- **D.** Programmable logic controllers, which are specialized devices used to automate machines and processes rather than to run whatever software someone installs — **correct.** The chapter defines programmable logic controllers as specialized computing devices used to automate machines and processes, in environments from manufacturing plants to ski lifts, and that fixed job is what separates them.

2. A hypothetical equipment yard wants to locate trailers parked across a site several hundred feet wide, and separately wants to identify low-value cartons as they pass through a doorway. What does the chapter's account of tags suggest?

- **A.** Active tags for the trailers, because they carry a battery and can be read from hundreds of feet, and passive tags for the cartons, because they cost a few cents and need only a few feet — **correct.** The chapter gives passive tags a range of up to several feet at a cost starting from a few cents, and active tags a battery, a cost upward of several dollars and a range of hundreds of feet, which matches the two requirements exactly.
- **B.** Passive tags throughout, because they cost a fraction of the alternative and the real difference between the two kinds is how long they survive being knocked about rather than how far away they can be read — Durability is not the distinction the chapter draws. Passive tags simply cannot be read from hundreds of feet, so the trailers across the yard would go unlocated however many tags were bought.

- **C.** Active tags throughout, because using a single kind of tag simplifies the readers and the extra cost per tag is trivial at any volume — Cost is not trivial when the tagged item is a low-value carton. Several dollars a tag against a few cents is the whole reason the chapter describes two kinds instead of one.
- **D.** Bar codes for both, because the chapter presents tags as a replacement that has not actually arrived in practice — The chapter actually claims tags are rapidly replacing bar codes, and this module softens that rather than reversing it. The doorway requirement alone rules bar codes out, since they need line of sight and hand scanning.

3. Why, according to the chapter, did hardware built for rendering images turn out to suit the training of machine-learning models?

- **A.** Because graphics hardware runs instructions one after another faster than a general-purpose processor does, and training a model is mostly one long chain of steps each of which waits for the one before it — This reverses the chapter's argument. Sequential processing is what the central processing unit is designed for, and the point about training is precisely that it does not have to happen in sequence.
- **B.** Because models are stored as images, so the hardware that displays pictures is also the hardware that can read them — Models are not stored as images, and nothing in the chapter suggests they are. The resemblance is in the shape of the calculation, not in the shape of the data.
- **C.** Because graphics hardware carries far more memory than a general processor, and memory is the binding constraint when training a model — Memory matters, but it is not the explanation the chapter gives. Its account rests on parallelism: thousands of cores each doing an independent piece of the same work.
- **D.** Because both jobs consist largely of the same calculation repeated over many pieces of data that do not depend on one another, so thousands of small cores can work at the same time — **correct.** Determining the colour value and intensity of a pixel is independent of the surrounding pixels, and the matrix and vector calculations underneath deep learning are independent in the same way, which is why parallel hardware suits both.

System software and storage

A machine full of hardware still does nothing on its own. Something has to tell the processor to read the keyboard, the drive to save the file, and the printer to wake up — and it cannot be the spreadsheet you are working in, because the spreadsheet has no idea what machine it is running on. This section covers the software that runs the machine rather than the work, and then the place all that work ends up: storage.

The software that runs the machine, not the work

Sort every program on a computer into two piles. In one pile is the software the computer was bought for: the word processor, the accounting package, the browser. In the other is the software that makes the first pile possible by driving the machine itself. That second pile has a name. **System software** is the collection of programs that control the basic operations of computer hardware.

The most prominent kind of system software is the **operating system**, and you already know several by name. The chapter's examples are Windows 11, macOS and Ubuntu Linux on computers, and Android and iOS on phones, the last two optimized for mobile devices. Its job is coordination, and it has four different things to coordinate.

Hardware components — the parts inside the machine that must work together, such as the processor and the monitor, which have no way of being aware of each other on their own.

Peripherals — devices attached to the computer, such as a printer, which is why any application can print without every application carrying instructions for every printer ever made.

Application software — the programs a person actually opens, which ask the operating system for what they need instead of reaching for the hardware themselves.

Users — the people giving the instructions, whose clicks and keystrokes have to be turned into something the electronics can act on.

The order in that list is also the shape of the machine. A request travels from a user, into an application, down to the operating system, and only then out to the hardware and the devices attached to it. Application software never talks to hardware directly, which is the fact the rest of this section is built on.

Two piles, two questions. When someone says “the software is slow,” the useful first question is which pile they mean. Application software is chosen by the business for the work it does; system software is chosen because of what hardware the organization owns and what it needs that hardware to support. They are bought differently, replaced on different schedules, and blamed for each other constantly.

One layer, many machines

Here is the idea worth carrying out of this section, because it explains far more than operating systems do. The operating system provides a common layer for different underlying devices, so that applications only have to be developed for different operating systems rather than for each different computer model.

Count what that saves. A software company whose customers own several hundred different models of laptop does not write several hundred versions of its product. It writes one version for each operating system it supports, and every machine running that operating system is covered. This is why software is sold as a version for one operating system and a version for another, rather than a version for your exact laptop.

Underneath the common layer, the differences between machines still have to be handled by somebody. **Device drivers** allow the computer to communicate with various different hardware devices: a small piece of software for one make and model, written once, that teaches the general layer how to talk to one particular thing. A printer released years after your operating system shipped works because a driver arrived for it, not because the operating system was rewritten.

Operating systems are often written in **assembly language**, a very low-level computer programming language that allows the computer to operate quickly and efficiently. Part of why that language stays out of sight is deliberate: the operating system is designed to insulate you from it and to make computer operations unobtrusive.

What it does while you are not looking

Many tasks are common to almost all computers, and the operating system performs every one of them: getting input from a keyboard or mouse, reading from and writing to a storage device such as a hard disk drive, and presenting information to you on a monitor. It also carries out the day-to-day operations nobody notices, such as updating the system clock, printing documents, and saving data to a hard drive.

The chapter’s comparison for all of that is a fair one. Just as our brain and nervous system control breathing, heartbeat and the senses without our conscious realization, the operating system transparently controls the computer’s basic operations. You notice it only when it stops.

The part you do notice has a name too. A **graphical user interface**, or GUI, is provided by operating systems such as Windows or iOS, as well as by different versions of Linux, and it enables you to send instructions to the computer by selecting or manipulating pictures, icons and menus rather than by typing them out.

The chapter's own example of that interface is worth walking through slowly. To copy a word processing file from a flash drive onto your computer, you point at a graphic icon of the file, then click and drag it onto an icon of your hard disk. That is the whole of what you do.

Underlying the icons and the simple dragging is a complex set of coded instructions telling the electronic components of the computer that you are transferring a set of bits and bytes from the flash drive to a location on the internal hard disk. Imagine having to type that set of instructions every time you wanted to copy a file.

The diagram below redraws the same relationship three ways: what the operating system stands between, why one layer can cover many machines, and what a single gesture actually sets in motion.

INTERACTIVE DIAGRAM

What the operating system sits between

Pick a view to see the same relationship from a different angle: who talks to whom, why one layer covers many machines, and what a single drag of the mouse actually starts.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

The operating system as coordinator — The centre of the machine, where every request from a person eventually arrives

Users (Clicks, keystrokes and gestures) → Application software (The word processor or accounting package the work is done in) → Operating system (Coordinates all four of the things around it) → Hardware and peripherals (Processor, monitor, drives, printers)

- The chapter's figure draws the operating system as a circle encompassing everything else, with arrows flowing between it and users, application software, hardware components and peripherals.
- Nothing in the outer ring communicates with anything else in the outer ring. An application does not talk to a printer; it asks the operating system, which knows how to reach the printer.
- That is why a **peripheral** only has to be understood once. Add a printer and every application on the machine can print, because printing was never the application's job.
- It also explains a familiar frustration: when one program cannot see a device, the fault is usually in the coordinating layer or its **device driver** rather than in the program complaining about it.

One common layer, many machines — The join between software that is written once and hardware that comes in hundreds of shapes

Applications (Written for an operating system, not for a model of computer) → Operating system (The common layer, plus a driver for each device) → Many different machines (Desktops, laptops and everything else running that system)

- The chapter's second figure stacks applications on top of the operating system with a double-headed arrow between them, then three arrows down to three different machines.

- The sentence to remember: the operating system provides a common layer for different underlying devices so that applications only have to be developed for different operating systems rather than for each different computer model.
- The arrow between applications and the operating system points both ways because the traffic runs both ways: the application asks, and the system reports back what happened.
- **Device drivers** are what make the bottom arrows possible. Each one teaches the common layer to communicate with one particular device, so hardware variety is absorbed in one place instead of in every program.

One gesture, a long sequence underneath — The chapter's own example: copying a file from a flash drive to a hard disk

You drag an icon (Point, click, drop it on the hard disk) → The graphical interface (Turns the gesture into an instruction) → The operating system (Issues the complex set of coded instructions) → The hardware (Bits and bytes move from one device to the other)

- A **graphical user interface** lets you send instructions by selecting or manipulating pictures, icons and menus, which is the only part of this sequence you ever see.
- Underlying the icons and the simple dragging is a complex set of coded instructions telling the electronic components that a set of bits and bytes is being transferred from the flash drive to a location on the internal hard disk.
- The chapter's challenge is a good one: imagine typing that whole set of instructions every time you wanted to copy a file. The interface is not decoration; it is the reason the machine is usable without training.
- The same hiding happens one level down, where the system is often written in **assembly language** and is designed to insulate you from it — the layer exists so that nobody above it has to learn the language below it.

Code somebody else already wrote

Operating systems are not the only software an organization needs in place before it can build anything of its own. A solid infrastructure also includes **frameworks and libraries**, which offer collections of prewritten code. The chapter names NodeJS, JQuery, React and TensorFlow; read those as its illustrations of a category rather than as a current shopping list, because the names in fashion change much faster than the idea does. Three things they buy an organization.

Development time — work already solved once by somebody else is reused instead of written again, which is how a small team ships something substantial in weeks rather than years.

Reach across platforms — some frameworks enable building applications that run on different platforms, so a firm is not paying to build the same product twice for two audiences.

Specialized capability — libraries for work such as developing machine learning models put techniques within reach of teams that could never have written them from scratch.

Notice the managerial shape of this, because it recurs everywhere in the module. Choosing a framework is choosing a dependency. It shortens the build, and it ties the organization to somebody else's decisions about what to keep supporting.

Before moving to storage, test the vocabulary this half of the section introduced.

CHECK YOURSELF

System software, drivers, and the common layer

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A colleague says the accounting package on her laptop “talks to the hard drive when it saves.” What does the chapter’s account of system software say is actually happening?

- **A.** She is right, because application software reaches the hardware directly and the operating system is only there to manage the screen, the keyboard and the mouse while the application gets on with the work — The operating system does manage the screen and the mouse, but it does far more than that. Reading from and writing to a storage device is one of the tasks the chapter lists as performed by the operating system itself.
- **B.** The accounting package asks the operating system, which is the system software controlling the basic operations of the hardware, and the operating system reaches the drive — **correct.** System software is the collection of programs that control the basic operations of computer hardware, and the operating system coordinates the interaction between hardware components, peripherals, application software and users, so the save request travels through it.
- **C.** The accounting package writes to the drive itself, and the operating system checks afterwards that the file arrived intact — If applications wrote to hardware themselves, every application would need to understand every make of drive. Verifying afterwards is not the arrangement; the operating system does the writing, not the inspecting.
- **D.** The hard drive requests work from whichever application is open, so the package responds rather than instructs — Hardware does not request work from applications. A drive is a device the operating system reaches on behalf of whatever asked, which is why one program cannot seize a device from another.

2. A hypothetical print shop buys a printer model released long after the operating systems on its computers were written, and the printer works. Which piece of the chapter's account explains that, and what is assembly language doing in the same story?

- **A.** A device driver lets the computer communicate with that particular piece of hardware, and the operating system is written in assembly language so that nobody else has to be — **correct.** Device drivers allow the computer to communicate with various different hardware devices, and the chapter says operating systems are often written in assembly language, a very low-level language, with the system designed to insulate you from it and make operations unobtrusive.
- **B.** The graphical user interface recognizes the printer visually, and assembly language is the language users type instructions in when the interface cannot help — A graphical user interface enables you to send instructions by selecting or manipulating pictures, icons and menus. It is how a person addresses the machine, not how the machine identifies an unfamiliar device.
- **C.** The printer contains its own operating system that replaces the computer's for the duration of the job, and assembly language is what applications are written in — Nothing in the chapter puts a replacement operating system inside a peripheral, and assembly language is described as the language operating systems are often written in rather than the language of applications.
- **D.** Frameworks and libraries supply the printing code to each application, and assembly language is a kind of storage medium used while a document waits — Frameworks and libraries are collections of prewritten code that reduce development time and help applications run on different platforms; they are not a substitute for a driver, and assembly language is a programming language rather than a medium.

3. A development team proposes building its new application on a widely used framework instead of writing everything itself. What does the chapter say frameworks and libraries provide, and what should the manager also notice?

- **A.** They provide finished applications the firm can rebrand, so the manager should notice that the product will look like competitors' products — A framework is a set of building blocks rather than a finished product, and the chapter describes it as supporting the development and operation of applications rather than as something to rebrand.
- **B.** They provide the operating system functions the application needs, so the manager should notice that the firm no longer needs to license an operating system — Frameworks sit above the operating system rather than replacing it. An application built on a framework still runs on an operating system, and the licensing question is untouched by the choice.
- **C.** They provide collections of prewritten code that cut development time and help an application run on different platforms, so the manager should notice the firm is taking on a dependency — **correct.** The chapter says frameworks and libraries offer collections of prewritten code, helping to reduce development time, enable building applications that run on different platforms, or develop machine learning models — and reusing somebody else's code means depending on their future decisions.

- **D.** They provide faster access to the hardware by going round the operating system, so the manager should notice that the application will need a device driver of its own for every machine it is installed on — Nothing bypasses the operating system in this account; applications reach hardware through it. Drivers belong to devices rather than to applications, so a framework choice does not create one.

Why storage is a component and not a detail

Processing and analyzing data is only half the job. Efficiently storing and retrieving it is the other half, and the amount organizations must hold keeps growing at an increasing rate. Two separate pressures push the same way. The business itself generates more data every year, and in some industries and jurisdictions regulations require organizations to keep records — business documents and internal communication such as email and instant messages included — long after anyone last had a use for them.

That second pressure is a requirement about time, not about speed, and the difference is the first clue that not all stored data wants the same treatment. Keeping a decade of old correspondence on the fastest storage the organization owns is spending money to solve a problem it does not have.

Three kinds of data, and the four questions that separate them

The chapter distinguishes three distinct types of data by their purpose, and gives each a definition worth keeping exactly as it is written.

Operational data — data used for managing business processes, such as processing transactions, or for data analysis; this is the data the business is running on right now.

Backup data — short-term copies of organizational data, used to recover from a system-related disaster, and frequently overwritten with newer backups as time passes.

Archival data — long-term copies of organizational data, often kept for compliance and reporting purposes rather than for any current operational use.

What actually separates them is a set of four requirements the chapter names: **timeliness**, **access speed**, **searchability** and **life span**. Ask those four questions about any pile of data and the right home for it usually answers itself.

REQUIREMENT	OPERATIONAL DATA	BACKUP DATA	ARCHIVAL DATA
Timeliness how current it must be	Current to the second; a stale figure is a wrong figure	Recent; yesterday's copy is useful, last year's is not	Old by definition; it is kept because it is old

REQUIREMENT	OPERATIONAL DATA	BACKUP DATA	ARCHIVAL DATA
Access speed how fast it must be reachable	Immediate, because a customer or a process is waiting	Fast enough to restore without stopping the business	Slow is acceptable; it is read rarely, if ever
Searchability whether it must be findable	Yes; the whole purpose is answering questions from it	Not really; it is restored wholesale, not queried	Not quickly, though it must be producible on request
Life span how long it must last	As long as it is current, then it is replaced	Short; a newer backup overwrites it	Years, which is the requirement everything else bends to

Read that table down the columns rather than across the rows. Each column is a different set of demands, and no single storage technology satisfies all three columns well, which is precisely why organizations run more than one.

Matching the data to the medium

These different uses of organizational data call for different physical storage technologies, and the chapter matches each type to one.

Operational data goes on fast disk — typically in databases or in files, on disk-based media such as hard drives, which offer high access speeds and are preferred where data is frequently accessed or where response time is of the essence, as on an e-commerce site.

The most demanding cases go on flash — flash-based storage such as solid-state drives is used where access speed is of crucial importance, and machine learning is one such case, since it requires fast storage to analyze vast quantities of different data.

Backups go somewhere secure but still quick — organizations periodically back up data to a secure location, and storing backups on hard drives enables quick recovery without slowing the company's operations; some run completely redundant systems so business continues seamlessly if the primary systems fail.

Archives go on cheap removable media — data no longer used for operational purposes, such as old internal email, is archived for long-term storage, typically on magnetic tape kept away from the working systems.

Tape deserves a closer look, because its weaknesses are the reason it is the right choice. Data is stored sequentially on magnetic tape, so access speed can be very slow and the contents are not quickly searchable. In exchange, the chapter reports a shelf life of up to 30 years, a cost very low compared with other storage media, and removability — meaning the archive is highly expandable and tapes can be stored in a secure, remote location.

A caution the chapter leaves out. The shelf life quoted for tape is a figure about the medium, and it is the least of an archivist's worries. Archives are lost far more often because nothing can still read them — the format was abandoned, or the last working drive failed — than because the tape itself decayed. An archive lives exactly as long as the equipment and software that can open it, which is the obsolescence problem this module returns to when it turns to managerial issues.

Sort a set of situations by which of the three types is in front of you, and the medium follows from the requirements rather than from habit.

SORT

Which kind of data is this, and where does it belong?

Place each situation with the type of data it describes, then read why the requirements push it onto that medium. Every organization here is a hypothetical practice situation.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Operational data — used for managing business processes or for analysis; needed now, fast, and searchable

- The stock figure a shopping site reads the instant a customer adds an item to a basket — This is data used for managing a business process as it happens, and response time is of the essence, so it belongs on hard drives or on flash-based storage such as solid-state drives.
- Customer records and transaction data that staff query all day from a database — The chapter puts transaction-processing data and customer data in databases on disk-based storage media, because hard drives offer high access speeds for data that is frequently accessed.
- Business documents, images and company brochures that employees open during the working day — Not all operational data sits in a database. The chapter names business documents, images and brochures as operational content stored as files, still on fast disk because people are waiting for them.
- The large volume of varied data a machine learning project reads over and over while training a model — The chapter says AI and machine learning require fast storage to analyze vast quantities of different data, which is exactly the case flash-based storage exists for.

Backup data — short-term copies for recovering from a disaster, frequently overwritten by newer ones

- Last night's complete copy of the order system, written to a secure location in case this morning's server fails — This is a short-term copy made to recover from a system-related disaster. Keeping it on hard drives enables quick recovery without slowing the company's operations.
- A completely redundant second set of systems, held ready so that business continues seamlessly if the primary systems fail — The chapter describes redundant systems as the strongest form of the same idea: the copy exists so operations continue, so its whole value is in being recent and fast to bring up.
- A copy that tomorrow's copy will overwrite, because what matters about it is that it is recent — Backups are frequently overwritten with newer backups, which is what separates them from archives; their life span requirement is short by design rather than by neglect.

Archival data — long-term copies kept for compliance and reporting, rarely read but required to last

- Internal email from several years ago that nobody expects to open, but that the organization must be able to produce if it is ever asked — The chapter's own example of archival data is old internal email, kept for compliance and reporting and stored long-term on magnetic tape rather than on working systems.
- Records written sequentially to low-cost removable media and taken to a secure site some distance away — Sequential writing, very low cost and removability are the properties the chapter gives magnetic tape, and removability is what lets the archive be both expandable and stored remotely.
- A store of finished records that is slow to read and cannot be searched quickly, and where nobody minds — Slow access and poor searchability would be disqualifying for operational data. For an archive they are an acceptable trade for a life span measured in years at a very low cost.

Files, folders, and a hierarchy that is not really there

Within most general-purpose computers, digital information is organized in a structured and accessible way using files and folders. Three terms carry that structure, and the third one is the one people rarely have a name for.

File — a block of data that stores specific content, such as a document, an image or a spreadsheet, and the unit people name, move, share and delete without thinking about drives at all.

Folder — a grouping of related files, also called a directory, although strictly speaking a folder is the graphical element that represents a directory on the screen.

Root directory — the top of the tree, typically the drive itself, from which subdirectories branch out, with each directory able to contain files as well as other directories.

Arranged that way they form a hierarchy shaped like a tree, and keeping it tidy pays off as it grows. Organizing directories in a logical, consistent manner — nesting course materials inside one folder, then by term, then by course — makes information easier to find, share and manage as the volume of digital content grows.

Then comes the caveat that matters more than everything above it. This is a *logical* organization. Although you can reach data using files and folders, the data is not physically organized that way on the drive at all: one file may be scattered across many separate places, and two files sitting in the same folder on screen may be nowhere near each other on the disk.

That gap between the arrangement you see and the arrangement that exists is the same move the operating system made at the start of this section. A common layer hides the machine so the person above it can work in ideas — folders, documents, drag and drop — instead of in hardware.

Almost everything in this section is one idea wearing different clothes: put a layer in between, and the thing above it stops having to know how the thing below it works.

Two questions on storage requirements and two on how data is arranged, with every option explained.

CHECK YOURSELF

Storage requirements, media, and the shape of a file system

Four options each, with an explanation for every one, including the options you rejected.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical charity is told to “buy faster storage” for everything it keeps. Which four requirements does the chapter use to separate the three types of data, and why does the instruction fail?

- **A.** Cost, security, ownership and location, and the instruction fails because faster storage is harder to secure — Cost and security are real concerns and cost is precisely why tape is used for archives, but these are not the four requirements the chapter names for distinguishing the three types of data.
- **B.** Timeliness, access speed, searchability and life span, and the instruction fails because archives need life span rather than speed — **correct**. The chapter distinguishes operational, backup and archival data by their requirements in terms of timeliness, access speed, searchability and life span, and archival data is defined by a long life span at low cost rather than by speed.

- **C.** Volume, format, department and vendor, and the instruction fails because different departments buy different equipment — These describe how equipment gets purchased rather than what the data needs. The chapter's categories are about purpose, and purpose is what decides the medium.
- **D.** Accuracy, completeness, timeliness and relevance, and the instruction fails because speed cannot improve data quality — Accuracy, completeness and relevance are qualities of information rather than the storage requirements this section names, although the closing observation about speed and quality is sound.

2. Which arrangement matches what the chapter says about matching each type of data to a physical medium?

- **A.** Operational data on magnetic tape because it is written continuously, backups on flash storage, and archives on hard drives so they stay searchable — This reverses the media. Data is stored sequentially on tape, so access can be very slow and the contents are not quickly searchable, which is why the chapter reserves tape for archives.
- **B.** Everything on flash-based storage, because a slow read is a business risk wherever it happens — Buying the fastest medium for data nobody reads is the waste the three categories exist to prevent, and the chapter reserves flash for situations where access speed is of crucial importance.
- **C.** Operational data in databases or files on hard drives or flash storage, backups on hard drives for quick recovery, and archives on magnetic tape — **correct.** The chapter stores operational data in databases or files on disk-based or flash storage, notes that storing backup data on hard drives enables quick recovery without slowing operations, and archives data no longer used operationally on magnetic tape.
- **D.** Operational data on hard drives, backups written to the same drives to keep recovery simple, and archives deleted once they stop being read — A backup on the same drive as the live data disappears with it in the failure it was meant to survive, and archives are kept precisely because they may still be required for compliance and reporting.

3. A manager assumes that because two documents sit in the same folder on screen, they are stored next to each other on the disk. What is the chapter's correction, and why does it matter?

- **A.** The folder structure is a logical organization; the data is not physically organized the same way on the drive, which is another case of a layer hiding the machine — **correct.** The chapter is explicit that although you can access data using files and folders, the data is not physically organized the same way on the drive, and that gap is the same abstraction the operating system performs everywhere else.
- **B.** The folder structure is physical, and moving a file between folders therefore moves it across the disk surface — This is the belief being corrected. Dragging a file between folders changes where it appears in the hierarchy, and says nothing about where its contents physically sit.

- **C.** Folders exist only in the operating system's memory and vanish when the machine is switched off, so nothing about them is stored at all — The hierarchy persists across restarts, since a drive that forgot its own structure would lose access to everything on it. What is logical is the arrangement, not its permanence.
- **D.** Folders are the same thing as directories in every respect, so the question has no answer worth giving — The two words are close but not identical: the chapter says a folder is strictly a graphical element representing a directory, and in any case the physical question still has a definite answer.

4. What does the chapter mean by the root directory, and what follows from a hierarchy shaped that way?

- **A.** The folder that holds the operating system itself, which is why deleting anything inside it stops the machine from starting and why ordinary users are kept out of it — System files usually do live somewhere below the root, but that is not what the word names. The root is the starting position of the whole tree, whatever happens to be stored in it.
- **B.** The oldest folder on the drive, from which every later folder inherits its settings and permissions — Age has nothing to do with it, and the chapter does not describe folders as inheriting settings from one another. The root is defined by position in the hierarchy, not by history.
- **C.** The top of the tree, typically the drive itself, from which subdirectories branch out and each directory can hold both files and other directories — **correct.** The chapter says that akin to a tree, the root directory, typically the hard drive, is at the top and subdirectories branch out, with each directory able to contain files as well as other directories.
- **D.** A hidden folder used by the archive process, which is why ordinary users never encounter it — The root is the most visible location on a drive rather than a hidden one, and archiving is about moving data to different media rather than about a special folder.

Finally, check what you could explain to somebody else without looking, since that is the standard the rest of the module assumes.

SELF-CHECK

Can you explain this section without the page in front of you?

Rate each statement honestly; anything you cannot do yet has a pointer to the exact place in this section to reread.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

- I can define system software and say what separates it from the applications a business chooses. The opening of the section, where every program is sorted into two piles.

- I can name the four things an operating system coordinates and give an example of each. The list of hardware components, peripherals, application software and users, just after the definition of the operating system.
- I can explain why software ships in one version for each operating system rather than one version for each computer model. The common-layer paragraphs, and the second view in the interactive diagram.
- I can say what a device driver does and why a printer released after the operating system still works. The paragraph on drivers, immediately after the common-layer argument.
- I can explain what assembly language is and why the operating system is designed to keep it away from me. The last paragraph before the section on what the operating system does unnoticed.
- I can name the four requirements that separate operational, backup and archival data. The section on three kinds of data, plus the table that reads down the columns.
- I can match each of the three data types to the medium the chapter puts it on, and justify the match. The list matching data to media, and the paragraph on why tape's weaknesses are acceptable for archives.
- I can explain why an archive can be lost even when the tape itself is perfectly readable. The highlighted caution after the paragraph on magnetic tape.
- I can say what the root directory is and why the folder hierarchy is described as logical rather than physical. The closing section on files and folders, and the correction at the end of it.

Networks: how machines share

Everything in the three sections before this one sits still. A machine, the software that drives it, a pile of stored data: none of them is worth much alone. The chapter is blunt about it — taken alone, each individual piece of technology has little value, and it is through connecting the different pieces that business value can be realized. This section is about the connecting, and it turns out to need only three things.

Why anything has to be connected at all

An organization runs on a great many separate pieces: servers and mainframes, personal computers and mobile devices, storage devices, even the systems that control the temperature of a room. The chapter's own illustration of why connecting them matters is the one worth keeping. The best-performing database in the world would be useless if the people and the applications that depend on its data could not reach it.

One reason information systems became as powerful as they did is precisely this ability to interconnect. Three consequences follow from it, and the chapter states all three plainly rather than leaving them to be inferred.

Stored data becomes reachable — a database nobody can get to is an expense rather than an asset, so the connection is what turns a store of records into something the organization can actually use.

People can work together — interconnection lets constituents inside the organization and constituents outside it communicate and collaborate, which is why the same technology ends up serving employees and outside partners alike.

New kinds of business become possible — the chapter's own claim is that many innovative business models would not exist without the internet, so connection changes what a company can sell and not merely how quickly it works.

Connection is not a feature bolted onto a system, then. For a great many organizations it is the system, which is why it is worth knowing exactly what a connection requires.

A network is a conversation, with equipment instead of people

Start with something you already do without thinking. When you speak to someone, you form a message in your head and then **code** it into a form that can travel, which for a voice means sounds. The message is **transmitted** along a pathway, in this case the air between you. The other person receives it and **decodes** it, using ears and a brain. Three moves: code, send, decode.

A computer network does exactly that with equipment. **Computer networking** is the sharing of data or services. A source produces a message, the message is encoded so that it can travel, it moves along a communication channel, and a receiver decodes it so the destination can understand it. Because the shape of the thing is the same, the requirements are the same, and the chapter names three of them.

Something to share — a sender and a receiver, which the chapter calls a source and a destination, that genuinely have a message between them; with nothing to share there is nothing for any equipment to do.

A pathway — the **transmission media**, meaning the physical route the data actually travels, whether that is a cable running through a wall or a radio signal crossing a room.

Rules both sides follow — **protocols**, which define the procedures that different computers follow when they transmit and receive data, so that whatever arrives can be interpreted at the far end.

The chapter makes the third requirement concrete with a job search. Imagine you are graduating and want work in France, so you write to employers there. One of them replies by email with everything you asked about its hiring process. The first requirement is met, because there is information to share. The second is met, because the internet carried it. But the reply is written in French, and you do not read French.

Nothing failed. The message was composed, sent and delivered intact, and it is still useless, because if the message is not understood by the receiver there is no communication. What is missing is an agreed language, and the agreed language is the protocol. You and the employer settle on English, and only then does the exchange do any work.

Arrival is not communication. Two systems can be wired together perfectly and share nothing at all, because agreeing on the wire is not the same as agreeing on the language.

The three requirements are worth being able to state in the chapter's own words, because most network failures you will ever be asked about are one of the three going missing.

COMPLETE IT

The three requirements, in the chapter's own words

Choose the wording the chapter uses in each sentence; the explanation appears once you pick, and it says what the other choices would actually have described.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

- Computer networking is **the sharing of data or services**, which is why a connection carrying nothing anybody wants is not doing any work. The chapter defines computer networking as the sharing of data or services, which keeps the definition about purpose rather than equipment. Cabling is only the second requirement, the pathway, and a network can use radio instead of any cable at all. Speed is bandwidth, which describes how well the sharing goes rather than what the sharing is.
- The first requirement is a sender and a receiver — a source and a destination — that **have something to share**. A message has to exist before anything can carry it, which is why the chapter states this requirement first. Machines of entirely different makes exchange data constantly, because that is what the third requirement, an agreed protocol, exists to make possible. Being in the same building describes a local area network, which is one type of network rather than a condition every network has to meet.
- The second requirement is **a pathway or transmission medium**, such as a cable, to send the message along. The transmission media are the physical pathway, cable or wireless, used to transmit data, and the chapter lists them second because a message with nowhere to go is going nowhere. A protocol is the third requirement, and it governs interpretation rather than travel. A server is a role a machine can play, not a requirement of a network: two peers with no server between them are still a network.
- The third requirement is **rules or protocols** governing communication between senders and receivers. Protocols define the procedures that different computers follow when they transmit and receive data, which is what makes an arriving message interpretable. Bandwidth decides how long a transfer takes and never decides whether it can be understood. A shared physical medium restates the second requirement, and the chapter is careful to keep the pathway and the rules apart precisely because a message can have the first without the second.
- In the chapter's example the reply arrives intact but in a language the reader cannot decode, which shows that **if the message is not understood by the receiver, there is no communication**. This is the chapter's own sentence, and it is the whole reason protocols are named as a separate requirement rather than assumed. The pathway plainly worked, because the email arrived complete and on time, so blaming the route misreads a delivery success as a delivery failure. The sender did encode the message, and encoded it perfectly well in French; the encoding and the decoding simply did not match.

What is actually being sent, and how much of it

Human communication is made of words. Computer communication is made of **bits**, the smallest unit of data a computer uses. Virtually any content can travel this way — a document, a photograph, a recording, a film — and as far as the network is concerned the only difference between them is how many bits there are. That number varies far more than most people expect, which is why some things appear instantly and others crawl.

The chapter gives four comparisons, and reading them together is more useful than memorising any one of them.

A customer's address — only a few thousand bytes, which is part of why a system can move millions of customer records around at night without anybody noticing it happened.

A page of text — roughly fourteen thousand bytes, still small enough that its size is almost never the reason a page is slow to appear on a screen.

A publication-quality photograph — possibly more than two hundred million bytes, which is more than ten thousand pages of text standing behind a single image.

A design file — an architectural drawing or a manufacturing plant layout can run to several billion bytes, which is why engineering firms think about their connections in a way that offices sending letters never have to.

The capacity of the path is the other half of the story. **Bandwidth** is the transmission capacity of a computer or a communications channel, measured in bits per second or in multiples of it, and it describes how much data can reliably be carried over the medium in one second. A network inside a building typically carries between a hundred million and a thousand million bits per second, which is usually far more than the connection leaving that building can manage.

Why one file takes eight seconds and the same file takes a day

The chapter takes a single file — a forty-five minute high-definition video, about one gigabyte — and asks how long it takes to arrive over six different connections. The tiers are the chapter's own, and they are anchored to an older broadband market, so read the ratios between the rows rather than treating any one label as current.

CONNECTION	TIME THE CHAPTER GIVES FOR ONE GIGABYTE
Dial-up telephone modem	More than a day and a half
Regular cable or DSL	About two hours and thirteen minutes
High-speed cable or DSL	About nine minutes
Fourth-generation mobile	About five minutes
Fifth-generation mobile	About twenty-seven seconds
Gigabit fiber	About eight seconds

One arithmetic sentence explains the entire table, and it is the part that will still be true when every figure above has been overtaken: **time equals size divided by rate**. The file never changed. Only the

capacity of the path did, and a path a thousand times wider finishes the job in a thousandth of the time.

There is one conversion to settle first, because storage is counted in bytes while networks are rated in bits. A byte is eight bits, so a gigabyte, being a thousand megabytes, is eight thousand megabits. That single number is why the exercise below divides eight thousand by a connection's rate in megabits per second and gets an answer in seconds.

- 1 Put both sides into the same unit.** Convert the file from bytes into bits, which for one gigabyte means eight thousand megabits, because a byte holds eight bits.
- 2 Divide the size by the rate.** Megabits divided by megabits per second leaves seconds, which is the answer anybody actually asked for.
- 3 Turn the answer into something a person can act on.** Seconds are precise and useless in a meeting; divide by sixty for minutes, and by sixty again for hours.

The sheet below lists the six connections with the capacity that produces each of the chapter's times. Write one formula for the first row and it is applied down the column, exactly the way a filled-down formula behaves in a real spreadsheet.

SPREADSHEET

How long does one gigabyte take?

Type one formula for row 2 in each column and run it; it is applied to every row the way a filled-down formula behaves in a real sheet.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Columns: Connection · Capacity in megabits per second · Seconds for one gigabyte · Minutes for one gigabyte · Times faster than the modem

1. Column C: how many seconds does one gigabyte take on each connection? One gigabyte is 8000 megabits and column B holds megabits per second. Round to the nearest whole second.

```
=ROUND(8000/B2, 0)
```

That is the whole table in one formula: time equals size divided by rate. Notice the first row runs to more than a hundred thousand seconds, which is where the chapter's day and a half comes from.

2. Column D: the same answer expressed in minutes, rounded to one decimal place.

```
=ROUND(8000/B2/60, 1)
```

Rounding at the end rather than partway through keeps the small rows honest: the fiber row is a fraction of a minute, and rounding it to whole minutes would have reported zero and hidden the answer.

3. Column E: how many times faster than the dial-up modem in row 2 is each connection? Round to the nearest whole number.

```
=ROUND(B2/$B$2, 0)
```

This column is the two before it read backwards. A connection roughly eighteen thousand times faster finishes in roughly an eighteen-thousandth of the time, and that relationship holds no matter what a typical connection happens to cost this year.

Two things in that column are worth pausing over. The first is that no judgment entered anywhere: the same three symbols produced a day and a half and produced eight seconds, so a claim about how long a transfer will take is checkable rather than arguable. The second is that the last column is simply the first two read backwards. A connection roughly eighteen thousand times faster finishes in roughly an eighteen-thousandth of the time, because that is what division does.

This is also the arithmetic behind a common and expensive mistake. A business that plans to move large files between sites every night, and buys the connection it needs for ordinary office traffic, has not bought a slow network; it has bought a network that cannot finish before morning. The size of the thing being moved and the capacity of the path have to be decided together, and neither number means much without the other.

Try the requirements, the bits and the arithmetic together before moving on to who is at each end of the wire.

CHECK YOURSELF

Requirements, bits, and the arithmetic

Four options, one best answer; read every explanation, including the ones for the options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. The chapter says business value comes from connecting the pieces rather than from the pieces themselves. Which statement is the chapter's own reason for that claim?

- **A.** A network costs more than the machines it joins together, so it deserves the larger share of management attention — Relative cost is not the chapter's argument, and it would be a weak one, since a network is usually a small fraction of what an organization spends on its machines and its software.
- **B.** Taken alone each piece of technology has little value: the best database in the world is useless if the people who depend on it cannot reach it — **correct.** The chapter states that taken alone each individual piece of technology has little value, and its own illustration is that the best-performing database would be useless if it could not be accessed by the people or applications throughout the organization that depend on the data.

- **C.** A network is the one part of an infrastructure that keeps working when an individual machine fails, so it is what the organization actually depends on — Some networks are built to survive the loss of a machine, but plenty are not, and a network is quite capable of failing on its own. Resilience is a design choice rather than a property of being connected.
- **D.** A network removes the need to keep data in any one place, because anything wanted can be requested from wherever it happens to be — Connecting machines does not remove the need to store anything. The data still has to sit somewhere, on somebody's storage, and the network only changes who can reach it from where.

2. A hypothetical architecture practice and a hypothetical insurance office both report that transfers to a second site are slow. The practice is sending design files; the office is sending pages of text and customer records. What does the chapter's comparison of content sizes suggest about the two complaints?

- **A.** The two complaints describe the same problem, because bandwidth affects every kind of content in exactly the same way once it has been reduced to bits travelling over the same wire — Bandwidth does apply to every kind of content, which is exactly why the amount of content decides the outcome. Two workloads that differ by a factor of a million on the same connection are not the same problem.
- **B.** The office should be affected more, because a large number of small items takes longer than one large item regardless of the total quantity involved — The chapter's comparison is about total quantity, not about how it is packaged. A few thousand bytes remain a few thousand bytes whether they arrive as one item or several.
- **C.** The practice is moving files of several billion bytes while the office is moving items of a few thousand, so the same connection is doing work that differs by orders of magnitude — **correct.** The chapter puts a customer's address at a few thousand bytes and a computer-aided design file at several gigabytes, and since time equals size divided by rate, the same rate produces wildly different waits.
- **D.** Neither complaint can be about the network, because all content is reduced to bits and bits travel at the same speed on any medium — Reducing everything to bits is what makes the sizes comparable rather than what makes them equal. The medium also matters, which is the entire content of the connection-speed table.

3. A hypothetical training centre is told that upgrading from a one megabit per second connection to gigabit fiber will help "a little" with a one gigabyte video. Using the chapter's own comparison, what should it actually expect?

- **A.** A change of roughly a thousandfold, turning a transfer of more than two hours into a matter of seconds — **correct.** Time equals size divided by rate, so multiplying the rate by a thousand divides the time by a thousand: the chapter's own table takes the same one gigabyte from about two hours and thirteen minutes down to about eight seconds.

- **B.** A modest improvement, because the size of the file is the main thing determining how long a transfer takes — Size is only half of the calculation. Holding the file constant and changing the rate is precisely the comparison the chapter's table makes, and it changes the answer by orders of magnitude.
- **C.** Almost no improvement, because bandwidth describes capacity rather than speed and the two are unrelated — Bandwidth is defined as how much data can reliably be transmitted over the medium in one second, which is what makes an estimate of the time possible in the first place.
- **D.** No change worth having, because a video takes as long to transfer as it takes to watch, its length being fixed — Transfer time and playing time have nothing to do with each other. The forty-five minute video in the chapter's example arrives in eight seconds over fiber and takes more than a day over a modem.

Three roles: server, client, and peer

Machines on a network are not equals by default. The chapter gives them three roles, and the only thing separating those roles is the direction in which requests travel.

A server — any computer on the network that makes access to files, printing, communications and other services available to the network's users, as the section on hardware described. Servers only provide services.

A client — any computer, or any software application such as a mail program on a laptop, that uses the services a server provides. Clients only request services, and a client usually has one user while many users share the server it talks to.

A peer — any computer that may both request and provide services, so the same machine can be a client at one moment of the day and a server at another.

Once you accept that the work can happen somewhere other than the desk, the machine on the desk can become very small. **Thin clients** are microcomputers with minimal memory, storage and processing capabilities, and they use **desktop virtualization** to give a worker a virtual desktop environment that is really running on a server elsewhere.

The reasons an organization does this are practical rather than technical. There are fewer copies of software to license and maintain, and it is easier to satisfy the strict privacy and data protection duties that apply in some industries and jurisdictions, because nothing of consequence is stored on the box the user actually touches. A terminal holding no data is a terminal that cannot lose any.

Which roles a network hands out is also what gives the network its name, and the chapter names two arrangements.

- **Client-server networks** — servers and clients have defined roles, and this is the typical business arrangement; with company networks and internet access everywhere, almost everyone at work is inside one whether they think about it or not.
- **Peer-to-peer networks** — any computer or device on the network can provide services as well as request them, which is what you find in small offices and homes, where no machine is set aside to serve the others.

Ask of any machine on a network whether it answers requests, makes them, or does both. That one question sorts servers from clients from peers, and it names the network at the same time.

Networks come in sizes, and the size is really a distance

Networks are commonly classified by size, by the distance they cover and by their structure. The three the chapter treats as the common ones are separated by nothing more complicated than how far apart the two ends are, which makes them much easier to keep straight than the abbreviations suggest.

TYPE	WHAT IT IS USED FOR	HOW FAR IT REACHES
Personal area network PAN	Wireless communication between one person's devices, using technologies such as Bluetooth	Under ten meters
Local area network LAN	Sharing data, software applications or other resources between several users	Typically within a building
Wide area network WAN	Connecting multiple local area networks, often with ownership and management distributed among several parties	From several buildings to a city to worldwide

The wide area network is the elastic one, and the chapter gives its middle cases names. A wide area network spanning several buildings on one site is a **campus area network**; one covering the area of a city is a **metropolitan area network**; and the largest of them all is the worldwide network the next section takes apart. Notice the phrase about distributed ownership: once a network leaves your buildings, parts of the path belong to other organizations, which is a management problem as much as a technical one.

The pathway can be radio rather than cable. A **wireless local area network**, almost always called a **Wi-Fi network**, is a local area network built on high-frequency radio-wave technology. Organizations install them to connect mobile devices, and wherever running a cable would be impractical or impossible; the ease of installation is why they became normal in offices and homes, and why public ones can now be found almost anywhere.

The chapter also flags fifth-generation mobile networks as promising for many applications, and particularly for large numbers of connected devices, because they offer extremely low latency and high transmission speed. Latency is the delay before a response begins, as distinct from how much can be carried once it does, and it is the measure that matters when a machine is waiting on an answer before it acts.

Pair each type with what it is for and how far it reaches; the distances are the part that does the work.

MATCH

Which network, over what distance

Pair each kind of network with what the chapter says it is for and how far it reaches; the explanation names what makes each one distinct.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Personal area network

Wireless communication between one person's own devices, using technologies such as Bluetooth, over a range under ten meters This is the smallest of the types and the one most people use without ever calling it a network: a phone and a set of headphones, or a laptop and a wireless keyboard on the same desk.

Local area network

Sharing data, software applications and other resources between several users, typically within a single building This is the ordinary office or home network. Its capacity is usually far higher than the connection leaving the building, which is why moving a large file to a colleague upstairs feels instant and sending it to another site does not.

Wide area network

Connecting multiple local area networks over a large physical distance, often with ownership and management distributed among several parties The distributed ownership is the part that matters managerially: once the path leaves your buildings, some of it belongs to other organizations, so availability is partly somebody else's decision.

Campus area network

A wide area network spanning several buildings belonging to one organization on a single site It is the middle case between one building and a city. A hospital group whose clinic, laboratory and administration buildings share one site is the ordinary example.

Metropolitan area network

A wide area network covering the area of a city Larger than a site and smaller than a country, this is the scale of a transit authority connecting stations, depots and a control room across one urban area.

Wireless local area network

A local area network built on high-frequency radio-wave technology instead of cable, also called a Wi-Fi network The reach is the same as a wired local network; only the pathway changed. Organizations install them for mobile devices and wherever running a cable is impractical, and the ease of installation is why public ones are now almost everywhere.

Fifth-generation mobile network

A mobile network offering extremely low latency and high transmission speed, which the chapter flags as promising for large numbers of connected devices Latency is the delay before a response begins, which is a different measure from capacity. It is the one that matters when a machine has to wait for an answer before it acts on something physical.

One last pass over the roles, the arrangements and the sizes before the next section takes the largest wide area network of all apart.

CHECK YOURSELF

Roles, arrangements, and distances

Three situations rather than definitions; every option is explained, including the ones that describe something real but different.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. In a hypothetical two-person design studio each computer can open files kept on the other, and neither machine is set aside to serve. In a hypothetical accounting firm, thirty desks open files kept on one machine that answers all of them and asks nothing of anybody. How does the chapter classify the two?

- **A.** Both are client-server networks, because in each case a machine is answering requests for files — Answering a request is not enough to make a machine a server in the chapter's sense. A server provides services and does not request them, whereas each studio machine does both, which is the definition of a peer.
- **B.** The studio is a peer-to-peer network and the firm is a client-server network, because a peer both requests and provides while servers and clients have defined roles — **correct**. The chapter defines a peer as any computer that may both request and provide services, and puts peer-to-peer arrangements in small offices and homes; client-server networks are those in which servers and clients have defined roles, and they are the typical business arrangement.
- **C.** The studio is a client-server network with two servers, and the firm is peer-to-peer because thirty machines are sharing one resource between them — This reverses both cases. Two machines that each provide and request are peers, not servers, and thirty clients depending on one provider is the textbook picture of a client-server network.
- **D.** Neither can be classified until it is known whether the connections are wired or wireless — The medium is a separate question from the roles. A wired network and a wireless one can each be arranged as client-server or as peer-to-peer, because the two decisions are independent.

2. A hypothetical call centre is choosing between three hundred ordinary desktop computers and three hundred thin clients running desktop virtualization. Which statement matches the chapter?

- **A.** Thin clients cost less because they are smaller and simpler, but each one still needs its own installed copy of every application the centre uses, so the whole saving is confined to the purchase price of the hardware — The licensing and maintenance saving is the chapter's stated reason for the arrangement, precisely because the software is not installed three hundred times. Confining the benefit to hardware misses the larger half of it.
- **B.** Thin clients remove the need for a network, because a stripped-down machine runs its software locally without asking anything of a server — The opposite is true. A thin client is almost entirely dependent on the network, because the environment the worker sees is running on a server somewhere else and reaching it is the only thing the terminal does.
- **C.** Thin clients are a kind of peer, because the desktop each one displays is assembled from the other terminals on the floor — A peer both requests and provides services. These terminals only request, and they request from a server rather than from each other, which makes the arrangement client-server.
- **D.** Thin clients have minimal memory, storage and processing of their own, and desktop virtualization runs each worker's desktop on a server instead, which cuts licensing and maintenance cost and keeps the data off the desk — **correct.** Thin clients are microcomputers with minimal memory, storage and processing capabilities, and the chapter says they use desktop virtualization to provide workers with a virtual desktop environment, helping reduce costs for software licensing or maintenance and comply with stringent privacy and data protection requirements.

3. A hypothetical hospital group has a network inside each of its three buildings on one site, a further network joining those three buildings together, and a wireless link between a nurse's tablet and the scanner clipped to her belt. Which three types has it just described?

- **A.** A wide area network in each building, a metropolitan area network joining them, and a wireless local area network between the tablet and the scanner — A wide area network covers a large physical distance and connects local networks, so it cannot be what sits inside a single building, and a metropolitan area network is specifically one covering the area of a city rather than one site.
- **B.** A personal area network in each building, a local area network joining them, and a wide area network between the tablet and the scanner — This inverts the distances. A personal area network reaches under ten meters, which is nowhere near a building, and a wide area network spans far more than the gap between a nurse and her own belt.
- **C.** A local area network in each building, a campus area network joining them, and a personal area network between the tablet and the scanner — **correct.** A network typically within a building is a local area network; a wide area network spanning several buildings on one site is what the chapter calls a campus area network; and wireless communication between one person's devices under ten meters is a personal area network.

- **D.** Three local area networks and nothing else, because everything described sits inside one organization on one site — Ownership is not what classifies a network. The chapter classifies by size, distance covered and structure, and on those measures the link to the belt scanner and the link between buildings are plainly different from the network inside one building.

How the internet and the web actually work

Everything in the sections before this one — the machines, the operating systems, the stored data, the wires between them — only becomes useful when something far away can reach it. So this section takes the most ordinary act in modern working life, opening a page, and pulls it apart. None of it is difficult. It is only unfamiliar, and every later argument in this module about cost, renting and risk quietly assumes you can picture what is happening here.

One network made out of many networks

The previous section sorted networks by how far they reach: a few metres, a building, a city, the world. The largest of them is the one you are almost certainly using right now. The **internet** is a large worldwide collection of networks that use a common protocol to communicate with each other. Notice what that definition does not say. It does not say one network, and it does not name an owner.

Three things are packed into that one sentence, and each of them explains something you have probably noticed without ever being able to name it.

Many networks, not one — the internet is a collection, so what you connect to is your own provider's network, which connects to others, and so on outward until the far end is reached.

A common protocol — the only thing every participant has to share is an agreement about how to talk, which is why equipment from different decades and different manufacturers still exchanges traffic without difficulty.

No owner named — nobody is in charge of the whole of it, so there is no central office to approve a new connection and equally none to telephone when something between two networks stops working.

The word itself records how the thing was built. **Internetworking** means connecting host computers and their networks together to form even larger networks, and that is exactly what happened: a university network here, a company network there, a home connection somewhere else, each separately owned and separately paid for, all agreeing to speak to each other the same way.

The practical consequence is worth sitting with. When you send a message from a kitchen table to an office server, it crosses equipment belonging to several companies that have never heard of you, and nobody arranged any of it in advance. That is not a weakness in the design. That is the design.

The web is not the internet

These two words are used as synonyms in ordinary speech, and they are not synonyms. The **World Wide Web** is a system of interlinked documents on the internet — or, as the chapter also puts it, a graphical user interface to the internet. The web runs on the internet in roughly the way an application runs on an operating system.

Five words do most of the work here, and each one names something you already handle every day:

Hypertext — documents that carry links to other documents, so a reader can move between them in any order rather than front to back. The chapter calls this the key feature of the web, and everything else is built on top of it.

A web page — a hypertext document, meaning it holds not only content but also the links that lead away from it. That combination is what makes it a page rather than simply a file somebody stored.

A hyperlink — a reference from one document to another that a reader can follow, which is what turns a heap of separate documents into a web instead of a library of unconnected ones.

A web browser — the software application used to locate and display web pages, including their text, graphics and multimedia content. It asks for documents and then decides how they look.

A website — a collection of interlinked web pages owned by the same organization or individual, usually hosted on a web server. The site is what an organization publishes; the page is what a visitor reads.

Keeping the two words apart is not pedantry. Email, a video call and a file transfer all travel over the internet without being the web at all, so “our supplier system is on the web” and “our supplier system is on the internet” describe two different arrangements carrying two different sets of risks.

CHECK YOURSELF

Two words that are not the same word

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical charity's new volunteer says the internet and the World Wide Web are two names for the same thing. Which correction matches the chapter?

- **A.** There is no real difference worth making; the chapter uses the two words interchangeably, and so may anyone else — The chapter defines the two separately, in different subsections, and the distinction does real work: a video call and a file transfer use the internet without touching the web at all.
- **B.** The web is the collection of separately owned networks, and the internet is the set of documents that travels across it — This has a sound pair of definitions attached to the wrong words. The network of networks is the internet; the interlinked documents are the web, which the chapter calls one of the most powerful uses of the internet.
- **C.** The internet is a worldwide collection of networks using a common protocol, and the web is a system of interlinked documents running on top of it — **correct.** The chapter defines the internet as a large worldwide collection of networks that use a common protocol to communicate with each other, and the World Wide Web as a system of interlinked documents on the internet.
- **D.** The internet is the part of the web that ordinary members of the public can reach, and the web also takes in private company systems — Private company systems appear later in this objective as intranets, and they are not a further layer of the web. Who may reach something is a question of authorization, not of which of these two words applies.

2. What makes a document a web page rather than simply a file of text somebody stored on a machine?

- **A.** It sits on a server rather than on a personal computer, which is what the word page is recording — Where a file sits does not decide what kind of thing it is. The chapter defines a web page by what it contains, and such a page can be written and read on one machine long before it is put on a server.
- **B.** It is a hypertext document, so it carries both content and links to other documents, which is what lets a reader move between pages in any order — **correct.** The chapter defines a web page as a hypertext document containing not only content but also hyperlinks, and calls hypertext the key feature of the web.
- **C.** It has been indexed by a search engine, since a document nobody can find is not yet part of the web — Indexing is something a search engine does afterwards, and the material on the deep web makes the opposite case: most of the web has never been indexed and is still the web.
- **D.** It is written in a language browsers understand, which is the whole requirement, and links are an optional extra on top — Being readable by a browser is necessary but is not the defining feature. Without links the result would be a library of unconnected documents rather than a web, which is precisely the distinction the word carries.

3. The same page looks slightly different on two machines, although both asked the same server for the same address. Which division of labour explains that?

- **A.** The server sends a finished picture of the page, so a difference has to be a fault in one of the two connections — A server sends a document, not a picture. If it sent finished images, the web could not adapt to a phone, a laptop and a projector without the site being rebuilt separately for each of them.
- **B.** The server sends the document and its markup, and the browser interprets that markup and decides how the result is drawn on that particular screen — **correct**. In the chapter's walkthrough the browser translates the markup, formats its visual appearance and displays the result, so the final drawing is the browser's decision rather than the server's.
- **C.** The browser keeps pages permanently, so the second machine is showing an older copy that the server has since replaced — Browsers do keep copies to save time, but the chapter's account of a request ends with the browser rendering what it has just received, and an older copy would not by itself explain a difference in appearance.
- **D.** The difference comes from the address, since a small variation in an address returns a different version of the page — Both machines used the same address, which is what the situation states. Different addresses can certainly return different pages, but that is not what happened here.

What a page is actually made of

Open any page, ask the browser to show you its source, and you will find the same words you were just reading, wrapped in short codes. That file is written in **Hypertext Markup Language**, or HTML: the standard method of specifying the structure and content of web pages. It is a text-based file format, which means a person can read it, and it uses codes called **HTML tags** to instruct the browser on how a document should be presented.

A tag is set apart from the surrounding text with angle brackets, and most tags come in pairs, one to begin an element and one to end it. Some tags say how something should look; others say what something is, such as the page title, a heading level, or the end of a paragraph. The chapter's own list is short, and it is enough to read the source of a real page.

TAG	WHAT IT DOES
<code><html> ... </html></code>	Delineates the whole HTML document, so everything else sits inside this one pair
<code><head> ... </head></code>	Holds the title, scripts, styles, metadata and other elements that are not displayed on the page itself
<code><body> ... </body></code>	Holds the visible portion of the document, which is everything a reader actually sees

TAG	WHAT IT DOES
<code> ... </code>	Creates bold text: the first tag turns bolding on, the second turns it off again
<code><p> ... </p></code>	Creates a new paragraph, which is how the browser knows where to break the running text
<code><table> ... </table></code>	Creates a table, so the content is laid out as rows and columns rather than as prose
<code> ... </code>	Creates a hyperlink, so the enclosed words carry the reader to another document
<code> ... </code>	Creates a link that opens a new email message rather than a page

The addresses inside those last two are placeholders. They are printed here as inert text so that you can read the shape of a tag without anything on this page ever reaching out to a network.

Two more names complete the picture. **HTML5** is the name the chapter gives the current version of the language, which added ways of structuring a page and of presenting modern multimedia without extra software installed alongside the browser. Treat that as a label rather than as a version number still being counted up: the web's markup is now revised continuously instead of being released in numbered editions, so there is no HTML6 waiting behind it.

Cascading style sheets, or CSS, are what developers use to specify the formatting and layout of the elements on a page.

The division between those two is the transferable idea, and it is the same abstraction argument the section on operating systems made. HTML says what the parts of a document are; CSS says how they should look. Separating them is why one style change can resize every heading across a site of a thousand pages without anyone editing a thousand pages, and why the same page can be legible on a phone and on a projector.

Anything you have met beyond the tags in that table — headings, images, embedded video — is a real part of modern HTML but is an addition to the chapter's list rather than part of it. Get the eight the chapter gives before you go looking for more.

MATCH

The chapter's eight HTML tags

Pair each tag with what it does. All eight come from the chapter's own table, and the addresses inside them are placeholders printed as inert text rather than working links.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

<html> ... </html>

Delineates the whole document, so every other element sits inside this one pair This is the outermost pair. A browser reading a file knows the markup starts here and ends at the closing tag, which is why it is the only tag that never sits inside another.

<head> ... </head>

Holds the title, scripts, styles and metadata that are not displayed on the page itself Everything in here is about the document rather than in the document. The title shows in the browser tab, the styles decide how things look, and none of it appears as text a reader can see.

<body> ... </body>

Holds the visible portion of the document, which is everything the reader actually sees The split between this and the head is the first structural decision in any page: information about the page in one place, the page itself in the other.

** ... **

Creates bold text, turning bolding on at the opening tag and off again at the closing one The chapter uses this to explain paired tags in general: one marks the beginning of an effect and the other marks its end, which is why leaving out a closing tag bolds the rest of the page.

<p> ... </p>

Creates a new paragraph, which is how the browser knows where to break the running text Pressing return in the source file does nothing at all. A browser only breaks the text where the markup tells it to, which is the first surprise for everyone who views the source of a page.

<table> ... </table>

Creates a table, so the enclosed content is laid out in rows and columns rather than as prose This tag changes the shape of the layout rather than the appearance of a few words, which is why it encloses a whole block of content instead of a phrase inside a sentence.

** ... **

Creates a hyperlink, so the enclosed words carry the reader to another document This is hypertext made concrete. The setting inside the opening tag names the destination, and the words between the two tags are what the reader sees and clicks.

** ... **

Creates a link that opens a new email message rather than opening a page The same tag with a different kind of destination. It is a useful reminder that a link names a resource in general, and a web page is only the most common kind of resource to name.

Where pages live, and how one is asked for

Pages are stored on **web servers**, which process user requests for those pages using the **Hypertext Transfer Protocol**, or HTTP. A protocol, from the previous section, is simply the agreed set of rules by which two machines talk. This one is the reason a browser written by one company can read a page served by software written by another company that has never heard of it.

The word “stored” is doing less work than it looks, and the next sentence is the most useful one in this section for anyone who will later touch a database. Web servers often employ scripting languages to assemble pages on the fly, or to retrieve data from databases and insert it into the page presented to the user.

That product page you looked at last week did not exist before you asked for it. It was built, on request, out of stored rows about that one product — which is a thread the supplement on databases later in this module picks up and follows.

One more thing rides along with every request, and it is the difference between privacy and none. Data travelling between a server and a user’s computer can easily be intercepted, so organizations now routinely use **HTTPS**, the secure form of the same protocol, for anything at all sensitive.

The padlock a browser shows beside an address is the browser reporting that the secure protocol is in use. Its absence on a page asking for a card number means that number crosses several unknown networks in a form anyone along the way could read.

How a request finds one machine out of billions

Every computer or device on the internet is identified by an **IP address**, which serves as that machine’s destination address and lets the network route messages to it. The older form is written as four numbers separated by periods, each of them between zero and 255. The newer form uses a far larger address, and it exists for a blunt reason: the world ran short of the older kind.

Numbers are hard to remember and they change from time to time, so people use names instead. A **uniform resource locator**, or URL, is the address you actually type, and the chapter dissects it into three distinct parts. Take a plain address such as `mail.example-museum.org` and read it from the right:

The top-level domain — the suffix at the end, here `org`, which indicates the family of names this one was registered under and something about the kind of registrant.

The domain name — here `example-museum`, the part that helps a person recognize the company or individual the address belongs to. This is the piece an organization registers and pays for.

The host name — here `mail`, naming the particular web server, or group of web servers, that will answer this request. The prefix `www` normally means the default site including the home page.

That third part carries an unexpectedly practical fact. An organization registers one domain and then puts as many host names in front of it as it has services, so `www.example-museum.org`, `mail.example-museum.org` and `photos.example-museum.org` can be three entirely separate systems on three different machines, all under one registered name.

A URL usually carries a path after the address as well, naming the particular resource wanted rather than the site as a whole. Something ending `example-museum.org/collections/textiles` is asking for one page in one part of the site, in the same way that a folder path names one file rather than a whole drive.

Top-level domains arrive in three families, and the chapter gives examples of each.

Suffixes for a kind of organization — `com` is reserved for commercial organizations, `edu` for educational institutions, `org` for organizations and typically non-profit ones, and `gov` for a United States government entity.

Country codes — more than two hundred and forty two-letter suffixes, one for each country, such as `de` for Germany. These are the reason an address can tell you where a name was registered.

Generic suffixes — added as names proliferated, each naming an industry or a use. The chapter's examples are `aero` for the air transport industry, `name` for individuals, `coop` for business cooperatives and `museum` for museums.

Names are registered through many different companies that compete with one another, and that is worth knowing for one reason: a suffix tells you which family of names something was registered in, not whether the registrant is honest.

An address need not be typed at all, either. A **QR code** is a two-dimensional bar code with a high storage capacity, most often holding an address so that scanning it with a camera opens the page. The same code can instead trigger an action, such as starting a call to a sales representative, sending a message to a set number, or authorizing a contactless payment through a banking or payment application.

Reading an address correctly is a small skill with an outsized payoff, and it is worth practising as code rather than as prose, because writing the rule down forces you to be exact about where one part of a name ends and the next begins.

WRITE THE CODE**Read an address the way a machine has to**

Write each function so every test passes; a failing test prints what your code returned beside what was expected.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. The chapter splits an address into a host name, a domain and a top-level domain. Implement **addressParts(address)**: return those three parts as an array, in that order. The domain and the top-level domain are the last two labels; anything before them is the host name, and an address with fewer than two labels names no domain at all.

Signature: `addressParts(address) -> [host, domain, topLevelDomain]`

```
function addressParts(address) {
  var labels = String(address).split('.').filter(function (l) { return
    l.length > 0; });
  if (labels.length < 2) return ['', '', ''];
  var n = labels.length;
  return [labels.slice(0, n - 2).join('.'), labels[n - 2], labels[n - 1]];
}
```

Counting from the right is the whole trick, and it is the reason this is worth writing down. Reading a name from the left tells you what somebody wants you to notice; reading it from the right tells you which name was actually registered.

2. Two addresses belong to the same site when they share a domain and a top-level domain, whatever host names sit in front. Implement **sameSite(a, b)**, returning true or false. An address that names no domain matches nothing, including another address that names no domain.

Signature: `sameSite(a, b) -> boolean`

```
function sameSite(a, b) {
  function registered(s) {
    var l = String(s).split('.').filter(function (x) { return x.length > 0;
  });
    return l.length < 2 ? '' : l[l.length - 2] + '.' + l[l.length - 1];
  }
  var ra = registered(a);
  return ra !== '' && ra === registered(b);
}
```

The fourth test is the one to remember. A convincing name in the host position proves nothing, because the host part is chosen freely by whoever owns the domain — and the domain is the last two labels, not the first two words you happen to recognize.

The second function in that exercise is the whole of one very common fraud. A name such as `login.example-bank.com.example-attacker.net` contains a familiar bank's name in full and reads convincingly at a glance, but the registered part is the last two labels, so the site belongs to whoever registered `example-attacker.net`. Reading an address from the right rather than from the left is the habit that catches it, and it is a habit a machine can be taught in about six lines.

What actually travels: one request, end to end

The internet moves web pages and everything else using **TCP/IP**, the transmission control protocol and the internet protocol taken together. Its first move is counter-intuitive. It does not send your request as one piece: it breaks it into small chunks called **packets**, sends those off separately, and reassembles them at the far end.

Breaking requests and files into small packets is what allows data to be routed and transferred efficiently across a network, and it buys three things at once:

Pieces travel independently — different packets belonging to one file may take different routes through the network, so no single congested link gets to decide how fast the whole thing arrives.

A failure costs a fragment rather than a file — when a packet goes missing it can be sent again on its own, instead of the entire transfer having to start over from the beginning.

Many conversations share one path — because everyone's traffic is in small pieces, a single large download cannot occupy a link for minutes on end while everybody else waits their turn.

You never see any of this happening, which is rather the point of it. It is also why trouble on a network so often shows up as something slow and intermittent rather than as something plainly broken.

The eight steps below make up one complete round trip, from your keystroke to the finished page on the screen. Put them in order, then read why each one has to sit where it does.

SEQUENCE**One request, from keystroke to page**

Put the eight steps into the order the chapter describes, then read why each one has to sit where it does.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

This is the chapter's own walkthrough of what happens between typing an address and seeing a page. It is worth getting the order exactly right, because almost every complaint you will ever hear about a slow or broken system is really a complaint about one of these eight steps.

1. **The user enters the address of a web page into the browser's address bar, and the browser turns it into a request for the page at that address.** — Nothing can begin until something has been asked for. Note that the person supplies a name, not a machine: everything that follows is the work of turning that name into a specific destination.
2. **TCP/IP breaks the request into small chunks called packets and routes them across the internet toward the web server where the requested page is stored.** — This is where the request stops being one thing. Breaking it up is what lets the pieces travel efficiently over networks that are carrying millions of other people's pieces at the same time.
3. **When the packets reach their destination, TCP/IP reassembles them into the original request and passes that request to the web server.** — Reassembly has to happen before the server sees anything, because a web server answers requests and knows nothing about packets. Each layer hides its own mess from the layer above it.
4. **The web server recognises that a web page is being requested, because the transfer protocol is named at the front of the address, and it retrieves the page.** — The protocol named in the address is what tells the server which kind of conversation this is. The same machine can answer several kinds, and it needs to know which set of rules applies before it can respond at all.
5. **The page is broken into packets by TCP/IP in turn and transmitted back across the internet toward the computer that asked for it.** — The journey home works exactly like the journey out, which is why a connection can be fast in one direction and slow in the other. Nothing about the reply is privileged; it is traffic like any other traffic.
6. **TCP/IP reassembles the packets at the destination and delivers the finished page to the web browser.** — The second reassembly restores the document the server sent. Only now does anything on your machine hold something that looks like a page rather than a pile of numbered fragments.
7. **The browser translates the HTML code, formats its visual appearance, and displays the results on the screen.** — This is the step people mistake for the whole process. The server sent structure and content; the decision about what it looks like on this screen was made here, at the last possible moment.

8. If the page contains a hyperlink and the user follows it, the whole process repeats from the beginning. — Hypertext is what makes the cycle a loop rather than a line, and it means the browser is not holding a session with one machine but starting a fresh conversation, often with a different organization entirely.

The same pipes carry calls, and let programs talk to each other

Once everything is packets, the difference between a telephone network and a computer network stops meaning very much. TCP/IP has allowed a convergence of computing and telecommunications, and the clearest evidence is **Voice over IP**, also called IP telephony: technologies that use internet infrastructure to place telephone or video calls. The chapter names WhatsApp, FaceTime, Zoom and Microsoft Teams among the consumer-facing examples, and notes that the versions sold to organizations add intelligent call routing, secure audio, and integration with other applications such as Salesforce or Slack.

The same web protocols carry something less visible and rather more consequential. They also let applications communicate with each other, which is the basis of an **application programming interface**, or API. The idea is simple and worth stating carefully, because a great deal of modern software is assembled out of it.

A service consumer — the program that wants something done, such as a ride-hailing application that needs a route across a city for which it holds no map of its own.

A service provider — the program that can do it, such as a mapping service that already holds the roads and the traffic and does little else but answer that kind of question.

Pre-specified parameters — the agreed values the consumer passes across, here a pickup location and a destination, and the result that comes back, here the best route between them.

No knowledge of the inside — the consumer never has to know how the provider produces its answer, which is exactly why a small team can ship a product resting on capabilities it could never have built itself.

A request of this kind is often just an address with values attached to the end of it, and the shape is worth seeing once.

The shape of an interface request. directions?

origin=CITY_A&destination=CITY_B&key=YOUR_KEY_GOES_HERE — three agreed values sent across, and a route sent back. The names before each equals sign are the parameters the provider has published; the values after them are what this particular caller wants. Nothing here is a real address and nothing here is a real key.

Now look at the last value in that request. The key is a credential: it identifies the account that will be billed for the work. Sitting where it does, in the middle of an address, it also travels into server logs, browser history, screenshots, chat messages and copied-and-pasted examples. Hold that thought. The cloud sections later in this module describe a company whose ordinary monthly bill became a ruinous one after exactly this kind of credential was taken and used by somebody else — and the reason that was possible is visible right here, on the page where the interface was first explained.

The part of the web you cannot search

Search engines work by following links. A program walks from page to page, collects what it finds and builds an index, so anything it can walk to becomes findable and anything it cannot walk to does not. That one mechanism explains a fact most people find surprising: the searchable part of the web is a small fraction of it.

The **deep web** is those parts of the web that conventional search engines cannot index. The chapter estimates the searchable surface at as little as a hundredth of the whole; treat that as an illustration rather than a measurement, since something unindexed is by definition uncountable. The structure is what matters, and the chapter enumerates it exactly.

Private areas requiring authentication — a course system, an electronic textbook, a bank statement. The pages are perfectly ordinary; they simply refuse to open for anyone without an account.

Pages generated from a database on request — a product listing, a library catalogue record, a government agency's weather archive. There is no stored page for a program to walk to, because the page comes into existence when somebody asks a question of the data.

Pages nothing links to — a document sitting on a public server with no link anywhere in the world pointing at it. A program that navigates by following links has no route by which to arrive.

None of that is sinister, and one consequence of it lands squarely on a manager. A company's own records are invisible to a search engine for the second and third reasons at once, which is why "just search for it" is never an answer to a question about internal information, and why organizations have to build their own ways of finding what they already hold.

The **dark web** is a different thing that happens to share a syllable, and the chapter is emphatic that the two are not to be confused. It is those areas of the web used for various nefarious purposes — the chapter names trading in drugs, in stolen payment card details and in illegal material — typically reachable only with specialized browsers that anonymize the user and hide their traces.

Put the two side by side and the confusion dissolves. The deep web is most of the ordinary web with a login screen or a database in front of it; the dark web is a small, deliberately hidden neighbourhood that most people will never have any reason to enter.

SORT

Surface, deep, or dark

Place each item in the layer it belongs to. Situations described as hypothetical are practice situations, and each one states the condition you need in order to decide.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Surface web — public pages that other pages link to, so a search engine can walk to them and put them in its index

- A news agency's front page, which dozens of other sites link to and any search engine can reach — Public and linked from elsewhere, which is the whole requirement. A program that navigates by following links has an obvious route to it and will index what it finds.
- An encyclopedia article that anyone can open and that thousands of other pages point at — The chapter names this kind of site among its examples of the common or surface web. Public plus heavily linked is the most indexable combination there is.

Deep web — ordinary content a conventional search engine cannot index: behind a login, generated from a database on request, or linked from nowhere

- The pages of a course system that open only after a student signs in with an account — This is the chapter's first category of deep-web content: private areas requiring authentication. A learning management system is its own example, and nothing about it is hidden or improper.
- An online store's product page, assembled from a database at the moment somebody asks for that particular product — The chapter's second category: dynamic pages created from connected databases. There is no stored page for an indexing program to walk to, because the page is made in response to a question.

- A document on a public server that no page anywhere links to — The chapter's third category: static pages that are not connected to other pages by hyperlinks. The document is public in the sense that anyone with the exact address can open it, and unfindable to anyone who does not have it.
- A hypothetical hospital's patient records, held behind its own firewall on its own network — Internal content behind a corporate or institutional firewall is not accessible to search engines. It is a variation on the authentication category, and it is why an organization has to build its own way of searching what it holds.
- A scientific journal archive that charges institutions for access and admits no indexing program — The chapter names subscription archives directly. Because the publisher charges for access, the contents are not made freely available for indexing, so they sit outside the searchable surface.
- A government agency's weather records, which appear only once somebody runs a query against its database — The chapter names public agency databases among deep-web content for exactly this reason: their contents are dynamically generated from databases and displayed on demand, so there is nothing standing there to be indexed.

Dark web — a deliberately hidden area used for criminal purposes, reachable only with software that anonymizes the user and hides their traces

- A marketplace trading stolen payment card details, reachable only through software that conceals who is visiting — Both halves of the chapter's definition are present: a nefarious purpose, and access only through specialized browsers that anonymize the user. This is the part that is deliberately hidden rather than merely unindexed.

THE FOUR CONFUSIONS WORTH FIXING BEFORE YOU MOVE ON

The internet is the network of networks, and the web is one thing running on it. HTTP is the rule for asking, and HTTPS is that same rule with the conversation sealed. The domain is what an organization registered, and the host name is which of its servers answers. And the deep web is ordinary content behind a login or a database, while the dark web is something else entirely.

CHECK YOURSELF

Protocols, addresses, packets and keys

Five situations drawn from this section; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical cooperative's membership page asks for a bank account number, and the browser shows no padlock beside the address. What does the chapter's account of the two web protocols say is at stake?

- **A.** Nothing much, since the padlock reports whether the organization behind the site is trustworthy rather than anything about the connection — The padlock reports on the protocol in use, not on the character of the organization. A dishonest site can be reached over a protected connection and an honest one over an unprotected connection.
- **B.** The page will load more slowly, because the secure form of the protocol is the one browsers are optimized for — Speed is not what separates them. The chapter's reason for the secure protocol is interception in transit, together with growing concern about both legitimate and illegitimate monitoring of web traffic.
- **C.** Data travelling between the server and the member's computer can easily be intercepted, and the secure protocol is what stops the account number crossing in readable form — **correct.** The chapter says data traveling between a server and a user's computer can easily be intercepted, which is why organizations now routinely use the secure protocol for anything sensitive, and the padlock is the browser reporting that it is in use.
- **D.** The account number is safe the moment it reaches the server, so the protocol matters for what the server does with it afterwards rather than for the journey — What the server does later is a separate question with separate safeguards. The exposure described here happens on the way, across several networks belonging to companies neither party has any relationship with.

2. A hypothetical garden centre sells forty thousand products and its site shows a page for every one of them, yet its web server stores only a few dozen files. How does the chapter explain that?

- **A.** The remaining pages are held on each visitor's own computer and assembled there by the browser out of files it has kept from earlier visits — A browser draws what it is sent; it does not hold a company's product data. Nothing on the visitor's machine knows what is in stock this morning or what it now costs.
- **B.** Web servers often employ scripting to assemble pages on the fly, retrieving data from a database and inserting it into the page when somebody asks for a particular product — **correct.** The chapter says web servers often employ scripting languages to assemble pages on the fly or retrieve data from databases and insert it into the page presented to the user, which is why the page did not exist until it was asked for.
- **C.** The site must be using compression, which is what allows forty thousand separate pages to be squeezed into the space that a few dozen ordinary files would otherwise occupy on the server — Compression makes stored files smaller; it does not conjure a page that was never written. The gap here is between stored files and reachable pages, not between large files and small ones.
- **D.** Forty thousand pages do exist as stored files, but a search engine reaches only the few dozen that other pages link to — This confuses storage with indexing. It is a true statement about why such pages sit in the deep web, but it assumes the pages were written and stored in advance, which is exactly what did not happen.

3. A hypothetical museum is told it can run its public site, its ticketing system and its staff mail under one registered name without any of the three interfering with the others. Which reading of an address explains how?

- **A.** Each service needs its own registered domain, and the three would merely be made to look similar to visitors — Registering three names is possible but is not what was described, and it throws away the shared identity that made the arrangement attractive. One registration can carry as many host names as an organization needs.
- **B.** The path written after the address is what separates one service from another, so all three would answer on the same single server out of three different folders sitting side by side on its disk — A path does name a particular resource inside a site, and it is a real part of many addresses, but it is not one of the three parts of the address itself and it cannot send a request to a different machine.
- **C.** An address names a host, a domain and a top-level domain, so different host names in front of one registered domain can reach different servers or groups of servers — **correct.** The chapter dissects an address into the host name, the domain and the top-level domain, and gives an organization's mail, photo and mapping services each answering on a different host name in front of the same domain.
- **D.** The top-level domain selects the service, so the museum would register the same name under three different suffixes — Suffixes indicate the family a name was registered in and something about the registrant. Using three of them would create three separate registrations rather than three services under one name.

4. Why does the internet break a file into packets and send the pieces separately, rather than sending it as one continuous stream?

- **A.** Because a network carries one message at a time and no more, so a long file has to be interrupted at intervals to let everybody else's traffic through before it is allowed to continue — Networks carry traffic from many senders at the same time, which is what they exist to do. Packets are what makes that sharing orderly rather than a workaround for a network that could otherwise manage one thing.
- **B.** Because breaking requests and files into small chunks lets them be routed and transferred efficiently, with pieces able to take different paths and a lost piece resent on its own — **correct.** The chapter's justification is that breaking requests and files into small packets allows for efficiently routing and transferring data across a network, and reassembly at the destination puts the pieces back before anyone sees a result.
- **C.** Because the receiving machine cannot hold a whole file at once and has to be given it in instalments — Storage is not the constraint. A machine with ample room for the entire file still receives it in packets, because the reason lies in the network between the two ends rather than at either end.
- **D.** Because encryption operates on small blocks, so a file has to be divided before a secure protocol is able to carry it — Secure transport and packet transport are separate matters. Traffic carrying nothing sensitive at all is packetized in exactly the same way, and the chapter gives efficiency as the reason.

5. In the interface request shown in this section, the final value passed across is a key identifying the account that will be billed for the work. Which conclusion follows?

- **A.** It is safe where it is, because values inside an address are discarded as soon as a page has finished loading — Addresses are among the most copied strings in computing. They turn up in server logs, browser history, screenshots, chat messages and pasted examples, any of which can outlive the page by years.
- **B.** It is a credential travelling in a place that is routinely logged, copied and shared, so treating it exactly like a password is the least that is required — **correct.** The key names the account that pays, and it is sitting in the part of a request most likely to be recorded and passed on, which is why this section flags it before the cloud material returns to a case built on exactly this failure.
- **C.** It identifies the provider rather than the consumer, so losing it would inconvenience the mapping service rather than the caller — The key is issued to the consumer so that the provider knows whose account to charge. Whoever holds it can spend against that account, which is why the consumer is the party exposed.
- **D.** It cannot be misused on its own, because a request is accepted only when the other agreed values are supplied alongside it — Those other values are supplied by whoever makes the request, so they are no obstacle whatever. Anyone holding the key can supply their own and have the work billed to somebody else.

Intranets, extranets, and where the machines live

The last two sections built the web out of parts: a browser asking, a server answering, an address that finds the right machine. Every one of those parts is indifferent to who is asking. This section adds the two decisions that turn that indifferent machinery into something a business can run on — who is allowed in, which turns one technology into three different tools, and where the machines physically sit, which is about weather, electricity and distance as much as about computing.

The same website, with a lock on the door

Start with something you have already used. A company publishes pages anybody can read: what it sells, where its offices are, how to apply for a job. Now picture the same company publishing a second set of pages that only its own staff can open — this month's announcements, the training you have to finish by Friday, the form for changing your health cover. Nothing about the technology has changed. What changed is who is let in.

That second set of pages has a name. An **intranet** looks and acts just like a publicly accessible website and uses the same software, hardware and networking technologies to transmit and display data, but it uses authentication techniques, firewalls, or both, to secure proprietary data held inside the corporate network so that the data can be viewed only by authorized users.

The plainest version never touches the public internet at all. In the simplest form of an intranet, communications take place only within the confines of organizational boundaries and do not travel across the internet — which is one reason an intranet can carry material a company would not put on the open web under any circumstances.

The chapter names five things organizations use intranets for. None of them is exotic; they are ordinary office work, moved onto web pages.

Disseminating corporate information — announcements, policies and reference material published once, in one place, so that nobody is working from a copy that was emailed round the building last spring.

Employee training — courses and materials staff work through on their own schedule, which is only practical if the material is reachable from every desk without also being reachable by the public.

Project management — the plans, schedules and current status of work in progress, kept where every member of the project sees the same version at the same moment.

Collaboration — the shared documents and discussions that let people in different departments and different buildings work on one thing together rather than separately and in parallel.

Employee self-service — portals where staff administer their own benefits, manage retirement plans and handle other human-resources tasks themselves, instead of asking somebody in an office to do it on their behalf.

The last of those is worth pausing on, because it is where an intranet usually pays for itself. Every benefit election a person makes for themselves is a form that nobody has to receive, read, key in and file. The work does not disappear; it moves to the person who already knows the answer.

Reaching the inside from outside

An intranet that only works from a desk in the building has an obvious problem: a growing share of the people who need it are not in the building. Increases in employee mobility mean an intranet has to be reachable from anywhere, and the answer is not to put it on the open web and hope a password is enough.

Instead, most companies let employees use a **virtual private network**, or VPN, to connect securely to the company's intranet while travelling or working from home. Picture the public internet as a road anybody may drive on and the VPN as an armoured van moving along it. The route is public and the contents are not, and anyone watching the road sees a van rather than what is inside it.

Before the architecture, check the two ideas you have just met.

CHECK YOURSELF

Intranets and the private tunnel into them

Four options, one best answer; read every explanation, including the ones for the options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A company's public site and its internal staff site look almost identical and were built by the same team with the same tools. What, according to the chapter, actually makes the second one an intranet?

- **A.** It runs on proprietary software written for internal use, rather than on the ordinary web technologies the public site uses — The chapter says the opposite in as many words: an intranet uses the same software, hardware and networking technologies as a publicly accessible website. Nothing has to be rewritten to make a site internal.

- **B.** It uses authentication techniques, firewalls, or both, so that proprietary data inside the corporate network can be viewed only by authorized users — **correct**. An intranet looks and acts just like a publicly accessible website and uses the same technologies to transmit and display data, but it adds authentication techniques and/or firewalls so that the data can be viewed only by authorized users.
- **C.** It is stored on a separate machine, and keeping the two sets of pages on different servers is what makes one of them internal — Separate machines are common and often sensible, but a second server with no access control on it is simply a second public site. Where the pages sit is not what restricts who may read them.
- **D.** It contains information about employees rather than about products, and the subject matter of the content is what decides the category — Subject matter is a good clue and a poor rule. A public site can carry a staff directory quite legitimately, and the chapter's summary table separates the three arrangements by users and access rather than by topic.

2. An organization puts benefit elections, retirement-plan changes and other human-resources tasks on its intranet so that staff complete them themselves. What is the chapter's name for this use, and what is the point of it?

- **A.** Collaboration, because several people in different departments now work on the same record at the same time — Collaboration is a separate use the chapter lists, and it means shared work on a common document or discussion. Here each person is completing their own record, and nobody else is meant to touch it.
- **B.** Disseminating corporate information, because publishing the current benefit rules in one place is the substance of what has been built — The rules almost certainly are published alongside the forms, which makes this tempting. Dissemination is one-way publication, though, and the distinctive thing here is that the employee changes something rather than reads something.
- **C.** Employee self-service through employee portals, which lets staff administer benefits and similar human-resources applications for themselves — **correct**. The chapter lists enabling employee self-service for administering benefits, managing retirement plans, or other human-resources-based applications through employee portals as one of the standard uses of an intranet.
- **D.** Project management, because enrolment runs to a deadline and the intranet is tracking progress towards it — Project management is another of the chapter's listed uses, and it covers plans, schedules and the status of work in progress. An annual deadline does not turn a form into a project.

3. Employees increasingly work from home and while travelling, and they need the intranet. Why does the chapter say companies use a virtual private network rather than simply publishing the intranet on the open web behind a password?

- **A.** Because a VPN lets employees connect securely to the company's intranet from the road or from home, keeping the traffic protected while it crosses public infrastructure — **correct**. The chapter says increases in employee mobility mean the intranet must be accessible from anywhere, so most companies allow employees to use VPNs to connect securely to the intranet while on the road or working from home.
- **B.** Because a VPN makes the intranet faster, and the speed of the connection is what stops remote staff from using the site — A tunnel adds work at both ends rather than removing it, so speed is not the argument. The problem being solved is that the path between the employee and the office is public, not that it is slow.
- **C.** Because an intranet cannot technically be reached from outside the building, so a VPN is what makes the pages exist for a remote worker at all — The site is perfectly reachable in a technical sense, which is exactly the danger. The VPN is chosen so that reachable does not mean reachable by everyone, not because remote access is otherwise impossible.
- **D.** Because publishing on the open web would require rewriting the intranet in different software, which a VPN avoids having to do — No rewriting is involved either way. An intranet already uses the same software, hardware and networking technologies as a public site; what a VPN changes is who can get to it and how the traffic travels.

Extranets: letting a partner in, deliberately

An intranet solves the problem of your own staff. It does nothing at all for the other companies you depend on, and a modern business depends on a great many: the supplier who has to know what to ship next week, the distributor who needs current prices, the laboratory running your samples. Those people are not employees, so they cannot be inside the intranet. They are also not the general public, so the public site tells them nothing they need.

The arrangement built for them is an **extranet**, which can be regarded as a private part of the internet cordoned off from ordinary users, and which enables two or more firms to use the internet to do business together. Although the content is on the web, only authorized users can reach it after logging on to the company's extranet website, which they do with an ordinary web browser rather than with special software.

Because an extranet uses the public — and normally insecure — internet infrastructure to connect two or more business partners, it often uses VPNs to ensure the secured transmission of proprietary information between them. Same road, same armoured van, a different passenger.

The chapter draws both arrangements. Work through the diagram one at a time and notice how few parts there are, and how much of the design is simply deciding where the boundary goes.

INTERACTIVE DIAGRAM

Three arrangements, drawn one at a time

Step through the public site, the intranet and the extranet, and watch where the boundary moves in each one.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Public website — The baseline: one server, one browser, and no question about who is asking

Who asks (Anyone with an internet connection) → The path (The public internet, unrestricted) → The check (None — the content is meant for everyone) → What is reached (A web server holding general, public content)

- This is the arrangement the previous section described, and the chapter's summary table puts it first: external communications, general public content, anyone with an internet connection, public and not restricted.
- There is no authentication step because there is nothing to protect. Everything published here has already been decided to be public, which is what makes the arrangement simple.
- It is the right design for products, opening hours, job openings and anything else the organization wants a stranger, and a search engine, to find.
- The other two arrangements are this one plus a decision about who is authorized. Nothing else about the technology changes, which is the point the whole section rests on.

Intranet, reached over a VPN — One organization's own network, used from a desk inside it and from a laptop anywhere else

Who asks (A corporate client outside the local network) → The path (The public internet, crossed inside a VPN tunnel) → The check (A firewall at the edge of the corporate network) → What is reached (The server holding the internal site and its data)

- Inside the building the picture is simpler still: a client on the local network reaches the same server directly, with no tunnel and no crossing of the internet at all.
- That simplest form is worth remembering on its own. Communications take place only within the confines of organizational boundaries and do not travel across the internet, so nothing is exposed to be intercepted.
- The **VPN** exists only because people move. It carries the outside client's traffic across public infrastructure to the firewall, so that working from home reaches the same server as working from a desk.
- The **firewall** is where the organization's boundary is drawn. Everything inside it is corporate and proprietary content for authorized employees; everything outside it has to be let through deliberately.

Extranet, between two firms — Two separate corporate networks, joined across the public internet for as long as the business relationship lasts

Who asks (A client on the supplier's own network) → First boundary (The supplier's own firewall) → The path (The public internet, crossed inside a VPN tunnel) → What is reached (The partner's firewall, and behind it the server and data)

- Two firewalls appear here rather than one, because there are now two organizations and each defends its own boundary. Neither company is inside the other.
- The partner logs on to the company's extranet website using an ordinary **web browser**. Nothing unusual is installed on the partner's machine, which is a large part of why extranets are practical between firms that share no technology.
- Because the arrangement uses the public and normally insecure internet infrastructure to connect two or more business partners, it often uses a VPN to secure the transmission of proprietary information between them.
- The content is genuinely on the web, and it is still private. Only authorized users reach it after logging on, which is how the chapter's table can call an extranet external communication and private, restricted access at the same time.

One technology, three arrangements

The chapter summarises the whole distinction in a single table, and it is worth reading carefully, because none of the four columns mentions technology. The internet, an intranet and an extranet differ by what they are for, what is on them, who may use them and how access is controlled — not by what they are built from.

ARRANGEMENT	FOCUS	TYPE OF CONTENT	USERS	ACCESS
Internet	External communications	General, public content	Anyone with an internet connection	Public and not restricted
Intranet	Internal communications	Specific, corporate and proprietary content	Authorized employees	Private and restricted
Extranet	External communications	Communications between business partners	Authorized business partners	Private and restricted

Read down the Users column and the design decision appears on its own. Anyone, then authorized employees, then authorized business partners. That is the whole difference, and it is an organizational question rather than a technical one: somebody has to decide which category a person falls into before any system can enforce it.

One set of web technologies, three audiences, separated by who is authorized — which is why the question “should this be an intranet or an extranet?” is always answered by naming the reader, never by comparing products.

SORT Internet, intranet, or extranet?

Place each item using the chapter’s four dimensions: focus, type of content, users, and access. Situations that are not the chapter’s own are hypothetical practice situations, and each states the conditions you need.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Internet — external communications, general and public content, anyone with an internet connection, public and not restricted

- External communications carrying general, public content, open to anyone with an internet connection — This is the chapter’s own summary row. All four dimensions point outward, and no authorization step exists because nothing here was meant to be withheld.
- A page describing the company’s products and where its offices are, which the company wants search engines to find — Wanting to be found is the giveaway. This is general, public content aimed at anyone with a connection, and adding a login to it would defeat its only purpose.
- A careers page inviting applications from people who do not work for the organization yet — The intended reader is by definition not an employee, so restricting it to authorized employees would make it useless. It is public content with an external focus.
- In a hypothetical practice situation, a public help article the company deliberately publishes so that customers can solve a common problem without telephoning — The whole value depends on customers finding it unaided, which means anyone with an internet connection has to be able to read it. That is public, unrestricted access.

Intranet — internal communications, specific corporate and proprietary content, authorized employees, private and restricted

- Internal communications carrying specific, corporate and proprietary content for authorized employees — The chapter’s intranet row. The content is proprietary and the users are on the payroll, so access is private and restricted even though the technology is ordinary web technology.
- The required safety training every member of staff must complete, published where the public cannot reach it — Employee training is one of the five intranet uses the chapter lists, and the audience is authorized employees, which is what makes the access private and restricted.
- A form on which employees change their own benefit elections and retirement-plan choices — This is employee self-service through an employee portal, the intranet use the chapter singles out, and only people on the payroll may have an election to change.

- In a hypothetical practice situation, a hospital's directory of which clinicians are on call tonight, visible only after signing in with a work account — Internal communication, proprietary content, authorized employees only. The sign-in is the authentication technique the chapter names as one of the two ways an intranet is secured.

Extranet — external communications between business partners, authorized business partners, private and restricted

- External communications between business partners, restricted to authorized business partners — The chapter's extranet row, and the one that catches people out: the focus is external, like the public internet, but the access is private and restricted, like an intranet.
- In a hypothetical practice situation, a live order-status screen a component supplier logs in to see, so it knows what to ship next week — The supplier is neither an employee nor the general public. This is two firms using the internet to do business together, with the partner logging on through an ordinary browser.
- In a hypothetical practice situation, a shared price list that two firms maintain together so that neither has to email a spreadsheet to the other — A single shared copy between two companies is the extranet benefit of central document management and a single point of data entry, avoiding the versions problem entirely.
- In a hypothetical practice situation, a manufacturer gives an outside haulage firm a login so that the haulier can update delivery dates directly in the manufacturer's system — The haulier is an authorized business partner rather than an employee, and letting it enter the data once, at source, is exactly the single point of data entry the chapter describes.

What an extranet is actually worth

Knowing what an extranet is does not tell you why a company would build one. The chapter gives seven reasons, and they are the clearest passage in the book on using technology to work with other firms, so they are worth having in full.

Timeliness and accuracy — extranets can dramatically improve the timeliness and accuracy of communications, reducing the potential for misunderstandings inside the organization as well as with business partners and customers.

Keeping changing information current — in business very little information is static, so it must be continually updated and disseminated as it changes, and an extranet is a cost-effective, global medium over which proprietary information can be distributed.

Central management of documents — one managed copy reduces the number of versions in circulation and the amount of out-of-date information stored in various places throughout the organization.

A workable level of security on a public path — security is still considered better on proprietary networks, but the chapter's judgment is that the internet can be used as a relatively secure medium for doing business.

Automating business transactions — moving orders, confirmations and invoices through a system rather than through people reduces both processing costs and cycle times, which is to say both the money and the waiting.

A single point of data entry — extranets reduce errors by giving data one place to be entered, from which it can be updated on disparate corporate computing platforms without having to be reentered anywhere.

Real-time data for management — because the transactions arrive as they happen, management can obtain real-time data with which to track and analyze business activities rather than waiting for a monthly summary.

The sixth of those is the one that surprises people, so hold on to it. Most errors between two companies are not caused by carelessness; they are caused by the same fact being typed in twice. A quantity entered by the supplier and reentered by the buyer is two chances to be wrong and no way of knowing which copy is right. Removing the second keystroke removes the disagreement, and that is why a benefit that sounds clerical shows up in the accounts.

Where the machines actually live

Everything so far has been about arrangements of software. Underneath all of it there are physical machines in a physical room, and once an organization has enough of them, that room becomes a management problem in its own right. Large organizations need hundreds, or even thousands, of servers, and the chapter gives four very different reasons why.

Moving things — a parcel carrier such as UPS needs tremendous amounts of computing power simply to route and track packages as they cross a network of vehicles and depots.

Selling things — an online store such as Zappos needs to provide product information to shoppers and to track customer orders from the moment one is placed until it arrives.

Watching what players do — a game developer such as Epic Games needs to track each and every action users take in a game as popular as Fortnite, which is an enormous number of very small records.

Training models — a company such as OpenAI needs the computing power to train its latest generative artificial intelligence models, which is a different and far heavier demand again.

Because the hardware and storage grow quickly, companies typically set aside dedicated space for their infrastructure components. That dedicated space is a **data center**, and the word covers an enormous range: such facilities run from a single dedicated server room at the back of an office to buildings the size of a large warehouse.

Two practical facts follow immediately, and neither is about computing. Storing and processing massive amounts of data requires a great deal of power, and it also requires air-conditioning to keep the equipment running within its optimal temperature range, which helps increase the life span of the equipment. So one upgrade buys two bills: the electricity the machines draw, and the electricity the cooling draws to remove the heat they produce.

Putting everything in one place is a deliberate choice rather than an accident of history. Keeping the infrastructure in one location helps in managing, repairing, upgrading and securing the equipment — one set of spare parts, one set of doors, one place to send an engineer at three in the morning.

Choosing which location, though, is a genuine trade-off with no clean answer. Organizations go to great lengths selecting locations that strike the optimal balance between protection from the elements, such as earthquakes or hurricanes, and proximity to the customers or users, in order to reduce latency. Distance costs time, and the safest ground is rarely the ground nearest the people you serve.

Availability, and what five-nines means in minutes

Today, almost any business can be considered an e-business. Data are the lifeblood of almost all organizations, so reliably accessing those data is a key concern — especially for data-intensive organizations, which the chapter illustrates with a range running from e-commerce companies through logistics companies to government agencies.

Such organizations strive for the highest level of availability of their hardware, storage and networking components, often reaching for **five-nines availability**: 99.999 percent, which translates into just over five minutes of downtime per year. Do the arithmetic slowly, because the number is easy to nod at and hard to believe. Five minutes is not five minutes per outage. It is the entire budget for every failure, every upgrade and every mistake across twelve months.

A target that severe reaches past the computers and into the building. The chapter separates what it demands of the parts from what it demands of the facility around them.

Demands on individual components — being able to swap a failed hard drive or another part quickly, so that a failure becomes a repair carried out while the service is still running rather than an outage.

Demands on the facility as a whole — connectivity, floor space, provision of energy and cooling, and physical security, because a working server in a room with no power and no air conditioning is not available to anybody.

Demands on the design for tomorrow — data centers need to be modular so they can be expanded easily as needs change, since a facility that cannot grow forces the whole decision to be made again in three years.

One facility, described

Abstractions about availability become concrete very fast when you look at what a high-availability facility actually contains. The chapter describes the parcel carrier's data centers in Atlanta, Georgia and Mahwah, New Jersey as prime examples, and the details are worth reading as a list of things somebody had to buy.

Its own electricity — the data centers are self-sufficient, and each one can operate for up to two days on power it generates itself if the supply from outside fails entirely.

Cooling on the scale of a neighbourhood — the power is needed not only for the computers but for air-conditioning, and each facility needs air-conditioning capacity equalling that of more than 2,000 homes.

Water, and then backup water — if power fails, cooling is provided using more than 600,000 gallons of chilled water, and the facilities even have backup wells in case the municipal water supply should fail.

A building designed against weather — the buildings are designed to withstand winds of 200 miles per hour, and raised floors protect the equipment from flooding.

Read that list again as a bill rather than as a boast. Generators, fuel, chillers, wells, engineered walls and raised floors are all bought before a single customer request has been served, and they are all bought in order to prevent something rather than to produce something.

There is an alternative, and it is the one most organizations take. Rather than building any of that, an organization can rent space for its own servers in a **collocation facility** — a data center managed by a third party that rents out space to several organizational customers. The tenant still buys, owns and manages its own machines. What it stops buying is the building, the power, the cooling and the physical security around them.

The decisions below are the ones somebody has to make before any of this exists. Work them in order.

DECIDE**Siting and running a facility**

Work the four decisions in order, and compare every outcome before moving on, including the outcomes of the options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

A hypothetical practice situation; no real organization is being described. You advise a growing logistics cooperative that has outgrown the server room behind its dispatch office. It now needs somewhere proper to put its machines, and four decisions have to be made before anything is built or rented.

Decision 1. Two candidate sites cost the same. One sits inland on stable ground, far from any concentration of the cooperative's customers. The other sits close to the largest group of customers but on an exposed coast. The board wants a rule for choosing.

- Treat it as a genuine trade-off between protection from the elements and proximity to users for lower latency, and decide it alongside power, cooling, connectivity, security and room to expand — **the stronger call**. Right. The chapter says organizations go to great lengths selecting locations that strike the optimal balance between protection from the elements, such as earthquakes or hurricanes, and proximity to the customers or users in order to reduce latency. Neither consideration outranks the other in advance, which is why the facility requirements have to be weighed in the same conversation.
- Choose the inland site automatically, because once a connection is fast enough distance no longer affects anything the user notices — Distance still costs time. That is precisely why the chapter names proximity to customers and users as a siting consideration in its own right, and it is the same physical fact that later justifies putting copies of content nearer to the people requesting it.
- Choose the coastal site automatically, because reducing delay for users is the only difference a customer will ever perceive — Latency is real, but a flooded facility is not slow, it is unavailable, and the chapter treats availability as the key concern for a data-intensive organization. A site that is fast on most days and absent on the worst day has not solved the problem.
- Decide it on land price alone, since the two sites are equally priced today and the cost of the building will dominate everything else — The two sites are already equally priced, so this rule does not even separate them. It also ignores the running costs the chapter emphasises — power, cooling and the consequences of downtime — which continue for as long as the facility does.

Decision 2. The design brief doubles the number of servers. The facilities manager proposes keeping the existing air-conditioning, arguing that the machines have their own fans and that cooling is a comfort item rather than an operating requirement.

- Reject it: processing that volume of data requires lots of power and also requires air-conditioning to hold the equipment within its optimal temperature range, which helps increase the life span of the equipment — **the stronger call**. Right. The chapter ties the two together explicitly. Heat is not a side effect to be tolerated; running outside the optimal range shortens the life of the hardware, so an upgrade that doubles the machines has quietly doubled a cooling requirement as well.
- Accept it, because cooling capacity only has to be increased once the equipment actually begins to fail in service — By the time components fail, the damage is already paid for. The chapter presents cooling as what keeps equipment inside its optimal range and therefore extends its life, which makes it a preventive cost rather than a repair triggered by a failure.
- Accept it, but move the servers further apart inside the room so the existing air-conditioning has less work to do — Spreading equipment out uses more floor space, which the chapter lists among the facility's own requirements, and it does not remove any heat from the room. The total energy going in still has to come out as heat somewhere.
- Defer it, on the grounds that electricity is a running cost and running costs are decided after the facility is built — Provision of energy and cooling is one of the demands the chapter places on the data center overall, alongside connectivity, floor space and security. Treating it as an afterthought is how a facility ends up unable to power the equipment it was built to hold.

Decision 3. The cooperative's dispatch system now runs continuously, and the board asks what a five-nines availability target would actually commit it to.

- Just over five minutes of unavailability in an entire year, covering every failure, upgrade and mistake, which constrains both the components and the building around them — **the stronger call**. Right. Five-nines means 99.999 percent availability, which the chapter translates into just over five minutes of downtime per year. That budget forces the specific demands it lists: parts such as hard drives that can be swapped quickly, and a facility with the connectivity, floor space, energy, cooling and security to keep running while that happens.
- Five minutes of unavailability per incident, after which the service is expected to be restored — This is the most common misreading and it makes the target sound comfortable. The chapter's figure is annual, so the whole year's worth of interruptions has to fit inside the same five minutes.
- A promise that no individual component will fail during the year, which is what the additional nines are counting — Availability is not the absence of failure. The chapter's own requirement is being able to swap a failed hard drive or other part quickly, which assumes components will fail and asks that the service survive it.
- A target about the speed of the service rather than its availability, since users judge a system by how quickly it responds — Response time matters and the chapter discusses it under latency, but availability answers a different question: whether the system can be reached at all. A fast system nobody can open is unavailable.

Decision 4. Finally, the board compares building a facility of its own against renting space for its servers in a data center run by a third party that also rents to other organizations.

- Recognize the second option as a collocation facility: the cooperative still owns and manages its own servers, but stops paying for the building, the power, the cooling and the physical security — **the stronger call**. Right. The chapter defines a collocation facility as a data center managed by a third party that rents out space to multiple organizational customers. It is a change in what you own around the machines, not in who owns the machines, and that distinction is what makes it a different decision from renting computing itself.
- Treat the two as identical in substance, since the servers are the same machines running the same software in both cases — The machines are indeed the same, which is what makes this tempting. Everything around them differs, though, and the chapter's facility example is a list of exactly those surroundings: generated power, chilled water, engineered walls and raised floors.
- Reject renting on the grounds that high availability can only be reached in a facility the organization has built and controls itself — The chapter presents collocation as a straightforward alternative rather than a compromise, and a shared facility built for many tenants may well exceed what a single small organization could justify building alone.
- Choose renting on the grounds that it removes the cooperative's responsibility for its servers and for the data on them — Renting space is not renting responsibility. In a collocation facility the tenant still owns, manages, patches and repairs its own servers; what the third party manages is the room they stand in.

WHERE THIS IS HEADING

Notice what the facility in that example actually spent its money on: generators, chillers, water, walls, floors, guards, spare parts and room to expand. Almost none of it is computing. Every item on that list is something a cloud customer stops buying, and that observation — not any argument about technology — is what the next objective is really about.

One last pass over the vocabulary of this section before the infrastructure starts aging.

CHECK YOURSELF

Facilities, availability, and the alternative

Three questions on where the machines live and what keeping them running demands; every explanation says what the option it describes would really be.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. What does the chapter mean by a data center, and how large is one?

- **A.** A building of at least warehouse size, since anything smaller is a server room and is not managed as a facility — Size is the one thing the chapter refuses to fix. It gives a range precisely so that a single dedicated server room counts, which matters because most organizations start there.

- **B.** Dedicated space set aside for an organization's infrastructure components, ranging from a single server room to a building the size of a large warehouse — **correct**. The chapter says companies typically set aside dedicated space for their infrastructure components, and that such data centers can range in size from a single dedicated server room to buildings the size of a large warehouse.
- **C.** The collection of servers an organization owns, wherever those machines physically happen to be located — Scattered machines are what a data center replaces. The reason for gathering them is stated plainly: keeping the infrastructure in one location helps in managing, repairing, upgrading and securing the equipment.
- **D.** A facility operated by a third party in which organizations rent space rather than building anything of their own, which is what the word has meant since renting became the usual arrangement — That describes a collocation facility, which the chapter offers as an alternative to building one. A data center may be your own or somebody else's, and the definition does not turn on who owns the walls.

2. A manager says the organization has reached five-nines availability because its main server has not failed since it was installed eighteen months ago. What is wrong with the reasoning?

- **A.** Nothing is wrong, because an unbroken run is the strongest possible evidence that the availability target has been met — An unbroken run of one machine is encouraging and is not the measure. The chapter attaches availability to the hardware, storage and networking components together, and to the data center overall.
- **B.** Availability is measured component by component, so a single server can never be described using a figure that belongs to the facility as a whole, and the two numbers should not be compared — The chapter does place demands on individual components, such as swapping a failed drive quickly, so component-level requirements are real. The target itself is about the service being reachable, which no single part can deliver alone.
- **C.** Five-nines describes response time rather than uptime, so an unbroken run says nothing about whether the target was reached — Response time is a separate concern, discussed under latency and proximity to users. Five-nines is 99.999 percent availability, which the chapter translates directly into just over five minutes of downtime per year.
- **D.** Availability covers whether the service can be reached at all, which depends on connectivity, energy, cooling and security as much as on the server itself — **correct**. The chapter ties the target to hardware, storage and networking components together and to the facility's connectivity, floor space, energy, cooling and security, so a healthy server proves only that one of those has held.

3. The chapter lists the parcel carrier's self-generated power, its chilled-water reserves, its wind-rated buildings and its raised floors. What is the managerial lesson that leads directly into the next objective?

- **A.** That reliable infrastructure is bought mostly outside computing, which is exactly the set of costs an organization stops carrying when it rents capacity instead of owning it — **correct.** Generators, cooling, water, engineered walls and raised floors are all bought to prevent interruption rather than to compute anything, and shedding that whole category of spending is the economic argument the cloud material rests on.
- **B.** That any organization needing reliable systems should expect to build a facility of this kind for itself before it can operate safely, and should treat the cost as the price of being taken seriously — The chapter offers the facility as a prime example of high availability, not as a template. It immediately names renting space in a collocation facility as the alternative most organizations take.
- **C.** That the largest risk to a data center is severe weather, so siting decisions can be settled by weather records alone — Weather is one half of the siting balance and proximity to users for lower latency is the other, and the chapter insists on both. A decision made on weather records alone has ignored everything the users experience.
- **D.** That equipment failure is the main cause of downtime, so the answer is to buy more reliable individual machines — Component reliability matters, which is why the chapter asks that a failed drive be swappable quickly. But it treats failure as expected and survivable, and it puts connectivity, energy, cooling and security beside the machines as equal requirements.

Why infrastructure goes out of date

Nothing you met in the last six sections wears out the way a delivery van wears out. A server that has run for six years without a single fault is still a working machine, and it can still be the wrong machine. That gap — between equipment that works and equipment that is enough — is what this objective is about. This section explains why the ground under an infrastructure keeps moving. The next one puts a cost on the movement.

The challenge, and why it lands on a manager

Obtaining, operating, maintaining and upgrading an information systems infrastructure is a tremendous challenge for an organization. The chapter then adds the clause that turns it from a technical problem into a management one: it is especially a challenge when these tasks are **not part of the organization's core business**.

A clinic is in the business of treating people. A freight company is in the business of moving pallets. Neither of them wants to be in the business of deciding when to replace three hundred computers, and yet both have to decide, because the equipment will not decide for them. It will simply keep running.

The chapter draws the pressures as arrows pointing inward at a single building, which stands for the organization and its data center. There are seven of them, and together they are the map of everything this objective covers.

Increasing obsolescence — hardware and software investments stop being current well before they stop working, which is the subject of this section.

Diminishing space — equipment accumulates while the room holding it does not grow, so floor area itself eventually becomes the binding constraint.

Expanding storage — more data is captured and kept every year, every retained record has to sit on a disk that somebody bought, and the richer content built on top of it pushes more traffic through the same network at the same time.

Expanding consumption — the electricity the machines draw in order to do the work, and the electricity drawn again to remove the heat that doing the work produces, which is the pressure the chapter's own text calls increasing energy needs.

Fluctuating demand — the work does not arrive evenly, so capacity sized for the busiest hour sits idle through most of the others.

Need for agility — the organization has to be able to try something new without first buying all the infrastructure a success would require.

AI infrastructure — the newest of the seven, because the machines that train models are not the machines an organization already owns.

This section takes the first of those seven on its own. The section after it takes five more of them together, because they share a shape that is much easier to see once obsolescence has been explained properly. Diminishing space is the sixth, and it gets no heading of its own there for the same reason the chapter gives it none: the room running out is discussed as part of the growth in stored data and in the facilities that have to hold it.

What obsolescence actually means

The everyday sense of the word is wrong here, and correcting it is most of the lesson. A thing is not obsolete because it is broken. It is obsolete because what it can do has fallen behind what is now needed. The machine has not changed at all; the requirement moved out from under it.

Over the past seventy-five years, information systems have gone through many radical changes, and rapid advances in both hardware and software capability have enabled or facilitated business processes that were simply not possible before. Organizations are therefore continuously faced with the need to upgrade the infrastructure in order to gain or maintain competitive advantage — the same phrase Module 2 used about strategy, arriving here as a question about equipment.

Ask what the organization can no longer do, not what has failed. Obsolescence is a capability gap, and a fleet with a perfect maintenance record can have an enormous one.

Before computers, so the change has something to be measured against

When an early mechanical computer driven by punched program cards was introduced in 1936, almost all business and government information systems consisted of file folders, filing cabinets and document repositories. Huge rooms were dedicated to storing those records. The consequences were not filing inconveniences; they were management problems.

Information was difficult to find — a fact existed somewhere in the building, and reaching it depended on knowing which cabinet and which year to open.

Corporate knowledge was difficult to maintain — only certain employees knew particular things, and when those employees left the firm, all their knowledge about the organization left with them.

History was difficult to reconstruct — the record of what had been done was only ever as complete as what somebody chose to file and could still retrieve years later.

The computer was the solution to the information storage and retrieval problems that organizations faced up to the 1940s. Everything in this module — the hardware, the operating system, the network, the data center — descends from that one answer. That is also why the answer keeps having to be bought again.

Generations of computing

Shifts in computing eras were made possible by fundamental changes in the way computing technologies worked. Each of those fundamental changes is a **generation of computing**, and the chapter tabulates them.

Read the table by column rather than by row. The first column is only a number. The third names the technology that changed. The fourth is the one that matters to a manager, because it says what organizations could suddenly do that they could not do before.

GENERATION	TIME PERIOD	MAJOR CHARACTERISTIC	WHAT CHANGED AROUND IT
1	1946–1958	Vacuum tubes	The mainframe era begins; ENIAC and UNIVAC are developed
2	1958–1964	Transistors	The mainframe era expands; UNIVAC is updated with transistors
3	1964–1990s	Integrated circuits	The mainframe era ends and the personal computer era begins; the IBM 360 arrives with a general-purpose operating system; the microprocessor revolution brings Intel, Microsoft, Apple, the IBM PC and MS-DOS

GENERATION	TIME PERIOD	MAJOR CHARACTERISTIC	WHAT CHANGED AROUND IT
4	1990s–2000	Multimedia and low-cost personal computers	The personal computer era ends and the interpersonal computing era begins; high-speed microprocessors and networks; high-capacity storage; low-cost, high-performance integrated video, audio and data
5	2000–2010	Widespread internet accessibility	The interpersonal computing era ends and the internetworking era begins; access to the internet from a broad variety of devices; prices keep dropping and performance keeps expanding
6	2010–present	Ubiquitous mobile connectivity	Powerful mobile devices arrive; the cloud computing era begins; Big Data, the Internet of Things and social networking
7	Present and beyond	Artificial intelligence and quantum computing	The rapid rise of AI and generative AI algorithms; quantum computing

Two honest notes belong beside that table. Its own caption says six generations while the table itself lists seven, the periods overlap or leave gaps, and the last row has no start year at all. Treat the periods as a teaching device that shows a direction of travel, not as settled history that could be defended date by date.

Before reading on, put the shifts into order yourself. The chain of era names is the clue: mainframe, then personal, then interpersonal, then internetworking, then cloud.

SEQUENCE

The generations of computing, in order

Put the seven shifts into the order the chapter gives them, then read why each one sits where it does.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Do not try to recall the years. Each shift ends one era and opens the next, and the chain of era names carries you through: mainframe, personal, interpersonal, internetworking, cloud. Work out which technology could only have come after which, and the sequence assembles itself.

1. **Vacuum tubes. The mainframe era begins, and the machines named ENIAC and UNIVAC are developed.** — This is the first generation because it is the first working answer to the filing-cabinet problem. The machines are enormous, few, and operated by specialists, which is exactly what a mainframe era means: the computer is a facility that an organization visits rather than a tool anyone holds.
2. **Transistors. The mainframe era expands, and UNIVAC is updated with the new components.** — Nothing about the role of the computer changes here, which is why it expands the era rather than ending it. Transistors do the same switching job as a vacuum tube while being smaller, cooler and far more reliable, so an organization can afford more computing than before without rethinking what computing is for.
3. **Integrated circuits. The mainframe era ends and the personal computer era begins; the IBM 360 arrives with a general-purpose operating system, and the microprocessor revolution brings Intel, Microsoft, Apple, the IBM PC and MS-DOS.** — Putting many components onto one chip is what makes a computer small and cheap enough to sit on a desk, so this is the shift that moves computing out of the machine room. Notice the general-purpose operating system in the same row: the moment hardware becomes plentiful, the layer that hides it becomes valuable.
4. **Multimedia and low-cost personal computers. The personal computer era ends and the interpersonal computing era begins, with high-speed microprocessors and networks, high-capacity storage, and low-cost integrated video, audio and data.** — Once machines are cheap and connected to one another, the interesting thing is no longer the machine but what passes between people using them. The era name says so: interpersonal computing follows personal computing because you need the personal machines to exist first.
5. **Widespread internet accessibility. The interpersonal computing era ends and the internetworking era begins, with access to the internet from a broad variety of devices while prices keep dropping and performance keeps expanding.** — This is the point at which the value of a machine starts to depend mostly on what it can reach. Local networks connected colleagues; widespread internet access connects an organization to everyone else, which is the precondition for every business use in the last section of this module.
6. **Ubiquitous mobile connectivity. Powerful mobile devices arrive and the cloud computing era begins, alongside Big Data, the Internet of Things and social networking.** — Connectivity stops being something you sit down to use and starts following the person, which is what makes renting computing sensible: if the device is small and always connected, the heavy work can happen somewhere else. That is the whole premise of the objective after this one.

7. Artificial intelligence and quantum computing, with the rapid rise of generative AI algorithms. — This row is the current shift rather than a finished period, which is why it has no start year in the chapter. It matters for infrastructure because it changes what an organization has to supply: different processors, different storage, and skills that a traditional technology department may not have.

The same table repays a second pass with a different question. Not when each technology arrived, but what it let an organization do that it could not do the week before it existed.

MATCH

Each generation and what it made possible

Pair each defining technology with the change it produced for organizations, then read why the pairing holds.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Vacuum tubes

Rooms of file folders begin to be replaced by a machine that can store records and find them again The chapter frames the first generation against what came before it: filing cabinets, difficult retrieval, and knowledge that walked out of the door with the employee who held it. The first computers were a solution to a storage and retrieval problem, not a productivity gadget.

Transistors

The same computing work is done by smaller, cooler and more reliable parts, so organizations can afford more of it This is the generation that changes the economics without changing the concept. More reliable components mean less downtime and more machines within reach of a normal budget, which is why the chapter records the era as expanding rather than ending.

Integrated circuits

Computing leaves the machine room: a computer becomes something a department, and then a person, can own Packing many components onto a single chip is what made a computer small and cheap enough to be personal. The general-purpose operating system in the same row is the other half of the change, because ordinary owners need a layer that hides the hardware from them.

Multimedia and low-cost personal computers

Machines become cheap enough to be everywhere and capable enough to carry video, audio and data together Cheap capable machines plus networks are what turn computing from something you do alone into something you do with other people, which is why the chapter names the era that follows interpersonal computing.

Widespread internet accessibility

The value of a machine starts to depend on what it can reach rather than on what it holds Once the internet is broadly available from many kinds of device, an organization is reachable

by anyone and can reach anyone, which is the foundation of every business use of the internet this module discusses later.

Ubiquitous mobile connectivity

Computing follows the person, and the heavy work starts moving to somewhere else that is always reachable. The chapter puts the beginning of the cloud computing era in this same row for exactly that reason. A small device that is always connected does not need to hold the capability itself, which is the argument the next objective develops in full.

Artificial intelligence and quantum computing

The infrastructure question changes shape, because the new work needs different processors, different storage and different skills. This is the only row that is still open, and the chapter lists it as a current shift rather than a completed period. It reappears in the next section as one of the seven drivers, since an organization new to this work faces large costs and integration challenges.

Moore's law, and what it is not

In 1965, Gordon Moore, a cofounder of Intel, hypothesized that the number of transistors on a chip would double at regular intervals, and he did not limit the prediction to any specified period of time. The prediction became known as **Moore's law**. The evidence the chapter offers is a comparison across decades: the first processor held a couple of thousand transistors, and current models hold billions.

The word "law" is what does the damage, because it invites a student to treat a business trend as physics. Four qualifications keep the idea usable.

It is an empirical observation, not a law of nature — nothing in physics requires the doubling, and the industry has partly treated it as a roadmap it agreed to keep hitting.

The interval has been restated — the original statement described roughly annual doubling, and the two-year figure the chapter quotes came from a later revision.

The pace has slowed — the chapter half concedes this when it says gains are now shaped by technological and economic limitations rather than by engineering alone.

The gains increasingly come from parallelism — from adding more computing cores that perform tasks at the same time, rather than from one core running proportionally faster each year.

The chapter also gives a figure for how many operations a current processor completes each second. It sits well below what modern processors actually do, so read it as an illustration of the idea rather than as a number worth carrying away.

What survives every one of those qualifications is the shape, and the shape is the part a manager needs. Capability for a given amount of money has risen relentlessly for decades. A machine bought five years ago is therefore competing against something considerably more capable at the same price, and nothing about it has to break for that comparison to start costing the organization something.

Faster cycles, and devices the organization never chose

This rising capability is both a blessing and a curse. Increases in processing power enable applications that were previously not possible; at the same time, managers must continuously think about when to upgrade the hardware components of the infrastructure. Two further pressures make that timing harder than the underlying trend alone would suggest.

IT cycles have shortened sharply — managers traditionally thought in terms of about five years, while manufacturers now release new versions of devices every six to twelve months.

Consumerization of IT — the devices people choose for themselves arrive at work, and the organization must then decide how to integrate, secure and support equipment it did not select.

The **consumerization of IT** moves part of the equipment decision away from the technology department and toward the individual employee. That is a governance change wearing the costume of a hardware change: the organization still carries every consequence of a device it never bought, never specified and cannot simply recall.

CHECK YOURSELF

Obsolescence, Moore's law and shortening cycles

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical veterinary practice is told its computers are fine because none of them has failed in four years. What is the chapter's objection to that reasoning?

- **A.** Equipment becomes unreliable at a predictable age, so a fixed four-year replacement cycle removes the need for judgment — Age and reliability are related but they are not the argument here. The chapter never claims machines fail on a schedule, and a replacement rule set by the calendar would still miss the reason for replacing them.
- **B.** Obsolescence is measured against what the organization now needs to do, so working equipment can still be too far behind to support the work — **correct**. The chapter describes accelerating obsolescence of hardware and software investments, driven by advances that enable business processes which were not previously possible. The gap is between capability and requirement, so a fault-free fleet can be badly obsolete.

- **C.** Obsolescence applies to software rather than hardware, so the age of the machines is not the relevant question at all — Both age. The chapter devotes one part of this objective to hardware obsolescence and the next to software obsolescence, and its central figure shows each one forcing the other along.
- **D.** The practice has not depreciated the equipment correctly, so the replacement decision is really an accounting matter — How the asset is carried in the accounts changes the timing of a cost, not the capability of the machine. A fully depreciated computer that does everything required is not obsolete, and a new one that cannot run the software is.

2. A student reads that transistor counts have doubled at regular intervals for decades and concludes that computing power is guaranteed to keep doubling. Which correction does the chapter's own wording support?

- **A.** The doubling describes storage capacity rather than transistor counts, so the conclusion applies to the size of disks instead of to the speed of processors, and the student has read a real observation against the wrong quantity — The prediction concerned the number of transistors on a chip. Storage capacity has its own history, and this module treats growing storage as one of the seven separate drivers rather than as the same claim.
- **B.** The prediction was made for a fixed number of years and then expired, so nothing follows from it about any period after that — The chapter says the opposite about the term: when the prediction was made, it was not limited to any specified period of time. What has changed is the pace, not an expiry date somebody wrote down.
- **C.** It is a prediction about how many transistors fit on a chip, and it stopped being reported once chips passed a billion transistors — Transistor counts are still reported, and the chapter uses exactly that comparison as its evidence, from a couple of thousand on the first processor to billions on current ones.
- **D.** It is an observation rather than a physical law, and its gains are now limited by technological and economic factors — **correct.** The chapter itself says that, given technological and economic limitations, today's gains in computing power are increasingly realized by adding more cores that perform tasks in parallel, which is a very different statement from a guarantee.

3. The technology manager of a hypothetical staffing agency has always planned equipment on a five-year horizon and finds it no longer works. Which two pressures does the chapter name beyond the underlying advance in capability?

- **A.** Shorter release cycles, with new versions of devices every six to twelve months, and the consumerization of IT bringing employees' own devices into the infrastructure — **correct.** The chapter names exactly these two beyond the capability trend: manufacturers now release new versions of devices every six to twelve months where managers once thought in terms of about five years, and consumerization means integrating users' own mobile devices.

- **B.** Longer vendor contracts and a shortage of technical staff, which together stretch every upgrade well beyond the window it was planned for and leave the fleet part-new and part-old for years at a time — Staffing and contract length are real difficulties, but they are not the two factors the chapter identifies here, and neither of them explains why the planning horizon itself has to shrink.
- **C.** Rising energy prices and shrinking floor space, which make each replacement cycle more expensive than the one before it — Energy and space are genuinely on the chapter's list of seven drivers, and they are covered in the next section. They are pressures on the infrastructure rather than reasons the replacement cycle has shortened.
- **D.** Falling equipment prices and improving reliability, which together make a five-year plan unnecessarily cautious — Prices falling and reliability improving would argue for a longer horizon, not a shorter one, which is the opposite of the situation the manager is describing.

Software gets old too, and it takes the hardware with it

Hardware is only half the story. Companies such as Microsoft are continuously developing new and improved software that uses the increased hardware power to help people be more productive, and each new generation of application software promises better performance and more features. That promise arrives with three strings attached.

New operating systems often require new hardware — a system such as Windows 11 can use new processor architectures and offer a richer feature set than an older one such as Windows XP, but only on machines that meet its requirements.

Older applications may stop working — older-generation application software may not be compatible with the new operating system, so an upgrade to one program can strand another.

The productivity gain is a claim, not a receipt — upgrades may increase productivity, and often they do not, while the cost of making them is incurred either way.

Some of that ageing is deliberate. **Planned obsolescence** means designing a product to last only for a certain life span, and manufacturers of hardware and software both apply the concept.

In hardware — certain components are not built to be serviceable, so the whole device has to be replaced once one of those components breaks down.

In software — older versions may be unable to open newer file formats, or a company may simply cease support for a product, which effectively forces users to switch.

The chapter's example of the second kind is a decision many organizations recently had to work through. Mainstream support for Windows 10 ended in October 2025, with paid extended security updates offered for a limited period afterwards, and the successor version carried very strict hardware requirements. Nothing about those machines changed on that date. What changed were the terms on which they could responsibly be used.

Why an upgrade is a project, not a purchase. Organizations are constantly faced with the decision of when and how to upgrade the current infrastructure, and the chapter is unusually blunt about the arithmetic. The costs run in two channels at once.

- The cost of the hardware and the software itself, which is the number that appears in the proposal.
- The time and resources needed for upgrading tens, hundreds or thousands of computers, which is usually the larger figure and rarely the one being argued about.

Multiply the second channel by a fleet and you can see why an organization delays, and why delaying eventually produces a single enormous replacement instead of a series of small ones.

The treadmill, and the bill it leaves outside the building

Put the two halves together and they form a loop the chapter draws as two arrows. More powerful hardware **enables** more powerful software; more powerful software then **requires** more powerful hardware. Each side is a perfectly reasonable engineering decision, and together they never stop turning.

Nobody has to be acting in bad faith for this to happen, which is what makes it worth naming. A vendor writing software that uses a capable machine well is doing its job. A manager who wants the features is doing hers. The loop is a property of the arrangement, not evidence of a conspiracy, and you will meet the same shape again in the next section when richer content drives demand for bandwidth and storage.

The loop also has a cost that appears on nobody's invoice. The rapid obsolescence of computer hardware carries a high price for the environment, in terms of the resources needed both to manufacture the new systems and to dispose of the old ones. That thread is picked up later in this module, where the responses to it are grouped under green computing.

Two pictures hold this section together: the map of seven pressures you met at the start, and the loop you have just read.

INTERACTIVE DIAGRAM

The seven pressures, and the loop underneath them

Open each view: the first is the map of what pushes an infrastructure to change, and the second is the mechanism that keeps it pushing.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

The seven drivers — Seven arrows pointing inward at one building, which stands for the organization and its data center

Space and storage (Diminishing space and expanding storage) → Consumption and demand (Expanding consumption and fluctuating demand) → Obsolescence (Hardware and software falling behind what is needed) → Agility and AI (The need to move fast and the need for AI infrastructure)

- The arrows all point inward, and that is the argument of the figure: none of these pressures is generated inside the organization, yet every one of them lands on it and has to be paid for by it.
- **Increasing obsolescence** is this section's subject. Hardware and software investments stop being current faster than they stop working, so the decision to replace is never forced by a failure.
- **Diminishing space** and **expanding storage** are the growth pressures: more data retained every year, and more equipment holding it inside a room that does not grow.
- **Expanding consumption** is the electricity pressure, and it is the one the label hides. More powerful machines draw more power to work and then more power again to be cooled, so the same upgrade appears twice on the same bill.
- **Fluctuating demand** is the shape pressure. The work does not arrive evenly, so capacity bought for the peak is idle the rest of the time and capacity bought for the average fails at the peak.
- **Need for agility** and **AI infrastructure** are the two that most directly argue for renting rather than owning, and both are taken up in the next section and then answered in the objective after it.

The upgrade treadmill — Two arrows forming a closed loop between hardware and software, each one labelled with what it does to the other

Powerful hardware (A more capable machine becomes available) → Enables (Software can now do things it could not do before) → Powerful software (The next release is built to use that capability) → Requires (So it will not run acceptably on the older machines)

- Read it as a circle rather than a line. There is no first cause and no end: each side of the loop is a sensible decision that makes the other side inevitable.
- Nobody in the loop is behaving badly. A vendor that writes software making full use of a capable machine is doing its job, and a manager who wants the new capabilities is doing hers.
- The loop is why obsolescence arrives on a schedule the organization did not set. The trigger is a release somewhere else, not a fault in anything the organization owns.

- The same shape appears again with data: richer content and more analysis require more storage and more bandwidth, which make richer content possible, which the chapter calls a vicious circle.
- It leaves a cost outside the accounts altogether, in the resources needed to manufacture the new equipment and to dispose of the old, which is where green computing enters later in this module.

An infrastructure is not bought once. It is rented from the future in instalments, and obsolescence is the name of the instalment that falls due whether or not anything has broken.

CHECK YOURSELF

Software obsolescence and the loop it belongs to

Three situations on the second half of this section; every option explains itself, including the three you do not pick.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical community college learns that the operating system on its lab machines will stop receiving mainstream support, that paid extended security updates are available for a limited period afterwards, and that the successor version has strict hardware requirements. Which concept does the chapter use for this?

- **A.** Consumerization of IT, since the decision about which system to run has effectively moved outside the organization and is now being made by the people who supply the devices staff already use at home — Consumerization concerns employees' own devices arriving at work and having to be integrated. Here the college chose and owns the machines; what it does not control is how long the vendor keeps supporting them.
- **B.** A generation of computing, since a change of operating system is one of the fundamental shifts in how computing technologies work — A generation of computing is a fundamental change in the way computing technologies work, such as the move from vacuum tubes to transistors. A new version of an operating system is an ordinary product release inside one generation.
- **C.** Planned obsolescence, in which a product is designed to last a certain life span and withdrawn support forces a decision the machines themselves would never have forced — **correct.** The chapter gives ceasing support for a product as one of its two examples of planned obsolescence, alongside older software that cannot open newer file formats, and notes that this effectively forces users to switch.
- **D.** Fluctuating demand, since the pressure on the college arrives at a specific date rather than evenly across the year — Fluctuating demand is about the volume of work arriving unevenly, which the next section covers. A support deadline is a date, not a spike in the amount of work the machines are being asked to do.

2. A hypothetical publisher notices that every hardware purchase is justified by software that will not run on the old machines, and every software upgrade is justified by capabilities the new machines made available. A director asks whether someone is manipulating them. What does the chapter's figure actually describe?

- **A.** A loop in which more powerful hardware enables more powerful software, which in turn requires more powerful hardware, with each step a reasonable decision on its own — **correct**. The chapter draws two arrows: powerful hardware enables powerful software, and powerful software requires powerful hardware. Naming it as a loop is what lets a manager plan for it rather than be surprised by each turn.
- **B.** A misunderstanding, because hardware capability and software capability advance quite independently of one another and any apparent link between the two is a coincidence of timing rather than a mechanism — They plainly do not advance independently, which is the entire content of the figure. New operating systems use new processor architectures, and older applications may not run on them at all.
- **C.** A purchasing failure, which disappears once the organization buys hardware and software from the same vendor under one agreement — Buying from one vendor changes who sends the invoices. It does not change the fact that a more capable machine invites more demanding software, which then makes the previous machines inadequate.
- **D.** A financing question, since the pattern stops mattering as soon as the equipment is leased rather than bought outright — Leasing changes how the cost is spread over time, and it may make replacement easier to schedule, but the equipment still has to be replaced on the same cycle for the same reason.

3. A hypothetical charity is deciding whether to replace six hundred aging computers and is comparing the vendor quotation against doing nothing. Which cost does the chapter say organizations most often leave out of that comparison?

- **A.** The purchase price of the software licences, which is normally quoted separately from the hardware and easily overlooked — Licence costs are part of the quoted cost of hardware and software, and they are visible in the proposal. The chapter separates the visible purchase from the invisible labour precisely because the second is the one that gets missed.
- **B.** The time and resources needed to carry out the upgrade across every machine, which for a fleet of that size usually exceeds the equipment cost — **correct**. The chapter counts both costs for hardware and software and the time and resources needed for upgrading tens, hundreds or thousands of computers, and it is the second that scales with the size of the fleet.
- **C.** The cost of retraining every member of staff, which the chapter treats as the single largest component of any upgrade — Training genuinely matters and often accompanies a major change, but the chapter does not single it out as the largest component, and an upgrade to a familiar system may need very little of it.

- **D.** The interest forgone on the money spent, since the comparison should be made against leaving the funds invested — The opportunity cost of the money is a legitimate part of any investment appraisal, and it is not the omission the chapter is pointing at in this passage.

Storage, spikes, energy, and the need to move fast

The section before this one explained why equipment stops being good enough long before it stops working. This one takes the other pressures on the chapter's diagram, and not one of them is about age. Data accumulates and is never thrown away. Demand arrives in bursts. One upgrade produces two electricity bills. Experiments need equipment before anybody knows whether the experiment is worth running. And artificial intelligence asks for a class of machine most organizations have never bought.

Read the five as one argument rather than five separate complaints. Each of them describes a way in which owning your own infrastructure forces a decision at the moment you have the least information. Buy for the peak and you own idle machines. Buy for the average and you fail on the busiest day. Buy before the experiment and you may be buying for a service nobody wants. Hold on to that sentence, because it is the whole reason the next objective exists.

EXPLORE

The five pressures, one card at a time

Open each card and read all four panels; the panels are the same four questions asked of every driver, which is what makes them comparable.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Expanding storage · More data, kept for longer

- **What the driver is:** More data are captured for business intelligence, for machine learning and artificial intelligence, and for purposes such as compliance, and the amount that feels sufficient is judged against what competitors do with theirs.
- **Where it shows up:** An organization analyzes what every visitor does on its own website in order to improve how the site performs, and runs algorithms over structured and unstructured material from media reports, social media and customer support calls.
- **What it costs you:** Three costs rather than one: storage space, more powerful computing hardware to work through the data, and database management systems to manage and analyze what has been kept. Richer content raises the bandwidth bill alongside them.
- **The move it argues for:** No single purchase closes this, because the chapter's vicious circle keeps turning — enhanced capability enables new applications, and the new applications require more capability again.

Fluctuating demand · Sized for the busiest hour

- **What the driver is:** Demand for computing resources rises and falls, which leaves an organization either short of resources at some moments or, far more often, holding idle resources most of the time.
- **Where it shows up:** Firms engaged in or supporting business-to-consumer electronic commerce meet a large spike before the December holidays; the chapter names Amazon and FedEx as its examples of who feels it.
- **What it costs you:** The chapter reports an estimate that as many as seven infrastructures in ten run at only about a fifth of their capacity. Growing costs again in money, in the time needed to locate facilities and negotiate contracts, and in long contracts that prevent scaling back down.
- **The move it argues for:** Its sharpest sentence sits here: it is relatively easy to hire temporary staff for a busy season, and typically not easy to make quick changes to an infrastructure.

Expanding consumption · One upgrade, two bills

- **What the driver is:** Processing data consumes electricity, and components such as the processor and the power supply convert part of it into heat that then has to be removed by cooling.
- **Where it shows up:** The chapter's snapshot has an ordinary desktop machine drawing tens of watts while idling and passing three hundred under load, and a single rack of servers using as much electricity as more than ten homes.
- **What it costs you:** More powerful hardware needs more energy to run and, at the same time, more energy to cool, so a single upgrade produces two increases on the same bill rather than one.
- **The move it argues for:** For an organization with hundreds or thousands of machines this stops being a curiosity, particularly since the chapter notes that power and heat keep rising as more processing is packed in without much gain in efficiency.

The need for agility · Deciding before you know

- **What the driver is:** Competing now means innovating constantly and working outside-in from the jobs customers are trying to get done, in rapid cycles rather than in long project plans.
- **Where it shows up:** The lean startup approach builds a minimum viable product, the smallest version real users can try, to find out quickly whether a solution has the effect it was meant to have.
- **What it costs you:** Experiments often need a different kind of infrastructure from the one already running, and committing to it means spending before any answer exists about acceptance or about how much demand there would be.
- **The move it argues for:** This is a strategy argument rather than a technical one, which makes it the cleanest link back to the previous module: how quickly an organization can learn is itself a competitive position.

AI infrastructure · A different class of machine

- **What the driver is:** The hardware and software used to support artificial intelligence work, which the chapter presents as having requirements of its own rather than being ordinary computing bought in larger quantities.
- **Where it shows up:** Training generally does not run on the traditional processors a business already owns; it runs on graphics processors or tensor processors, which the chapter says are not normally part of a traditional infrastructure.
- **What it costs you:** A unit price several times that of the dearest personal computer in the chapter's own comparison table, and more than twice that of a workstation, together with storage that has to be both larger and faster than usual and people who work with particular machine learning libraries and frameworks.
- **The move it argues for:** The chapter is blunt about the result for a newcomer — large costs plus development and integration difficulties — which is exactly the situation where renting rather than owning deserves a hearing.

The data keeps arriving, and nothing is thrown away

The first pressure is simply volume. Organizations now collect and analyze very large amounts of data for business intelligence, for machine learning and artificial intelligence, and for other purposes such as compliance. The amount that feels like enough keeps rising, because it is judged against what competitors are doing with their data rather than against any fixed requirement.

The chapter gives two ordinary examples of where the volume comes from, and neither one involves anything exotic.

Watching the website — an organization can analyze each visitor's actions on its own site in order to improve how that site performs, which means recording something about every click rather than only about every sale.

Reading everything else — organizations increasingly run machine learning and artificial intelligence algorithms over structured and unstructured material drawn from media reports, social media, customer support calls and other sources.

Now look at what capturing all of that actually costs, because it is three things and most people budget for one. It requires ever more storage space. It requires ever more powerful computing hardware, because the data have to be worked through and not merely held. And it requires **database management systems** for managing and analyzing what has been kept, which is software somebody has to license, configure and run.

The same story runs along the wires. Internet bandwidth grew enormously during the dot-com boom, which allowed organizations to deliver richer and far more bandwidth-hungry content, and the chapter names video services such as YouTube and the films streamed by Netflix as creating the need for more bandwidth again.

The chapter's name for this: a vicious circle. Enhanced capabilities enable new applications, and those new applications in turn require a certain level of capability in both the data and the communications infrastructure. Each round of improvement manufactures the demand for the next one.

If that shape looks familiar, it should. It is the same loop as the hardware-and-software treadmill in the previous section, drawn with storage and bandwidth in place of processors and operating systems. Recognizing the shape is worth more than either example, because it tells you that a purchase which ends the problem does not exist.

Demand does not arrive evenly

The second pressure is timing. Demands for computing resources fluctuate, which leaves an organization with either too few resources at some moments or, far more often, too many idle resources most of the time. According to estimates the chapter reports — and it attributes them only to estimates, with no source of its own — as many as seven organizational infrastructures in every ten run at only about a fifth of their capacity.

Read that as the upper end of a range rather than as a typical figure. The chapter gives a ceiling and not an average, so the honest sentence is that as many as seven in ten may be running that far below capacity, not that a typical one is.

That sounds like bad purchasing until you notice the constraint underneath it. Capacity is bought in whole units. A server is a whole server, a rack is a whole rack, and a room is a whole room, so the size you buy has to cover the busiest hour of the year rather than the average hour. Everything above the average is the price of being available on the day it matters.

The chapter's example of the busiest hour is seasonal. Companies engaged in or supporting business-to-consumer electronic commerce — it names Amazon and FedEx — face large spikes in demand in the pre-holiday season in December, and need increased capacity to handle them.

The one sentence to carry into the cloud section. The chapter puts the asymmetry plainly: while it is relatively easy to hire temporary staff to handle an increase in orders, it is typically not that easy to make quick changes to the information systems infrastructure based on changing needs.

People can be added for a season and released afterwards. A server room cannot. Every argument in the next objective is a way of making infrastructure behave more like the temporary staff and less like the building.

Growth is the same problem stretched over years instead of weeks, and it is where the diminishing space on the chapter's diagram actually appears: equipment accumulates while the room holding it does not. For an organization whose customer or user base is expanding, the facilities have to grow

along with the computing needs, and the chapter uses the ordinary case of a company that outgrew a friend's garage and now operates somewhere upward of two dozen major data centers.

Three separate costs make that growth painful, and only the first is obvious.

Money — renting additional facilities is expensive, and building them is far more expensive still, so the decision competes with everything else the organization might do with the same funds.

Time — significant time is needed for locating the right facilities, negotiating the contracts, and then setting up the hardware and software inside them, none of which can be compressed by wanting it more.

Commitment — long-term contracts limit the flexibility to scale the infrastructure back down in periods of lower demand, so a decision made in a good year is still being paid for in a bad one.

CHECK YOURSELF

Volume and timing

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. The chapter calls the relationship between capability and data a vicious circle. What does it mean by that?

- **A.** Data volumes grow at a steady annual rate, so capacity planning is a matter of extending a straight line far enough into the future and buying whatever the line says the fifth year will need — A straight line would make this an ordinary budgeting exercise, and the chapter's worry is precisely that it is not one. Growth here is driven by what new capability makes possible, which arrives in steps rather than at a predictable rate.
- **B.** Storage becomes cheaper faster than data volumes rise, so the circle closes itself and no management decision is required — Falling unit prices are real and they do not close the circle, because the chapter names three costs rather than one: space, more powerful hardware to process the data, and database management systems to manage and analyze it.
- **C.** Enhanced capabilities enable new applications, and those applications then demand more capability again, so each round of improvement creates the demand for the next — **correct.** Enhanced capabilities enable new applications, which in turn require a certain level of capability in both the data and the communications infrastructure — the same shape as the hardware-and-software loop in the previous section.

- **D.** Organizations collect data they have no use for, so the circle is broken simply by deciding to collect less of it — Some collection is certainly wasteful, but the chapter's examples are not: analyzing every visitor's actions to improve a website, and reading unstructured material from support calls and media, are uses that produce the volume on purpose.

2. A hypothetical county records office runs its own servers. Its technology lead reports that the machines are busy for a few hours a week and close to idle the rest of the time, and asks whether that means the equipment was bought badly. What does the chapter's treatment of demand fluctuation say?

- **A.** It is the ordinary consequence of owning capacity that has to cover the busiest hour, and the chapter estimates that as many as seven infrastructures in ten do the same — **correct.** Capacity is bought in whole units and has to cover the peak, so everything above the average hour is the price of being available on the day it matters; the chapter's own estimate describes exactly this pattern across organizations generally.
- **B.** It means the machines were oversized, and a correctly specified purchase would have produced steady high utilization through the whole week — This assumes a size exists that is right for both the quiet hours and the busy ones, and that is the assumption the chapter denies. Sizing for steady high utilization means being short of capacity every time demand rises above it.
- **C.** It means the office should move other work onto the machines whether or not that work is needed, so that the investment is seen to be used — Manufacturing work to justify a purchase raises utilization on a report and adds cost in reality. The chapter is describing a structural mismatch between fixed capacity and variable demand, not a presentation problem.
- **D.** Utilization is not a meaningful measure for owned equipment, because the money was spent once and does not change according to how busy the machines are on any given afternoon — The money was spent once, and it was spent on something that is now depreciating, drawing power and being cooled whether or not it is busy. That is why idle capacity is treated as a cost rather than as a neutral fact.

3. A hypothetical parcel depot handles its December surge by hiring temporary staff, but its systems slow to a crawl in the very same weeks. Which reading matches the chapter's account of why the two problems are not equally easy to solve?

- **A.** Software slows under load in a way that hardware does not, so the depot needs a different application rather than more capacity — Applications certainly differ in how well they cope with load, and that is a separate question from the one the chapter is asking here. Its point is about how quickly each kind of resource can be added, not about software quality.
- **B.** Temporary staff cost less than equipment, so the depot should hire more people and ask them to work around the slow systems — Cost is not what separates the two cases in the chapter's argument; speed of adjustment is. People can also work around a slow system only up to the moment the system stops responding at all.

- **C.** The depot should have signed a longer equipment contract, because a long-term commitment is what persuades a supplier to make additional capacity available quickly when the December weeks arrive — Long-term contracts do the opposite in the chapter's account: they limit the flexibility to scale the infrastructure back down in periods of lower demand, which is listed as one of the costs of growth rather than a remedy.
- **D.** It is relatively easy to hire temporary staff to handle an increase in orders, and typically not that easy to make quick changes to the infrastructure based on changing needs — **correct.** That asymmetry is the chapter's own sentence and the best single justification for the cloud material that follows, because every idea in the next objective is a way of making infrastructure adjust at the speed staffing already does.

One upgrade, two electricity bills

The third pressure is energy, and the reason it deserves a heading of its own is that the cost arrives twice. As computers process data they consume electricity. Various components, the processor and the power supply among them, turn part of that electricity into heat, which is why most computers contain several fans. So more powerful hardware needs more energy to do the work, and at the same time needs more energy to be cooled while doing it.

The chapter offers a snapshot of the quantities involved, and the snapshot is worth reading for its ratios rather than its digits.

One desktop machine — the chapter puts it between forty and a hundred and seventy watts while idling, rising to three hundred watts or more when it is working under full load.

One rack of servers — a single rack in a data center can easily consume fifteen to seventeen kilowatts, which the chapter compares to the electricity needed by more than ten homes.

A room full of racks — that comparison is the durable part, and it is why a data center is discussed as an electrical facility with computers in it rather than as a large office.

Read the rack figure as a floor, not a ceiling. Those kilowatt figures predate the current generation of machines built for artificial intelligence work, which the chapter itself describes only two subsections later. Racks assembled for that kind of processing draw materially more than the chapter's number. Keep the household comparison, which still teaches the right intuition, and treat the number as the chapter's snapshot rather than as a current specification.

Why is this a management question rather than a facilities one? Because of multiplication. You may never notice one personal computer on a household electricity bill, but an organization running hundreds or thousands of them is in a different position entirely. The chapter adds an unwelcome observation: power consumption and heat emissions continue to rise as manufacturers pack more processing power into servers, often without providing much improvement in energy efficiency. Power and cooling are, in its words, significant cost factors.

Moving fast, and why that is an infrastructure problem

The fourth pressure is the only one that comes from strategy rather than from equipment, and it is the reason this subsection sits in an infrastructure chapter at all. Modern organizations have to innovate constantly to stay abreast of the competition. Rather than building the business around the competencies they happen to have, they are pushed to take an **outside-in approach**: start from the jobs current and future customers are trying to get done, then work back to a solution.

Being ahead of competitors requires speed, so traditional ways of developing a product or service will not hold up over time. The chapter therefore points at an agile mindset borrowed from the startup world, and names one approach in particular.

Lean startup methodology — an approach built on rapid cycles of devising new solutions and developing minimum viable products, so that whether a solution has the desired effect can be tested quickly rather than argued about.

Minimum viable product — the smallest version of a solution that real users can actually try, built to produce an answer cheaply rather than to be the finished thing.

The consequence for infrastructure — such experiments often need a different kind of infrastructure from the one the organization already runs, and that infrastructure has to be decided on before any answer exists.

The chapter states the conclusion directly: it makes little sense to invest time and money into infrastructure changes before even knowing whether the experiment will be a success. It then gives two examples that are deliberately unglamorous.

The mobile application — there is little sense in purchasing a quantity of hardware and installing web servers to support an app without knowing whether the market will accept it, or how much demand there would be to meet.

The analytics project — there is equally little sense in purchasing new workstations to run a piece of large-scale data analysis before anyone knows whether the analysis will produce the results it was expected to produce.

This is a strategy argument wearing technical clothes, which is what makes it the cleanest link back to the previous module. If competitive position now depends on how quickly an organization can learn from a real attempt, then anything that slows an attempt down is a competitive problem, and a procurement cycle is exactly such a thing.

DECIDE

One year of capacity decisions

Work the four decisions in order, and after each one read every outcome rather than only the one you chose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

A hypothetical practice situation. You are the operations manager of an invented regional food bank network that runs its scheduling, inventory and volunteer systems on machines in a room at its own warehouse. Nothing here describes a real organization, and each situation states the conditions you need in order to reason about it.

Decision 1. Deliveries and volunteer sign-ups run at a steady level for ten months of the year. In the fortnight around the winter holidays, traffic to the sign-up system runs roughly eight times higher, and last year the system was unusable for two of those days. Servers are bought whole, and the smallest useful step up is a machine that would carry the December load on its own. You are keeping the equipment in your own building. What do you buy?

- Buy the machine sized for the December peak, and accept that it will be lightly used for most of the year. — **the stronger call.** Right, and notice how uncomfortable the right answer is. Capacity is bought in whole units, so owning means choosing between idle resources most of the time and too few resources at the peak. What you are really buying is the two worst days, which is exactly the pattern behind the chapter's estimate that as many as seven infrastructures in ten run at only about a fifth of their capacity.
- Buy for the average level of demand and ask volunteers to try again later during the December fortnight. — This trades a known cost for an unknown one. The days you would be failing on are the days the organization exists for, and the failure is invisible in the budget while the idle machine is visible in it. Sizing for the average means being short every time demand rises above the average, which is most of December.
- Commission a more careful forecast so that a single capacity figure can be found that fits the whole year. — No such figure exists, and looking for it misreads the driver. This is fluctuating demand rather than a forecasting error: the shape of the year has a spike in it, and a forecast that describes the spike accurately still leaves you choosing between covering it and not covering it.

Decision 2. Your analytics volunteer wants to keep a record of every interaction with the sign-up system, not just completed sign-ups, so that the network can work out where people give up. Separately, some of the network's records must be retained for several years to satisfy requirements that apply to organizations of this kind in its jurisdiction. What do you put in the plan?

- Add disk capacity, since storage is the cheapest part of any system and the rest of the plan is unaffected. — Storage space is only the first of the three costs the chapter names. Keeping the data is not the same as being able to ask it anything, and a pile of records nobody can query will produce neither the analysis nor the compliance answer you kept it for.
- Plan for storage space, for enough computing power to work through the data, and for a database management system to manage and analyze what is kept. — **the stronger call.** Right. That is the chapter's own list, and the second and third items are the ones people leave out. Processing capacity is what turns a stored record into an answer, and the database software that makes the records searchable is a licence, a configuration and a running cost of its own.
- Record only completed sign-ups, and delete anything older than the current month to keep the volume flat. — That deletes the abandoned sign-ups, which are the entire subject of the analysis, and it also collides with the retention requirement stated in the situation. Volume can be managed, but not by discarding the two categories of record you were specifically told to keep.

Decision 3. The plan doubles the processing power in the warehouse room. Your finance lead has already added the extra electricity that the new machines are rated to draw. The facilities manager reads the plan and says the energy line is still short. Who is right, and why?

- Finance is right; the rated draw of the machines is the whole of the additional energy cost. — This is the mistake the chapter is written to prevent. The rated draw covers doing the work and not removing the heat that doing the work produces, and in a room rather than on a desk that heat has to be taken out by equipment which draws power of its own.
- The facilities manager is right, because more powerful hardware needs more energy to run and, at the same time, more energy to cool. — **the stronger call.** Right. One upgrade produces two increases on the same bill. The chapter spells out the mechanism: components such as the processor and the power supply generate heat, cooling has to remove it, and the cooling is itself a consumer of electricity. It also warns that packing more processing into servers often arrives without much improvement in efficiency.
- The facilities manager is right, but only because electricity prices rise every year, so any energy figure written in advance will be too low. — Price inflation is a real budgeting risk and it is not the chapter's argument here. The second cost would appear even if the tariff never moved, because it comes from the physics of the room rather than from the market for electricity.

Decision 4. A board member proposes a volunteer-matching app that would pair skills to shifts automatically. Nobody knows whether volunteers would use it. A consultant recommends buying servers now, sized for the demand the app would generate if it succeeded, so that success is not wasted. What is the chapter's position?

- Buy the servers now, because an app that succeeds and then fails under load has lost its chance and equipment is cheap next to that risk. — This is the reasoning the agility material rejects. The chapter says plainly that it makes little sense to invest time and money in infrastructure changes before even knowing whether the experiment will be a success, and its own example is buying hardware and web servers for an app whose acceptance and demand are both unknown.
- Build the smallest version real volunteers can actually try, run it on capacity you have not committed to, and decide about equipment once there is an answer. — **the stronger call.** Right. That is a minimum viable product inside the lean startup approach: rapid cycles that produce an answer cheaply. The infrastructure argument follows from it — the experiment may need a different kind of infrastructure entirely, and committing to that before the answer exists spends money on a question rather than on a result.
- Drop the idea, since an organization that cannot fund the success case should not begin the experiment. — That treats the whole cost of success as the entry price of asking the question, which is the same error as the first option seen from the other side. The point of a small testable version is that the question can be answered for far less than the answer is worth.

Artificial intelligence asks for a different machine

The fifth pressure is the newest one on the chapter's diagram. **AI infrastructure** is the hardware and software used to support artificial intelligence work, and the chapter's claim is that it has special requirements — that it is not ordinary computing bought in larger quantities.

The reason is the shape of the work. Training a model means processing very large amounts of data efficiently, which takes an enormous quantity of arithmetic. Because of the nature of that arithmetic, training generally does not run on the traditional processors a business already owns; it runs on graphics processors or tensor processors, which the chapter says are not normally part of a traditional infrastructure. The requirements in processing, storage and analysis differ vastly from ordinary business workloads.

Four consequences follow for whoever has to write the budget, and each one is a place where an estimate built from last year's prices will be wrong.

Unit price — the chapter prices a single widely used training processor in the region of twenty-five thousand US dollars and its successor a little above that. Set that beside the comparison table this module worked through under the previous objective, where a personal computer reaches five thousand dollars and a workstation ten thousand: one such processor costs several times the dearest personal computer and more than twice a workstation, and a training rig needs many of them.

Quantity and time — researchers estimated that training an early version of a well-known generative model might have taken more than a month running on a thousand such processors at once.

Storage — the storage requirements, in capacity and also in speed, often exceed those of traditional systems, so the usual disks are wrong on two counts rather than one.

People — the data scientists doing the work draw on particular machine learning libraries and frameworks, which is a hiring and training cost sitting behind the hardware cost.

Handle the very large figures with care. Prices for these processors move constantly and street prices diverge sharply from list prices, so treat the chapter's numbers as a snapshot that fixes the order of magnitude and nothing finer.

The chapter also reports that the company it calls Facebook, whose corporate parent renamed itself Meta in 2021, would build data centers with computing power equivalent to hundreds of thousands of these processors. That was an announced plan rather than a finished building, and this module says so rather than reporting it as an accomplished fact.

What survives all of that qualification is the shape of the argument, and the shape is what you are being taught: this work needs different silicon, at a unit price far above a business machine, in quantities that only make sense to own if you intend to train continuously.

The chapter's bottom line for a manager is short. For an organization new to this kind of work, an AI infrastructure means large costs together with real development and integration difficulties, which is precisely the situation in which renting rather than buying deserves a hearing.

CHECK YOURSELF

Energy, agility, and a different machine

Three questions on the last three pressures; every option carries its own explanation, so read the ones you rejected too.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. Why does the chapter say that more powerful hardware raises the energy bill twice rather than once?

- **A.** Because electricity is charged at a higher rate for commercial than for domestic use, so the same consumption costs an organization more than it would cost a household — Tariffs do differ between kinds of customer, and that is not the doubling the chapter describes. Its second cost appears on the same tariff as the first, because it comes from what happens inside the room rather than from how the electricity is priced.
- **B.** Because the components draw more electricity while they work and, as they work, convert part of it into heat that has to be removed by cooling equipment drawing electricity of its own — **correct.** Components such as the processor and the power supply generate heat, most computers run fans to control temperature, and in a data center the cooling is a substantial consumer in its own right — so one upgrade produces two increases.
- **C.** Because a more powerful machine runs continuously at its maximum rating rather than idling between tasks the way an older machine did — Utilization does affect consumption, which is why the chapter quotes an idle figure and a loaded figure for a desktop machine. But a machine that idles still has to be cooled when it is busy, so this does not explain a second, separate cost.
- **D.** Because heat shortens the life of components, so the second cost is replacing hardware early rather than paying for more energy — Early replacement is a genuine consequence of running hardware hot, and it is a hardware cost rather than an energy one. The chapter's doubling is about energy in both halves: energy to compute and energy to cool.

2. A hypothetical arts council is deciding whether to build an online booking service. Its adviser recommends starting with the smallest version real users can try, on capacity the council has not committed to buying. Which of the chapter's ideas is that, and what is the infrastructure argument behind it?

- **A.** Planned obsolescence, since the first version is deliberately designed to be discarded, which is how a product life span is managed — Planned obsolescence is the previous section's term for a product designed by its manufacturer to last only a certain life span. A minimum viable product is built by you, to answer a question, and it is kept if the answer is good.
- **B.** A pilot study, which the chapter recommends only for projects too large to fund in a single step, since ordinary projects should be fully specified before any equipment is chosen — The chapter does not treat this as a special case for large projects. Its argument is that speed of learning is now a competitive requirement, which makes a full specification before any real test the slower and riskier path.
- **C.** An argument about cost alone, since the smallest version is the cheapest one available and the chapter treats cost as the only criterion for how to begin — Cost is part of it and not the whole of it. The reason for starting small is that the experiment produces information about acceptance and demand, and that information is what the later spending decision actually needs.

- **D.** The lean startup approach and a minimum viable product: rapid cycles produce an answer cheaply, before any infrastructure has been committed to a service nobody may want — **correct.** The lean startup approach runs rapid cycles of devising solutions and building minimum viable products to test whether they have the desired effect, and the infrastructure consequence is the chapter's own: do not buy for a demand nobody has measured.

3. A hypothetical veterinary network wants to train a model on its own case records. Which description matches the chapter's account of what an AI infrastructure requires?

- **A.** Processors of a kind not normally part of a traditional infrastructure, storage that differs in both capacity and speed, and people who work with particular libraries and frameworks — **correct.** The chapter names all three: graphics or tensor processors rather than traditional ones, storage requirements that exceed traditional systems in capacity and speed, and data scientists drawing on specific machine learning libraries and frameworks.
- **B.** The same processors as ordinary business computing, simply a great many more of them, because the difference between training a model and running a payroll lies in the quantity of the work rather than in its kind — This is the assumption the chapter sets out to correct. It says training typically does not take place on traditional processors at all, and that the requirements for processing, storage and analysis differ vastly from ordinary workloads.
- **C.** Mainly additional network bandwidth, because training work is limited chiefly by how quickly data can be moved between an organization's sites — Moving data matters, and it is not what the chapter identifies as the distinguishing requirement. The distinguishing requirement is the arithmetic itself, which is why a different class of processor appears rather than a faster link.
- **D.** No special equipment at all once the work is rented, because renting removes the technical differences along with the ownership — Renting changes who owns and who maintains the equipment; it does not change what the work needs. A rented AI environment is still built on the specialized processors and faster storage, which is a large part of why it is priced the way it is.

Five pressures, one shape

Put them side by side and the same structure appears in every one, which is the reason to learn them together rather than as a list to be memorized.

- 1 Each one is continuous** — data keeps arriving, demand keeps moving, energy keeps being consumed, competitors keep experimenting, and none of that stops when a purchase order is signed.
- 2 Each one forces a commitment early** — capacity, facilities, contracts and specialized hardware all have to be chosen before the information that would settle the choice is available.

- 3 Each one punishes both directions** — too much capacity is money sitting idle, too little is a service that fails, and there is no single figure that is safe in both a quiet March and a busy December.

Before moving on, test whether you can spot which pressure is at work from the symptom alone. That is the skill the rest of this module keeps asking for, because a manager is almost never told which driver they are looking at.

SORT

Which pressure is at work here?

Every situation below is an invented practice case rather than a real organization; drop each one into the driver the chapter's definitions call for, then read why it belongs there.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Expanding storage — the volume of data being captured and kept is what is growing, and it drags hardware and database software along behind it

- A hypothetical museum starts recording every interaction with its online catalogue in order to improve the catalogue, and finds it now needs a more capable database system to ask questions of what it has kept. — This is the chapter's own website example, and it shows the driver's full cost. Capturing more produces three needs rather than one: space to keep it, processing power to work through it, and a database management system to manage and analyze it.
- A hypothetical broadcaster replaces its audio archive with high-resolution video and discovers that the links between its editing rooms are now the slowest part of the working day. — The chapter treats richer content and the bandwidth it demands as the same driver seen along the wires: capability enabled bandwidth-hungry content, and the content then demanded more bandwidth again.

Fluctuating demand — the need is uneven over time, so owned capacity ends up either idle or short

- A hypothetical examinations board sees traffic to its results system rise sharply on two mornings a year and sit close to zero for the rest of it. — Owned capacity has to cover the busiest morning, which leaves it idle the rest of the year. This is the seasonal spike the chapter describes for December shopping, moved to a different calendar but structurally identical.
- A hypothetical charity signs a three-year agreement for machines and space to support a fundraising campaign, and cannot reduce the commitment when the campaign finishes. — The chapter lists this among the costs of growth: long-term contracts limit the flexibility to scale the infrastructure back down in times of lower demand, so a decision made at the peak is still being paid for afterwards.

Expanding consumption — the cost being described is electricity, whether for running the machines or for cooling them

- A hypothetical laboratory doubles the number of processors in its server room and finds that its air conditioning can no longer hold the room within its temperature range. — One upgrade, two increases. The processors draw more electricity to do the work and turn part of it into heat, and the cooling that removes the heat is a separate consumer that also has to grow.
- A hypothetical accountancy firm running several hundred desktop machines notices that the line growing fastest in its accounts this year is the electricity bill rather than the hardware budget. — The chapter makes exactly this point about scale: one personal computer is invisible on a household bill, but for an organization with hundreds or thousands of them, rising energy cost becomes a major issue in its own right.

The need for agility — a commitment has to be made before anybody knows whether the idea works or how much demand it would attract

- A hypothetical transit authority wants to try a trip-planning tool with a few hundred riders before deciding whether to build it properly. — This is a minimum viable product inside the lean startup approach: build the smallest version real users can try, in order to learn quickly whether it has the desired effect before committing to an infrastructure for it.
- A hypothetical publisher is asked to approve the purchase of analytics workstations for a study whose usefulness nobody has yet been able to demonstrate. — The chapter's second worked example is this one almost exactly: there is little sense in buying new workstations to run analytics before anyone knows whether the analytics will produce the expected results.

AI infrastructure — the requirement is a different class of processor, storage or skill from ordinary business computing

- A hypothetical insurer discovers that training a model on its own claim records will not run usefully on the ordinary servers it already owns. — Training generally does not take place on traditional processors. It runs on graphics or tensor processors that the chapter says are not normally part of a traditional infrastructure, which is why the existing fleet cannot simply be reused.
- A hypothetical geology institute is told that the storage its modelling work needs must be not only larger than what it runs today but also considerably faster. — The chapter is specific that storage requirements for this work exceed traditional systems in capacity and in speed, so ordinary disks are the wrong answer on two counts rather than one.

Where the chapter goes next

The chapter closes this objective with its own transition, and it is worth reading as a question rather than a summary. Given all of these issues, organizations have been looking for ways to manage their infrastructure that enhance flexibility and agility while reducing costs. The next objective is the answer the industry settled on.

Every driver in this section is a mismatch between how fast a business need changes and how slowly an owned infrastructure can change with it.

That is why the next objective is not about better machines. It is about somebody else owning them.

Renting instead of owning

The two sections before this one were a list of the costs of ownership: machines that age, software that stops being supported, data that piles up, demand that spikes before the holidays, and a power bill that arrives whether or not anybody used the equipment. This section is the answer the industry settled on. Stop owning most of it, and buy the service instead.

Nobody actually wants a mail server

The chapter opens this objective from the manager's side rather than the technology's. Running an infrastructure is hard because hardware and software keep evolving, because the demand for storage and for network bandwidth keeps rising, and because the cost of the energy underneath all of it rises too. An organization also needs dedicated staff to support the equipment, and that is a further cost on top of the equipment itself.

Then comes the sentence that stings. Managing an information systems infrastructure is often **not among an organization's core competencies**, so other organizations may simply be better at doing it. A hospital is good at medicine and a bank is good at credit. Neither one set out to become good at cooling a room full of machines.

There is a second problem in any organization old enough to have bought equipment twice. The infrastructure has grown over the years, one purchase at a time, so it ends up fragmented and difficult to consolidate. Meanwhile efficiency, effectiveness and agility are what the firm actually competes on, and those need a flexible, scalable infrastructure rather than an accumulation of past decisions.

Out of that the chapter names the shift this whole objective rests on. Over the past decades there has been **a shift away from thinking about developing and maintaining the infrastructure toward thinking about what services the infrastructure should deliver**. Its own illustration is the one everybody recognises: people and organizations want to *use email*. Nobody ever wanted to purchase an email server and then deal with its administration, its maintenance, its storage and its energy consumption.

Four consequences follow from that shift, and the chapter states each of them plainly.

Buy or rent rather than build — organizations increasingly obtain their applications from other parties and assume that those applications will work, instead of constructing and maintaining everything themselves.

Except where the system is the advantage — the exception the chapter names is the highly specialized system that helps a firm gain or sustain competitive advantage, and it points to Amazon and Dell as companies that build such systems for themselves.

The infrastructure still decides what is possible — even when rented, it determines how quickly new systems can be implemented and how well they will run, so handing over the lower levels frees attention rather than ending responsibility.

Customers notice before the finance department does — any lack of robustness or integration is immediately noticed by customers and other stakeholders, and the chapter names the price as lost business, lost trust and lost goodwill.

THIS IS MODULE 2'S ARGUMENT, APPLIED TO PLUMBING

Rent the commodity, build the differentiator. A system any competitor could order this afternoon is not what makes a company different, so owning it wins nothing. A system that encodes what this firm knows and rivals do not is worth building and worth keeping. Most of what follows in this objective is that one distinction turned into a purchasing policy.

What cloud computing actually is

Two technical advances made the rest possible. Internet bandwidth grew large enough to carry real work across it, and virtualization matured enough to let one physical machine host many separate computing environments. The word itself is not technical at all: the chapter says flatly that **the cloud is a metaphor for the internet**, which is why every diagram of it is drawn as a cloud with arrows going in and coming out.

The definition worth learning word for word. It is not the chapter's own wording. The chapter quotes the definition published by the United States National Institute of Standards and Technology, the federal standards body whose formulation the rest of the industry settled on.

It is worth learning as that body wrote it. Cloud computing is "a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."

Read it as four promises. You can reach it from anywhere. You can have it when you ask. The resources are shared rather than yours. And you can start and stop without a negotiation.

Notice what the definition says is being rented: networks, servers, storage, applications and services. That is the same component list this module walked through under objective 3.2, which is the honest way to understand the cloud. It is not a new kind of computing. It is the same five components, in somebody else's building, billed by use.

Three pieces of vocabulary carry the money side of that definition, and a manager needs all three.

Utility computing — the model cloud computing uses: renting processing, data storage or networking from an outside provider on an as-needed basis and paying only for what is actually used, the way a household pays for electricity.

Capital expenditure — money spent up front to own equipment, committed once, years before anybody can know whether the amount of capacity purchased turned out to be the right amount.

Operational expenditure — money spent continuously to use a service, rising and falling with what the business actually did that month, and approved as a running cost rather than as a purchase.

The chapter's summary of the financial effect is one clause long: cloud computing helps transform infrastructure costs **from a capital expenditure to an operational expenditure**. That sounds like accounting trivia and is not. It changes which budget the money leaves, who has to approve it, and how far in advance the decision has to be made. It also converts a fixed cost, paid whether or not the capacity was used, into a variable one that follows the work.

CHECK YOURSELF

The shift, the definition, and the money

Four options, one best answer; every organization described here is hypothetical, and only companies the chapter itself names are real, so read every explanation including the ones you did not pick.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. Which statement best captures the shift the chapter says has taken place over the past decades?

- **A.** Organizations have moved from buying applications to writing their own, because writing software has become cheaper than purchasing it — The chapter describes the opposite direction of travel: organizations increasingly buy or rent applications rather than building them, and assume that those applications will work.
- **B.** Organizations have moved from thinking about developing and maintaining the infrastructure toward thinking about what services that infrastructure should deliver — **correct.** That is the chapter's own sentence, and its illustration is that people want to use email rather than to purchase an email server and deal with its administration, maintenance, storage and energy consumption.

- **C.** Organizations have concluded that the infrastructure no longer affects how quickly new systems can be built, so it can safely be left to somebody else — The chapter says the reverse in the same passage: the infrastructure determines how quickly new systems can be implemented and how well they will function, whoever happens to own the equipment.
- **D.** Organizations have moved every system to outside providers, including the specialized ones that make them different from their rivals — The chapter names an explicit exception. Highly specialized systems that help a firm gain or sustain competitive advantage are the ones organizations still build for themselves, and it names Amazon and Dell as examples.

2. A hypothetical publisher stops replacing its servers every few years and instead rents processing and storage from an outside provider, paying only for what it uses. Which description of the change matches the chapter?

- **A.** It has adopted a utility computing model, and its infrastructure costs move from a capital expenditure to an operational expenditure — **correct.** Utility computing is renting processing, storage or networking from an external provider on an as-needed basis and paying only for what is actually used, and the chapter says cloud computing helps transform infrastructure costs from a capital expenditure into an operational expenditure.
- **B.** It has adopted resource pooling, which is the chapter's term for the arrangement in which a customer pays according to usage — Resource pooling is a real characteristic, but it describes the provider assigning distributed resources dynamically to many customers. Paying according to usage is measured service, and the financial model behind it is utility computing.
- **C.** It has eliminated its infrastructure costs, because nothing is being purchased and therefore nothing appears in the budget — The costs have changed shape rather than vanished. They now arrive monthly instead of every few years, and the chapter's point about metering is precisely that a running cost still has to be managed.
- **D.** It has converted a variable cost into a fixed one, because a monthly bill is more predictable than an occasional large purchase — This reverses the chapter's claim. Buying equipment is the fixed cost, incurred whether or not the capacity was used; renting produces variable costs that rise and fall with the work that actually ran.

3. A hypothetical logistics firm has one system no competitor can match: a routing engine built on years of its own delivery data. Following the chapter's reasoning about what to rent and what to build, what should it do?

- **A.** Rent the routing engine from a provider, since the chapter recommends buying or renting applications rather than building them — The chapter's buy-or-rent advice comes with an exception attached in the same sentence, and this system is the exception: it is the specialized capability that helps the firm gain or sustain competitive advantage.
- **B.** Build everything itself, because a firm with one distinctive system should keep control of its whole infrastructure — Nothing in the argument requires that. Turning over the lower levels of the infrastructure is described as what allows a business to focus on the applications that make it different, so owning the routing engine is compatible with renting the machines under it.

- **C.** Keep building the routing engine while renting the ordinary infrastructure beneath it, because the exception the chapter names is the specialized system that gains or sustains advantage — **correct**. Organizations buy or rent rather than build, except for the highly specialized systems that help gain or sustain competitive advantage, and renting the commodity underneath is what frees the attention to keep improving the part that matters.
- **D.** Sell the routing engine to competitors, because the chapter's opening case is a company that turned the infrastructure it built for itself into a revenue stream, and the same move is open to this firm — The chapter reports that some firms monetized infrastructure they had already built, which is a description of what happened rather than advice. Selling a rival the capability that distinguishes you is the opposite of the argument being made.

Two companies the chapter follows

The first one built the market. Having assembled an immense infrastructure — in information technology and in logistics both — to support its own online store, **Amazon** decided to use those resources to generate additional revenue. Individuals and organizations can now rent storage space and computing time from it on an as-needed basis. That is the same move the chapter used to open the chapter: a company selling the capability it had to build anyway.

The second is the chapter's fullest cloud case. As **Airbnb** grew popular with travelers around the world, it found itself limited by the constraints imposed by its original service provider. Moving to a rented infrastructure let it obtain two hundred servers without negotiating service contracts and without committing to a minimum level of usage.

The chapter is careful to say what the alternative would have demanded. Scaling that flexibly would have been close to impossible in a data center of its own, because of both the time and the money needed to acquire that many machines — and, as the chapter asks, at the time who knew whether the business would actually take off?

That question is where the argument stops being about technology. With its own equipment the company would have had to add capacity **in chunks**, because you cannot buy a third of a server. Capacity therefore moves in steps while demand moves as a curve, the two lines cross each other constantly, and two gaps open that are never both closed at once.

Capacity above demand — equipment that was purchased, powered, cooled and staffed while doing nothing, which is the underused capacity the chapter estimates as many as seven infrastructures in ten are carrying.

Demand above capacity — customers arriving at a system that cannot serve all of them, which costs an order today and may cost the customer for good.

Rented capacity closes both gaps by following the demand line instead of stepping past it. The chapter draws that as two panels side by side, and it is worth walking through one panel at a time before

reading anything else about the cloud.

INTERACTIVE DIAGRAM

Capacity against demand, two ways

Step through the owned picture, then the rented one, then the two gaps between them; the gaps are where the money is, so read that panel last and slowly.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

In-house infrastructure — The left panel of the chapter's comparison: a stepped capacity line drawn over a demand line that keeps crossing it

Forecast (Guess next year's busiest hour) → Buy in chunks (Capacity arrives as whole machines) → Wait (Weeks to deliver, days to configure) → Live with it (Until the next purchase is approved)

- Capacity can only be added in **chunks**, because a server is not divisible. Whatever the forecast says the business needs, the purchase is rounded up to whole machines, and the rounding is paid for in full.
- Between purchases the capacity line is flat while demand carries on moving, so the relationship between what the organization has and what it needs is wrong in one direction or the other almost all of the time.
- The chapter notes that servers and other elements of an infrastructure typically take several weeks to be delivered and days or weeks to configure, because system software, databases and applications all have to be installed and set up.
- Scaling back down is the part people forget. Owned equipment cannot be handed back, and where facilities are rented instead the long-term contracts limit the flexibility to shrink when demand falls.

Cloud infrastructure — The right panel: one capacity line that closely follows the demand line instead of stepping past it

Ask (A control panel, not a negotiation) → Receive (Minutes rather than weeks) → Track demand (Up and down, often automatically) → Release (Stop paying when the work stops)

- The chapter's case is the travel platform that obtained two hundred servers without needing to negotiate service contracts or commit to minimum usage, which is what the first two boxes look like in practice.
- Because resources can be scaled up or down almost instantaneously and often automatically, the capacity line can be kept just above what is needed to keep users satisfied rather than far above it or below it.
- There is no purchase to approve before an experiment, which is why the same chapter argues it makes little sense to invest in infrastructure changes before knowing whether the experiment will succeed at all.

- The release step is the one with no counterpart in the owned picture. Capacity that is given back stops being billed, so a quiet month costs less than a busy one instead of costing the same.

The two gaps — The two shaded areas in the left panel, which between them carry the entire economic argument for renting

Gap one (Capacity above demand) → The bill for it (Purchase, power, cooling, staff, all idle) → Gap two (Demand above capacity) → The bill for it (Orders not taken, customers not served)

- The first gap is visible in the accounts and invisible to customers. Equipment that was bought, powered, cooled, insured and staffed while doing nothing is the underused capacity the chapter estimates as many as seven infrastructures in ten are carrying.
- The second gap is the reverse: invisible in the accounts and extremely visible to customers, because nothing on an invoice records the order that was never placed by somebody who gave up waiting.
- Both gaps come from the same cause, which is why closing one by buying more equipment simply widens the other. Capacity that moves in steps cannot follow demand that moves as a curve.
- This is the sentence to carry out of the section: with a traditional in-house infrastructure the organization ends up either holding unused resources or being unable to satisfy its users, and adding capacity in chunks is what forces that choice.

The same reasoning is why the chapter says cloud computing has become a popular way of building an infrastructure for artificial intelligence. Those workloads fluctuate hard — heavy resource needs while a model is being trained, much lighter needs while it is merely being used — and the specialized hardware they want is expensive to own and idle between training runs. Renting a spike is cheaper than owning one.

The five characteristics

The chapter says the cloud model has several unique and essential characteristics that distinguish it from an in-house infrastructure. Five are named, they are quoted from the same national standards definition as the wording above, and they are worth learning as a set rather than one at a time.

On-demand self-service — a customer obtains resources in a buffet-style fashion as they are needed, without lengthy negotiations and in many cases without any human interaction with the provider.

Rapid elasticity — resources are scaled up or down almost instantaneously and often automatically, in step with what the organization's users are actually doing.

Broad network access — the services are reached over the internet, so they are available from almost anywhere and from almost any web-enabled device.

Resource pooling — the provider manages many distributed resources and assigns them dynamically to many customers, so a customer rents capacity without knowing or controlling which machine supplied it.

Measured service — usage is monitored and the customer pays for what it used, which turns the fixed costs of an infrastructure into variable costs that are easy to track and monitor.

Every one of those reads as an advantage, and every one of them has a failure mode that the chapter also describes elsewhere. Open the cards below and read all four panels of each characteristic, especially the last one.

EXPLORE

The five characteristics, benefit and failure mode together

Open each card and read all four panels; the fourth panel is what the same characteristic looks like when nobody is managing it, which is the half most descriptions leave out.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

On-demand self-service · Getting resources without asking a person

- **The chapter's definition:** Users can access cloud resources in a buffet-style fashion on an as-needed basis without lengthy negotiations with the service provider, and in many cases with no human interaction with the provider at all.
- **What it looks like in practice:** The chapter's example is a customer who needs only a payment card for billing and can then set up server instances or expand storage through a web-based control panel, which is what makes an experiment possible this afternoon.
- **Why owning could not do this:** Obtaining equipment used to begin with a supplier, a quotation and a contract, so the earliest an idea could be tested was measured in weeks. The negotiation, not the technology, was usually the slow part.
- **What it looks like when nobody is watching:** The same door is open to everybody who holds an account. Resources can be created that nobody approved, forgotten by the person who created them, and billed every month afterwards to a company that no longer knows what they are for.

Rapid elasticity · Capacity that follows the work

- **The chapter's definition:** In a cloud environment, computing resources can be scaled up or down almost instantaneously and often automatically, based on what users need at that moment.
- **What it looks like in practice:** The chapter's case is the pre-holiday surge: there is no need to purchase expensive equipment in advance for a spike that may never materialize, and if it does materialize the resources can be obtained instantly in almost any quantity.
- **Why owning could not do this:** Servers typically take several weeks to be delivered and days or weeks to be configured, because system software, databases and application software all have to be installed and set up before anything runs on them.
- **What it looks like when nobody is watching:** Capacity that grows automatically will also grow automatically for work that should not be happening. The mechanism does not ask whether the demand is legitimate, only whether there is more of it than there was a minute ago.

Broad network access · Reachable from wherever the work is

- **The chapter's definition:** Because cloud services are accessed over the internet, they are accessible from almost anywhere and from almost any web-enabled device.
- **What it looks like in practice:** The chapter describes real-time management of business processes: an application can be reached whenever it is needed and from any location, so a knowledge worker can respond to something urgent without first travelling back to an office.
- **Why owning could not do this:** An application on a machine inside one building is reachable inside that building. Everything else — the private tunnel, the remote desktop, the laptop carried home — was an arrangement bolted on to work around that limit.
- **What it looks like when nobody is watching:** A service reachable from anywhere is reachable by anyone holding a credential for it. The question of who may come in moves from the door of a building, which the organization controls, to an account, which it can only manage.

Resource pooling · Somebody else's machines, assigned as needed

- **The chapter's definition:** Rather than renting space or time on one specific physical machine, providers manage multiple distributed resources that are dynamically assigned to many customers according to their needs, so the customer rents a resource with no knowledge or control of how it is provided or where it is located.
- **What it looks like in practice:** The chapter notes that providers often do allow a geographic area to be specified: a company may want its resources near its customers to reduce response latency, or its storage held in a particular region to satisfy data protection duties.
- **Why owning could not do this:** An organization sizing its own equipment has to size it for its own busiest hour, and every other organization does the same, which leaves a great deal of machinery standing mostly idle across all of them at once.
- **What it looks like when nobody is watching:** You cannot say exactly where the data physically sits, which is uncomfortable when a duty about where records may be held applies to your industry or your region. The default is convenient; the exception has to be asked for.

Measured service · Paying for what actually ran

- **The chapter's definition:** Providers monitor usage and customers pay only for what they use, with different resources metered differently — server instances charged by the hour according to their computing power, memory and operating system, or storage charged by the volume held and moved.
- **What it looks like in practice:** For the customer, the fixed costs associated with an infrastructure become variable costs that are easy to track and monitor, which is the same transformation the chapter describes as moving from a capital to an operational expenditure.
- **Why owning could not do this:** An owned machine costs the same on its busiest day and its quietest one. There is no meter to read, so there is also no way to discover from the bill that half the capacity has not been touched since it was installed.
- **What it looks like when nobody is watching:** The meter runs whether or not anybody is reading it. An invoice is the first place an unnoticed mistake becomes visible, and by the time it is visible the resources have already been consumed and charged for.

Now read the same list a second way and it stops being a brochure. Each characteristic is an answer to a specific problem raised in the two sections before this one, which is exactly why the chapter puts the list here rather than at the start of the chapter.

Pairing them off is not decoration. Naming the problem a characteristic solves is what stops an organization from paying for a capability it has no use for, and it is the fastest way to tell a real reason for moving from a fashionable one.

MATCH

Each characteristic and the problem it answers

Pair each of the five characteristics with the infrastructure problem from the previous objective that it is answering; the explanation names where that problem was raised.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

On-demand self-service

Equipment that took weeks to arrive, with locating facilities and negotiating contracts happening before that, so an idea sat a quarter away from being tested The chapter lists significant time for locating the right facilities, contract negotiations and setting up hardware and software among the reasons growth hurts. Self-service removes the negotiation entirely, which is why an experiment can start the same day.

Rapid elasticity

Demand that spikes before the holidays and falls away afterwards, against capacity that can only be purchased in fixed chunks This is the demand-fluctuation problem in its clearest form: firms supporting consumer electronic commerce face large spikes in the pre-holiday season, and equipment bought for the spike is idle for the rest of the year.

Broad network access

Work that no longer happens at one desk, because people travel, work away from the office, and bring their own devices into the organization. Mobility and the consumerization of information technology were raised as drivers of infrastructure change. A service reachable from any web-enabled device answers them without the organization owning the device in question.

Resource pooling

Machines standing mostly idle, because every organization sized its own equipment for its own busiest hour and then waited for that hour. The chapter reports an estimate that as many as seven infrastructures in ten run at only about a fifth of their capacity. Pooling is what allows one provider's machines to be busy for somebody at almost any moment, which is where the saving comes from.

Measured service

A large sum committed up front for capacity, plus the power and the cooling to run it, incurred whether or not the work ever arrived. Rising energy costs and the capital cost of equipment were both named as pressures on the infrastructure. Metering is what converts those fixed costs into variable ones that follow the work, and it is what makes the spending visible at all.

What renting does not do

Two things survive the move, and the rest of this objective is about both of them. The work of choosing, integrating and managing services stays with the customer, so the technology function changes shape rather than disappearing. And the meter that makes measured service attractive keeps running whether or not anybody in the organization is reading it.

Owning means paying in advance for a guess about capacity. Renting means paying afterwards for what actually happened — which is better arithmetic and a worse habit, because a cost nobody approved in advance is a cost nobody is necessarily watching.

CHECK YOURSELF

Characteristics, chunks, and the two gaps

Three questions on the mechanics of renting capacity; every organization described is a hypothetical practice situation, and each option explains itself whether or not you chose it.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical seed supplier needs about four times its usual computing capacity for six weeks each spring and its ordinary capacity for the rest of the year. Which characteristic addresses that most directly, and why?

- **A.** On-demand self-service, because the extra capacity can be obtained through a control panel without negotiating with anybody at the provider — Self-service describes how the resources are obtained rather than how they grow and shrink. It genuinely helps here, but the question is about capacity following a seasonal curve, which is a different characteristic.
- **B.** Resource pooling, because the provider holds distributed resources that many customers draw on as their needs change — Pooling is the precondition that makes spare capacity available to be assigned, and it is present. It does not describe the speed of the change or the fact that the capacity is given back in June.
- **C.** Rapid elasticity, because resources scale up and down almost instantaneously and often automatically, where purchased equipment takes weeks to arrive and days to configure — **correct.** The chapter contrasts equipment that takes several weeks to deliver and days or weeks to configure with a cloud environment where resources scale up or down almost instantaneously and often automatically, and it uses a seasonal surge to make the point.
- **D.** Broad network access, because the supplier's staff can reach the extra capacity from anywhere and from any web-enabled device, which is what makes the spring arrangement workable at all — Broad network access explains who can reach a service and from where. The supplier's problem is the size of the capacity in March compared with October, not the location of the people using it.

2. The chapter presents measured service as a benefit. What is the same characteristic's cost?

- **A.** The meter runs whether or not anybody is reading it, so a cost that nobody is monitoring can grow until the invoice makes it visible — **correct.** Measured service transforms fixed costs into variable ones that are easy to track and monitor, but tracking has to be done by somebody; the chapter's own cautionary case is what happens when nobody does it.
- **B.** Metering obliges the customer to commit to a minimum level of usage for the length of the contract — This is the opposite of what the chapter describes. A cloud customer can typically obtain resources without committing to minimum usage, which is exactly what it credits for the travel platform's expansion.
- **C.** Metering means the provider decides which physical machine the work runs on and where that machine is located — That is resource pooling, not measured service. It is a genuine consequence of renting, but it concerns where the work runs rather than how the customer is charged for it.
- **D.** Metering prevents capacity from being reduced again once it has been increased, because the meter is set at the higher level — Releasing resources and having the charge fall is the point of the model. Metering records what was used in each period, so a reduction in usage appears in the next bill rather than being locked out.

3. The chapter draws a stepped in-house capacity line beside a cloud capacity line that follows demand. What are the two gaps in the in-house picture, and why does each one cost money?

- **A.** Capacity above demand and capacity above budget, which are both forms of overspending, one on equipment and one on the staff to run it — Only one of these is a gap in the figure. Staffing is a real cost of ownership, and the chapter names it, but the two shaded areas in the diagram are both about the distance between capacity and demand.
- **B.** Capacity above demand and demand above capacity: one is equipment being paid for while idle, the other is business the system cannot serve — **correct**. With a traditional in-house infrastructure an organization ends up either holding unused resources or being unable to satisfy its users, and the two shaded areas in the chapter's figure are those two situations.
- **C.** Demand above capacity and demand above forecast, which are both planning failures rather than problems with the infrastructure itself — A better forecast narrows the gaps but cannot close them, because capacity that arrives in whole machines cannot trace a demand curve however good the forecast is. The problem is the shape of the supply, not the quality of the guess.
- **D.** Capacity above demand and demand above capacity, though only the first has a cost, because unserved demand usually returns later — The second gap has the larger and less visible cost. Nothing on an invoice records an order that was never placed by a customer who gave up waiting, and the chapter treats unsatisfied demand as a real loss rather than a delay.

Service models and kinds of cloud

“We moved to the cloud” describes two decisions, not one. The first is how much of the machinery you hand over — some of it, most of it, or all of it. The second is whose building it sits in and who else is renting space beside you. This section takes them in that order, because the first decides what your own staff still have to do on Monday morning, and the second decides who else is on the same hardware.

Renting covers three very different deals

Think about renting somewhere to live. You could rent a bare plot of land and put up your own building on it. You could rent an empty flat, where the walls, the wiring and the heating are somebody else’s problem but the furniture is yours and you decide what happens in each room. Or you could take a hotel room, where everything down to the towels is provided and the only thing you bring is yourself.

All three are renting, and the rent is the least interesting difference between them. What actually separates them is **how much of the work has already been done for you** and how much you are still allowed to change. Cloud computing divides along exactly that line, into the three arrangements the chapter calls **service models**. The comparison with renting a home is this module’s way of making the idea concrete; the chapter states the models directly.

To see the three cleanly, first lay out the pile of machinery they divide up. Every application anybody uses sits on top of the same stack of layers, whether the machines are in your building or somebody else’s.

LAYER, FROM THE GROUND UP	WHAT IS ACTUALLY THERE	WHAT IT LOOKS LIKE WHEN IT
The facility	A room or a building, the electricity feeding it, and the cooling that carries the heat back out	The machines are fine and none switched on
The hardware	Physical servers, storage, the network between them, and a firewall in front of the lot	There is nowhere for the work to and nothing to hold the data
The operating system	The program that lets everything above it use the machine without knowing anything about the machine	Nothing you install has any way the processor, the disk or the ne

LAYER, FROM THE GROUND UP	WHAT IS ACTUALLY THERE	WHAT IT LOOKS LIKE WHEN IT
The platform	The web server that answers requests, the database management system that keeps the records, and the tools an application is built with	Your application has no way to p page or to store a row
The application	The thing a person actually opens: the mail, the records screen, the booking form	There is a working computer wi on it that anyone in the busines
Your settings and your data	Which accounts exist, what the fields are called, and the records themselves	The application runs perfectly a nothing about your organization

Each service model draws a line across that stack at a different height. Everything below the line is the provider's job; everything above it stays yours. One sentence per model is enough to hold all three in your head, and the rest of this section is those three sentences unpacked.

Infrastructure as a service — you manage the operating system and everything above it, including your own software licences, and the provider stops at the hardware.

Platform as a service — you manage your own application and nothing beneath it, because the operating system, the web server and the database engine arrive already running.

Software as a service — you manage your settings and your data, and nothing else at all, because even the application is maintained and updated by somebody else.

Infrastructure as a service: you rent the machines

In **infrastructure as a service**, usually shortened to **IaaS**, only the basic capabilities of processing, storage and networking are provided. The provider owns the building, the power, the cooling and the hardware. Everything above that is yours: you choose the operating system, you install and configure what you need, and you keep track of your own software licences. Of the three models this one gives the customer the most control over the resources, and for exactly the same reason it leaves the customer the most work.

The chapter's example is Amazon Web Services, where a customer chooses computing power, memory, operating system and storage to suit its own needs and can therefore build almost its entire infrastructure in the cloud rather than in a room of its own. Netflix moved its infrastructure onto that

service in order to convert films into the many formats different devices need, to run its customer-facing website, and to host other applications it cannot operate without.

Three consequences follow from renting at this level, and the chapter names all three.

The greatest flexibility of the three models — if your software needs a particular operating system, or a particular version of one, this is the model that lets you have it.

Software licences remain your responsibility — the provider rents you a machine, not permission to run things on it, so every licence you needed before you moved is a licence you still need.

Setup costs are relatively high — somebody in your organization still has to install, configure and maintain everything above the hardware, which is skilled work that takes time.

Notice what the model has and has not removed. It has removed the building, the electricity, the cooling and the wait for equipment to be delivered. It has not removed a single decision about how the machine is set up, which is why the chapter pairs “most control” and “highest setup cost” in the same breath.

Platform as a service: you rent the machines and the software that runs them

In **platform as a service**, or **PaaS**, customers run their own applications, typically designed using tools the service provider supplies. The user has control over the applications but limited or no control over the infrastructure underneath. What arrives is a working platform: the operating system, the web server, the database management system and the development tools, all installed, all updated, all kept running by the provider.

The saving is not only in effort. Because the computing platform is provided, the customer does not have to worry about purchasing software licences — for the web servers’ operating systems, for instance, or for the database management systems — and the service provider manages the functioning and the updating of the platform. The chapter’s example is Microsoft Azure, which it describes under its older name of Windows Azure, and which customers use to deploy applications they wrote themselves.

Serverless computing, the current trend in this model. A newer arrangement goes one step further and frees the customer from setting up machines at all. On a serverless platform such as Amazon’s AWS Lambda, the customer supplies small, specific functions that can be assembled into an application; the service takes care of all management of the underlying platform, scales the work continuously, and charges on the basis of the time the functions actually run. The servers have not gone anywhere. The customer simply never meets one.

The trade this model makes is worth saying out loud, because it is the one people misjudge. You have given up the ability to choose or tune the platform — its version, its configuration, sometimes its language — and in exchange you have given away the whole job of keeping that platform alive. For an organization whose distinctive work is its own application and not its own plumbing, that is usually a good trade.

Software as a service: you rent the finished application

In **software as a service**, or **SaaS**, the customer uses only applications provided via a cloud infrastructure. There is no machine to choose, no operating system to patch, no platform to keep current. Typically the customer cares only about the application, with no knowledge of or control over the infrastructure beneath it, and typically has only a limited ability to control or configure application-specific settings.

This is the model you have used already, probably today. The chapter groups its examples into five familiar kinds of application.

Web-based email services — the chapter names Google's Gmail, and nobody in your building has ever patched a mail server for you to read your mail.

Document storage — services such as Dropbox and WeTransfer, where the file goes somewhere you never see and comes back when you ask for it.

Collaboration software — the chapter names Slack and Zoom, both of which run a conversation between people in different places with no equipment on either side.

Web-based productivity suites — Zoho and Google Docs, where the document editor itself is delivered to you rather than installed by you.

Advanced business applications — customer relationship management systems such as those provided by Salesforce.com, which run a whole business function as a rented service.

Software as a service is typically the easiest of the three to deploy, because the customer does not have to worry about maintaining or updating the software, the underlying platform or the hardware infrastructure. Some companies have gone further still and offer an entire solution as a service, providing not only software but other things around it such as expertise. The price of that ease is configurability: you get the application the provider built, adjusted only as far as its own settings allow.

The quickest way to fix the three models in place is to sort real arrangements into them, so try that now before the second decision arrives.

Sort

Which service model is this?

Place each arrangement under the service model it describes, then read why it belongs there. Some are the chapter's own examples; the ones marked as practice situations are hypothetical and no real organization is being described.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Infrastructure as a service — Processing, storage and networking are provided; the customer chooses the operating system and manages everything above it, licences included

- A streaming company rents raw computing capacity, chooses the operating systems that run on it, and uses it to convert films into the many formats different devices need. — This is the chapter's Netflix example. Only processing, storage and networking were bought; every layer above them, starting with the operating system, is still the company's own responsibility.
- A customer selects computing power, memory, operating system and storage from a provider's control panel and builds most of its infrastructure there rather than in a room of its own. — Choosing the operating system is the giveaway. That choice exists in exactly one model, because in the other two the provider has already made it and will not let the customer change it.
- A practice situation: a hypothetical architecture firm needs one licensed design package that runs on a specific older operating system, and needs control of the machine image it sits on. — A requirement about the operating system itself can be met only where the customer still chooses it, and the licence for the design package stays the firm's to manage in this model.

Platform as a service — The operating system, web server, database engine and tools arrive running; the customer controls its own application and little else

- A team deploys software its own developers wrote onto a rented platform built with the provider's tools, and nobody on the team has ever installed or patched a database engine. — The customer controls the application and has limited or no control over the infrastructure underneath, and the provider manages the functioning and the updating of the platform. That is the definition.
- Small, specific functions are run by the provider on demand, assembled into an application, and billed for the time they actually run, with no machine ever set up by the customer. — This is serverless computing, which the chapter presents as a current trend within the platform model: the platform is still provided, and the customer has simply stopped seeing even the machine.
- A practice situation: a hypothetical charity has two developers who wrote a booking application, and wants the provider to keep the web server and database engine current while the booking rules stay the charity's own. — The booking rules are the application, which the charity keeps; the web server and database management system are the platform, which the provider runs. The line falls exactly where the platform model draws it.

Software as a service — The customer uses only the finished application, with no control over the infrastructure and limited ability to configure it

- Staff open a web-based email service through a browser, and nobody in the building has ever installed, patched or updated a mail server. — The chapter names web-based email as its first example of the model. The customer uses only the application and has no knowledge of or control over the infrastructure beneath it.
- A firm keeps shared documents in a hosted storage service and sends large files through a hosted transfer service, having bought no equipment for either. — Document storage services are among the chapter's own examples of the model, and nothing about the arrangement gives the firm a machine, an operating system or a platform to look after.
- A practice situation: a hypothetical sales team logs in to a hosted customer-records application, can rename fields and add users, but cannot change how the application works. — Limited ability to control or configure application-specific settings is precisely what the chapter says distinguishes this model. The team manages its settings and its data, and nothing else.

The line that decides everything: who manages what

Learners reliably go wrong here in the same way: by trying to remember which brand name belongs to which model. Brands move between models and providers sell in all three. The durable test is a question about your own organization — **what do my people still have to look after on Monday morning?** Answer that, and the model names itself.

Read the ladder below from the bottom. Each rung hands one more layer to the provider, and each rung takes one more decision away from you.

ARRANGEMENT	THE PROVIDER LOOKS AFTER	YOU STILL LOOK AFTER	WHAT YOU GIVE
Owning it outright	Nothing — there is no provider	The facility, the power, the cooling, the hardware, the operating system, the platform, the application, the data	Nothing, except money and the years
Infrastructure as a service	The facility, the power, the cooling and the hardware	The operating system, your licences, the platform, the application, the data	Physical control the machines and the ability to specify the hardware exactly

ARRANGEMENT	THE PROVIDER LOOKS AFTER	YOU STILL LOOK AFTER	WHAT YOU GIVE
Platform as a service	Everything up to and including the operating system, the web server, the database engine and the tools	Your own application, and the data inside it	The choice of platform, its version and much of its configuration
Software as a service	Everything up to and including the application itself	Your settings and your data	Nearly all configuration — application works the way the provider built it

Three things about that ladder catch people out, and all three are worth stating flatly rather than discovering later.

No rung takes away your data or your decisions — even in the model that leaves you the least to manage, which records exist, who may see them and what they mean is still entirely your problem.

Higher up the ladder is not the same as better — it is less work and less control, and which of those two an organization needs more depends on whether the thing being run is its distinctive work or its plumbing.

An organization sits on several rungs at once — the same company can rent finished applications for its office work, a platform for the system its developers wrote, and raw machines for the one awkward package that needs a particular operating system.

Handing over a layer hands over the labour of running it and the freedom to change it, in one movement, and you cannot accept the first half without accepting the second.

MATCH

The responsibility ladder

Pair each arrangement with the honest description of what your own organization is still responsible for under it.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Owning the infrastructure outright

You look after every layer, from the electricity and the cooling up to the application your staff log in to. With no provider anywhere in the picture, the facility, the hardware, the operating system, the platform, the application and the data are all in-house work, which is the arrangement objective 3.3 spent two sections describing the cost of.

Infrastructure as a service

You manage the operating system and everything above it, licences included, while the provider stops at the hardware. Only the basic capabilities of processing, storage and networking are provided, so the customer has the most control over the resources — and managing software licences remains the customer's responsibility.

Platform as a service

You manage your own application and nothing beneath it, because the operating system, web server and database engine are the provider's to run and update. The customer runs its own applications, typically designed with the provider's tools, and has limited or no control over the underlying infrastructure; platform licences and platform updates both move to the provider.

Software as a service

You manage your settings and your data, and nothing else, because the application itself is maintained and updated for you. The customer uses only the application, has no knowledge of or control over the infrastructure, and typically has only a limited ability to configure application-specific settings. It is the easiest model to deploy for exactly that reason.

Serverless computing

You supply individual functions and never set up a machine at all, and you are charged for the time those functions actually run. The chapter presents this as a new trend designed to free users from having to set up virtual machines: the service takes care of all management of the underlying platform and enables continuous scaling, billed by usage time.

Before moving to the second decision, check that the three models are genuinely separate in your mind and not just three names in a row.

CHECK YOURSELF

Telling the three service models apart

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical logistics firm rents computing capacity, installs an operating system of its own choosing on it, and runs its warehouse software on top. It is surprised to find it must still account for the operating system licences. Which model is this, and is the surprise justified?

- **A.** Platform as a service, and the surprise is justified, because the provider supplies the platform and should therefore supply the licences that come with it — The platform model does remove platform licences, which is what makes this tempting, but the firm chose and installed its own operating system. That choice does not exist in the platform model, where the operating system arrives already running.
- **B.** Infrastructure as a service, and the surprise is not justified, because only processing, storage and networking are provided and managing software licences remains the customer's responsibility — **correct.** In infrastructure as a service only the basic capabilities of processing, storage and networking are provided, the customer therefore has the most control over the resources, and managing software licences is still the responsibility of the customer.
- **C.** Software as a service, and the surprise is justified, because a rented application should arrive with everything it needs to run already paid for — Nothing here resembles software as a service. The firm is running its own warehouse software on an operating system it selected, which means it is managing layers that this model would have removed entirely.
- **D.** Infrastructure as a service, and the surprise is justified, because renting hardware from a provider transfers the cost of anything installed on that hardware to the provider — The model is identified correctly, and this is the closest wrong answer for that reason. The reasoning fails: renting a machine buys you the machine, not permission to run licensed software on it, and the chapter says so explicitly.

2. A hypothetical insurer wants its own developers' software running in production without anyone on staff installing or patching an operating system or a database engine, and without buying licences for either. Which model does the chapter describe for this?

- **A.** Infrastructure as a service, because the insurer is running software it wrote itself and therefore needs the greatest possible control over the resources — This model gives the most control, but control is not what the insurer asked for. It asked to stop looking after operating systems and database engines, and that is exactly the work this model leaves with the customer.
- **B.** Software as a service, because the insurer wants an application to run without maintaining anything, which is what this model exists to provide — The insurer does want to maintain nothing, which makes this look right, but the application is its own. Software as a service supplies the provider's application, so there would be nowhere for the insurer's code to run.
- **C.** Platform as a service, because the provider supplies the operating system, web server, database management system and tools while the insurer controls only its own applications — **correct.** Customers run their own applications on tools the provider supplies, they have control over the applications but limited or no control over the underlying infrastructure, and they avoid buying licences for the web servers' operating systems or the database management systems.
- **D.** A private cloud, because software written in-house has to run on infrastructure the organization owns and controls itself — A private cloud is an answer to a different question, about whose machines these are rather than how much of the stack is provided. Software written in-house can run perfectly well on rented capacity, and often does.

3. Why does the chapter describe software as a service as typically the easiest model to deploy, and what does that ease cost the customer?

- **A.** It is easiest because the customer maintains and updates nothing at all, and the cost is configurability, since what can be adjusted in the application is typically limited — **correct**. The customer does not have to worry about maintaining or updating the software, the underlying platform or the hardware infrastructure, and typically has only limited ability to control or configure application-specific settings.
- **B.** It is easiest because it is the cheapest of the three models, and the cost is reliability, since a shared application is more likely to fail than one the customer runs itself — Cost is a separate question the chapter treats under evaluating providers, and it does not rank the models by price. Nor does it claim that a shared application fails more often; availability is a criterion to examine, not a property of the model.
- **C.** It is easiest because the customer keeps control of the operating system, and the cost is the licences, which the customer must continue to purchase — This describes infrastructure as a service, where the customer does keep the operating system and its licences. In software as a service there is no operating system in the customer's hands to keep.
- **D.** It is easiest because it removes the need to keep any data at all, and the cost is that reports have to be requested from the provider each time — The data is emphatically still the customer's, and the model changes nothing about which records exist or what they mean. Only settings and data remain the customer's to manage, which is a small list but not an empty one.

Public, private, and the mixture most organizations end up with

The second decision is a different question entirely: not how much you rent, but whose cloud it is and who else is in it. The chapter separates two kinds and describes a third that organizations arrive at in practice.

Before the three make sense, one word they all turn on needs a definition. **Scalability** is the ability to adapt to increases or decreases in demand for processing or data storage. It is a promise rather than a physical fact, which is why it is something to test in a particular provider rather than to assume from the word "cloud".

Public cloud — services that any interested party can use on a pay-per-use basis, which suits work that has to scale rapidly and organizations without the capital or the people to build capacity of their own.

Private cloud — a cloud internal to one organization, which still offers self-service access to a shared pool and still bills departments for what they use, but keeps everything inside the organization's own control.

Hybrid cloud — some applications kept in-house and others moved to a public cloud, which the chapter describes as the approach organizations often choose once they stop looking for a single answer.

The private cloud is the one most often misunderstood, so be precise about what it does and does not do. It helps an organization balance the demand and supply of computing resources across its own departments, and it gives a high degree of customizability, flexibility and control over its data and applications. What it does not do is free the organization from the issues associated with managing a cloud infrastructure. Somebody still owns the building, the power, the cooling and the machines, and that somebody is still you.

The chapter contrasts the two kinds by the attributes each one tends to bring, and the contrast is more useful than either definition on its own.

THE PRIVATE CLOUD TENDS TO BRING	THE PUBLIC CLOUD TENDS TO BRING
Capital expenditure — the equipment is bought	Operational expenditure — the capacity is billed as it is used
Owned by the client	Owned by the service provider
Control, privacy and security under one roof	Flexibility, elasticity and measured service
Centralized and increasingly efficient use of what the organization already has	Standardized offerings, fast and easy setup

Set beside each other, those columns are the same trade the service models made, at a different level. The private column buys control and pays for it in money and effort; the public column buys speed and pays for it in standardization and in depending on somebody else. The reason the hybrid arrangement is so common is that most organizations have some work that belongs in each column, and no rule says a company has to answer once for everything it runs.

Four signals tell you which column a particular application belongs in, and they are worth applying one application at a time rather than to the organization as a whole.

Demand that swings hard and without warning points at the public column, because capacity that grows in minutes and shrinks again is exactly what rented, pooled equipment can do and owned equipment cannot.

A rule about where data may sit points at the private column, because internal policy or a duty that applies in a particular industry may require records to stay on machines the organization itself controls.

No capital and no technical staff points at the public column, since building a private cloud is equipment bought up front and an infrastructure somebody in the organization must then run.

Equipment already bought and sitting idle points at the private column, because pooling what the organization owns and letting departments provision from it recovers value from capacity already paid for.

CHECK YOURSELF

Public, private, and what each one really buys

Four options, one best answer; every explanation is worth reading, because the wrong answers here are the ones people act on.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical local charity has no capital to spend on equipment and no technical staff, but needs a service that can grow quickly if a campaign succeeds. Which kind of cloud does the chapter point to, and why?

- **A.** A private cloud, because a charity handling donor details needs the control and privacy that only its own machines can provide — A private cloud does give a high degree of control and privacy, but it does not free an organization from managing an infrastructure — and this charity has neither the capital to build one nor the staff to run it.
- **B.** Neither kind, because a charity of that size should buy a single server outright and avoid a recurring bill altogether — Buying a server outright is the arrangement with the largest capital cost and the least ability to grow, which is the opposite of what a campaign that may or may not succeed calls for.
- **C.** A hybrid arrangement, because splitting the work between owned and rented capacity is the safest starting position for an organization with no experience — Hybrid is what many organizations settle on, which is why this sounds sensible, but it means running both arrangements at once. An organization with no technical staff has taken on the harder half rather than avoided it.
- **D.** A public cloud, because its services can be used by any interested party on a pay-per-use basis and it suits applications that need rapid scalability — **correct.** Services in a public cloud can be used by any interested party on a pay-per-use basis, and the chapter recommends it for applications needing rapid scalability and for organizations with insufficient capital or other resources to build or expand an infrastructure.

2. A hypothetical manufacturer builds a private cloud so its departments can provision servers from a shared internal pool in minutes. Its finance director expects the infrastructure management work to disappear. What does the chapter say?

- **A.** The expectation is wrong: a private cloud still leaves the organization managing the infrastructure, though it does give real customizability and control — **correct**. The chapter is explicit that a private cloud provides self-service access on a utility computing model but does not free the organization from the issues of managing the infrastructure; what it buys is customizability, flexibility and control over its data and applications.
- **B.** The expectation is right, because self-service provisioning and a utility billing model are what define a cloud, and both remove the underlying management work — Self-service access and metered billing are genuinely present in a private cloud, which is what makes the reasoning attractive. They change who requests capacity and how it is charged internally, not who has to keep the equipment running.
- **C.** The expectation is right for the machines but wrong for the network, since networking is the one layer a private cloud cannot pool across departments — Networking is not a special case here, and a private cloud can pool network resources like any other. The management work that remains covers every layer the organization still owns, not one of them.
- **D.** The expectation is wrong, because a private cloud is simply a public cloud with a contract that restricts who else may use the same equipment — This confuses the two kinds. A private cloud is internal to the organization and owned by the client; the public cloud is owned by the service provider and open to any interested party on a pay-per-use basis.

3. What does the chapter mean by scalability, and what follows from that definition when choosing a provider?

- **A.** The number of machines a provider owns, which follows directly from how large the provider is, so the largest provider is the appropriate choice — Provider size is not the definition, and it is a poor proxy for the thing that matters. What an organization needs to know is whether capacity moves with its own demand, which is a question about the service it is buying.
- **B.** The speed at which data can be moved from one provider to another, so the organization should choose whichever provider transfers data fastest — That is closer to openness, which the chapter treats separately as a question about moving between providers and the cost of transferring large volumes out. It is a real criterion, but it is not this one.
- **C.** The ability to adapt to increases or decreases in demand for processing or data storage, so it is a promise to test against your own demand rather than to assume — **correct**. The chapter defines scalability as the ability to adapt to increases or decreases in demand for processing or data storage, and because it is claimed rather than guaranteed it belongs on the list of things a customer examines about a provider.
- **D.** The ability of an application to serve users in more than one country, so it matters to international organizations and not to local ones — Serving several countries is a matter of where resources are located and which rules apply to them, which the chapter treats under resource pooling and under compliance. Scalability is about volume of demand, not geography.

Two choices, taken together

Put the two decisions side by side and the shape of a real arrangement appears. A service model says how far up the stack somebody else is responsible; a kind of cloud says whose machines those are. They are independent, which is why an organization can rent finished applications from a public provider for its office work and still run a platform of its own inside the building for the system nobody else is allowed to touch.

That is also why the chapter says there is typically no single provider that can meet all of an organization's needs. Organizations often have to partner with several, choosing among the three service models according to the business's needs and frequently combining public and private clouds. Three questions, asked about one application at a time rather than about the organization as a whole, get you most of the way to an answer.

- 1 What must we still be able to change?** If the answer includes the operating system, you are looking at infrastructure as a service; if it is only your own application, a platform will do.
- 2 Is this work distinctive or is it plumbing?** Rent the plumbing as finished software, and keep your effort for the system that makes the organization different from its competitors.
- 3 Where is this data allowed to sit, and who else may be near it?** That question, and not the technology, is what usually pushes one particular application out of a public cloud and back inside the building.

The situations below work those three questions through four cases. They are practice situations, and the last one is the arrangement the chapter says organizations most often reach.

DECIDE

Choosing a model and a kind, one application at a time

Work the four decisions in order, and after each one read every outcome before moving on, including the ones you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Four hypothetical practice situations, invented for this exercise; no real organization is described in any of them. In each case ask the three questions from the section: what must we still be able to change, is this work distinctive or is it plumbing, and where is this data allowed to sit.

Decision 1. A hypothetical veterinary group with six clinics needs email, shared documents and video meetings for its staff. It employs nobody technical, and none of this work is what makes the group different from any other veterinary practice. What should it rent?

- Software as a service: rent the finished applications and manage only accounts, settings and the records inside them. — **the stronger call.** Right. This is plumbing rather than distinctive work, so the group should hand over as much of it as possible. Web-based email, document storage and collaboration software are three of the chapter's own examples of the model, and it is the easiest to deploy precisely because nothing has to be maintained or updated by the customer.
- Infrastructure as a service: rent machines and install mail, file and meeting software on them. — This would work and it would be a poor use of the group's money. Renting at this level leaves the operating system, the licences, the installation and the patching with a customer that has nobody to do any of it, in exchange for control over software that six veterinary clinics have no reason to customize.
- Platform as a service: rent a platform and have someone build the group a mail and document system on it. — The platform model is for running an application you have and want to keep. Commissioning one that already exists in mature form, and then owning it forever, converts a solved problem into a permanent development project.
- A private cloud: buy equipment and pool it across the six clinics so each one can provision what it needs. — A private cloud is the largest commitment on the table. It is capital expenditure, it is owned by the client, and the chapter is explicit that it does not free an organization from the work of managing the infrastructure — work this group has no one to do.

Decision 2. A hypothetical university research team has written its own analysis software and needs it available to collaborators. Nobody on the team wants to patch an operating system or a database engine, and the team has no licence budget. The software itself is ordinary code that any current platform can run.

- Platform as a service, so the operating system, web server and database engine are provided and the team looks after only its own application. — **the stronger call.** Right. The team controls its application and has limited or no control over the infrastructure underneath, which is exactly what it asked for. Because the computing platform is provided, it also avoids buying licences for the web servers' operating systems and for the database management system, and the provider manages the platform's updates.
- Infrastructure as a service, so the team can guarantee its software will run by choosing the machine and the operating system itself. — The guarantee is real, but it is being bought with the exact work the team said it would not do. Choosing the operating system means installing, configuring, patching and licensing it, which is why the chapter pairs this model's flexibility with relatively high setup costs.
- Software as a service, because a research team should use an existing analysis application rather than maintaining one of its own. — That may be sound advice about research, and it is not an answer to the question asked. The team has written the software; software as a service supplies the provider's application, so there is nowhere in this model for the team's own code to live.

- Serverless computing, because it removes machines entirely and charges only for the time functions run. — Serverless is a real option within the platform model and would suit work that arrives in short bursts. It asks the team to restructure its software into small individual functions first, which is a rewrite nobody has asked for and which the situation gives no reason to want.

Decision 3. A hypothetical film archive digitises old reels using a specialised package that runs only on an older operating system, and its work arrives in unpredictable bursts: nothing for a month, then thousands of hours at once. It has the licence for the package already.

- Infrastructure as a service in a public cloud: rent capacity as the bursts arrive, install the older operating system and the package on it. — **the stronger call.** Right. A requirement about the operating system itself can be met only in the model where the customer still chooses it, and the licence stays the archive's to manage, which it already holds. Renting publicly means the idle month costs nothing, which is the whole answer to work that arrives in bursts.
- Platform as a service, because the archive should let the provider handle the operating system and stop worrying about it. — That is usually good advice and here it is impossible. The platform model supplies the operating system already running and does not let the customer specify an older version, so the archive's one indispensable package would have nowhere to run.
- Buy a machine outright and run it in the archive's own building, since the work is specialised and the software is already licensed. — Owning removes the rental bill and reintroduces every problem objective 3.3 described. Capacity would be sized for either the quiet month or the burst: idle equipment most of the year, or a queue when the reels arrive.
- Software as a service, choosing whichever hosted digitisation application is closest to the package the archive uses today. — This throws away a licence the archive already owns and a workflow built around it, in exchange for an application it cannot configure beyond the provider's settings. The chapter's point about limited configurability is exactly the risk here.

Decision 4. A hypothetical city transit authority runs two things: a journey planner whose traffic multiplies whenever the weather turns, and a fare-card records system holding personal details that internal rules say must stay on machines the authority controls. A board member insists on one arrangement for both.

- A hybrid arrangement: the journey planner on rented public capacity, the fare-card records on the authority's own controlled infrastructure. — **the stronger call.** Right, and the chapter describes this as what organizations often choose — certain applications in-house while others move to the public cloud. There is typically no single provider that meets all of an organization's needs, and the two applications here are asking for opposite things: elasticity for one, control for the other.

- Everything in a public cloud, since a capable provider can secure the fare-card data as well as the authority can. — The provider may well be more capable, and that is not what the constraint says. An internal rule requiring the data to stay on controlled machines is a decision about control and location, and the chapter treats where data physically sits as a genuine limit rather than a technical detail.
- Everything in a private cloud, so the authority controls both applications and can still provision capacity on demand internally. — This satisfies the fare-card rule and strands the journey planner. Internal pooling can only redistribute equipment the authority has already bought, so the weather-driven surge still has to be paid for in advance and sits idle the rest of the time.
- Everything as rented finished applications, so neither system needs any infrastructure decision at all. — Neither system fits. The fare-card rule is about where data sits, which this model settles in the provider's favour, and a journey planner built around one city's network is distinctive work rather than something to buy off a shelf.

WHAT THIS SECTION DOES NOT ANSWER

Knowing which model and which kind you want tells you nothing about whether a particular provider is any good at delivering it. Three questions are still open, and the next section is about all three:

Will it be there when we need it — not even the largest providers are immune from hardware failures, programming errors and network outages, and a promise of uptime is not the same thing as availability.

What happens to the data — once records are on somebody else's equipment, who can reach them, who may be compelled to hand them over, and how hard would it be to take them somewhere else.

What will it actually cost — the rented figure is exact and arrives monthly, while most organizations have never worked out what the same work truly costs them in-house.

Deciding well: what to ask before you move

Renting an infrastructure can be arranged in an afternoon, with a card, a web page and nobody's signature. That speed is the appeal, and it is also the difficulty: a decision that once took months of meetings now takes minutes, so the thinking has to happen somewhere else. This section is that thinking — the chapter's own cautionary case, the questions worth putting to a provider before depending on one, and the honest argument about what any of it costs.

The bill nobody was watching

The chapter introduces cloud computing with a warning rather than a sales pitch, and the warning is worth taking seriously precisely because nobody in it does anything obviously foolish.

A small software company — the chapter deliberately does not name it, and neither will this page — ran a translation application. It did not own the translation engine. Its application sent text out to another company's service and received the translated text back, reaching that service with an **interface key**: a short secret string that identifies the paying customer on every single request.

The company's usual spend was around US\$1,500 a month. Then one bill, covering roughly six weeks, came to about US\$450,000. According to the founders, somebody appeared to have stolen the key and used it to request nineteen billion characters of translation. Every one of those characters was charged to the company that owned the key, because charging the key holder is exactly what the key is for. The budget the company had built its year around was gone, and it was left asking the provider for leniency.

The chapter sets a second, quieter case beside it. A startup doubled its user base and expanded its rented infrastructure to keep up, but never put cost management practices in place while it was growing. Resources were over-provisioned, there was no real-time monitoring, and the rising cost went unnoticed until the monthly bill arrived. Nothing was stolen in that one. It was simply nobody's job to look.

Read both cases against the five characteristics from the earlier sections and the mechanism is plain. Three of them are doing the damage, and all three are features that were sold as benefits.

On-demand self-service — resources can be obtained without negotiating with anybody, which also means they can be obtained without anybody inside the organization approving the spending.

Rapid elasticity — capacity scales up almost instantly and often automatically, so a runaway stream of requests meets no ceiling of the kind a room full of owned machines would have imposed.

Measured service — the meter runs for whoever presents the credential, and it never asks whether the work being requested is work the business actually wanted done.

That is why the chapter's conclusion is about vigilance rather than avoidance. Cloud computing offers scalability and flexibility, and without proper oversight a company can quickly find itself facing financial trouble that threatens its operations and its growth. The five defences it recommends are ordinary management practices rather than technical measures, which is the part worth noticing.

- 1 **Put cost monitoring tools in place**, so that what is being spent is visible while it is being spent rather than only when the billing period closes.
- 2 **Review resource allocations regularly**, because the second case was not an attack at all: it was capacity that made sense once and was never revisited.
- 3 **Set budget alerts** that fire on anomalies, so an unusual pattern raises a flag on the day it starts instead of the day the invoice arrives.
- 4 **Educate teams on cost awareness** and build a culture of accountability, since the person who provisions a resource in seconds is rarely the person who later reads the bill.
- 5 **Read the fine print** to know what you are getting into and what you may be liable for, which is the only one of the five that has to happen before anything is switched on.

One thread ties this back several sections. The thing stolen in the first case was an interface key of exactly the kind shown earlier sitting inside a request, in plain readable text, where anything able to see the request can see it too. The security failure and the spending failure were one event, not two.

The exercise below walks a hypothetical firm through four moments where one of those five defences was the thing that was missing. The firm is invented; only the shape of the failure is the chapter's.

DECIDE Four moments in a runaway bill

Work the four decisions in order, and after each one read every outcome, including the ones for the options you did not pick.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

A hypothetical practice situation, built on the shape of the chapter's own cautionary case rather than on any real company. You look after operations at a small analytics firm that has just started sending work to an outside provider's metered service. Four moments arrive over the following year, and each one maps onto one of the chapter's five prevention strategies.

Decision 1. Nothing is switched on yet. The engineering lead has an account ready and asks whether there is anything to do before the first request goes out. What matters most right now?

- Read the terms first: how usage is charged, what happens when usage is not yours, and what the firm may be liable for. — **the stronger call.** Right. Reading the fine print is the one item on the chapter's list that has to happen before the service is running, because it is the only moment when you can still decline the arrangement. It is also where you find out whether requests made on a stolen credential are billable to you, which is the question the chapter's case turned on.
- Switch it on and watch the first bill, since real numbers teach more than a contract does. — Real numbers do teach, but the first bill arrives only after a whole billing period of usage has already happened. That is exactly the interval in which the chapter's company ran up a charge hundreds of times its normal size, and the interval it had no way of seeing into.
- Ask the provider to cap the account so that overspending is impossible, and treat the matter as settled. — A spending cap is a useful control where a provider offers one, and it is worth having. Treating it as the whole answer skips four of the chapter's five recommendations, and a cap set high enough not to interrupt normal work is still high enough to hurt.
- Have the finance team approve every individual request the service receives. — This would defeat the arrangement entirely. On-demand self-service is what makes the service worth using, and a metered service may handle thousands of requests an hour; the chapter's answer is oversight of the pattern, not approval of each event.

Decision 2. Three months in, a manager asks what the service has cost so far this quarter. Producing the answer takes two days, because somebody has to open the provider's console, export the usage and add it up by hand. What is the missing practice?

- Cost monitoring, so that spending is visible while it is happening rather than reconstructed afterwards. — **the stronger call.** Right. The chapter puts cost monitoring tools first for this reason: a figure that takes two days to assemble will be assembled rarely, and a cost nobody looks at is a cost nobody manages. This is the practice whose absence produced the chapter's second case.
- Nothing important. A monthly total is enough oversight for a service of this size. — That is precisely the reasoning behind the chapter's second case, where a growing startup let costs escalate unnoticed until the monthly bill arrived. A monthly total tells you what already happened; it cannot tell you about it while you could still act.
- Switch to a provider whose invoices are simpler to read. — Invoice formatting is not the problem here. The firm has no view of usage between invoices whatever their layout, and changing provider to solve a monitoring gap replaces a small piece of work with a migration.
- Have the engineering lead estimate the monthly spend from memory at each management meeting. — An estimate from memory is the thing cost monitoring exists to replace. It will be roughly right while usage is steady, which is the only period in which nobody needed it, and roughly right is exactly the wrong answer during an anomaly.

Decision 3. Over a weekend, usage rises to many times its normal level and stays there. Nobody at the firm notices until the invoice appears eighteen days later. What should have been in place?

- Budget alerts set to fire on an anomaly, so the pattern raises a flag on the day it starts. — **the stronger call.** Right. This is the third of the chapter's five strategies, and it is the one that shortens the exposure. Cost monitoring makes spending visible to whoever looks; an alert removes the requirement that somebody happens to be looking on a Saturday.
- A standing monthly review meeting where usage is examined alongside the other operating costs. — A monthly review is worth having and belongs to a different strategy, reviewing resource allocations. As a detector it is too slow: a monthly cycle can let an anomaly run for most of a billing period, which is roughly the interval that produced the chapter's case.
- A request to the provider to refund the unusual usage, since the firm did not authorise it. — The meter recorded real requests made with a valid credential, which is what a metered service bills for. The company in the chapter's case was left pleading for help and leniency rather than exercising a right, which is the difference between a remedy and a favour.
- A larger prepayment to the provider, so that a spike is absorbed without an unexpected invoice. — Prepaying changes when the money leaves rather than whether it is spent. The firm would still lose the same amount, and it would lose it without ever receiving the invoice that was the only signal anything had happened.

Decision 4. The investigation finishes. The credential had been readable in a place a departed contractor could still reach, and separately about half the capacity the firm was paying for had been provisioned for a product launch that never happened. What does the chapter's list say the firm still owes itself?

- Both a regular review of what is provisioned and cost awareness across the teams, because the two failures have different causes and different owners. — **the stronger call.** Right. One failure was a credential and the other was capacity nobody revisited, and the chapter's five strategies cover both: regular review of resource allocations catches the idle capacity, and educating teams on cost awareness with a culture of accountability addresses the gap between whoever provisions a resource and whoever pays for it.
- A security review only. The unused capacity is a finance matter and belongs on a different agenda. — Splitting them is how the second one survives. The chapter's own second case involved no attacker at all, only over-provisioned resources and no monitoring, and it produced the same result: money leaving the business without anybody deciding that it should.
- A move back to machines the firm owns, since a metered service has proved too dangerous to use. — The chapter draws no such conclusion. Owned machines would replace a visible metered bill with the fixed costs and idle capacity of objective 3.3, and the firm would still have had a credential where a contractor could read it.
- A stricter approval process for new services, so that no team can start using a provider without sign-off. — Sign-off before the first use is worth having and overlaps with reading the fine print. It does nothing about either failure here, though: both happened long after the service was approved, and neither would have been prevented by an extra signature at the start.

Ten questions to put to a provider

Before any criterion matters there is a prior decision, and the chapter states it first: **which applications, services or data should move at all**. That is not one answer but many, because there is typically no single provider that can meet all of an organization's needs. Organizations end up partnering with several, choosing an infrastructure, a platform or a finished application for each need and combining public and private arrangements, for the plain reason that no one solution fits everything.

Once something is a candidate to move, the chapter lists the long-term, strategic issues management should weigh when comparing public providers. There are ten of them. They are easier to use as questions than as a row of nouns, because a criterion you cannot phrase as a question is one you will not actually ask.

CRITERION	THE QUESTION IT MAKES YOU ASK	WHAT A WEAK ANSWER SOUNDS LIKE
Availability	What uptime is promised, and how would we keep trading during the hours it is missed?	"They are one of the biggest providers in the world, so they do not go down."
Reliability	What backups are taken of the servers and the storage, and will we have enough bandwidth to reach large amounts of our own data?	"It is all in the cloud, so presumably it is backed up."
Scalability	Can this provider carry our storage and transaction volumes as they are now and as we expect them to be?	"It is the cloud. Scaling is what it does."
Viability	If this provider stopped trading, what would it cost us in money and time to stand everything up somewhere else?	"They have plenty of investors, so that will not happen."
Security	How is our data protected from outside intruders, and how would we audit who has reached it?	"Their website says security is their highest priority."
Privacy	How is the privacy of the people whose records these are protected once the records sit on shared hardware?	"It is our account, so nobody else can see any of it."

CRITERION	THE QUESTION IT MAKES YOU ASK	WHAT A WEAK ANSWER SOUNDS LIKE
Compliance	Does storing this particular data here satisfy the duties that apply to our industry and to the places we operate in?	"The provider has a compliance page, so we are covered."
Diversity of offerings	Can one provider cover what we need now and what we will need next, or are we about to manage several relationships?	"We will pick the best tool for each job and sort the rest out later."
Openness	If we wanted to leave in three years, how would the data get out, how long would that take and what would the transfer be charged at?	"We would just copy it across."
Cost	What does the metered bill actually cover, and what does the same work truly cost us when we do it ourselves?	"The monthly figure is less than we paid for the server."

Those ten are not a checklist invented for this page. They are the chapter's direct answer to the third question its opening case asked the reader: what factors should organizations consider when building on infrastructure provided in the cloud. The rest of this section is the discussion behind four of them — availability, security, openness and cost — because those are where the chapter has most to say and where managers most often go wrong.

SORT

Which criterion is this question testing?

Each question below is one a manager might put to a provider. Place it under the criterion it is really testing; the organizations described are hypothetical.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Availability and reliability — will it be running, what is backed up, and how do we keep trading on the day it is not

- What uptime is promised for this service, and what will we be doing during the hours that promise is not met? — Availability is the primary concern for most organizations, and the chapter notes that not even the largest public providers are immune from hardware failures, programming errors or network outages. Because of that it expects the customer to plan ahead and replicate its cloud-based infrastructure in different locations.

- What backups are taken of the servers and storage, and will there be enough bandwidth for us to reach large amounts of our data quickly? — These are two of the three questions the chapter tells organizations to examine when judging whether an application can be moved: the promised uptime, the backups made to servers and storage, and whether sufficient bandwidth will be provided for large amounts of data.

Scalability and viability — can the provider grow with us, and will the provider still be there in a few years

- Our transaction volume triples every December. Can this provider absorb that, and the volumes we expect to reach in three years? — Scalability is one of the biggest promises of cloud computing, yet the chapter warns that not every provider will meet every organization's demands, so the evaluation has to cover both current and future needs in storage and transaction volume.
- If this provider were to stop trading, what would it cost us in money and time to set up somewhere else and move everything across? — That is viability: the stability of the provider in the long run. The chapter lists the repercussions precisely as the costs and efforts of setting up a new infrastructure, migrating applications and transferring the data to it.

Security, privacy and compliance — who can reach the data, whose duties apply to it, and where may it physically sit

- How is our data protected from outside intruders, and how would we ever establish who had access to a particular record? — The first half is the chapter's security question. The second half is its structural warning: because a public cloud is shared among different companies on the same hardware, auditing who has access to the data is extremely difficult, if not impossible.
- May the records of the people we serve be held on machines outside this country, given the duties our sector works under? — This is privacy and compliance together. The chapter treats compliance duties as applying in particular industries and jurisdictions, and notes that some providers allow a customer to specify geographic areas for exactly this kind of requirement.

Diversity of offerings and openness — how many providers must we manage, and how hard would it be to leave one

- Will one provider cover the services we need now and the ones we will need next, or are we about to manage several separate relationships? — Diversity of offerings is about the management burden: a larger number and diversity of providers is more difficult to manage, so many organizations prefer to deal with fewer providers that can meet all their needs.
- How would we get several terabytes back out again if a better offer arrived, how long would that take, and what would the transfer be charged at? — That is openness. Providers use different infrastructures and different ways to store data, which makes migration extremely difficult and produces lock-in, and bandwidth plus the charges providers levy for moving data into or out of their infrastructure make it slow and expensive as well.

Cost — what the meter shows, set against what the same work truly costs us

- The rented figure is on the invoice. What does the same machine truly cost us once licences, electricity, the facility and the staff are counted? — This is the cost comparison the chapter calls hard. Calculating the monthly cost of a rented server is easy, while many organizations do not know what a comparable server of their own costs once the machine, the licence fees, the electricity, the data center and the staff are all included.
- If we rent capacity only for our few peak weeks each year, who actually operates that arrangement and what does the extra complexity cost us? — The chapter raises exactly this balance: using a cloud infrastructure only for periods of peak demand is sensible, and, needless to say, it adds another layer of complexity to the IT operations, which is a cost like any other.

What an agreement actually promises

Availability is where most organizations start, and the chapter is blunt about it. As examples from the largest public providers have shown, not even they are immune from problems: a hardware failure, a programming error, or a network outage somewhere in between. The useful question is therefore not whether an outage will happen but what the customer has arranged for the day it does.

The instrument that is supposed to answer that is the **service-level agreement**: a contract specifying the level of service to be provided in terms of performance, measured for instance as uptime, along with warranties, disaster recovery and so on. It reads like a guarantee. It is not one, and the chapter says so plainly.

It does not guarantee that resources will be available — it promises certain service levels and provides refunds or discounts when those promises are missed, which makes it mostly a way of settling disputes once something has already gone wrong.

A refund is not compensation — refunds and discounts normally cover only the fees paid for the service, and can never offset the opportunity cost of the business lost while the service was unavailable.

Self-service cuts both ways — doing everything yourself is an advantage until something breaks, because help is not always guaranteed to be there, so support arrangements matter most for the applications the organization cannot trade without.

The customer still has to plan — being caught by a negative event is usually too costly in lost business or goodwill, so organizations are expected to take their own precautions, including replicating their cloud-based infrastructure in more than one location.

The chapter then insists on a counterpoint, and leaving it out would make the criticism dishonest. An organization also has to ask how well it could hold that uptime itself, and at what cost. Often the honest

answer is that even when a provider misses its promised levels, it still delivers better uptime than a poorly managed operation in the organization's own building would have. Neither arrangement is safe by default. One of them simply puts the failure somewhere you cannot walk to.

The usual resolution in practice is a hybrid one: keep certain applications in-house and move others to the public cloud. That is the same answer the previous section arrived at from the other direction, and it is worth noticing that the chapter reaches it twice by two different routes.

Sharing a building you cannot see

Security, privacy and compliance are three separate criteria that get discussed together because they all follow from one structural fact. A public cloud is, by definition, shared: different companies running different applications on the same hardware.

Two consequences follow, and neither is a criticism of any particular provider.

You cannot say exactly where the data physically are — resources are pooled and assigned dynamically, so a customer rents capacity without knowing which machine in which building holds its records at any given moment.

Auditing who has touched the data is extremely difficult — the chapter goes as far as saying it may be impossible, which matters because knowing who reached a record is often the entire substance of an obligation.

Control is the other half of it. With an infrastructure of its own a company has complete control over its own data. That control is reduced in a rented one, organizations have fewer legal rights over data stored in the cloud, and providers may be asked to hand data held on their servers to law enforcement, leaving the customer with little say in the matter.

The weight of this falls hardest on organizations holding sensitive records — the chapter names medical and legal work — which is why some of them keep precisely those records in their own building while renting everything else.

Compliance needs care in the other direction. The chapter names rules on financial reporting, rules on the handling of health information, and an industry standard for card payments as examples of duties an organization may have to satisfy. What matters for a decision is the shape of such a duty rather than its name.

Duties of this kind are not universal — requirements apply in particular industries and particular jurisdictions, so the first question is which of them reach this organization and this data at all.

They can decide geography — a duty may dictate where records are permitted to sit physically, which is one reason some providers let a customer specify the geographic region its resources are drawn from.

A provider cannot answer them on your behalf — whether an arrangement satisfies a duty is a question for the organization's own advisers rather than for a marketing page, and the accountability stays with the organization either way.

Getting out again

Openness, sometimes called interoperability, is the criterion noticed last and felt hardest. Most providers use different infrastructures and different ways of storing data, which makes moving data between them extremely difficult and can leave a company locked in to whichever provider it happened to start with. The chapter makes a dry observation worth repeating: providers do offer tools to make moving data easier — into their own infrastructure.

A second force is physical rather than commercial. Moving terabytes from one provider to another takes real time on whatever bandwidth exists, and providers often charge for transferring data into or out of their infrastructure. A migration that was budgeted as a weekend of copying can turn out to be months of transfer with an invoice of its own attached.

Ask the exit question at the beginning, while you still have the leverage to negotiate the answer. Once the data are large and the applications are built around one provider's way of doing things, the cost of leaving stops being a choice and becomes a fact.

CHECK YOURSELF

Agreements, exposure, and getting out again

Four options, one best answer; read every explanation, including the ones for options you did not choose. Any organization described is hypothetical.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A provider's agreement promises a very high uptime figure and pays a discount whenever that figure is missed. What has the customer actually obtained?

- **A.** A guarantee that the resources will be available, which can be enforced if they are not — The chapter is explicit that such agreements do not guarantee the availability of resources. A commitment with a penalty attached is a different instrument from a guarantee, and confusing the two is what leaves an organization with no plan.

- **B.** A promise of certain service levels together with refunds or discounts when they are missed, which works mostly as a way of resolving disputes — **correct**. The chapter describes service-level agreements as promising certain service levels and providing refunds or discounts if the promises are not met, which makes them mostly a vehicle for resolving conflicts in case of problems.
- **C.** Protection against the business lost during an outage, since the discount is calculated from the revenue affected — Refunds and discounts normally cover only the fees paid for the service. They can never offset the opportunity costs arising from lost business, which is usually the larger figure by a wide margin.
- **D.** A reason to stop planning for outages, because responsibility for uptime has now moved to the provider — This is the conclusion the chapter warns against. Because being affected by a negative event is often too costly in lost business or goodwill, organizations are expected to plan ahead and replicate their cloud-based infrastructure in different locations.

2. In the chapter's cautionary case, an enormous volume of work was requested using a credential that did not belong to the requester. Which feature of the arrangement turned that into a financial event of that size?

- **A.** Broad network access, because the account could be reached from anywhere and from almost any web-enabled device, which is what put it within reach of somebody it did not belong to — Broad network access explains how the service was reachable, which is true of every cloud service and of the ones that never produce a surprise bill. Reachability alone does not decide what anything costs.
- **B.** Resource pooling, because the customer's work was running on hardware shared with other companies — Resource pooling explains where the work physically ran and why the customer could not say which machine held it. Nothing about sharing hardware makes a request more expensive than it would otherwise be.
- **C.** Measured service together with rapid elasticity: the meter runs for whoever presents the credential, and capacity rose to meet every request without a ceiling — **correct**. Metering is what makes usage billable to whoever holds the credential, and elasticity is what let the requested volume actually be served rather than refused, so the two together convert a stolen key into a bill hundreds of times the usual size.
- **D.** On-demand self-service by itself, because the account could be opened without negotiating with the provider — Self-service explains how quickly the arrangement could be set up in the first place, and it is genuinely part of why nobody had to approve the spending. It does not explain why nineteen billion characters of work were served and charged.

3. A hypothetical research institute holds several terabytes with one provider and is offered better terms by another. Its analysts budget a weekend for the move. Which criterion is about to correct them, and why?

- **A.** Viability, because the real question is whether the new provider will still be trading in three years — Viability is a real criterion and worth asking about before signing anything, but it concerns whether the provider survives rather than what makes this particular migration difficult.
- **B.** Openness, because providers store data differently, and bandwidth limits plus transfer charges make moving very large volumes slow and expensive — **correct.** Most providers use different infrastructures and different ways to store data, which makes migrating between them extremely difficult, and the chapter adds that bandwidth and data transmission costs limit it further because providers often charge for moving data into or out of their infrastructure.
- **C.** Scalability, because the new provider may not be able to hold the volume the institute expects to accumulate over the next few years, and a second migration would cost more than this one — Scalability concerns whether a provider can meet current and future demands for storage and transaction volume. It says nothing about the cost or the duration of the move itself.
- **D.** Availability, because the institute cannot work while its data are in transit between the two providers — Working around a transition is a genuine operational problem, but it is a consequence of the migration rather than the criterion that predicts how long and how expensive the migration will be.

4. A hypothetical clinic is told by a provider that its patient records will be completely secure and fully compliant in the provider's public cloud. What does the chapter say the clinic still cannot obtain there?

- **A.** Encryption of the records, which is unavailable on any shared infrastructure — Nothing in the chapter says protective measures are unavailable in a shared infrastructure. The difficulty it identifies is about location and auditing, not about whether the data can be protected in transit or at rest.
- **B.** Knowledge of exactly where the data physically sit, together with a workable way to audit who has reached them — **correct.** Because a public cloud is shared among different companies with different applications on the same hardware, it is impossible for organizations to know exactly where the data are physically located, and auditing who has access is extremely difficult if not impossible.
- **C.** Any protection at all from outside intruders, because the hardware is shared with other companies — This overstates the chapter considerably. It treats security as a question to put to the provider, not as something that cannot exist in a public cloud, and providers are described as being well aware of these concerns.
- **D.** The freedom to keep some records in its own building, because a provider agreement has to cover all of an organization's data — The chapter recommends the opposite. Organizations are expected to weigh which applications and data to move and which to keep in-house, and it names medical and legal work as fields where that judgment matters most.

The chapter's own case teaches the vocabulary. Five new organizations, none of them real and none of them the one above, ask you to use it cold: pick the service model, name the pressure, or judge the migration before you read why.

APPLY IT

Five decisions you have not seen before

Each question drops you into a different, invented organization mid-decision. Any organization described is hypothetical.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A four-person architecture firm wants shared project boards, file storage and e-signature for contracts, with no interest in writing or customizing any software themselves. Which service model fits what they are actually asking for?

- **A.** Software as a service, since they want finished applications to use, not infrastructure or a platform to build on — **correct.** Project boards, storage and e-signature are complete applications delivered over the internet; the firm wants to use them, not to manage the machines or the platform underneath them.
- **B.** Infrastructure as a service, since it gives them the most control over how the applications run — Control over how applications run is exactly what this firm has no interest in. Infrastructure as a service would hand them a set of virtual machines to administer, which is more responsibility than four people asked for.
- **C.** Platform as a service, since it lets them build exactly the workflow they want — Nothing here asks to build a custom workflow. A platform to develop on solves a problem this firm does not have; they want to sign in and use existing software.
- **D.** None of the three; a firm this size should keep everything on its own office server — Size is not the deciding factor. Even a very small firm can reasonably run entirely on finished cloud applications rather than maintaining a server at all.

2. A mobile game studio's servers sit mostly idle, then strain under ten times the normal traffic for the first week after every major update, then fall quiet again. Which pressure on their infrastructure is this, most precisely?

- **A.** Rising energy costs, since more players means more electricity drawn by the servers — Energy cost would rise during the spike, but that is a consequence of the real pressure, not the pressure itself, and it says nothing about why capacity is hard to plan for.
- **B.** Obsolescence, since the hardware clearly cannot keep up with a growing player base — Obsolescence is hardware falling behind over time regardless of traffic. This studio's problem repeats on a predictable weekly pattern around each release, which is a sizing problem, not an aging one.

- **C.** Demand fluctuation, since capacity has to be sized for a spike that does not last — **correct.** The studio faces short, sharp spikes it has to be ready for without paying to keep that much capacity running the other fifty weeks of the year, which is exactly the fluctuating-demand pressure the chapter describes.
- **D.** Growing storage needs, since more players generate more data to keep — Storage would grow steadily as data accumulates. A spike that rises and falls within a week is about processing and network capacity in the moment, not about how much has been saved up.

3. A regional accounting firm plans to move several years of client records to a new provider over a single weekend, and has budgeted for the new provider's fees but nothing else. What is most likely to go wrong?

- **A.** The new provider is likely to be less reliable, since switching providers tends to increase the chance of downtime — Reliability is a property of the provider chosen, not a consequence of switching itself; nothing here suggests the new provider is less dependable.
- **B.** The volume of data will make the transfer itself slow and possibly costly, a fact the weekend budget does not account for — **correct.** Providers store and structure data differently, and bandwidth limits plus transfer charges make moving a large volume slow and often expensive — a cost this budget never priced in.
- **C.** Client records cannot legally be moved between providers without a court order — No such requirement exists in general; moving business records between commercial providers is a contractual and technical matter, not one that needs judicial approval.
- **D.** Nothing is likely to go wrong, since fees are the only real cost a migration of this kind involves — This ignores exactly the cost the scenario is testing. Fees are one line item; the data transfer itself is frequently the larger and less visible one.

4. A boutique hotel-booking platform owns its own servers, sized to handle the single weekend each year when a local festival triples normal traffic. The rest of the year, most of that capacity sits unused. What does this tell you about the fit between the cost model and the workload?

- **A.** It fits well, since owning hardware is generally the cheaper option once a company can afford the upfront cost — Ownership is not cheaper by default; a fixed asset sized for one weekend a year is paid for in full whether it is used or idle, which is precisely the mismatch here.
- **B.** It fits poorly; a metered, elastic arrangement would charge mostly for the one busy weekend, not for capacity idle all year — **correct.** Rapid elasticity and a measured, pay-for-what-you-use model exist for exactly this kind of workload: capacity that is needed briefly and would otherwise sit idle almost all year.
- **C.** It fits poorly, but only because the servers should be replaced on a faster schedule than this company follows — Replacement frequency is a separate question from utilization. A brand-new server sized this way would sit idle just as often as an old one.

- **D.** It fits well, since owning the hardware means the company fully controls every machine during the busy weekend — Control during the festival is not what is being asked. The company already controls its own hardware today; the problem is what that control costs the other fifty-one weeks of the year.

5. An environmental testing lab has written its own image-analysis software for soil samples and runs it in intense bursts a few times a month. It wants to stop maintaining physical servers, but still needs to install and configure its own software stack. Which service model matches that?

- **A.** Software as a service, since the lab wants to stop managing hardware of any kind entirely — Wanting to stop managing hardware is necessary but not sufficient here: software as a service would hand the lab someone else's finished application, and this lab has already written its own.
- **B.** Platform as a service, since the lab would be handed a ready-made analysis application to run — The lab already has its analysis software; it is not looking for one to be provided. A platform to develop on also usually constrains how software is built, which is not what a bursty, already-written workload needs.
- **C.** None of the three fit well, since custom-written software cannot run in a rented arrangement — Custom software runs routinely on rented infrastructure; this is one of the most common uses of the cloud in fields with bursty, specialized computing needs.
- **D.** Infrastructure as a service, since it removes the hardware while leaving the software up to the lab — **correct.** Infrastructure as a service supplies the computing, storage and networking while leaving the operating system and everything installed on it to the customer, which is exactly the fit for software the lab already wrote and wants to keep running its own way.

What it actually costs

The upside is real and easy to state. Under a metered model an organization pays for the resources it uses and can scale up or down as it needs to, which gives it control over what is being consumed and unusual transparency about what each part of the infrastructure costs. Owned equipment tells you nothing comparable. A server in a cupboard costs the same on a busy day as on an idle one, and no invoice ever breaks that down for you.

What the chapter refuses to do is claim the metered bill is smaller. Its own words are that there is considerable disagreement over whether moving to the public cloud is ultimately cheaper than maintaining an infrastructure of your own. It also explains why the argument never settles, and that explanation is the most useful thing in this section.

The comparison is unfair in one specific direction. The rented figure is exact, because the monthly cost of a rented server is a number printed on a bill. The in-house figure usually is not, because many organizations do not know what it truly costs to run a comparable server of their own once everything is counted.

The machine itself — the one element nearly every quick comparison includes, usually as a purchase price divided by however many years somebody expects the machine to last.

Software licences — for the operating system, the database engine and anything else the machine has to run, which under an infrastructure service remain the customer's responsibility.

Electricity — counted twice over, because the machine draws power to do the work and the cooling draws power again to remove the heat that work produced.

The facility — the share of floor space, power distribution, cooling plant and physical security that this one machine occupies inside a building somebody has to pay for.

The staff — the fraction of the people who install, patch, monitor, back up and eventually replace it, often the largest of the five figures and the one least likely to have been written down anywhere.

Work the comparison honestly and it may still come out either way. The chapter names one company, the online game developer Zynga, that moved out of a public cloud and back into an infrastructure it owned. Read that as one company reaching one conclusion about its own workload at one moment, not as evidence about the general case — which is exactly what the chapter's own sentence about considerable disagreement concedes.

Two consequences are worth carrying away. Using rented capacity only for periods of peak demand is a genuinely sensible arrangement, and the chapter notes that it adds another layer of complexity to the operation, so somebody has to run it and that somebody costs money too. And moving to a rented infrastructure shifts the cost structure towards operational expenditure, which changes the organization's financial model rather than merely changing where its invoices come from.

The exercise below builds the in-house figure one element at a time, which is what an analyst has to do before any comparison means anything at all. It is a hypothetical sheet: every figure in it is invented for practice.

SPREADSHEET

Build the in-house figure before you compare it

Type one formula for row 2 in each column, then run it; the formula is applied down every row the way a filled-down formula behaves in a real sheet.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Columns: Cost element · Cost per year · In the quick comparison? · Cost per month · Counted per month · Full cost so far, per month

1. Column D: put every element on the same footing as a monthly rented bill by turning the yearly figure into a monthly one.

```
=B2/12
```

Comparisons are only possible between figures covering the same period. A rented bill arrives monthly, so the owned costs have to be expressed monthly before either number means anything next to the other.

2. Column E: show only what a quick comparison would have counted. Where column C says yes, show that element's monthly figure; otherwise show zero.

```
=IF(C2="yes", B2/12, 0)
```

Only the first row survives, which is the whole problem. A comparison built on the purchase price of the machine alone is being made against a rented bill that already includes the licences, the power, the building and the people.

3. Column F: accumulate the full monthly cost as you go down the list, so the last row shows what this one machine truly costs each month.

```
=SUM($B$2:B2)/12
```

The final row is several times the figure the quick comparison produced. That gap is the chapter's point exactly: the rented number is known precisely and the owned number usually is not, so the two are rarely being compared on equal terms.

The chapter's own conclusion is that there are various issues to consider and that each organization must make informed choices about how to harness what the cloud offers while minimizing the drawbacks. That is not a dodge. It is the honest shape of the decision: ten criteria, no single provider that answers all of them well, and a hybrid arrangement waiting at the end of most of the arguments.

Nobody signs up for a runaway bill, a migration they cannot afford, or an outage they have no plan for. They sign up for something quick and cheap, and postpone the questions that would have surfaced all three.

SELF-CHECK**Could you actually run this decision?**

Rate each statement honestly; anything you cannot do yet has a pointer to the exact place in this section to reread.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

- I can retell the chapter's cautionary case and name the three cloud characteristics that turned a stolen credential into a bill of that size. The opening of this section, through to the list of the three characteristics that were doing the damage.
- I can list the five things the chapter recommends to prevent a runaway bill, and say which one has to happen before anything is switched on. The numbered defences immediately after the two cases; the last of the five is the one that has to come first in time.
- I can name the ten criteria for evaluating a provider and turn at least six of them into a question I would actually ask. The table of criteria, questions and weak answers, in the part of the section on what to put to a provider.
- I can explain what a service-level agreement does and does not promise, and why a refund is not the same as compensation. The section on what an agreement actually promises, especially the first two items in the list there.
- I can state the chapter's counterpoint about in-house uptime without it sounding like an argument against ever renting anything. The paragraph after that list, where the chapter compares a missed promise against a poorly managed operation of your own.
- I can say why an organization cannot know exactly where its data physically sit in a public cloud, and what that makes difficult. The section on sharing a building you cannot see, where pooling and auditing are set side by side.
- I can explain lock-in to somebody who assumes moving providers is a weekend of copying. The section on getting out again: different ways of storing data, bandwidth, and charges for transferring data out.
- I can list the five elements of the true in-house cost of one server and say which of them a quick comparison usually leaves out. The list in the closing part of the section, and the sheet that follows it.

What the cloud made possible

Renting infrastructure changed more than the invoice. Once processing, storage and network capacity could be summoned in minutes and dropped again just as quickly, a set of arrangements that used to be impractical became ordinary. This section covers the four the chapter names: software assembled from blocks rather than written as one piece, many small machines pointed at one enormous problem, copies of content kept close to the people asking for them, and a deliberate effort to use less power doing all of it.

One program that does everything

The traditional way to build business software was to write one large program containing every feature it would ever need. The chapter calls this a **monolithic application**: built as a single unit, with the entire application code held in a single code base.

That sounds like a criticism, and at first it is not. A single code base is relatively easy to develop, test and deploy, because there is one thing to build, one thing to check and one thing to install.

The cost arrives later. The very property that makes the application easy to ship makes it inflexible and very difficult to adapt to changing business or user needs, because the part you want to change is bolted to every part you do not.

Two consequences follow from that single code base, and they pull in opposite directions.

Everything ships together — one build, one test run and one installation cover the whole application, which is genuinely simpler than coordinating a dozen separate pieces.

Everything changes together — altering one feature means rebuilding, retesting and redeploying the entire program, so a small business change carries the risk of the whole system with it.

Building out of blocks instead

The alternative is to stop writing one program and start assembling one. The chapter's image is a model built out of Lego blocks: a **modular architecture** means choosing individual building blocks that are best suited for a particular need and connecting them using APIs.

The blocks have names. A **packaged business capability**, usually shortened to PBC, is an individual building block designed to perform a well-specified business function — processing payments, say, or managing orders. On a technical level a PBC is itself composed of smaller blocks called **microservices**, each doing one narrow job.

Because different vendors offer competing blocks for the same function, an organization can pick whichever suits it, an approach the chapter calls **best of breed**. Its own figure shows four such capabilities side by side: managing orders, managing inventory, managing customers, and processing payments.

Why you do not care how the oil gets changed. The chapter's explanation of an interface is the plainest one in the book. You cannot be an expert in everything, so it is probably more effective to have someone change your car's oil for you — the dealership, an independent garage, an oil-change service, or a friend.

All that matters to you is that the service will be provided at the expected level of quality and cost, and you typically do not care if different providers do things differently or use different tools. An API is that same agreement written down for software: what you ask for, what comes back, and silence about everything in between.

The chapter's worked case is Uber, which built its systems as separate components for driver management, passenger management, trip management, billing and payment processing, all connected through APIs. Because the components are separate, each one could be changed relatively easily when the business process behind it changed.

Then comes the caveat, which the chapter does not bury. Assembling an application out of hosted blocks avoids reinventing the wheel, but a modular approach demands tremendous effort and expertise to plan the architecture, to select the right components from hundreds or thousands of available blocks, and to orchestrate and deploy them together.

Modularity moves the difficulty rather than removing it: far less code to write, far more to choose between and connect — and the chapter says that integration can be extremely complex, time-consuming, and well beyond the means of small enterprises.

Three questions on that trade-off, before the section turns to a very different use of many machines.

CHECK YOURSELF

Monoliths, blocks, and what an interface promises

Four options, one best answer; every organization described here is a hypothetical example written for practice, and every company named is one the chapter names, so read all four explanations including the ones you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical distributor runs its whole ordering system as one large program containing every feature it needs. Which pair of consequences does the chapter attach to that design?

- **A.** It is difficult to develop, test and deploy, which is precisely why organizations began assembling software from separate blocks — The chapter says the opposite about building it: because the entire application code sits in a single code base, a monolithic application is relatively easy to develop, test and deploy. The difficulty arrives later, when the thing has to change.
- **B.** It is relatively easy to develop, test and deploy, and correspondingly inflexible and very difficult to adapt to changing business or user needs — **correct.** The chapter grants the monolith its advantage and then names the price. The same single code base that makes the application easy to build and ship makes it inflexible and very difficult to adapt to changing business or user needs.
- **C.** It cannot be hosted on rented infrastructure, because providers run applications that have been split into separate components — Nothing about a single code base prevents it from running on rented infrastructure. The infrastructure service model described earlier in this objective rents an operating system upward and lets the customer run whatever it likes on top.
- **D.** It costs less to run than a modular equivalent, because a single code base consumes less storage and network capacity than many blocks would — Storage and network capacity are not the comparison the chapter draws, and it says nothing about a monolith being cheaper to run. The trade-off it names is ease of building set against the ability to change.

2. A hypothetical clinic buys a component that handles patient billing and connects it to the rest of its systems through an interface. In the chapter's vocabulary, what has it bought, and what is that component built from?

- **A.** A packaged business capability, which on a technical level is composed of smaller blocks called microservices — **correct.** A packaged business capability is an individual building block designed to perform a well-specified business function, such as processing payments or managing orders, and the chapter says such capabilities are composed on a technical level of smaller building blocks referred to as microservices.
- **B.** A microservice, which is composed of several packaged business capabilities working together — The relationship runs the other way around. Microservices are the smaller parts that a packaged business capability is assembled from, not the larger thing assembled out of capabilities.
- **C.** A monolithic application, because billing is a complete piece of software in its own right — A monolithic application contains every feature the system needs inside one code base. A component bought to perform one well-specified function and connected through an interface is the opposite arrangement.
- **D.** A grid, because the work of producing each bill is distributed across many networked machines — Grid computing combines many small computers to attack a single large computation. Producing a bill is ordinary business work, not a calculation that has to be divided across machines.

3. The chapter explains interfaces by comparing them to having someone else change your car's oil. What is that comparison actually teaching?

- **A.** That the cheapest supplier is usually adequate, since the same task produces the same result wherever it is carried out — Price is not what the comparison turns on. The chapter lists a dealership, an independent garage, an oil-change service and a friend as equally acceptable, which is a statement about the agreed result rather than about cost alone.
- **B.** That what matters is the service arriving at the expected quality and cost, while how the provider does the work is not the customer's concern — **correct**. The chapter says all that matters is that the service will be provided at the expected level of quality and cost, and that you typically do not care if different providers do things differently or use different tools. That is exactly what an interface promises between two pieces of software.
- **C.** That specialist work belongs in-house, because a customer cannot judge whether an outside provider did the job properly — The chapter argues the reverse. Its starting sentence is that you cannot be an expert in everything, so it is probably more effective to have the work done by someone who is.
- **D.** That a modular approach removes the need for planning, since each provider takes responsibility for its own block — Planning is precisely what a modular approach demands. The chapter warns that it takes tremendous effort and expertise to plan the architecture, select the right components from hundreds or thousands available, and orchestrate and deploy them.

Many small machines, one enormous problem

Some organizations need more computing performance than any ordinary arrangement can supply. The chapter's examples are all simulations of events that are expensive, dangerous or impossible to stage for real.

Crash and design simulation — car manufacturers such as General Motors' German subsidiary Opel and the Japanese manufacturer Toyota use large supercomputers to simulate automobile crashes and to evaluate design changes for vibrations and wind noise.

Research modelling — facilities such as Oak Ridge National Laboratory use supercomputers to model neutron transport inside nuclear reactors and to study climate change scenarios.

Earth science — other organizations use comparable capacity to simulate earthquakes, where the alternative to a simulation is waiting for the real event and measuring the damage afterwards.

The obvious answer is a supercomputer, and the chapter spends a paragraph explaining why that answer so often fails. Speed alone does not settle it, because some complex simulations take a year or longer to calculate even on a supercomputer.

Cost settles the rest. The fastest supercomputers can cost more than US\$200 million, and the chapter is careful to say that the purchase price does not represent the total cost of ownership, which also carries personnel, facilities, storage and software. An organization needing such a machine only occasionally struggles to justify any of that, so historically it either rented time on somebody else's supercomputer or decided not to solve the problem at all.

Grid computing is the third answer: combining the computing power of a large number of smaller, independent, networked computers — often regular desktop machines — into a cohesive system able to solve problems that only supercomputers could previously solve. Large computing tasks are broken into small chunks, and individual computers complete them.

The line to remember. Grid computing and cloud computing both make use of distributed resources, which is exactly why they get confused. The chapter separates them by what the resources are aimed at: in a grid, the resources are typically applied to a single large problem, whereas a cloud serves many separate customers doing many separate things.

The chapter's exemplar machine, the Summit supercomputer, is described as performing more than 200,000 trillion calculations per second. Read that as an illustration of scale rather than a record, since it has been surpassed several times since the figure was written.

The sequence below is the chapter's argument in order. Reassembling it is the quickest way to see that a grid is a last answer rather than a first one.

SEQUENCE

Why a grid, and how the work gets divided

Put the six stages into the order the chapter's own argument runs, then read why each one sits where it does.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

A hypothetical materials research group needs a simulation that would take far longer than it can wait on any machine it already owns. Two answers get ruled out before the chapter reaches grid computing, and the ruling out is the part worth following, because it explains what a grid is actually for.

1. **The work is stated: one simulation, far too large for the machines the organization already owns to finish in a useful time.** — Grid computing answers a particular shape of problem, so the shape has to be named first. The chapter's own cases are crash simulation, reactor and climate modelling, and earthquake simulation, each of them a single large calculation rather than many small unrelated ones.

2. **A supercomputer is considered, and speed alone turns out not to settle the question, because some complex simulations take a year or longer to calculate even on a supercomputer.** — This is the step most people skip. Buying the fastest available machine does not automatically make the problem tractable, so the organization cannot treat raw performance as the whole answer before it has looked at the size of its own task.
3. **Cost rules the purchase out. The fastest supercomputers can cost more than US\$200 million, and the purchase price does not represent the total cost of ownership, which also carries personnel, facilities, storage and software.** — The chapter separates the sticker price from the running commitment on purpose. A machine also needs people who can operate it, somewhere to put it, storage to feed it and software to run on it, and an organization needing it only occasionally cannot justify any of that.
4. **The historical fallback is named and found wanting: rent time on somebody else's supercomputer, or simply decide not to solve the problem.** — This is the state of affairs grid computing improved on, and the second half of it is the honest one. The chapter says organizations sometimes decided not to solve the problem at all, which is what an unaffordable capability actually costs a research programme.
5. **The large computing task is broken into small chunks, each of which can be completed on its own.** — Divisibility is the precondition, not an afterthought. If the pieces cannot be worked on independently then handing them to separate machines gains nothing, which is why not every heavy calculation is a candidate for this treatment.
6. **The chunks are distributed to a large number of smaller, independent, networked computers, often ordinary desktop machines, and the completed pieces are combined into the finished result.** — This is the definition itself: combining the computing power of many smaller networked computers into a cohesive system able to solve problems that only supercomputers could previously solve. Note that the resources are all aimed at one problem, which is what separates a grid from a cloud.

When the computers are inside a moving object

The hardware section of this module described cyber-physical systems and the chapter's warning that their faults land in the physical world. Its own security box makes that concrete, and it repays reading as a supply-chain argument rather than as a scare story.

Car hacking. A modern vehicle contains a number of control units — essentially small computers — operating and integrating the engine and transmission, airbags, steering and braking, remote keyless entry and much else, all coupled together through a maze of networks. Vehicles also carry Bluetooth for hands-free calling, and sometimes a Wi-Fi hotspot for passengers.

Researchers have repeatedly demonstrated breaking into these onboard systems. The chapter's explanation is not about clever attackers: there is a tremendous amount of programming code in a modern vehicle, most of it from a broad range of vendors, which makes it virtually impossible for a manufacturer to understand every potential vulnerability in what it ships.

The counter-argument is given fairly. A successful break-in typically is not easy, often requiring several researchers several months, so some regard the danger as more hype than reality. The chapter's own verdict is that the truth is likely somewhere in the middle, because others can learn from a successful effort and greatly shorten the learning curve.

The response the chapter reports is collective rather than clever, and it comes in three parts, none of which any single manufacturer can carry out alone.

Standardize the approach — technology and vehicle component vendors work together on common ways of building and securing these systems, so that every supplier is not quietly inventing its own.

Share what gets found — information about vulnerabilities moves between vendors and manufacturers, which is what shortens the gap between a flaw being discovered somewhere and being repaired everywhere.

Secure the supply chain — making supply chains harder to tamper with keeps counterfeit parts out of the vehicle, which is the physical version of the same code-provenance problem.

The transferable lesson for a manager is that a security question about a connected product is usually a question about where the code came from. If most of what runs inside the product arrived from suppliers, then knowing what you shipped is a purchasing and record-keeping problem before it is a technical one.

Keeping the content near the people who want it

Distance costs time. The chapter states it plainly: the larger the geographical distance between a user and the web server hosting some content, the longer that content takes to transmit. Streaming media

makes the delay obvious, but every part of a page pays it.

A **content delivery network** answers that by providing a network of servers in various geographical locations which store copies of particular websites. When a request arrives, the delivery server closest to that user answers it, which speeds delivery significantly and is normally unnoticed by the person browsing. It also saves bandwidth, and offers performance that would otherwise be too expensive for an organization to provide alone.

Content delivery networks have since evolved into **edge computing**, where not only data storage but also processing moves away from a centralized location out to the edges of the network, in order to minimize latency. Reduced network traffic and lower costs come with it, and both matter more as the number of connected devices grows.

Why that delay matters is clearest in the chapter's four examples, which escalate deliberately.

- 1 **Annoying** — higher latency reduces the usability of a home assistant such as Amazon's Alexa, because a pause before every answer turns a conversation into a wait.
- 2 **Impractical** — it makes automated stock trading nearly impossible, since the entire purpose of that arrangement is acting before somebody slower does.
- 3 **Dangerous** — it makes the operation of autonomous vehicles nearly impossible, because a vehicle deciding what to do next cannot wait for a round trip across a continent.
- 4 **Fatal** — the chapter ends with a patient wearing an internet-connected EKG vest monitoring vital functions, where the delay can result in life or death.

Notice what the last two share with the vehicle in the security box. Once computing is embedded in something physical, the network stops being a convenience and becomes part of the safety argument.

Four arrangements are now on the table, along with the interfaces that connect them. Each answers a particular problem, and pairing them is how you keep them apart.

MATCH

Which arrangement answers which problem

Pair each arrangement from this section with the problem it exists to solve; the explanations name the chapter's own reasoning for each pairing.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Modular architecture

An application that has to change often is built as one unit, so every small change carries the whole system with it. The chapter presents modular architectures as the answer to the monolith's inflexibility: choose the blocks best suited to a particular need and connect them, rather than holding every feature in one code base.

Packaged business capability

A well-specified business function such as processing payments would otherwise have to be written from scratch inside your own application. That is the definition: an individual building block designed to perform a well-specified business function, with processing payments and managing orders as the chapter's two examples.

Microservices

Even a single business capability is too large to replace in one piece, so it is assembled from smaller parts each doing one narrow job. The chapter says a packaged business capability is composed on a technical level of smaller building blocks referred to as microservices, which is what makes a capability adjustable from the inside.

Best-of-breed selection

Several vendors offer a block for the same function, and their strengths and prices differ. The chapter notes that you can typically find capabilities offered by different vendors, allowing you to choose the one that best meets current or future needs.

Connecting blocks through APIs

Software written by different people, often at different companies, has to work together without either side knowing how the other is built. This is the oil-change comparison in software form: what matters is the service arriving at the expected quality and cost, not whether the provider does things differently or uses different tools.

Grid computing

A single large problem needs more computing power than the organization can justify buying, and would take too long on the machines it owns. The chapter reaches grid computing only after ruling out a supercomputer on cost and on total cost of ownership, and it stresses that the pooled resources are aimed at one problem.

Content delivery network

Users far from the one server holding the content wait noticeably longer for it to arrive. The chapter names distance itself as the cause, and answers it with servers in various geographical locations holding copies, the closest one delivering the content.

Edge computing

A round trip to a distant central location is too slow for work that has to react immediately. Edge computing is what content delivery networks evolved into: not only storage but also processing moves to the edges of the network in order to minimize latency, which is what a vehicle or a monitoring device needs.

Virtualization

A room full of servers each uses a small fraction of its capacity while drawing power and needing cooling. The chapter gives virtualization as its first green computing tactic, consolidating many underutilized machines onto far fewer and cutting both the energy bill and the carbon footprint.

IT asset disposition

Retired equipment cannot simply go to a landfill, and may still hold user data when it leaves the building. The chapter says drives must be wiped, and describes third-party vendors that wipe them and then either refurbish and resell the equipment or dismantle it to recycle materials and dispose of hazardous waste.

Using less power to do the same work

Energy is the last of the trends, and the chapter is careful to name two separate reasons organizations act on it.

The reputational driver — organizations are increasingly scrutinized for their contribution to societal issues such as climate change, and executives conclude they cannot afford the consequences of inaction for the company's reputation.

The financial driver — efforts of this kind can save money on energy and water use, waste disposal and carbon taxes, and may attract grants, rebates or free technical advice, so they can reach the bottom line rather than only the brochure.

Green computing is the chapter's name for both halves of the response: using computing resources more efficiently to reduce environmental impacts, and using information systems to reduce negative environmental impacts elsewhere. The short version is doing the same work, or more of it, with less.

Four tactics carry most of the weight, and the first is the reason this discussion sits inside a chapter about the cloud.

Virtualization — running several separate computing environments on one physical machine, so that many servers each using a fraction of their capacity are consolidated onto a few well-used ones, which lowers the power bill and the cooling bill together.

Shared provider infrastructure — cloud computing has been argued to reduce energy consumption, because the provider's infrastructure is shared among many users instead of sitting idle in each of their buildings.

Power management — software on individual desktops recovers the energy otherwise wasted by machines left idling or on standby overnight, which stays invisible until somebody measures it.

Printing less — discouraging staff from printing emails and business documents reduces paper waste, and the chapter supports this with an unsourced figure for annual paper use that is not repeated here.

Retiring the equipment is the part organizations forget. Companies cannot simply send old machines to a landfill, so the first decision is when equipment should be retired and the next is what becomes of it.

Old computers must be wiped of all user data before they leave the building. Specialist firms, which the chapter calls IT asset disposition vendors, wipe the drives and then either refurbish and resell usable equipment or dismantle it to recycle valuable raw materials and dispose of hazardous waste properly.

A tension the chapter leaves open. Green computing credits the cloud with lowering energy consumption, because one provider's shared infrastructure replaces many idle private ones. The same chapter describes an enormous new appetite for computing power to train AI models, running on hardware that draws far more than the machines beside it.

Both statements can be true at once, and the chapter never reconciles them. So carry the question with you: when a workload moves to a shared provider, is less energy actually being used, or has the saving simply been spent on work nobody was doing before?

That is the fourth and last of the trends. The cards below set all four side by side, including what each one demands in return, which is the half a proposal usually leaves out.

EXPLORE

The four trends, and what each one asks for in return

Open each card and read all four panels; the third panel is the one worth lingering over, because it names what the arrangement costs rather than what it saves.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Modular architectures · Assembling software rather than writing it

- **The idea in one line:** Building software by choosing individual blocks best suited to a particular need and connecting them through interfaces, in place of one large program holding every feature in a single code base.
- **What it looks like in practice:** A system whose ordering, inventory, customer and payment functions are separate packaged business capabilities, each chosen from whichever vendor suits it best, and each replaceable without disturbing the others.
- **What it demands in return:** Tremendous effort and expertise to plan the architecture, to select the right components from hundreds or thousands of available blocks, and to orchestrate and deploy them so they work as one thing.
- **The limit worth knowing:** The chapter says the integration can be extremely complex and time-consuming, and well beyond the means of small enterprises, so modularity is not automatically the cheaper road for a small organization.

Grid computing · Many small machines, one large problem

- **The idea in one line:** Combining the computing power of a large number of smaller, independent, networked computers into a cohesive system able to solve problems that previously needed a supercomputer.
- **What it looks like in practice:** A crash, reactor or earthquake simulation broken into small chunks, each completed by an ordinary machine somewhere on the network and returned to be combined with the rest.

- **What it demands in return:** A problem that genuinely divides into independent chunks, and a population of machines whose spare capacity can be borrowed without disrupting whatever else those machines are doing.
- **The limit worth knowing:** It is not a general substitute for renting infrastructure. The resources in a grid are typically applied to a single large problem, whereas cloud resources serve many separate customers doing many separate things.

Content delivery and edge computing · Moving the work closer to the person

- **The idea in one line:** Keeping copies of content on servers in several geographical locations so the nearest one answers a request, and then moving processing as well as storage out towards the edges of the network.
- **What it looks like in practice:** A page or a video that arrives promptly for viewers on three continents because each of them is served from a machine a few hundred miles away, a process normally unnoticed by the viewer.
- **What it demands in return:** Somewhere to hold the copies and a way to keep them current, plus the judgement to decide which work has to happen near the user and which can afford a round trip to the centre.
- **The limit worth knowing:** Distance is the constraint it removes, not capacity. An application that is slow because of its own design, or because its origin is overloaded, is not repaired by holding copies nearer to anybody.

Green computing · Doing the same work with less power

- **The idea in one line:** Using computing resources more efficiently in order to reduce environmental impacts, and using information systems to reduce negative environmental impacts elsewhere in the organization.
- **What it looks like in practice:** Consolidating underused servers through virtualization, shifting work to a shared provider, managing power on idle desktops, printing less, and routing retired equipment through a specialist rather than a skip.
- **What it demands in return:** A decision about when equipment is retired, and a process that wipes user data from every machine before it leaves the building, whether it is resold, refurbished or dismantled.
- **The limit worth knowing:** The chapter credits shared cloud infrastructure with lower energy use while describing an enormous new demand for computing power to train models, and it does not reconcile the two claims.

One last check across the whole section, mixing the four trends with the security box that sits between them.

CHECK YOURSELF**Grids, distance, vehicles, and watts**

Four questions across the whole section; every organization described is a hypothetical example written for practice, and each explanation says what the option you did not pick would actually describe.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical university research group splits one very large simulation into thousands of small pieces and has hundreds of ordinary networked desktop machines complete them. How does the chapter separate this from cloud computing?

- **A.** Both use distributed resources, but the resources in a grid are typically applied to a single large problem, whereas a cloud serves many separate customers and purposes — **correct**. The chapter says that similar to cloud computing, grid computing makes use of distributed resources, but that in contrast to cloud computing the resources in a grid are typically applied to a single large problem.
- **B.** A grid uses machines the organization owns itself, whereas a cloud uses machines belonging to somebody else — Ownership is not the line the chapter draws, and a grid can be assembled from machines belonging to several organizations or contributed by volunteers. What the machines are aimed at is the distinguishing feature.
- **C.** A grid is a way of storing data across many machines, whereas a cloud is a way of processing it — Grid computing is defined as combining computing power to solve problems that previously required a supercomputer, which is processing rather than a way of storing data across machines.
- **D.** There is no real difference between them, because grid computing is simply the earlier name for what is now sold as cloud computing, and the change of word marked a change of marketing — The chapter sets the two side by side precisely in order to separate them, and it defines cloud computing separately as renting configurable resources on demand with minimal management effort.

2. The chapter's security box argues that vehicle manufacturers cannot know every vulnerability in their own vehicles. What reason does it give?

- **A.** Because attackers work faster than manufacturers can test, so any list of known vulnerabilities is out of date before it is finished — The chapter says the opposite about speed. A successful break-in typically is not easy and often requires several researchers several months, which is why some observers regard the danger as more hype than reality.
- **B.** Because a modern vehicle contains a tremendous amount of programming code, most of it supplied by a broad range of outside vendors — **correct**. The chapter states that there is a tremendous amount of programming code in modern vehicles and that most of this code comes from a broad range of vendors, making it virtually impossible for manufacturers to understand all potential vulnerabilities.

- **C.** Because the control units in a vehicle are not connected to one another, so a fault in one cannot be traced from another — The box describes the control units as coupled together through a maze of networks, alongside Bluetooth for hands-free calling and sometimes a passenger hotspot, so connection is part of the problem rather than absent from it.
- **D.** Because vehicles are tested for safety rather than for security, and that testing is carried out by regulators rather than by manufacturers — Testing regimes are not what the box discusses. Its recommendation is that component vendors standardize approaches, share information and secure supply chains against tampering and counterfeit parts.

3. A hypothetical museum streams video to visitors on three continents from a single server in one country, and distant viewers wait noticeably longer for it to start. Which arrangement does the chapter offer, and why does it work?

- **A.** A content delivery network, because copies held on servers in several geographical locations let the nearest one answer, which removes the delay caused by distance — **correct.** The chapter says the larger the geographical distance between a user and the web server hosting the content, the longer transmission takes, and that a content delivery network reduces this latency with servers in various geographical locations holding copies, the closest one delivering the content.
- **B.** A private cloud, because owning the infrastructure gives the museum direct control over how quickly the video is delivered — A private cloud changes who owns and controls the machines, and the chapter values it for customizability and control. It does nothing about the distance between one location and a viewer on another continent.
- **C.** Grid computing, because dividing the video among many small machines lets them deliver the pieces in parallel — Grid computing applies many machines to a single large computation. Delivering a video to a distant viewer is a transmission problem, not a calculation that can be divided into chunks.
- **D.** A faster connection at the museum's own server, because the delay comes from the capacity of that connection rather than from distance — Capacity at the origin can matter, and the chapter treats bandwidth as a real constraint elsewhere. The cause it names for this particular delay is the distance itself, which is why the remedy is to move copies closer rather than to widen one connection.

4. A hypothetical charity runs forty servers in its own building, each using a small fraction of its capacity, and wants to cut both its power bill and its environmental impact. Which tactic does the chapter name first, and what does it do?

- **A.** Power management software, which stops individual machines idling overnight and so reduces the load on every server in the room — Power management software is one of the chapter's tactics and does recover energy wasted by machines left idling or on standby overnight. It is aimed at individual desktops rather than at a room of servers each running well below capacity.

- **B.** Virtualization, which runs several separate computing environments on one physical machine so underused servers can be consolidated onto a few well-used ones — **correct**. The chapter says organizations can save large amounts of money on power and cooling by using virtualization to replace many individual servers with far fewer machines, and notes that computing resources in organizations are often very much underutilized.
- **C.** Printing less, since paper waste is the largest environmental cost that an organization of this kind carries — Printing less is a genuine tactic on the chapter's list, but it addresses paper waste rather than the power and cooling drawn by underused servers, and the chapter makes no claim that paper is the largest cost.
- **D.** Disposing of the least used servers immediately, since equipment that is doing almost nothing still draws power and still has to be cooled, and costs more to keep than it would cost to replace — Retirement is a decision the chapter treats carefully rather than quickly. Companies cannot just send retired equipment to a landfill, drives must be wiped of user data, and usable machines are refurbished or dismantled for recycling and safe disposal.

Where the data itself lives: databases and SQLite

Everything in this module so far has been about the container: the machines, the software that drives them, the wire between them, and the room they sit in. None of it says anything about the thing all of it exists to hold. An organization's records are the part that cannot simply be bought again after a bad day. This supplement follows the data itself, from a folder of files to a stored set of records you can question.

How this supplement relates to the chapter. The chapter keeps brushing against databases without ever stopping on them. It defines a server as a machine that provides others with access to databases. It says web servers often assemble a page by pulling data out of a database at the moment the page is requested. It counts database management systems among the costs that growing data drives up, and then names them again as a licence the platform service model quietly removes. This section gathers those scattered mentions into one place and carries them as far as reading a stored table and asking it a question. It is a labeled supplement rather than a textbook objective, and it prepares for the database work the course does separately in SQLite rather than replacing it.

What the chapter already told you about data

Four sentences, spread across four different sections of the chapter, hold most of what a manager needs. Read together they make an argument the chapter itself never quite assembles.

A server is defined by the fact that it serves data — the chapter describes that class of computer as one built to give many users at once their websites, their applications and their access to databases, which is why it is optimized for connections rather than for one person's screen.

Web pages are often built out of a database as you ask for them — the page that arrives was not sitting on a disk in that form, because a program queried stored records and assembled the page around the result.

Keeping data is a running cost rather than a purchase — the chapter says that capturing more data demands more storage space, more powerful computing hardware, and database management systems to manage and analyse whatever has been kept.

Renting a platform removes the licence, not the design — on the platform service model the provider supplies and updates the database software, and the chapter names that licence as one of the things the customer no longer has to buy.

Put those four together and the shape appears. Data is kept once, on a machine built to serve it; other programs ask that machine for what they need; and the arrangement costs money continuously, in space, in electricity and in software. What the chapter never says is why data is kept that way rather than in files, which is where this supplement starts.

Why a pile of files eventually fails

The earlier section on system software and storage described files and folders: a file is a block of data holding specific content, and folders group related files into a tree. For documents that works beautifully. It fails, quietly and expensively, the moment several people have to record the same kind of fact.

Take a hypothetical community music school that lends instruments to its learners. Three members of staff each keep a spreadsheet of loans. Nothing about that is unreasonable: every file is small, readable, and belongs to the person who made it. The trouble starts with the first question that spans them.

The same fact ends up stored more than once — a learner's name and address are typed into whichever file the person on shift happened to open, so a correction made in one of them never reaches the others.

Nothing decides which copy is authoritative — when two files disagree about how many instruments are out, no rule settles it, and the disagreement is resolved by whoever sounds more certain rather than by evidence.

The shape of a record becomes a matter of habit — one file writes a date as a word and another as digits, one gains a column in March that the others never get, and a heading that meant one thing in the autumn means something slightly different by the spring.

Questions that span the files cannot really be asked — counting a year of loans means opening every file and adding by hand, which is work nobody repeats often enough to notice when a total comes out wrong.

A **database** is the answer to exactly that failure, and it is worth defining in those terms rather than as a piece of software you install. A database is one stored, shared set of records that everyone reads and writes, held in a declared shape, and asked questions of instead of copied. The program that keeps it is a **database management system**: it enforces the shape, lets many people work at once without overwriting each other, and answers the questions.

The difference from a file is that shape. A file is a block of content whose meaning lives in the head of whoever made it. A **table** is a declared shape: named columns, each holding one kind of value, and one row for each thing the table is about. Because the shape is declared rather than assumed, the system can refuse a record that does not fit — a loan with no borrower, or a second row claiming an identifier that already belongs to another row.

CHECK YOURSELF

Files, records, and what a database actually changes

Four options, one best answer; read every explanation, including the ones for options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. Two of the hypothetical music school's spreadsheets disagree about how many instruments are out on loan. A volunteer proposes settling it by declaring one of the three files the master copy. What is the strongest objection?

- **A.** Nothing is wrong with the idea, because declaring a master copy is exactly what a database does, only without needing any software — A database does something stronger than nominate a favourite copy: it removes the other copies, so there is nowhere else for a loan to be recorded. A nominated file is a convention, and a convention only holds while everybody remembers it.
- **B.** The other two files still exist and are still being edited, so the totals part again the first time somebody records a loan in the wrong one — **correct**. The problem is not which file is trusted but that three places can accept a record. As long as the other two remain open on somebody's screen, the school is one distracted afternoon away from the same disagreement.
- **C.** Spreadsheets cannot store dates reliably, so the same disagreement would return in a different form within a term or two — Spreadsheets handle dates perfectly well. The inconsistency in the files came from three people typing them differently, which is a question of who may write where rather than of what the software can store.
- **D.** A single master file would be too large for one person to keep up to date, so the work has to stay divided across the three files — Size is not the difficulty here; a year of instrument loans would be a small file by any measure. The cost of the arrangement is inconsistency between copies, and dividing the work is what created it.

2. In the school's loan records, every loan carries a member identifier rather than the member's name. Why is it arranged that way?

- **A.** An identifier occupies less storage space than a name, and storage is the constraint that shapes how tables are designed — The space saved is trivial and would not justify the extra table. Facts are separated for correctness, and the join you pay for afterwards is the price of that correctness rather than a saving.

- **B.** A query cannot compare text, so a numbered identifier is required before two tables can be matched at all — Queries compare text quite happily, and plenty of tables are matched on text columns. The trouble with matching on a name is that two members may share one, not that the language cannot do it.
- **C.** The name is a fact about the member, so it is stored once on the member's row and every loan simply refers to it — **correct**. One fact, one home. Correcting a spelling on the member's row makes every loan correct at once, because no loan ever contained a copy that could fall out of step with it.
- **D.** Typing a name onto every loan would take longer, and saving that typing is the reason the tables were split — It does save typing, but that is a side effect rather than the reason. Even with perfect typists, copying the name onto every loan creates as many chances to disagree as there are loans.

3. The chapter says web servers often assemble a page by retrieving data from a database as the request arrives. What follows from that for someone managing the site?

- **A.** The finished page already sits on the server's disk in the form the visitor sees, and the database holds only a spare copy of it — For a page assembled on request there is no such stored copy: the layout is stored, but the prices, the availability and the names are fetched and dropped in as the page is built for that visitor.
- **B.** What the visitor sees is produced from stored records at the moment of asking, so a slow or unavailable database is a slow or unavailable site — **correct**. The page is a view of the records, made on demand, so the quality of the records is the quality of the page and the availability of the store is the availability of the site.
- **C.** The site cannot be copied to servers nearer the visitor, because the records behind it can exist in one location only — Copies of content can certainly be pushed closer to visitors, which is what the earlier section on content delivery networks described. What travels is usually the parts that do not change per visitor.
- **D.** The database matters for internal reporting, while the public site is a separate system fed by an overnight export — This describes one possible arrangement rather than the one the chapter names. When the page is assembled from a query at request time, the public site and the store are the same system by definition.

How to read a stored table

A table has only two directions and both of them are ordinary, which is why a table is easier to read than most people expect on first meeting one.

A row is one of the things the table is about — one loan, one learner, one payment, recorded once, with a value in every column that applies to it.

A column is one kind of fact — every value in it means the same thing and is the same kind of thing, which is exactly what makes it possible to filter on it, sort by it, or total it.

A key is the column that identifies the row — a value no two rows may share, so that one record can be referred to from elsewhere with no ambiguity about which one is meant.

A relationship is a column that names a row in another table — a loan carries a learner identifier rather than a learner's name, so the name is stored once and borrowed by every loan that needs it.

That last one is the move the files could not make. Correct a name on the learner's own row and every loan is correct the same instant, because no loan ever held a copy of it in the first place.

Asking a table something is then three separate decisions, and keeping them separate is most of the skill:

- 1 **Which rows?** Every condition in the question — still out, this term, longer than a month — narrows the set of rows before anything else is allowed to happen.
- 2 **Which columns?** A question rarely wants a whole record. Naming the columns you actually need is what turns a screen of data into an answer somebody can read aloud.
- 3 **Rows, or a summary of rows?** How many, how much and on average are not questions about rows at all. They collapse many rows into a single value for each group.

The language that writes those decisions down is **SQL**, the structured query language, and it is broadly the same language whichever database system you meet. Each decision has its own word, and reading them in this order makes a query far less mysterious than it looks:

FROM names the table the rows come from, and so decides what is being counted before anything else in the query happens at all.

WHERE keeps only the rows that satisfy a condition, and because it runs before any total exists it silently determines every number that follows.

SELECT names the columns that come back, and a calculation such as sessions booked minus sessions attended can be one of them, given a name of its own.

GROUP BY collapses the surviving rows into one row for each distinct value, which is what lets a count or a sum report per category instead of per record.

HAVING filters those grouped rows, and it exists because WHERE has already finished its work by the time any total has been calculated.

ORDER BY and LIMIT sort the result and keep only the first few rows, which together answer anything phrased as the busiest, the longest, or the worst three.

Here is one whole query over a table of lesson sign-ups: `SELECT Learner, Fee FROM signups WHERE Term = 'Spring' ORDER BY Fee DESC`. Read it as four decisions in a row. The rows come from the sign-up table; only spring sign-ups survive; two columns come back, the learner and the fee; and the result arrives dearest first. Nothing in it names a particular learner, which is why the same query still works after another hundred sign-ups arrive.

Before writing one yourself it is worth being deliberate about the order of thinking, because a query that runs is not the same thing as a query that answers the question you were asked.

SEQUENCE

From a business question to a query you can trust

Put the five steps into the order that produces an answer to the question actually asked, then read why each one sits where it does.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

The hardest part of querying is not the words. It is resisting the urge to start typing before the question has been stated precisely enough to be answered. These five steps are the same discipline the earlier modules asked for when turning evidence into a claim.

- 1. State the question in ordinary business words, and name the decision the answer feeds.**
“Which instruments are out right now, and who has them?” is a question; “have a look at the loans” is not. — A query can only be as precise as the sentence behind it. Naming the decision the answer serves is what stops you producing a screen of rows that is technically correct and useless, because nobody can act on it.
- 2. Find where the fact is stored: which table has one row for each thing the question is about, and which other table holds the extra detail the question also mentions.** — The question mentions instruments and people, and those are two different kinds of thing. The table you count from is the one whose rows are the units being asked about, which here is loans rather than members.
- 3. Decide which rows. Every condition in the question — still out, this term, longer than a month — has to be written down as a filter before anything is counted.** — The filter runs before any total exists, so whatever it excludes is missing from every number that follows. Most confidently wrong answers were produced by a filter that nobody went back and read.

4. **Decide what comes back: the rows themselves with the columns the question mentions, or one summary value for each group when the question asks how many, how much, or how long on average.** — Choosing rows and choosing columns are separate decisions, and summarising is a third. A question about loans per town does not want the loans at all; it wants one line per town, which is a different result rather than a different layout.
5. **Run it, then read the result back against the original question and satisfy yourself that it answers that question rather than a neighbouring one.** — A query that runs is not a query that is right. The commonest failure is a clean answer to a slightly different question — every loan rather than every loan still out — and only rereading the question catches it.

Asking one table a question

The table below records lesson sign-ups at that same hypothetical music school. Everything in it is invented for practice, every person named included. Each row is one sign-up, carrying the learner, the instrument, the level, the term, how many sessions were booked, how many were attended, and the fee.

Write a query for each question and run it. Any query that returns the right answer is accepted rather than only the wording that happened to be expected, so there is more than one way through. When a query cannot run, the page says why in plain language, which teaches more than an empty result would.

DATABASE

Question one table

Write a SELECT for each question and run it. Any query that returns the right answer is accepted, so there is more than one correct way through.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

signups — RecordID, Learner, Instrument, Level, Term, Sessions, Attended, Fee (10 rows)

1. List the learner and the instrument for every sign-up at the beginner level.

```
SELECT Learner, Instrument FROM signups WHERE Level = 'beginner'
```

SELECT chooses the columns and WHERE chooses the rows. They are two independent decisions, and confusing them is the most common early mistake: asking for fewer columns never removes a row.

2. Show every sign-up with the learner, the instrument, and the number of sessions missed, with the worst attendance first.

```
SELECT Learner, Instrument, Sessions - Attended AS Missed FROM signups ORDER
```

BY Missed DESC

Missed sessions are nowhere in the table; they are worked out as the query runs. Naming the calculation with AS is what lets ORDER BY refer to it, and it also gives the reader a heading that says what the number means.

3. Which instruments does the school actually teach? List each one once.

```
SELECT DISTINCT Instrument FROM signups
```

DISTINCT answers a question about the set of values rather than about the records. It is the smallest example of the difference between listing rows and describing them, which is what grouping does at greater scale.

4. Name the three most expensive sign-ups, showing the learner, the instrument and the fee.

```
SELECT Learner, Instrument, Fee FROM signups ORDER BY Fee DESC LIMIT 3
```

Questions phrased as the worst three or the busiest five are a sort followed by a cut, not a filter. Nobody could have written a WHERE for this, because the threshold depends on the data rather than on the question.

Notice what the second question needed. The number of missed sessions is not stored anywhere; it is worked out from two columns at the moment you ask. Storing it as well would create a third value that can disagree with the other two, which is the trouble with the files all over again in miniature. Facts that can be derived are generally derived rather than kept.

Questions no single row can answer

Everything so far returned rows that already existed. Two kinds of question do not work that way, and between them they cover most of what management actually asks for.

Summaries ask about groups rather than records — how many loans per instrument, or how long the average loan runs, produce one line for each group and no underlying rows at all.

Joins ask about facts that were deliberately kept apart — who the borrower is and what they borrowed are two different kinds of fact living in two tables, and a join matches them on the value they share.

Filters can apply before or after the grouping — keeping only the loans still out is a condition on rows, while keeping only the instruments borrowed more than once is a condition on a total that does not exist until the grouping has finished.

The two tables below belong to the same hypothetical school, and again every member and every loan in them is invented. One holds a row for each member, with the town they live in and the date they joined; the other holds a row for each loan, carrying a member identifier rather than a name.

DATABASE

Two tables, and questions about groups

Each question needs either a join, a grouping, or both; run each query and compare the result with what the question actually asked for.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

members — MemberID, Name, Town, JoinedOn (6 rows)

loans — LoanID, MemberID, Instrument, Status, Weeks (10 rows)

1. Which instruments are out at the moment, and who has them? Show the member's name and the instrument.

```
SELECT m.Name, l.Instrument FROM loans l JOIN members m ON l.MemberID =  
m.MemberID WHERE l.Status = 'out'
```

A join puts related rows from two tables side by side on a matching value. Notice that the name is stored once and borrowed by the query, which is precisely why correcting it in one place corrects every answer.

2. How many times has each instrument been loaned? Show the most borrowed first.

```
SELECT Instrument, COUNT(*) AS Loans FROM loans GROUP BY Instrument ORDER BY  
Loans DESC
```

Grouping turns a list of events into a description of them. The result has one row per instrument and none of the original loans, which is the clearest illustration that a summary is a different answer rather than a tidier one.

3. Which instruments have been borrowed more than once? Show the instrument and the number of loans, busiest first.

```
SELECT Instrument, COUNT(*) AS Loans FROM loans GROUP BY Instrument HAVING  
COUNT(*) > 1 ORDER BY Loans DESC
```

WHERE filters rows and HAVING filters groups. Putting the count condition in WHERE fails because at that stage nothing has been counted yet, and the distinction becomes intuitive once you read a query in execution order rather than in writing order.

4. How many loans has each town produced? Show the busiest town first.


```
SELECT m.Town, COUNT(*) AS Loans FROM loans l JOIN members m ON l.MemberID = m.MemberID GROUP BY m.Town ORDER BY Loans DESC
```

This answer exists in neither table. Nobody recorded a town against a loan, yet the question is answerable because the two facts were stored in a way that lets them be brought together on demand.

5. For loans that have already come back, how long did they run on average?

```
SELECT AVG(Weeks) AS AvgWeeks FROM loans WHERE Status = 'returned'
```

The filter decides the answer here as much as the function does: including loans that are still out would average an unfinished length against finished ones. An average is only as honest as the set of rows underneath it.

The last questions are the ones worth sitting with. Neither answer exists in any row: the count for each town is assembled at the moment of asking, out of records that were never entered with towns in mind. That is the practical difference between a set of files and a database — files answer the questions they were built for, and a database answers questions nobody had thought of when the records were written.

What renting takes off your hands, and what stays yours

The service models earlier in this module decide who installs and patches the database software. They do not decide who is responsible for the data, and that confusion is what this section exists to prevent.

- 1 **Infrastructure as a service.** You are given processing, storage and networking, and everything above that is yours: the operating system, the database software, its licence, its updates and its backups.
- 2 **Platform as a service.** The provider supplies and updates the platform, database management system included, so the licence and the patching stop being your problem. The chapter names that saving explicitly.
- 3 **Software as a service.** You use a finished application, so you choose settings and you own the records inside it, and nothing underneath is visible to you at all.

What none of the three takes away is the design. Which records exist, how they relate to one another, what has to be unique, what a column is allowed to contain, and which questions the store must be able to answer are decisions about the organization rather than about the software. A rented platform will hold whatever shape you give it, including a poor one, and will hold it very reliably indeed.

CHECK YOURSELF**Who is responsible for the data**

Each question is about where responsibility sits rather than about syntax; read every explanation before moving on.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical town archive moves its catalogue onto a rented platform, where the provider supplies and updates the database software. Which statement is accurate?

- **A.** The archive no longer has to decide what counts as a catalogue record or which questions the catalogue must be able to answer, because a rented platform arrives with both of those already settled — A platform supplies a general-purpose place to keep records. It has no view about what an archive record ought to contain, and it will accept an incoherent design as readily as a good one.
- **B.** The archive keeps paying the database licence but gives up control of how the catalogue is designed — This has the trade-off backwards. The platform model is defined by the licence and the platform maintenance moving to the provider, and design was never something the provider held.
- **C.** The archive stops buying and patching the database software, and still decides which records exist, how they relate, what must be unique, and which questions the catalogue has to answer — **correct.** The licence, the patching and the underlying machines move to the provider; the meaning of the data does not move anywhere, because it was never a technical question in the first place.
- **D.** A rented platform cannot host a catalogue, because a catalogue needs database software the archive installs on its own machines — Hosting a catalogue is ordinary platform work. Nothing about a catalogue requires locally installed software, which is why so many organizations run this kind of store on rented platforms.

2. A hypothetical youth theatre runs its booking system on the infrastructure service model. Who is responsible for the database software it depends on?

- **A.** The theatre, because that model supplies processing, storage and networking, and everything from the operating system upward is the customer's to install, licence and patch — **correct.** The chapter describes that model as providing only the basic capabilities of processing, storage and networking, which is also why it says the customer keeps the most control and keeps responsibility for managing software licences.
- **B.** The provider, because supplying storage necessarily includes supplying the software that organizes what is stored — Storage in this sense means space and the machines that hold it. Organizing what goes into that space is exactly the work the customer takes on in exchange for the extra control.

- **C.** Neither, because on that model the database runs as a shared service used by every customer on the same machines — That describes a shared service somebody else operates, which is a different model. Here the customer receives raw capacity and decides what runs on it.
- **D.** The theatre for the software and the provider for the data held inside it, which is the division of responsibility that this service model was designed around and the reason it costs less than the alternative — The split is the other way round in one half and wrong in the other: the customer runs the software, and the data inside it is the customer's responsibility on every one of the three models.

3. A hypothetical dental practice proposes recording far more detail about every appointment so that it can analyse patterns later. Which cost picture matches what the chapter says?

- **A.** A single one-off cost, since the records are captured by staff who are already there and stored on machines the practice already owns — The capture is close to free and the keeping is not. Data recorded once has to be stored, backed up, moved and eventually archived for as long as the practice keeps it, which is a commitment rather than a purchase.
- **B.** A rise in storage only, because processing and software costs are fixed regardless of how much data is kept — Storage is the visible cost and rarely the only one. Analysing a much larger set of records needs hardware that can work through it, which is why the chapter names the two together.
- **C.** A rise in bandwidth only, because the extra detail has to travel between the practice and wherever it is stored — Bandwidth does grow with richer content, and the chapter makes that point about video. It is one strand of the story rather than the whole of it, and the practice would still be paying to keep and process everything it sent.
- **D.** More storage space, more capable hardware to process the data, and database software to manage and analyse it — three running costs from one decision — **correct.** The chapter states it plainly: capturing more data requires ever more storage space, ever more powerful computing hardware, and database management systems to manage and analyse it. Deciding to record something is deciding to pay for it repeatedly.

The hand-off to SQLite

Everything above is about reading. Building is the other half, and it is where the course goes next.

SQLite is a database system that keeps an entire database in a single ordinary file and runs inside the program using it, with no separate server to install or keep running. That makes it an unusually honest place to learn, because nothing between you and the data is hidden behind an administrator.

The queries on this page are the same language you will write there. What changes is that you will also decide what the tables are: which things deserve a table of their own, which column identifies a row, and which questions the design has to answer on the day somebody finally asks. Creating a database application in SQLite is the course's own separate work, and this supplement is the bridge to it rather than a substitute for it.

The container can be rented; the meaning cannot. Somebody in the organization still has to know what a record is, which is why a section about tables belongs in a module about machines.

WHAT TO CARRY OUT OF THIS SECTION

One stored set of records, one home for each fact, and questions asked of the store rather than answers copied out of it. If you can say which rows a claim rests on and which columns it reads, you can defend it. If the claim arrived as a file somebody mailed you, all you can do is repeat it.

The internet as a business tool

Everything in this section has already appeared somewhere in this module: the internet, the web, intranets and extranets, the copies of pages held near users, the interfaces that let one application ask another a question. What the chapter never does is set those pieces side by side and ask the plain business question — what does a company actually use all of this for? The answer is three things, and telling them apart comes down to one decision, taken by a person rather than by a machine.

How this supplement relates to the chapter. Chapter 3 teaches each of these parts where it belongs, as infrastructure. This supplement reassembles them around the audience being served, because that is the shape the course's own objective takes: using internet technology for electronic commerce, for communicating with employees, and for working with business partners. It is a labeled application supplement rather than a textbook learning objective, it adds no claim the chapter does not already make, and every organization used for practice in it is hypothetical and invented.

Three jobs, one set of parts

Begin with the fact that makes this genuinely confusing rather than merely new. A browser asking a server for a page behaves in exactly the same way whether the person reading it is a shopper, a member of staff, or a supplier's dispatcher. Same request, same protocol, same page arriving. The technology has no opinion about who is at the other end of it.

What differs is who is authorized, and that is a decision somebody in the organization has to take before any system can enforce it. The chapter's comparison of the internet, an intranet and an extranet, which you met earlier in this module, separates the three by focus, content, users and access rather than by what they are built from. Read that table again and it turns out to be a table of audiences.

Those audiences are the three jobs, and they are worth naming plainly before anything is built for them.

Selling to customers — general, public content that anyone with an internet connection may read, which is the arrangement the chapter calls the internet, and which almost every organization now depends on to take money at all.

Communicating with employees — specific, corporate and proprietary content restricted to authorized employees, which is the arrangement the chapter calls an intranet, and which quietly replaced an enormous amount of paper.

Working with business partners — communications between two or more firms doing business together, restricted to authorized partners, which is the arrangement the chapter calls an extranet and carries over the public internet.

Three audiences, one technology, one decision. Everything that follows is a consequence of that, including the expensive mistakes at the end of the section.

Selling to customers: almost any business is an e-business

The chapter states it without qualification: today, almost any business can be considered an e-business. That sentence is stronger than it first looks, because it does not say almost any business sells online. It says the business now runs on its data being reachable — and once that is true, the wire and the server are part of the trading position whether or not one item is ever bought on the site.

A hypothetical example makes the range obvious. A veterinary practice that publishes nothing but its address, its opening hours and a booking form has still put its bookings on the internet. When that page fails to load on a Monday morning, appointments are not made and the money is not taken. Data are the lifeblood of almost all organizations, so reliably accessing them is a key concern, which is the chapter's reason for caring about availability at all.

Four demands follow from that, and every one of them was taught earlier in this module as a piece of infrastructure rather than as a commercial requirement. Assembled, they are what selling over the internet actually asks of a company.

Answering while the customer is still there — operational data for a site where response time is of the essence belongs on fast disk-based storage, and on flash-based storage where access speed is of crucial importance, because a page that arrives late has already lost the sale.

Surviving the busiest fortnight of the year — firms engaged in or supporting business-to-consumer electronic commerce meet a large spike in demand before the December holidays, and the capacity that felt generous in March is what decides how that fortnight goes.

Closing the distance — the greater the geographical distance between a user and the server holding the content, the longer that content takes to arrive, which is the delay a network of servers holding copies in several locations exists to remove.

Being available at all — organizations that depend on trading electronically strive for the highest availability of their hardware, storage and networking components, and the five-nines target the chapter describes leaves just over five minutes of downtime in a year.

Notice what kind of decisions those are. Not one of them is a marketing decision, and not one of them can be taken after the campaign has been booked. For a business selling over the internet the shopfront *is* the infrastructure, which is the whole of the chapter's argument for why a manager should care about plumbing.

CHECK YOURSELF

Selling over the internet, and what it demands underneath

Four options, one best answer; read every explanation, including the ones for the options you did not choose.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. A hypothetical veterinary practice takes no payments on its website and sells nothing there. Its owner says the practice is therefore not an e-business and does not need to think about availability. Which reading matches the chapter?

- **A.** Almost any business can be considered an e-business, so the real question is what happens to the practice on a morning when its data are not reachable — **correct**. The chapter says that today almost any business can be considered an e-business, and that because data are the lifeblood of almost all organizations, reliably accessing those data is a key concern rather than an optional refinement.
- **B.** The owner is right, because being an e-business means selling goods or services over the internet and the practice does neither — This reads e-business as a synonym for online retailing, which is narrower than the chapter's claim. The practice's bookings, records and opening information already depend on the same infrastructure a retailer's orders depend on.
- **C.** The practice becomes an e-business only once it accepts card payment on the site, since a completed transaction is what makes trading electronic — Payment is one thing a site can do, not the thing that makes a business dependent on the internet. A booking form that stops working costs the practice appointments long before any card is presented.
- **D.** The distinction does not matter, because availability targets apply to organizations large enough to run a data center of their own — Availability is about whether the organization can reach and be reached, and the chapter attaches it to being data-intensive rather than to owning a facility. A small practice renting everything still loses the morning when the pages do not load.

2. A hypothetical ticket seller's pages take several seconds to appear while a customer sits waiting, and the delay is in reading the records the page is built from. Which of the chapter's storage decisions is at stake?

- **A.** Archival storage, since these records are being kept for the long term and it is the archive the customer is really waiting on while the assistant apologizes for the delay — Archival data are long-term copies held for compliance and reporting, typically on tape, where data sit sequentially so access is slow and not quickly searchable. That is the wrong medium for anything a person is waiting on.

- **B.** Backup storage, since the recovery copy is what would be read if the main records were slow to answer — Backup data are short-term copies for recovering from a disaster, and the chapter's reason for keeping them on hard drives is quick recovery. They are not the copy a page is built from during normal trading.
- **C.** Operational data belongs on fast disk-based storage where response time is of the essence, and on flash-based storage where access speed is of crucial importance — **correct.** The chapter puts operational data on disk-based media precisely because hard drives offer high access speeds where response time matters, and names an e-commerce site as its example, with flash used where speed is crucial.
- **D.** None of these, because the delay a customer experiences is decided by bandwidth rather than by where the records are kept — Bandwidth is the capacity of the pathway, and widening it does not help when the time is being spent reading a record before anything is sent. The two delays have different causes and different remedies.

3. A hypothetical gift retailer's site is slow for two separate reasons: everybody arrives at once in the fortnight before the holidays, and customers on another continent wait noticeably longer than customers at home. Which pairing matches the chapter's account?

- **A.** Both are capacity problems at heart, so one larger server at the retailer's own site answers the seasonal slowness and the overseas delay together, and no second arrangement is needed — A larger server does nothing about distance. The overseas customer is waiting on transmission across a long path, and that wait is unchanged by how quickly the far end could have replied.
- **B.** Both are distance problems, since a network carrying more traffic behaves like a longer one — Congestion and distance both add delay, but the chapter treats them separately: one is demand exceeding capacity in a fortnight, the other is the geographical gap between a user and the server holding the content.
- **C.** The seasonal slowness is a bandwidth shortage and the overseas delay is a storage shortage — Neither label fits. The December problem is a fluctuation in demand rather than a fixed shortage, and the overseas delay is caused by the length of the path, not by where the records are stored.
- **D.** The seasonal slowness is a demand fluctuation answered by capacity that can grow and shrink; the overseas delay is distance, answered by copies of the content held near those users — **correct.** The chapter treats the pre-holiday spike as a demand fluctuation, and treats distance as its own cause of latency answered by a network of servers in various geographical locations holding copies, the nearest one delivering the content.

Communicating with employees: the inside of the building, from anywhere

The second audience is the organization's own staff, and the arrangement built for them is the intranet you met earlier: the same web technologies, secured with authentication techniques, firewalls, or both, so that proprietary material is visible only to authorized users. What is worth adding here is not the definition but the accounting — what each internal use replaced, and why the replacement costs less.

The chapter names five internal uses. Read them as a column of things that somebody used to do by hand.

INTERNAL USE	WHAT IT REPLACES	WHAT CHANGES AS A RESULT
Disseminating corporate information	A notice sent round the building and kept in whichever copy each person happened to save	One current copy in one place, so arguments about what the policy says stop happening
Employee training	Scheduling everybody into a room on a day that suits almost nobody	Staff work through the material from any desk, on their own schedule
Project management	Status reported upward, rewritten, and then redistributed downward	Every member of the project sees the same plan and the same state at the same moment
Collaboration	Documents mailed back and forth between departments and buildings	People in different places work on one thing rather than on parallel copies of it
Employee self-service	A form completed by a member of staff and then keyed in by somebody in an office	The work moves to the person who already knows the answer, and one keystroke disappears

Only the last row saves money you could point at in a budget, and it is the row to hold on to, because the same move reappears in the next audience. Nothing there was automated. A step was deleted, because the person who held the information was finally allowed to record it directly.

Then there is the question of where the employee is standing. An intranet reachable only from a desk inside the building serves a workforce that is increasingly not inside the building, so most companies let employees use a virtual private network to connect securely to the intranet while travelling or working from home.

The audience has not changed there; only the road has. It is the same promise the cloud characteristic called broad network access makes when it says a service is available from almost anywhere and from almost any web-enabled device — the same internal material, on whatever device the person happens to have with them.

Working with partners: the second keystroke

The third audience is the other companies the organization depends on: the supplier who has to know what to ship, the haulier who has to know what to collect, the laboratory running the samples. They are

not employees, so the intranet is shut to them. They are not the public, so the public site tells them nothing they need. The chapter's answer is the extranet — a private part of the internet reachable by authorized business partners after logging on, and usually secured with a tunnel.

The interesting part is not the arrangement itself but the failure it removes. Follow one hypothetical order through two companies that share nothing, and watch where the cost appears.

- 1 The buyer records the order.** A purchasing clerk enters the quantities into the buyer's own system, which becomes one authoritative record, and emails a copy of it across to the supplier.
- 2 The supplier records it again.** Somebody at the supplier reads that email and types the same quantities into a different system, creating a second record that merely resembles the first one.
- 3 The buyer changes something.** A quantity is revised and a second email goes out, so the supplier's record is now a version behind, and nothing in either system says which of the two numbers counts.
- 4 The goods arrive.** The disagreement surfaces in the receiving bay, the most expensive place in either company to find it, and settling it costs a phone call, a credit note, and sometimes a second lorry.

Nobody in that story was careless. The error was manufactured by step two, and step two exists only because two firms keep separate copies of one fact. The chapter's name for removing it is a single point of data entry, from which data can be updated on disparate corporate computing platforms without having to be reentered — and the benefit that sounds clerical is the one that turns up in the accounts.

Three more of the chapter's partner benefits are worth reading as business outcomes rather than as features of a product.

Automating business transactions — moving orders, confirmations and invoices through a system rather than through people reduces processing costs and cycle times, which is to say both the money spent and the time spent waiting.

Real-time data for management — because transactions arrive as they happen, management can obtain real-time data with which to track and analyze business activities, instead of waiting for a summary written after the month it describes.

Central management of documents — one managed copy reduces the number of versions in circulation and the amount of out-of-date information stored in various places, which matters most when the other reader works for a different firm.

There is a version of this same relationship with the people taken out of it altogether. An interface between applications lets one organization's system use a service another organization provides, without knowing how that service is produced, by passing agreed values and receiving an answer back. A shared web area lets two firms read one record; an interface lets their two systems do it unattended. The principle is identical — an agreed shape, one authoritative answer — with the browser and the clerk removed.

The situations below are all hypothetical and invented. Sort each one by the audience it is really serving, because that is the decision that settles everything else about it.

SORT

Which audience is this really serving?

Place each situation with the use it serves. Every organization below is a hypothetical practice situation, and each one states the conditions you need in order to decide.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

Selling to customers — general, public content that anyone with an internet connection may read, with access public and not restricted

- A hypothetical bakery publishes its product range, its opening hours and an order form that anybody who finds the site can use — General, public content readable by anyone with an internet connection, with access public and not restricted — the chapter's own description of the internet, and the reason nothing goes here that the bakery would not tell a stranger.
- A hypothetical bus operator publishes its timetable for any passenger to check, and the pages are copied to servers in several regions so they load quickly wherever the passenger is — Public content plus a content delivery network, which the chapter describes as the way to reduce the delay caused by geographical distance. The copies change how fast the public is served, not who is served.
- A hypothetical museum lets a visitor reserve a timed entry from home at eleven at night, weeks before arriving — The museum sells nothing on the page beyond a place in a queue, and it is still trading electronically: almost any business can be considered an e-business, and the booking is taken from a member of the public with no account and no prior relationship.

Communicating with employees — specific, corporate and proprietary content, restricted to authorized employees or members, with access private and restricted

- Staff at a hypothetical building society open a page from their own desks and enrol in this year's health cover without sending a form to anybody — Employee self-service through a portal is one of the five internal uses the chapter names for an intranet, and the content is proprietary and restricted to authorized employees.

- Shift supervisors at a hypothetical parcel depot read the week's safety notices and finish a required training module from any depot in the country — Two of the chapter's named intranet uses at once, disseminating corporate information and employee training, both restricted to the organization's own staff wherever in the country they are working.
- Site surveyors at a hypothetical engineering firm open the internal drawing archive from a hotel, through a secured tunnel, rather than from the office — A virtual private network changes where the employee is standing, not who the audience is. The chapter treats the tunnel as the way an intranet reaches a mobile workforce, and the content remains internal.

Working with partners — communications between two or more firms doing business together, restricted to authorized business partners

- A hypothetical dairy cooperative gives each of its two haulage contractors a login to see tomorrow's collection schedule and confirm the volumes lifted today — Two firms using the internet to do business together, reachable only by authorized business partners after logging on. The confirmation is also the single point of data entry that spares both sides from keying the same volume twice.
- A hypothetical seed laboratory posts every test result it produces into an area that its two client growers can read the moment the result exists — Content whose focus is communication between business partners, private and restricted to authorized partners, and an example of the real-time visibility the chapter lists among the benefits of working this way.
- A hypothetical grain merchant's system asks a haulier's system directly for a delivery slot and receives an answer, with nobody at either firm involved — This is the machine-to-machine version of the same partner relationship: a service consumer uses a provider's service without knowing how it is produced, by passing agreed values. The browser and the clerk are gone; the audience is not.

Getting the audience wrong

In practice the wrong answer here is almost never a technology. It is publishing to the wrong audience, and there are three familiar ways of doing that, each of which looks like a shortcut on the day it is taken.

Internal material on the public site — usually done to avoid the trouble of managing accounts, and it works right up until somebody notices that proprietary content has become general content and cannot be called back.

Obscurity mistaken for restriction — an unlinked page that no search engine has indexed is still reachable by anyone holding the address, which is why the chapter treats authentication and firewalls, rather than not being linked to, as what makes content private.

Partners handed employee access — the quickest way to let an outside firm see one schedule is to give it a staff login, and doing so grants that firm everything else an employee can reach along with it.

Each of those is a decision about people wearing a technical costume. That is precisely why the person who should be answering it is somebody who understands the business rather than somebody who understands the servers.

Work the four decisions below in order, and read every outcome before moving on — including the outcomes of the choices you did not make.

DECIDE

Four requests in one week

Work the four decisions in order, and after each one compare every outcome before moving on.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

A hypothetical practice situation. You have taken over systems and communications at an invented regional food cooperative that sells produce boxes to the public, employs about forty people across three sites, and works with two haulage contractors. Four requests land in your first week. Each one looks like a technology question and is really a question about which audience is being served.

Decision 1. The membership secretary wants the annual report, the monthly member notices and the compulsory food-handling training module put on the public website, so that members can always find them and nobody has to manage accounts.

- Put them behind authentication as an intranet for authorized members, and leave the public site to the catalogue. — **the stronger call.** Right. Same web technologies, different audience, different access rule: internal communications, specific and proprietary content, authorized users, private and restricted. Notice that you decided this by naming the reader, not by comparing products — which is what makes it a management decision rather than a technical one.
- Publish them on the public site as asked, since nobody outside the cooperative would go looking for member notices. — The chapter treats authentication and firewalls, not the unlikelihood of being found, as what makes content private. Member notices and a training module are proprietary content, and once they are public they are readable by any competitor, any journalist and any search engine, permanently and without your knowing.
- Publish them on the public site but leave them unlinked from the menu, so only members who were sent the address can reach them. — An unlinked page is exactly the third kind of content the chapter puts in the deep web: invisible to a search engine and reachable by anyone holding the address. It is one forwarded email away from being fully public, and nothing in the arrangement will tell you when that happens.

- Issue every member an account on the extranet the cooperative already runs with its two haulage contractors. — It would work on the day, and it dissolves the boundary you will later be asked to audit. The chapter defines an extranet's users as authorized business partners; putting several hundred members inside it gives them the partner content too, and leaves you unable to say who may see the collection schedules.

Decision 2. The public produce-box shop runs at a steady level for most of the year. In the fortnight before the December holidays orders run several times higher, and last year the site was slow enough that customers abandoned their baskets. The equipment currently sits in a room at the main site.

- Move the public shop onto rented capacity that grows and shrinks with demand, and serve the pages and images from copies held near customers. — **the stronger call.** Right, and note that it is two answers to two different problems. Capacity that can grow and shrink answers the seasonal fluctuation the chapter describes for December electronic commerce; copies held in several geographical locations answer the delay caused by distance. Neither one fixes the other.
- Buy a second server sized for the December fortnight and keep it in the room with the first. — This is the demand-fluctuation problem the chapter describes, met in the most expensive way available. Capacity bought in whole units leaves idle resources for most of the year, and what you have actually purchased, permanently, is a fortnight. It may be the only option if you must own the equipment, but it should be chosen knowingly.
- Ask customers to place their holiday orders earlier in the season so the load is spread out. — This asks the customer to solve the cooperative's capacity problem, and most of them will simply buy elsewhere in the fortnight when they were going to buy anyway. For a business selling over the internet the site is the shop, and telling shoppers to come at a quieter time is closing the door at the busiest hour.
- Reduce the size of the product photographs so that each page is smaller. — It genuinely helps, and it is not enough. Smaller pages reduce the bytes each visitor pulls but not the number of visitors arriving at once, so the peak still exceeds what the room can serve. It is also a poor trade for a business whose product is food that people buy by looking at it.

Decision 3. The two haulage contractors currently receive collection schedules by email and retype them into their own systems. Twice this year a lorry arrived for a volume that had been revised after the email went out, and both times the cooperative and the contractor each produced a document proving they were right.

- Give both contractors access to the schedule where it is maintained, and let them confirm the lifted volume in the same place. — **the stronger call.** Right. This is the chapter's single point of data entry, from which data can be updated on disparate corporate computing platforms without being reentered. One record exists, so there is nothing left for the two firms to disagree about, and management gets real-time data on what was actually lifted rather than a reconciliation after the fact.

- Send the schedule twice, once as an email and once as an attached spreadsheet, so each contractor can check one against the other. — Two copies do not settle a disagreement; they create a third version and a new question about which one is authoritative. The chapter's benefit is the removal of the reentry, not better documentation of it, and every extra copy is another thing that can be a version behind.
- Publish the collection schedule on the public site so the contractors can always see the current version. — It does make the schedule current, and it also tells every competitor what this cooperative moves, from where, and on which days. The content's focus is communication between business partners, and the chapter makes that category private and restricted for exactly this reason.
- Require both contractors to telephone the depot each morning to confirm the day's volumes before loading. — A call is simply a third place for the number to be transcribed, and it adds a person to a process that is failing because of people copying figures. It also produces nothing management can track or analyze, since the confirmation exists only in somebody's memory of the conversation.

Decision 4. Six delivery drivers and two field buyers are almost never at a desk. They need the week's notices, the compulsory training module, and a way to file expense claims from wherever they happen to be.

- Let them reach the same internal pages the office staff use, over a secured tunnel from whatever device they carry. — **the stronger call.** Right. Increases in employee mobility are the chapter's own reason for virtual private networks: staff connect securely to the intranet while travelling or working away from the office. The audience did not change, so the arrangement should not either — only the road into it is different.
- Email the notices out weekly and have the drivers post paper expense forms back to the office. — This is precisely the arrangement the intranet replaced. Notices sit in eight separate mailboxes with no way to tell who is reading a superseded one, and every posted form is a keystroke for somebody in an office — the work that employee self-service was named for removing.
- Publish the notices and the training module on the public site and accept expense claims by email, since these people are outside the building anyway. — Being outside the building is a location, not an audience. The chapter separates these arrangements by who is authorized, and a driver in a lay-by is still an employee, so putting internal material in front of the general public to reach eight of them is a large exposure for a small convenience.
- Give each of them a laptop with a copy of the internal pages stored on it, so no connection is needed at all. — A copy taken on Monday is wrong by Tuesday. The chapter's observation that very little information in business is static is the whole reason internal communication moved onto web technologies in the first place, and eight stored copies recreate exactly the problem one current copy was there to solve.

Reading it back to the objective

The course objective this supplement serves has three clauses, and each one now has an arrangement, a set of demands on the infrastructure, and a question that decides which of the three you are in. That is the whole of it, and it fits in one table.

THE USE	THE ARRANGEMENT	WHAT THE INFRASTRUCTURE HAS TO DELIVER	T
Electronic commerce	The public internet	Fast storage, capacity that survives the peak, copies held near the customer, and availability	V s
Communicating with employees	An intranet, reached from outside through a secured tunnel	Authentication, one current copy of everything, and reach to wherever the staff actually are	Is m
Working with partners	An extranet, or an interface between the two firms' systems	One authoritative record both firms work from, and transactions that arrive as they happen	Is d e

One set of web technologies, three audiences, separated by who is authorized — and the separation is drawn by a manager naming the reader, not by a product being purchased.

One last pass over the three uses before the module closes.

CHECK YOURSELF

Three uses, one technology

Four options, one best answer; the explanations are where the distinctions live, so read all four.

Lesson and answer summary — the interactive version of this activity loads when scripting is available.

1. According to the chapter, what actually separates the internet, an intranet and an extranet from one another?

- **A.** The software and networking technologies each is built from, since the three arrangements use different products — The chapter says the opposite: an intranet uses the same software, hardware and networking technologies as a publicly accessible website. Nothing in the comparison table mentions a product at all.

- **B.** Whether the traffic crosses the public internet, since intranet traffic stays inside the organization and extranet traffic does not — This is nearly true and still unreliable. The simplest form of an intranet does stay within organizational boundaries, but most companies let employees reach the intranet over a tunnel across the public internet, so the route cannot be the divider.
- **C.** Who is authorized to see the content, since the three carry different content for different audiences under different access rules — **correct.** The chapter's table separates the three by focus, type of content, users and access: public content for anyone with a connection, corporate and proprietary content for authorized employees, and partner communications for authorized business partners.
- **D.** Which department owns the pages, since internal pages belong with human resources and partner pages belong with purchasing — Ownership inside the organization is a question of who maintains the pages, not of who may read them. Two arrangements owned by the same department can still face entirely different audiences.

2. A hypothetical cannery gives its two ingredient suppliers a shared area in which to confirm each shipment as it leaves. Its operations manager notices she no longer waits for a monthly summary to know what is in transit. Which of the chapter's stated benefits is she describing?

- **A.** Central management of documents, which reduces the number of versions in circulation and the amount of out-of-date information stored around the organization — That benefit is about which copy of a document is the current one. It is real and it is not what changed here: the manager's complaint was about when she learned something, not about which version she was reading.
- **B.** Management can obtain real-time data with which to track and analyze business activities — **correct.** The chapter lists exactly this among the benefits of working with partners this way, and it follows from the transactions arriving as they happen rather than being gathered up and reported afterwards.
- **C.** Automating business transactions, which reduces processing costs and cycle times — Automation lowers what each transaction costs and how long it takes to complete. That is a benefit of the same arrangement, but the manager described visibility of activity rather than cheaper or faster processing.
- **D.** The internet can be used as a relatively secure medium over which to do business with another firm — Security is the chapter's judgment that the public internet can carry this traffic acceptably, which is a precondition for the arrangement existing at all rather than the improvement the manager noticed.

3. A hypothetical wholesaler's system asks a courier's system for a delivery slot and receives one back, with nobody at either firm involved. How is this best described, and how does it relate to the shared partner area the two firms already use?

- **A.** It is a content delivery arrangement, since the courier is supplying content to the wholesaler across a distance — A content delivery network holds copies of a site closer to its users to reduce the delay caused by distance. Nothing here is about distance, and the courier is answering a question rather than serving a stored page.
- **B.** It is a form of intranet, because the two systems are communicating privately rather than publicly — An intranet serves an organization's own authorized employees. These are two separate firms, which is what makes the relationship a partner arrangement no matter how private the exchange is.
- **C.** It replaces the shared partner area entirely, since once two machines exchange the figures directly there is nothing left for a person to log in and read, and the web area can be switched off — The two do different jobs. A shared area lets people at both firms read and confirm records; an interface lets their systems exchange a specific answer unattended, and most partnerships end up using both.
- **D.** It is an interface between applications: one program uses another's service by passing agreed values, without knowing how the result is produced — **correct.** The chapter defines an interface between applications as letting a service consumer use a service provider's service without having to know how the underlying service is created, by passing pre-specified values and receiving the requested service back.

REFERENCE

Chapter vocabulary

Every term the chapter defines, in one place, written so it makes sense on its own with an example beside it. Search by word, or filter by the objective the term belongs to. This chapter has more vocabulary than the two before it, and a good deal of the difficulty is pairs of terms that sound almost the same and mean genuinely different things — the deep web and the dark web, an intranet and an extranet, grid computing and cloud computing, a client and a server. Use this while you read rather than after.

AI infrastructure

3.3

The hardware and software systems used to support artificial intelligence work. It has special requirements: specialized processors rather than ordinary ones, storage that is both larger and faster than usual, and particular machine learning libraries and frameworks for the people building the models.

For example: A company that wants to train its own model discovers it cannot simply run it on the servers it already owns, and has to budget for a different class of machine entirely.

application programming interface (API)

3.2

An arrangement that lets one piece of software use a service provided by another without knowing how that service works inside. The consumer passes agreed-on values across and gets a result back, which is why software can be assembled out of parts built by different companies.

For example: A ride-hailing app sends a pickup point and a destination to a mapping service and receives the best route, without its developers ever having written a map.

archival data

3.2

Long-term copies of organizational data, kept mainly for compliance and reporting rather than for daily use. Because it is rarely read, it can live on slow, cheap, removable media, and it needs a life span measured in years rather than days.

For example: Internal email from several years ago that nobody expects to open, but that the organization must be able to produce if it is ever asked to.

assembly language

3.2

A very low-level programming language that lets a computer operate quickly and efficiently, and the language operating systems are often written in. Part of an operating system's job is to insulate everyone else from it.

For example: Dragging a file from a flash drive to a hard drive is one gesture for you and a long sequence of low-level instructions underneath, which you never have to see.

backup data

3.2

Short-term copies of organizational data, made so the organization can recover from a failure or disaster. Backups are frequently overwritten by newer ones, so their value is in being recent and quick to restore rather than in lasting.

For example: Last night's copy of the order system, held on fast storage so that if this morning's server fails the business is running again in minutes rather than days.

bandwidth

3.2

The transmission capacity of a computer or communications channel, measured in bits per second or multiples of it. It is how much data can reliably be carried in one second, and it is what decides whether a file takes eight seconds or a day and a half to arrive.

For example: The same video downloads in seconds over a fiber connection and takes hours over an old telephone modem, because only the capacity of the path changed.

bit

3.2

The smallest unit of data used by computers. Human communication is made of words; computer communication is made of bits, and everything sent over a network — a document, a song, a film — is bits, differing only in how many of them there are.

For example: A page of text runs to roughly fourteen thousand bytes while a print-quality photograph can be more than two hundred million, which is why one arrives instantly and the other does not.

broad network access

3.4

One of the five characteristics of cloud computing: because the services are reached over the internet, they are available from almost anywhere and from almost any web-enabled device. It is what lets a manager act on something without going back to an office.

For example: Approving an urgent order from a phone in an airport, using the same system that sits on the desktop back at work.

campus area network

3.2

A wide area network that spans several buildings belonging to one organization. It is a middle case between a network inside a single building and one that crosses a city or the world.

For example: A hospital group whose clinic, laboratory and administration buildings sit on one site and share one network across all three.

capital expenditure and operational expenditure

3.4

Two ways of paying for the same computing. A capital expenditure is money spent up front to own equipment; an operational expenditure is money spent continuously to use a service. Cloud computing converts one into the other, which changes the budget it comes out of and who has to approve it.

For example: Buying servers is a large purchase in one year; renting the same capacity is a monthly bill that rises and falls with how much the business actually used.

cascading style sheets (CSS)

3.2

The standard used to specify the formatting and layout of the elements on a web page. It separates how a page looks from what the page says, so the wording and the appearance can be changed independently.

For example: Changing one style rule makes every heading on a thousand-page site larger, without anyone editing a thousand pages.

central processing unit (CPU)

3.2

The general-purpose processor that performs a computer's calculations. It is built to handle instructions one after another very quickly, which suits ordinary software where each step depends on the result of the last one.

For example: Recalculating a spreadsheet, where each formula may need the answer from the formula above it before it can run.

client

3.2

Any computer or software application on a network that uses the services a server provides. Clients only request; they do not serve. A client usually has one user, whereas many users share the server it talks to.

For example: The mail program on a laptop asks a mail server for new messages; it never answers requests from anyone else.

client-server network

3.2

A network in which servers and clients have defined roles, some machines providing services and others only requesting them. It is the normal business arrangement, and nearly everyone at work is using one whether they know it or not.

For example: Every desk in an office opening files kept on a shared server rather than on each other's machines.

cloud computing

3.4

A model for enabling convenient, on-demand access over a network to a shared pool of configurable computing resources — networks, servers, storage, applications and services — that can be provisioned and released quickly with minimal management effort or interaction with the provider. In plainer words: computing rented by the hour from somebody else's building.

For example: A company that needs two hundred servers for a week gets them the same afternoon with a payment card and a web page, and stops paying when it stops using them.

collocation facility

3.2

A data center managed by a third party that rents space to several organizational customers. The tenant still owns and manages its own servers; what it stops paying for is the building, the power, the cooling and the physical security.

For example: A regional firm that puts its own machines in a rented, professionally run room instead of building a server room and a generator of its own.

computer networking

3.2

The sharing of data or services between machines. It needs three things and no fewer: a sender and a receiver with something to share, a pathway between them, and rules both sides follow.

For example: Two people can only have a conversation if there is something to say, a room to say it in, and a language they both speak; a network is the same three requirements in equipment.

consumerization of IT

3.3

The trend in which the devices people choose for themselves arrive at work and have to be integrated into the organization's infrastructure. It shifts part of the equipment decision away from the technology department and toward the individual.

For example: Employees expecting to read work email on their own phones, which the organization must then decide how to secure and support.

content delivery network

3.4

A network of servers in different geographical locations that store copies of the same content, so a request is answered by the copy nearest the user. It reduces the delay caused by distance, saves bandwidth, and is invisible to the person browsing.

For example: A video that plays instantly for viewers on three continents because each of them is actually being served from a machine a few hundred miles away.

cyber-physical system

3.2

A system that integrates computing with physical processes, coordinating software, sensors and mechanical components closely enough to monitor and control something in the real world with a high degree of autonomy. The consequences of a fault are physical, which is what makes securing them different.

For example: A factory where sensors, controllers and machines run most of the production line between them and reorganize the work without a person intervening.

dark web

3.2

Those areas of the web used for various criminal purposes, typically reachable only with specialized browsers that anonymize the user and hide their traces. It is deliberately hidden, and it is not the same thing as the deep web.

For example: A marketplace trading stolen payment card numbers, which no ordinary browser or search engine will take you to.

database

3.2

An organized collection of related data held so that it can be stored and retrieved reliably. The chapter treats databases as the thing hardware exists to run and storage exists to hold, which is why a storage decision is really a decision about how well questions can be answered later.

For example: A shop's record of every product, every supplier and every sale, kept once so that a price change is made in one place rather than in four spreadsheets.

database management system

3.3

The software that stores data, keeps it consistent, and answers questions about it. The chapter counts it as a third cost of capturing more data, alongside the storage space and the processing power, because somebody has to license, configure and run it.

For example: Two clerks record a sale at the same moment and the totals still agree afterwards, which is the software doing work nobody sees.

data center

3.2

Dedicated space set aside for an organization's infrastructure, ranging from a single server room to a building the size of a warehouse. Keeping the equipment in one place makes it easier to manage, repair, upgrade, power, cool and secure.

For example: A parcel carrier's facilities that can run for days on their own generated power, with cooling capacity comparable to that of a whole neighbourhood of houses.

digital density

3.2

The rising proportion of an organization's activity that exists as data and is carried out through connected devices. The chapter names it as the reason a well-functioning infrastructure has become essential rather than merely useful.

For example: A delivery that once left only a signed paper docket now leaves a scan at every handover, a location trace and a photograph, so the same journey produces a hundred times the record it used to.

deep web

3.2

Those parts of the web that conventional search engines cannot index: pages behind a login, pages generated from a database when somebody asks for them, and pages nothing links to. Most of the web is like this, and none of it is necessarily sinister.

For example: Your own course pages, your bank statements and an online store's product listings are all invisible to a search engine, for three different reasons.

desktop virtualization

3.2

Providing workers with a desktop environment that actually runs on a server rather than on the machine in front of them. It reduces the cost of licensing and maintaining software and helps with strict privacy and data protection duties, because the data never leaves the data center.

For example: A hospital where every screen shows the same working environment, and nothing is stored on the terminal itself if it is stolen.

device driver

3.2

Software that allows a computer to communicate with a particular piece of hardware. It is the translation layer between one general operating system and the specific behaviour of one make and model of device.

For example: Plugging in an unfamiliar printer and having the computer fetch the small piece of software that teaches it how to talk to that printer.

domain name

3.2

The part of a web address that helps a person recognize the company or individual it belongs to. Names exist because the numeric addresses machines actually use are hard to remember and can change.

For example: Seeing a familiar name in the address bar tells you whose server you have reached, which a string of numbers would not.

e-business

3.2

A business whose ordinary operations depend on internet technology rather than merely using it at the edges. The chapter's position is that almost any business today can be considered one, which is what makes infrastructure everyone's problem rather than the technology department's.

For example: A family bakery that takes orders on a website, buys flour through a supplier portal and rosters staff in a shared calendar is running on the internet even though nobody there would call it a technology company.

edge computing

3.4

What content delivery networks grew into: moving not only stored data but also the processing away from one central location out to the edges of the network, close to where the data is produced. The point is to cut the delay, and secondarily to cut traffic and cost.

For example: A monitoring device that decides for itself that a reading is dangerous, instead of sending every reading across the country and waiting for an answer.

embedded system

3.2

A computer designed to perform one specific set of tasks well, built into something that is not thought of as a computer. It has no general-purpose life; it does the one job.

For example: The computer that manages a car engine, a traffic light or a media player, which nobody ever installs software on.

extranet

3.2

A private part of the internet, cordoned off from ordinary users, that lets two or more firms do business together. The content is on the web, but only authorized partners can reach it after logging in, and it usually travels through a secured tunnel because the path itself is public.

For example: A supplier logging in to a manufacturer's system to see current orders, confirm shipments and update prices without anyone rekeying the data.

field-programmable gate array (FPGA)

3.2

A processor whose circuitry can be reconfigured after it is built, offering customizable hardware acceleration. It sits between a general-purpose processor and a chip designed for a single task.

For example: A team that needs one unusual calculation run very fast reshapes the hardware around that calculation instead of buying a chip that will never exist.

file

3.2

A block of data that stores specific content, such as a document, an image or a spreadsheet. It is the unit people name, move, share and delete, and it is the reason ordinary users never have to think about where anything physically sits.

For example: The report you saved yesterday, which you can find by its name without knowing anything about the disk it is on.

five-nines availability

3.2

Availability of 99.999 percent, which works out at just over five minutes of downtime in a year. It is the target data-intensive organizations aim for, and reaching it constrains everything from component design to the choice of site.

For example: A payment system that is expected to be unavailable for less time in a whole year than an average lunch break.

folder

3.2

A grouping of related files, shown as a folder in a graphical interface and called a directory underneath. Folders nest inside folders, which is how a large amount of digital content stays findable.

For example: Keeping coursework in one folder, then a folder per term inside it, then a folder per course inside that.

frameworks and libraries

3.2

Collections of prewritten code that developers assemble rather than write. They shorten development time, help one program run on several platforms, and supply the building blocks for specialized work such as machine learning.

For example: A small team shipping a working application in weeks because most of what it does was already written by somebody else and is being reused.

generation of computing

3.3

Each fundamental change in the way computing technologies work. The usual account runs from vacuum tubes through transistors and integrated circuits to low-cost multimedia machines, widespread internet access, ubiquitous mobile connectivity, and now artificial intelligence and quantum computing.

For example: The move from a room-sized machine that a few specialists operated to a phone in everyone's pocket is several such changes stacked on top of one another.

graphical user interface (GUI)

3.2

An interface that lets you instruct a computer by selecting or manipulating pictures, icons and menus rather than by typing commands. It is the reason using a computer no longer requires learning its language first.

For example: Dragging a file onto a disk icon instead of typing a copy instruction with the exact path of both places.

graphics processing unit (GPU)

3.2

A processor optimized for doing the same operation on very many pieces of data at once, using thousands of small cores. It was built for rendering images, where every pixel can be worked out independently, and turned out to suit machine learning for the same reason.

For example: Training a model, where enormous numbers of similar calculations have no need to wait for one another and can all run side by side.

green computing

3.4

Using computing resources more efficiently to reduce environmental impact, and using information systems to reduce environmental impact elsewhere. The chapter's tactics are consolidating underused machines, shifting work to shared providers, managing power on idle desktops, and printing less.

For example: Replacing a room of servers that each ran at a fraction of capacity with a few machines doing the same work, and paying less for power and cooling as a result.

grid computing

3.4

Combining the computing power of many smaller, independent, networked computers into one system in order to solve a problem that previously needed a supercomputer. Like cloud computing it uses distributed resources, but unlike cloud computing those resources are all applied to a single large problem.

For example: A simulation broken into thousands of small pieces, each finished by an ordinary desktop machine somewhere and sent back to be reassembled.

hardware

3.2

The physical machines that run the applications and databases an organization needs in order to process transactions and analyze business data. Because organizations perform many different kinds of activity, they need several different kinds of computer to support them.

For example: The register at the front of a shop, the server in the back room and the phone in the manager's pocket are all hardware doing three different jobs.

host name

3.2

The part of a web address naming the particular server, or group of servers, that will answer the request. The prefix www usually means the default site, and larger organizations use other host names for their separate services.

For example: One organization's mail service, map service and document storage each answer on a different host name in front of the same domain.

HTML tag

3.2

A code set apart with angle brackets that tells a browser how part of a document should be presented, or what that part means. Most tags come in pairs, one to start an element and one to end it.

For example: A pair of tags around a phrase can make it bold, mark it as a heading, or turn it into a link to another document.

HTML5

3.2

The chapter's name for the current version of the web's markup language, which added ways of structuring a page and of presenting modern multimedia without extra software. Treat it as a label for the modern language rather than as a version number still being counted up, because the markup is now revised continuously rather than released in numbered editions.

For example: Video that plays in the browser itself, with no separate plug-in to install, because the page can simply say that this is a video.

HTTPS

3.2

The secure version of the web's transfer protocol, now used routinely for anything sensitive because data travelling between a server and a browser can otherwise be intercepted and read. The padlock in the address bar is the browser reporting that it is in use.

For example: Entering a card number on a page without the padlock means the number crosses several unknown networks in a form anyone along the way could read.

hybrid cloud

3.4

An arrangement in which an organization keeps some applications and data in-house and moves others to a public cloud. The chapter presents it as what organizations usually settle on, because no single provider meets every need and some data is not worth moving.

For example: A firm running its public website on rented capacity while its patient records stay on machines it owns and controls.

hyperlink

3.2

A reference from one document to another, which the reader can follow. Links are what turn a collection of separate pages into a web rather than a library of unconnected documents.

For example: Clicking a word in an article and arriving at the source it was quoting, on a completely different organization's server.

hypertext

3.2

The key feature of the web: documents that contain links to other documents, so a reader can move between them in any order rather than front to back. Everything else about the web is built on top of this one idea.

For example: Reading three paragraphs of one page, following a link, and finishing on a page written by somebody who has never heard of the first author.

Hypertext Markup Language (HTML)

3.2

The standard method of specifying the structure and content of web pages. It is a text-based format that uses codes called tags to tell the browser what each part of the document is and how it should be presented.

For example: Choosing to view the source of any page shows the same words you were reading, wrapped in codes that gave them their headings, links and emphasis.

Hypertext Transfer Protocol (HTTP)

3.2

The protocol web servers use to process requests for pages. It is the agreed set of rules by which a browser asks for a document and a server answers, and it is the reason a browser made by one company can read a page served by another.

For example: Typing an address and pressing return is a request in this protocol, whether the server at the other end runs the same software as your machine or not.

infrastructure as a service (IaaS)

3.4

The cloud service model in which only the basic capabilities of processing, storage and networking are provided. The customer has the most control and the most responsibility: it still chooses the operating system, manages its software licences, and bears relatively high setup costs.

For example: A streaming company renting raw computing capacity to convert films into different formats and to run its own website on top of it.

information systems infrastructure

3.2

The hardware, system software, storage, networking and data centers an organization needs in order to operate fully and to meet its goals. It is the foundation everything else in the organization is built on, and the chapter's point is that it creates value by enabling efficiency, effectiveness and increasingly agility.

For example: Nobody buys a bank because of its servers, and every service the bank sells stops the moment those servers do.

input device

3.2

Hardware that supplies data to a computer. Keyboards, mice, touch screens and cameras are the familiar ones, and sensors have widened the category enormously.

For example: A scanner at a receiving dock, which is doing nothing more than getting a number into a system without anybody typing it.

internet

3.2

A large worldwide collection of networks that use a common protocol to communicate with one another. It is not one network but an agreement that lets many separately owned networks behave as though they were one.

For example: A message from a home connection to an office server crosses several companies' equipment on the way, and nobody involved had to arrange it in advance.

Internet of Things (IoT)

3.2

A vast network of interconnected physical devices that collect and share data, often producing very large amounts of information that can be gathered and analyzed centrally. It extends the reach of information systems to objects that were never previously connected to anything.

For example: Refrigerated containers reporting their temperature continuously across a supply chain, so a spoiled shipment is known about before it is opened.

internetworking

3.2

Connecting host computers and their networks together to form larger networks. The word internet is a contraction of it, and it describes exactly what the internet does.

For example: A company network, a university network and a home network all reaching each other without any of them merging into a single owner's system.

intranet

3.2

An internal site that looks and behaves like a public website and uses the same technologies, but uses authentication or firewalls so that only authorized employees can see it. In its simplest form the traffic never leaves the organization at all.

For example: The internal pages where staff read announcements, take required training and change their own benefit elections without contacting anyone.

IP address

3.2

The address of a computer or device that enables a network to route messages to the right destination. The older form is four numbers between 0 and 255 separated by dots; the newer, much larger form exists because the world ran short of the older kind.

For example: Every device on a home network has one, which is how the router knows which machine asked for the page that just came back.

lean startup methodology

3.3

An approach built on rapid cycles of devising a solution, building the smallest version of it that can be tested, and finding out quickly whether it has the effect it was supposed to have. It matters for infrastructure because it means committing to equipment before the answer is known makes no sense.

For example: Putting a rough version of a new service in front of real customers for a month before deciding whether it deserves a permanent system.

local area network (LAN)

3.2

A network for sharing data, applications and other resources between several users, typically within one building. It is the ordinary office or home network, and its capacity is usually far higher than the connection leaving the building.

For example: Everyone in an office printing to the same printer and opening the same shared drive, all without anything crossing the internet.

mainframe

3.2

A large computer used as the main central computing system of a major organization, optimized for high availability, heavy use and security. It is the machine of choice for mission-critical work such as processing very large volumes of transactions.

For example: A bank whose account balances all live on one heavily protected system that thousands of tellers and cash machines use at the same time.

measured service

3.4

One of the five characteristics of cloud computing: the provider monitors usage and the customer pays only for what it uses. Fixed costs become variable costs that are easy to track — and easy to run up if nobody is tracking them.

For example: A bill that is small in a quiet month, large in a busy one, and catastrophic in a month when somebody else was using your account.

metropolitan area network

3.2

A wide area network covering the area of a city. It is larger than a network in one building or on one site, and smaller than a network spanning a country or the world.

For example: A city transit authority connecting stations, depots and its control room across the whole urban area.

microservices

3.4

The small building blocks that packaged business capabilities are made of on a technical level. Each does one narrow job and communicates with the others across defined interfaces, so one can be replaced without rebuilding the rest.

For example: Changing how a company calculates delivery charges without touching the part of the system that takes payment.

minimum viable product (MVP)

3.3

The smallest version of a solution that can be put in front of real users to find out whether it has the desired effect. It exists to produce an answer cheaply, not to be the finished thing.

For example: A single page and a form, launched to see whether anybody signs up, before a system is built behind it.

mobile device

3.2

Tablets and smartphones, which have become part of an organization's information systems infrastructure rather than a personal convenience beside it. They have one user, travel everywhere, and are increasingly where work is actually done.

For example: A field technician who receives the day's jobs, records what was done and takes payment on the same handheld device.

modular architecture

3.4

Building software by choosing individual blocks best suited to a particular need and connecting them through interfaces, rather than writing one large program that contains everything. It buys flexibility at the cost of having to plan, select and orchestrate the pieces.

For example: Assembling a service out of separate components for orders, inventory, customers and payments, each of which can be swapped later.

monolithic application

3.4

An application built as a single unit that includes all the features it needs, with the whole thing in one code base. That makes it relatively easy to develop, test and deploy, and correspondingly hard to change one part of without disturbing the rest.

For example: A long-serving system where adding one field to a form means retesting and redeploying the entire program.

Moore's law

3.3

The prediction, made in 1965 by one of Intel's cofounders, that the number of transistors on a chip would keep doubling at regular intervals. It is an observation and an industry roadmap rather than a law of nature, and its pace has slowed, with recent gains coming increasingly from adding more cores that work in parallel.

For example: The first processors held a few thousand transistors and current ones hold billions, which is why a phone outperforms the machines that once filled a room.

neuromorphic chip

3.2

A processor whose design is inspired by the architecture of the brain, aiming to improve energy efficiency and speed for artificial intelligence work. It is one of several attempts to escape the power cost of doing this work on general-purpose hardware.

For example: Research hardware that tries to do recognition tasks at a small fraction of the electricity a conventional processor would need.

on-demand self-service

3.4

One of the five characteristics of cloud computing: a customer can obtain resources as needed, without lengthy negotiation and in many cases without speaking to anyone at the provider at all. It is what makes an experiment possible this afternoon rather than next quarter.

For example: Setting up new servers through a web control panel with a payment card, and having them running before the meeting that requested them has ended.

operating system

3.2

The most prominent kind of system software. It coordinates the interaction between hardware components, attached devices, application software and users, and it provides a common layer so that programs can be written for the operating system rather than for every individual model of machine.

For example: The reason a word processor ships in one version for one operating system rather than a separate version for every laptop ever made.

operational data

3.2

Data used to manage business processes as they happen, such as processing transactions, or to analyze the business. It needs to be current and fast to reach, which is why it lives on the quickest storage an organization has.

For example: The stock level a shopping site checks the instant you add something to a basket, which would be worthless if it took a minute to read.

output device

3.2

Hardware that presents results to a person. Monitors, printers and speakers are the familiar examples, and the category matters because a system nobody can read the results of has not finished its job.

For example: A picking list printed in a warehouse, which is where the computer's decision finally becomes something a person can act on.

packaged business capability (PBC)

3.4

A building block designed to perform one well-specified business function, such as processing payments or managing orders. Different vendors offer competing blocks for the same function, so an organization can choose the one that best fits and connect it to the others.

For example: Using a specialist payments component inside your own ordering system instead of writing payment handling yourself.

packet

3.2

One of the small chunks a request or a file is broken into for transmission. Breaking data up allows it to be routed and moved efficiently, and the pieces are reassembled at the far end before anyone sees them.

For example: A photograph you send arrives as thousands of fragments that may take different paths and are put back in order at the other end.

peer

3.2

Any computer that may both request and provide services, rather than being only a client or only a server. Peer arrangements are common in small offices and homes, where no machine is dedicated to serving the others.

For example: Two laptops sharing a folder directly with each other, with neither of them being the office server.

peer-to-peer network (P2P)

3.2

A network that lets any computer or device on it both provide and request services. It needs no dedicated server, which makes it cheap and simple, and correspondingly harder to control centrally.

For example: A household where any machine can reach the shared printer and the shared drive without a server sitting between them.

peripheral

3.2

A device attached to a computer, such as a printer, whose interaction with everything else is coordinated by the operating system. From the software's point of view a peripheral is something the operating system knows how to reach.

For example: Any application can print, because the operating system rather than each program deals with the printer.

personal area network (PAN)

3.2

Wireless communication between devices over a very short range, under about ten meters, using technologies such as Bluetooth. It is the smallest of the network types and the one most people use without noticing.

For example: Headphones connected to a phone, or a laptop connected to a wireless keyboard on the same desk.

personal computer

3.2

A computer used for personal computing and small business computing, serving one user at a time. It is the reference point against which the other classes of computer are usually described.

For example: The machine on an office desk, which does one person's work and would be overwhelmed if a thousand people asked it questions at once.

planned obsolescence

3.3

Designing a product to last only for a certain life span. In hardware it appears as components that cannot be serviced, so the device is replaced when one fails; in software it appears as older versions that cannot open newer files, or support that is simply withdrawn.

For example: A vendor announcing an end-of-support date, which turns a working fleet of machines into an upgrade decision whether the organization wanted one or not.

platform as a service (PaaS)

3.4

The cloud service model in which the provider supplies the computing platform — operating system, web server, database management system and development tools — and the customer runs its own applications on it. The customer controls the application and little else, and stops buying platform licences.

For example: A team deploying software it wrote onto a rented platform, without anyone on that team ever installing or patching a database engine.

private cloud

3.4

A cloud internal to one organization, which still provides self-service access to pooled resources on a pay-for-use basis but keeps everything inside the organization's own control. It does not remove the work of running an infrastructure; it buys customizability, control and privacy instead.

For example: A large firm letting its own departments provision servers from a shared internal pool in minutes, rather than from an outside provider.

programmable logic controller (PLC)

3.2

A specialized computing device used to automate machines and processes, from a factory line to a ski lift. It is built to run one control task reliably for years in conditions no office machine would survive.

For example: The device that decides when a conveyor starts and stops, which has no screen, no keyboard and no user.

protocol

3.2

A definition of the procedures computers follow when they transmit and receive data. It is the agreed language of a network, and without it a message can arrive perfectly and still mean nothing to the receiver.

For example: An email that reaches you in a language you cannot read has travelled the whole way successfully and still failed to communicate.

public cloud

3.4

Cloud services that any interested party can use on a pay-per-use basis. It suits work that needs to scale rapidly, and organizations that do not have the capital or the people to build capacity of their own.

For example: A new company launching a service worldwide on rented capacity, having bought no equipment at all.

QR code

3.2

A two-dimensional bar code with a high storage capacity, usually holding a web address so that scanning it with a camera opens a page. It can also trigger an action, such as starting a call, sending a message, or making a payment.

For example: A code on a table in a restaurant that opens the menu on your phone, with no address to type and no application to install.

radio frequency identification (RFID)

3.2

A technology that uses electromagnetic fields to transmit data between a reader and a small tag attached to an object. Unlike a bar code it does not need line of sight or hand scanning, can hold more data, and can be read at a distance.

For example: A pallet passing through a doorway and being counted automatically, without anyone pointing a scanner at each carton.

rapid elasticity

3.4

One of the five characteristics of cloud computing: resources can be scaled up or down almost instantly, often automatically, in step with demand. Equipment ordinarily takes weeks to arrive and days to configure, which is why owning means guessing in advance.

For example: A retailer whose capacity grows through a holiday rush and shrinks again in January, without a single purchase order.

resource pooling

3.4

One of the five characteristics of cloud computing: the provider manages distributed resources that are assigned dynamically to many customers, so a customer rents capacity without knowing which machine or which building it came from. Providers often do allow a region to be specified.

For example: A European company insisting its storage stay on servers in Europe, which is a data protection decision rather than a technical one.

RFID tag

3.2

The small transponder attached to an object so it can be identified by a reader. Passive tags cost a few cents and are read from a few feet away; active tags contain a battery, cost several dollars, and can be read from hundreds of feet.

For example: A tag under an animal's skin carrying an owner's details, and a tag on a shipping container carrying a manufacturing history, are the same technology at two sizes.

root directory

3.2

The top of a file hierarchy, typically the drive itself, from which every subdirectory branches out. It is the reason a location can be described as a path from a known starting point.

For example: Following a chain of folders from the drive down to one document is walking the tree from the root to a leaf.

scalability

3.4

The ability to adapt to increases or decreases in demand for processing or data storage. It is one of the main promises of cloud computing, and not every provider can meet every organization's idea of it, so it is something to test rather than assume.

For example: A ticket site that stays up when a popular event goes on sale, because capacity grew in the minutes when it was needed.

sensor

3.2

An input technology that collects data about the physical environment — temperature, pressure, vibration, proximity — and supplies it in real time. Sensors are what let information systems observe the world rather than only record what people type.

For example: A vibration sensor on a motor that reports a change in behaviour days before the motor fails.

server

3.2

Any computer on a network that makes files, printing, communications and other services available to other users of the network. Servers are optimized for many simultaneous users, so they carry more processing power, memory, storage and network capacity than a single-user machine.

For example: One machine answering thousands of visitors at once, some of them getting the identical page and others getting a page assembled for them individually.

serverless computing

3.4

A newer platform arrangement that frees the customer from setting up machines at all: small, specific functions are run by the provider on demand, scaled automatically, and billed by the time they actually run. The servers still exist; the customer just never meets them.

For example: A function that resizes an uploaded photograph, which costs nothing on a day when nobody uploads anything.

service-level agreement (SLA)

3.4

A contract specifying the level of service to be provided in terms of performance, warranties and disaster recovery. It does not guarantee that resources will be available; it promises service levels and provides refunds or discounts when they are missed, which makes it mainly a way of settling disputes.

For example: A refund covering the fee for the hours a service was down, which does not cover the orders the business lost while it was down.

software as a service (SaaS)

3.4

The cloud service model in which the customer uses only the application, with no knowledge of or control over the infrastructure beneath it and usually limited ability to configure it. It is the easiest model to adopt because nothing has to be maintained or updated by the customer.

For example: Company email, shared documents, meetings and customer records that staff simply log in to, with nobody in the building patching anything.

supercomputer

3.2

The most expensive and most powerful class of computer, used mainly for massive scientific problems rather than by business organizations. Even at the top of the range some problems take a year to compute, which is what motivated spreading work across many ordinary machines instead.

For example: Modelling a climate scenario or a nuclear reaction, work that no ordinary machine could finish in a useful amount of time.

system software

3.2

The collection of programs that control the basic operations of computer hardware. Application software does not talk to hardware directly; it talks to the system software, which talks to the machine.

For example: A spreadsheet asks to save a file and never has to know what kind of drive is fitted or how it is wired.

tensor processing unit (TPU)

3.2

A processor custom-designed to optimize the kind of matrix calculations that dominate deep learning. It is more specialized than a graphics processor and correspondingly faster on the work it was built for.

For example: Hardware built by a large provider specifically to run its own language models, and then rented out to customers doing similar work.

thin client

3.2

A machine with minimal memory, storage and processing capability, used as a window onto software that actually runs elsewhere. It is cheap, replaceable, and holds nothing worth stealing.

For example: A terminal in a public area that has no data on it at all, so losing it costs the price of the box and nothing else.

top-level domain

3.2

The suffix at the end of a domain name. Some indicate a kind of organization, such as commercial, educational, non-profit or government; more than two hundred are two-letter country codes; and a growing number of generic ones name particular industries or uses.

For example: The last part of an address tells you something about who registered it, which is why an unexpected suffix on a familiar-looking name is worth a second look.

transmission control

3.2

protocol/internet protocol (TCP/IP)

The protocol suite the internet uses to move web pages and other information. It breaks a request into packets, routes them, reassembles them at the destination, and does the same in reverse with the reply.

For example: Every page you open is this cycle running twice: a request out in pieces, a page back in pieces, reassembled before you see anything.

transmission media

3.2

The physical pathway — cables and wireless — used to carry data. It is the second of the three requirements of a network, and the one that most directly limits how much can be sent and how quickly.

For example: The difference between a fiber line and an old copper connection is nothing but the pathway, and it changes a download from seconds to hours.

uniform resource locator (URL)

3.2

The address people actually use to identify and locate a web page. It names the host, the domain and the top-level domain, and usually a path to the particular resource wanted within that site.

For example: A long address ending in a series of words after a slash is naming a specific page inside a site, not just the site itself.

utility computing

3.4

The model cloud computing uses: renting processing, storage or networking from an outside provider as needed and paying only for what is actually used, the way a household pays for electricity. It is where the shift from buying capacity to buying service comes from.

For example: Paying for computing by the hour that it ran, rather than paying once for a machine that then sits idle most nights.

virtual private network (VPN)

3.2

A technology that lets someone outside the office connect securely to the organization's internal network across the public internet. It is what makes an intranet reachable from home or on the road without putting it on the open web.

For example: Working from home and reaching the same internal systems as a colleague at a desk, over a connection that is encrypted the whole way.

virtualization

3.4

Running several separate computing environments on one physical machine, so that many underused servers can be consolidated onto a few well-used ones. It is the technique underneath both desktop virtualization and most of what cloud providers sell.

For example: Twenty servers that each ran at a fraction of capacity replaced by three that run properly loaded, with a large fall in power and cooling cost.

Voice over IP (VoIP)

3.2

Technologies that use internet infrastructure to place telephone or video calls. Their existence is the clearest sign that computing and telecommunications have converged: the call and the web page now travel the same way.

For example: A team meeting held over an internet application rather than a conference call, with the organization's telephone system quietly becoming software.

web browser

3.2

A software application used to locate and display web pages, including their text, graphics and multimedia. It requests documents, interprets the markup it receives, and decides how the result looks on your screen.

For example: The same page looks slightly different on two machines because the browser, not the server, does the final drawing.

web page

3.2

A hypertext document containing both content and links to other documents. It is the unit the web is made of, and the thing a web server actually sends when it is asked for something.

For example: An article you are reading, together with the links inside it that would take you somewhere else entirely.

web server

3.2

A computer that stores web pages and processes requests for them. Many of them do not store finished pages at all: they run a program that assembles the page when it is asked for, often by querying a database.

For example: A product page that did not exist a second before you asked for it, because it was built from stored rows about that product.

website

3.2

A collection of interlinked web pages owned by the same organization or individual, usually hosted on a web server. The site is the unit an organization publishes; the page is the unit a visitor reads.

For example: One organization's address covering thousands of pages, all reachable from each other and all under the same name.

Wi-Fi network

3.2

The everyday name for a wireless local area network, which connects devices using high-frequency radio waves instead of cables. Ease of installation made it the default in offices, homes and public places.

For example: A cafe where a dozen strangers are all on one network that took an afternoon and one box to install.

wide area network (WAN)

3.2

A network connecting multiple local area networks over a large distance, often with ownership and management distributed among several parties. It ranges from a few buildings to a city to the whole world, and the internet is the largest example of one.

For example: A retailer whose stores, warehouses and head office are all on one network although no single organization owns every link between them.

wireless local area network (WLAN)

3.2

A local area network that uses high-frequency radio waves rather than cables, installed to connect mobile devices or where running cable is impractical. It is the same idea as a wired local network with the pathway changed.

For example: A warehouse where handheld scanners work anywhere in the building because no cable could reach a moving person.

workstation

3.2

A computer built for medical, engineering, architectural, animation or graphics work, optimized for visualizing and rendering three-dimensional models. It typically serves one user, but with far more processing power, memory and graphics capability than an ordinary desktop.

For example: The machine an architect uses to walk a client through a building that has not been built, which would defeat an ordinary office computer.

World Wide Web

3.2

A system of interlinked documents on the internet, and in effect a graphical interface to it. The web is not the internet: the internet is the network of networks, and the web is one very successful thing running on top of it.

For example: Email, video calls and file transfers all use the internet without being the web at all, which is why the two words are not interchangeable.

Final challenge

Situations rather than definitions. Each one describes something happening in an organization and asks what is actually going on, which is the form this material takes once you are the person being asked. Every company described is invented. Answer all of them, then score: the result breaks out by objective, so a weak area points at the section to reread instead of disappearing into a single number.

FINAL CHALLENGE

Final challenge

Answer key — the interactive, self-scoring version loads when scripting is available.

1. **3.2** A hypothetical national insurer processes millions of policy transactions a day against one central system that ten thousand staff and a public website all use at once. It is being replaced, and four options are on the table. Which class of computer does the chapter describe for this work, and why that one?
 - **A.** A supercomputer, because the volume is enormous and this is the most powerful class of machine available — Supercomputers are the most expensive and most powerful class, but the chapter says they are used primarily to assist in solving massive scientific problems and are typically not used by business organizations. Raw speed is not what this workload needs.
 - **B.** A mainframe, because it is optimized for high availability, heavy shared use and security, and is the chapter's machine for mission-critical transaction processing — **correct.** The chapter defines a mainframe as a large computer used as the main central computing system for major corporations, optimized for high availability, resource utilization and security, and typically used for mission-critical applications such as transaction processing.
 - **C.** A rack of workstations, because workstations carry fast processors and large memory and several of them together would cover the load — Workstations are optimized for visualization and rendering of three-dimensional models for medical, engineering, architectural and graphics work, and typically serve one user each. Their strengths are the wrong strengths for shared transaction processing.
 - **D.** A larger personal computer, because the difference between the classes is mostly physical size, and a desktop machine can be ordered with as much memory and as fast a processor as the nightly run needs — The classes differ by what they are optimized for, not by size alone. A personal computer serves one user; the chapter's comparison has mainframes serving ten thousand or more at once, which is a difference in kind.
2. **3.2** A hypothetical architecture practice buys two machines in the same month. One will let a designer walk clients through unbuilt buildings in three dimensions. The other will hold the practice's drawings and be reachable by everyone in the office. How does the chapter classify them?

- **A.** Both are servers, because both are more powerful than an ordinary desktop and both are shared resources of the practice — Cost and power do not decide the class. The chapter separates them by what they are optimized for: one machine visualizes and renders for the person sitting at it, the other answers many people at once.
- **B.** The first is a server because rendering is a service, and the second is a workstation because it stores the working files people work on — This reverses both definitions. The word service in the chapter means making files, printing and communications available to users of a network, not the professional service the practice sells to its clients.
- **C.** Both are workstations, because a workstation is any professional machine used for architectural or engineering work and both were bought for the practice — The intended use of the practice does not classify the machine. The second one has no single user sitting at it and exists to answer requests, which is the chapter's definition of a server rather than a workstation.
- **D.** The first is a workstation and the second is a server, because one is optimized for rendering for a single user and the other for access by many concurrent users — **correct.** A workstation is optimized for visualization and rendering of three-dimensional models and typically has one user; a server makes files, printing and other services available to users of the network and is optimized for many concurrent users, with more memory, storage and network capability.

3. **3.2** On a hypothetical bottling line, small dedicated controllers start and stop conveyors, sensors report temperature and vibration continuously, and the whole line reorganizes itself when a batch changes without an operator intervening. Which description matches the chapter?

- **A.** The controllers are embedded systems and the sensors are output devices, and because they exchange readings continuously over the factory network the arrangement is best described as an intranet on the production floor — The controllers are indeed a kind of purpose-built computer, but sensors supply data rather than present results, which makes them input devices. An intranet is a private internal website, not a factory network.
- **B.** This is a peer-to-peer network, because the machines request and provide services from one another with no central server involved — Peer-to-peer describes machines that both request and provide services on a general-purpose network, and the chapter places it in small offices and homes. It says nothing about coordinating physical processes.
- **C.** The controllers are programmable logic controllers, the sensors are input devices supplying environmental data in real time, and the whole arrangement is a cyber-physical system — **correct.** Programmable logic controllers are the chapter's specialized devices for automating machines and processes, sensors are input technologies collecting environmental data such as temperature and vibration in real time, and a cyber-physical system integrates computing with physical processes through tight coordination of software, sensors and mechanical components.
- **D.** This is grid computing, because many small independent machines are combining their computing power to complete one large task — Grid computing combines many independent computers, often ordinary desktops, on a single large computational problem. Nothing here is being computed collectively; the machines are running a physical process.

4. 3.2 A hypothetical equipment hire firm wants to identify returned items automatically as they come through a doorway, without anyone pointing a scanner at each one. Items are low in value and arrive in bulk. Which reading of the chapter's material is accurate?

- **A.** Passive tags are the obvious fit, because they cost a few cents and are read from a few feet, and the doorway can be the reader — **correct**. The chapter describes passive tags as small and inexpensive, starting from a few cents, with a typical range of up to several feet, and lists eliminating line-of-sight reading and reading at greater distances among the advantages over bar codes.
- **B.** Active tags are required, because only a tag with its own battery can be read without the item being pointed at a scanner — Active tags include a battery and can transmit hundreds of feet, but they cost several dollars each. Paying that for a low-value item is the trade-off the chapter is asking you to notice, not a requirement.
- **C.** Bar codes are the better answer, because the chapter says radio-frequency identification cannot be read unless the tag is plainly visible — This inverts the chapter. Not needing line of sight is precisely the advantage it lists first: the data is readable regardless of the item's position or whether the tag is plainly visible, which is what bar codes cannot do.
- **D.** Neither technology applies, because identification at a distance requires a network connection on each item — A tag is not a networked computer. It is a small transponder that answers a reader's electromagnetic field, which is why a passive one needs no battery and no connection of its own.

5. 3.2 A hypothetical software company sells one product that runs on two different operating systems. A new manager asks why the company does not instead ship a version for each of the several hundred computer models its customers own. What does the chapter's account of system software say?

- **A.** Each model would need its own build, and testing every combination of processor, graphics card and disk on the market would be far too expensive, so the company ships two versions and accepts that some machines will run them badly — Cost is not the reason, because the per-model builds are not needed in the first place. The chapter's point is structural: the operating system removes the need for them entirely.
- **B.** Applications talk to hardware directly, so the company writes one version and relies on device drivers supplied by customers to make up the difference — The chapter states the opposite. Application software cannot interact directly with hardware; it interacts with the system software, which in turn interacts with the hardware, and drivers are part of that layer rather than a customer responsibility.
- **C.** Modern hardware is standardized enough that any program will run on any machine, so the two versions exist only for commercial reasons — Hardware is not uniform, which is why drivers exist at all. Two versions are needed because the two operating systems present different layers, not because of marketing.

- **D.** The operating system provides a common layer over different underlying devices, so applications only have to be built for the operating system rather than for each computer model — **correct.** The chapter says the operating system provides a common layer for different underlying devices so that applications only have to be developed for different operating systems rather than for each different computer model, which is exactly why a product ships in two versions and not two hundred.

6. **3.2** A hypothetical veterinary group must keep three kinds of data: today's appointments and charges, a nightly copy in case a server fails, and years of old internal correspondence it must be able to produce if asked. Which arrangement matches what the chapter says about matching data to media?

- **A.** All three belong on the fastest available storage, because data that cannot be reached instantly has no value to the practice — The chapter separates the three types precisely because their requirements differ along timeliness, access speed, searchability and life span. Paying for instant access to correspondence nobody reads is the waste the distinction exists to prevent.
- **B.** Operational data on fast disk or flash storage, backups on hard drives so recovery is quick, and archival data on low-cost removable media such as magnetic tape — **correct.** The chapter puts operational data on disk-based or flash storage for speed, notes that storing backups on hard drives enables quick recovery without slowing operations, and archives data that is no longer used operationally on magnetic tape, which is slow and not quickly searchable but very low cost and removable.
- **C.** Operational data on tape because it is written continuously, backups on flash storage, and archival data on hard drives so it stays searchable — This reverses the media. Tape stores data sequentially, so access can be very slow and the contents are not quickly searchable, which is why the chapter reserves it for archives rather than for live transactions.
- **D.** Operational and backup data together on one drive, and archival data deleted once the retention period is over, since old email has no operational use — Keeping the backup on the same drive as the live data defeats its purpose, since one failure takes both. Whether old correspondence may be deleted is a compliance question, and archival data exists precisely because the answer is often no.

7. **3.2** Two hypothetical laboratories agree to exchange results automatically. The cable is installed, the files leave one system and arrive intact at the other, and the receiving system rejects every one of them. Which of the chapter's three requirements of a network has not been met?

- **A.** The pathway, because a cable that delivers unusable files has not really transmitted the data — The pathway is the physical medium, cable or wireless, used to transmit data, and here it did its job: the files arrived intact. Arrival is a separate question from comprehension.
- **B.** The sender and receiver, because a system that rejects a file is not genuinely a receiver for that message — Both ends exist and one of them has something to share, which is what the first requirement asks for. Rejecting a message you cannot interpret is what a receiver without a shared protocol does.

- **C.** The protocol, because the two sides have not agreed on the procedures they will follow when transmitting and receiving — **correct**. The chapter's third requirement is rules or protocols governing communication between sender and receiver, and its own illustration is an email that arrives perfectly in a language the reader cannot decode: if the message is not understood by the receiver, there is no communication.
- **D.** None of them, because all three are present and the problem is simply a software defect at the receiving laboratory — Calling it a defect skips the diagnosis. The chapter's framework says a message can travel the whole way successfully and still fail, and the missing element in that case is the agreed procedure, not the code.

8. **3.2** A hypothetical training department must move a one gigabyte video file to a site that has an old low-speed connection. Someone proposes buying a much faster connection for the site instead. Which statement follows from the chapter's own comparison of connection speeds?

- **A.** Transfer time falls roughly in proportion to the increase in capacity, so a connection a thousand times faster turns a transfer of well over a day into seconds — **correct**. Bandwidth is the transmission capacity measured in bits per second, so time is size divided by rate: the chapter's own table takes the same one gigabyte video from more than a day and a half on an old modem down to a few seconds on gigabit fiber.
- **B.** Transfer time depends mainly on the size of the file, so a faster connection will make only a marginal difference to a file this large — Size is only half of the calculation. Holding the file constant and changing the rate is exactly the comparison the chapter's table makes, and it changes the answer by orders of magnitude.
- **C.** Transfer time is set by the slower of the two connections only when both ends are wireless, and by the faster one otherwise — The slower link limits the transfer regardless of whether either end is wireless. Nothing in the chapter makes the medium decide which end governs.
- **D.** Transfer time cannot be estimated in advance, because bandwidth describes capacity rather than speed and the two are unrelated — Bandwidth is defined as how much binary data can reliably be transmitted over the medium in one second, which is precisely what makes an estimate possible.

9. **3.2** A hypothetical clinic replaces the computers in its examination rooms with cheap terminals that hold no data and run everything from a server in the basement. What has it done, in the chapter's terms, and what does it gain?

- **A.** It has built a peer-to-peer network, gaining resilience because any terminal can serve any other if the basement server fails — Peer-to-peer means any device can both request and provide services. These terminals only request, and if the server fails none of them can serve anything, so no resilience has been gained.
- **B.** It has installed embedded systems, because each terminal now performs one specific set of tasks rather than being a general-purpose computer — An embedded system is built into a device to perform a specific set of tasks, such as controlling an engine. A thin client is still a general-purpose window onto general-purpose software running elsewhere.

- **C.** It has moved to a public cloud, because the applications now run somewhere other than where the user is sitting — A public cloud is a service any interested party can rent on a pay-per-use basis. The clinic still owns the server in its own basement, so the resources were never pooled with anyone else's.
- **D.** It has adopted thin clients with desktop virtualization, reducing software licensing and maintenance costs and helping with strict privacy and data protection duties — **correct.** Thin clients are microcomputers with minimal memory, storage and processing capability, and the chapter says they use desktop virtualization to provide workers with a virtual desktop environment, helping reduce costs for software licensing or maintenance and comply with stringent privacy and data protection laws.

10. **3.2** A hypothetical university librarian is asked why a search engine cannot find the library's catalogue records, its subscription journals, or the student portal, and whether that means they are on the dark web. What is the accurate answer?

- **A.** They are on the dark web, which is the chapter's name for everything a conventional search engine cannot index — The chapter is explicit that the dark web is not to be confused with the deep web. The dark web is the part used for nefarious purposes and reachable only with specialized anonymizing browsers.
- **B.** They are on the deep web because somebody hid them deliberately, and material reaches the surface web only when an organization pays a search company to index it, which is why public bodies with no advertising budget stay invisible — Nothing here is hidden in the sense the dark web is hidden, and indexing is not something an organization purchases. A catalogue is unindexable because its pages are generated from a database when somebody asks.
- **C.** They are on the deep web: pages behind authentication, pages generated from databases on demand, and pages nothing links to are not indexable, and that is unrelated to the dark web — **correct.** The chapter enumerates the deep web as private areas requiring authentication such as a learning management system, dynamic pages created from connected databases, and static pages not connected to others by hyperlinks — and separately warns that the dark web is something else entirely.
- **D.** They are on the surface web but rank too low to appear, so better search terms would eventually reach them — Search terms cannot reach a page that was never indexed. A search engine cannot traverse content that only exists once a database query is run for a specific request.

11. **3.2** A hypothetical charity is told that its address, its mail service and its donation service can share one domain while behaving as separate systems. Which explanation of a web address matches the chapter?

- **A.** The three parts of an address are the protocol, the domain and the file path, and only the file path can vary between one service and another, so the two names must be reaching the same machine and the same software — The chapter's dissection names the host name, the domain and the top-level domain. A path to a particular resource may follow, but it is not one of the three parts of the address itself.

- **B.** An address is one indivisible name registered as a whole, so a separate service requires a separately registered domain name — Registrars register domains, and the organization is then free to use as many host names in front of that domain as it needs. The chapter's own example lists several services on one domain.
- **C.** The address names a host, a domain and a top-level domain, and different host names in front of the same domain reach different servers or groups of servers — **correct**. The chapter says a URL has three distinct parts — the host name, the domain and the top-level domain — and gives examples of one organization's mail, photo and mapping services each answering on a different host name in front of the same domain.
- **D.** Host names are decorative, since the numeric address underneath is what actually identifies the organization to a browser — The host name is what tells the network which server or group of servers should answer, so it is doing real routing work rather than decoration.

12. **3.2** A hypothetical manufacturer wants three things: a public site, a private site where employees enrol in benefits, and a site where its suppliers see live order status. It asks whether these are three technologies or one. What does the chapter say?

- **A.** One set of technologies used three ways: an internet site, an intranet restricted to authorized employees, and an extranet restricted to authorized partners — **correct**. The chapter says an intranet looks and acts just like a publicly accessible website and uses the same software, hardware and networking technologies, but adds authentication or firewalls; an extranet is a private part of the internet enabling two or more firms to do business together. Its summary table separates them by focus, content, users and access, not by technology.
- **B.** Three different technologies, because an internet site is built with web standards while an intranet and an extranet run on proprietary software that the organization buys separately for each audience — The chapter explicitly says the same technologies are used. What differs is who is authorized and whether the content ever crosses the public internet.
- **C.** One technology used twice, since an extranet is simply the name for an intranet once suppliers are given accounts on it — The chapter's table gives them different users, different content and different focus: internal communications with corporate content for employees, versus external communications between business partners. Adding supplier accounts to an internal site is not the same design.
- **D.** Three different networks, because internal communication cannot travel over the same infrastructure as public content — Internal communication in the simplest intranet stays inside organizational boundaries, but an extranet deliberately uses the public internet infrastructure and protects the traffic with a secured tunnel.

13. **3.2** A hypothetical logistics company is choosing where to build a facility that will hold its servers. Two candidate sites are equally priced: one is far from any customers but very safe from severe weather, the other is close to its largest customer base but on a more exposed coast. What does the chapter say the decision involves?

- **A.** Proximity is irrelevant once a connection is fast enough, so the safer site wins automatically — Distance still costs time, which is why the chapter says organizations weigh proximity to customers to reduce latency rather than treating location as settled by the network.

- **B.** The exposed site wins automatically, because reducing delay for users is the only thing customers ever notice — Latency matters, but a facility that floods is unavailable, and the chapter's availability discussion makes downtime the more expensive failure. Both considerations are live.
- **C.** Neither site should be built on, because reliability of this order is obtainable only by renting space in a facility a third party already operates, and a company of this size has no business running a data center at all — Renting space in a collocation facility is a real option the chapter names, but it is an alternative to be weighed rather than the only way to reach high availability.
- **D.** A genuine trade-off between protection from the elements and proximity to users for lower latency, decided alongside power, cooling, connectivity, expandability and security — **correct.** The chapter says organizations go to great lengths selecting locations that strike the optimal balance between protection from the elements and proximity to customers or users to reduce latency, and separately lists connectivity, floor space, energy, cooling, security and modular expandability as requirements of the facility overall.

14. **3.3** A hypothetical municipal utility is told that its five-year-old server fleet is fine because nothing has broken. Its technology manager disagrees and points to the chapter's account of obsolescence, in which the planning horizon of five years managers once worked to has given way to new versions of devices every six to twelve months. Which argument is the chapter's?

- **A.** Capability keeps advancing underneath a fleet that is standing still, so the question to put to the machines is not whether they still work but what the utility can no longer do with them — **correct.** The chapter says rapid advances continuously force organizations to upgrade to gain or maintain competitive advantage, and that whereas managers once thought in terms of five years, new versions of devices are now released every six to twelve months.
- **B.** Equipment that has not failed has not yet been used up, so replacement should wait until something actually stops working and the money already committed to these machines has been earned back in full — Whether a machine still runs is not the chapter's test. It frames obsolescence as the gap between what current technology enables and what the installed base can do, which opens long before a failure.
- **C.** Machines become unreliable at a known age, so a fixed replacement interval removes the need to make a judgment — The chapter gives no such interval, and a fixed schedule would replace machines that are still capable while leaving others in place after they have become a constraint.
- **D.** Obsolescence is a hardware problem only, so a fleet that runs current software is by definition current — The chapter devotes a whole subsection to software obsolescence: new operating systems often require new hardware, and older applications may not be compatible with the new operating system.

15. **3.3** A hypothetical school district hears that its operating system will stop receiving mainstream support, with paid extended updates available for a limited period afterwards. The new version has strict hardware requirements. What is the managerial shape of this situation, in the chapter's terms?

- **A.** A software licensing question, since paying for the extended updates converts a fleet-wide upgrade into an ordinary recurring subscription the district can carry in its operating budget for as long as it likes — Extended updates buy time rather than removing the decision, and they are a bridge with an end date rather than a permanent arrangement.
- **B.** Planned obsolescence, in which a vendor's end-of-support decision forces a fleet-wide upgrade whose real cost is the hardware and the labour of upgrading every machine — **correct.** The chapter defines planned obsolescence as designing a product to last only a certain life span, notes that a company may cease support for a product and effectively force users to switch, and says the cost lies both in hardware and software and in the time and resources needed to upgrade tens, hundreds or thousands of computers.
- **C.** A security question only, because support and updates concern vulnerabilities rather than capability — Security is part of it, but the chapter's emphasis is the compulsory upgrade: a new operating system that requires new hardware and may not run the organization's older applications.
- **D.** A negotiation, because vendors routinely extend support indefinitely for large institutional customers who ask — Nothing in the chapter suggests support dates are individually negotiable, and building a plan on that assumption would leave the district without a path when the date arrives.

16. **3.3** A hypothetical publisher notices a pattern: each hardware refresh is justified by software that will not run on the old machines, and each new software release is justified by capabilities the new hardware made possible. A director asks whether someone is being manipulated. What does the chapter's figure describe?

- **A.** A loop in which more powerful hardware enables more powerful software, and the more powerful software then requires hardware more powerful still — **correct.** The chapter draws exactly this two-arrow loop: powerful hardware enables powerful software, which requires powerful hardware. It names the same pattern a vicious circle when describing how enhanced capabilities enable new applications that then demand more data and communications capacity.
- **B.** A vendor conspiracy between the hardware makers and the software houses, which is why the chapter recommends refusing every upgrade until a business case for it arrives from outside the technology function — The chapter does not allege bad faith. It observes that upgrades may increase productivity and often do not, while remaining a large cost factor, which is a management problem rather than a conspiracy.
- **C.** A misunderstanding, because software capability and hardware capability advance independently of one another — The chapter's whole point is that they are coupled: new operating systems use new processor architectures and often require new hardware to run at all.
- **D.** A financing decision, since the pattern disappears once equipment is leased rather than purchased — Leasing changes who owns the equipment and how it is paid for. It does not change the fact that new software needs capability the installed machines do not have.

17. **3.3** A hypothetical parcel service sizes its own equipment for the busiest week of December. For the remaining eleven months much of that capacity sits idle, and it still ran short during one unexpected week. Which reading matches the chapter?

- **A.** The sizing was simply wrong, and a more careful forecast would have produced a capacity figure that fits the whole year — No single figure fits both a December peak and a February trough. The chapter describes the mismatch as structural rather than as a forecasting error.
- **B.** This is evidence against owning any infrastructure, since the chapter concludes that owned capacity can never be justified — The chapter presents renting as a way to address the problem and later notes considerable disagreement about whether it is ultimately cheaper, including one company that concluded owning suited its workload better.
- **C.** This is a storage problem, because the shortage appeared at the moment the volume of parcel records being captured and kept exceeded the disk space that had been provisioned for the year — The shortage was of capacity to serve a peak in activity, which is a different problem from the growth in data volumes the chapter treats separately.
- **D.** This is the demand fluctuation problem: capacity bought in fixed increments is either idle most of the time or short at the peak, and it cannot be changed as quickly as temporary staff can be hired — **correct.** The chapter says demands for computing resources fluctuate, leaving either too few resources at some times or too many idle most of the time, cites estimates that a large share of infrastructure runs at a small fraction of capacity, and notes that while it is relatively easy to hire temporary staff, it is typically not that easy to make quick changes to the infrastructure.

18. **3.3** A hypothetical research institute doubles the processing power in its server room and budgets for the extra electricity the new machines will draw. Its facilities manager says the budget is short. Why, according to the chapter?

- **A.** Because electricity prices rise every year, so any energy budget written in advance will be short — Price movements are real but are not the chapter's argument, which is about the physical consequence of adding processing power rather than about tariffs.
- **B.** Because more powerful hardware needs more energy to run and, at the same time, more energy to cool, so one upgrade produces two increases — **correct.** The chapter spells out that components such as the processor and power supply generate heat, that more powerful hardware needs more energy to deliver the extra computing power, and that at the same time more powerful hardware requires more energy for cooling — and it puts a busy rack's draw at the equivalent of more than ten homes.
- **C.** Because servers draw their maximum power continuously regardless of load, so idle figures cannot be used in a budget — The chapter gives a range between idling and full load for a desktop precisely because draw varies with load, so a constant maximum is not the assumption it makes.
- **D.** Because energy is charged to the technology budget rather than to facilities, so the figure was recorded in the wrong place — Moving a cost between budget lines does not change its size. The shortfall exists because a second cost was never counted.

19. **3.3** A hypothetical media startup wants to test whether anyone will use a new service. A consultant proposes buying servers and building the capacity the service would need if it succeeded. What does the chapter's treatment of agility say?

- **A.** Buy the capacity in advance, because a service that succeeds and then falls over under the load has lost its chance for good, and the equipment is cheap set against the cost of that one bad week — The chapter names this specific case — buying hardware and installing web servers for a mobile app before knowing whether it will be accepted or how much demand must be met — as an example of what makes little sense.
- **B.** Buy half the capacity, because the chapter recommends provisioning to the midpoint between failure and success — No such midpoint rule appears anywhere in the chapter, and provisioning to a number nobody has evidence for is guessing with more steps.
- **C.** Neither: the chapter argues that startups should not concern themselves with infrastructure, because the question only arises once a company is established — The chapter draws the opposite conclusion: the infrastructure question is most acute for organizations experimenting, which is why it uses startups to make the point.
- **D.** It makes little sense to spend time and money on infrastructure before knowing whether the experiment will succeed or how much demand it would attract, which is why a minimum viable product comes first — **correct.** The chapter says being ahead of the competition necessitates speed, describes the lean startup approach of rapid cycles and minimum viable products to test whether solutions have the desired effect, and states that it makes little sense to invest in infrastructure changes before knowing whether the experiment will be a success.

20. **3.3** A hypothetical charity plans to train a model on its own case records and budgets for it using the price of the servers it bought last year. What does the chapter say makes that estimate unsafe?

- **A.** Training work runs on specialized processors that are not normally part of a traditional infrastructure and cost far more per unit, and the storage requirements differ in both capacity and speed — **correct.** The chapter says training machine learning models typically does not take place on traditional processors, that an AI infrastructure requires graphics or tensor processing units not normally part of the traditional infrastructure, and that data storage requirements in both capacity and speed often exceed those of traditional systems — with unit prices in the tens of thousands of dollars.
- **B.** Training work runs on ordinary processors but for much longer, so the estimate is right in unit price and wrong only in duration — The chapter's point is that the hardware is different in kind, not merely used for longer. Sequential processors are the wrong shape for work that can be done in parallel.
- **C.** The estimate is safe, because the chapter says the processing, storage and analysis requirements of machine learning work are broadly similar to those of the transaction processing a business already runs — The chapter states the requirements differ vastly from traditional computing workloads, which is why it treats an AI infrastructure as a separate topic at all.

- **D.** The estimate is unsafe only because prices for computing equipment fall over time, so last year's figure is above the current one — Falling prices would make an old estimate too high; the chapter's warning is that the required equipment is a different and far more expensive category altogether.

21. 3.4 A hypothetical training provider stops buying servers and instead rents processing and storage from an outside provider, paying only for what it uses. Its finance director asks what has actually changed. What is the chapter's answer?

- **A.** Nothing has changed except the supplier, since the same resources are being used in the same quantities for the same work — The quantity is no longer fixed in advance, which is the change. Under the previous arrangement the organization paid for capacity whether or not it was used.
- **B.** The organization has adopted a utility computing model, which transforms infrastructure costs from a capital expenditure into an operational expenditure and makes fixed costs variable — **correct.** The chapter defines utility computing as renting resources such as processing, storage or networking from an external provider on an as-needed basis and paying only for what is actually used, and says cloud computing helps transform infrastructure costs from a capital expenditure to an operational expenditure.
- **C.** The organization has outsourced its technology function altogether, so what looks like an infrastructure decision is really a staffing decision, and the finance question is which contracts of employment end — Renting infrastructure does not by itself remove the technology function, and the chapter notes that responsibility for choosing, integrating and managing services remains with the customer.
- **D.** The organization has moved to a private cloud, because the resources it rents are reserved for its own use — A private cloud is internal to an organization. Renting from an outside provider on a pay-per-use basis is the chapter's definition of a public cloud.

22. 3.4 A hypothetical ticketing company doubles its capacity in the minutes after a popular event goes on sale and releases it an hour later, all automatically. Which of the five cloud characteristics is doing the work, and what would the alternative have looked like?

- **A.** Broad network access, because the extra capacity was reachable from anywhere over the internet, on whatever device staff had with them, without a single new machine arriving at the company's own site — Broad network access describes services being reachable from almost anywhere and from almost any web-enabled device. It explains who can use the system, not how the capacity grew.
- **B.** Resource pooling, because the company was given capacity from a shared pool it does not own and cannot locate — Resource pooling is what makes the spare capacity available to be assigned, and it is genuinely present. It does not describe the speed of the change, which is what the question is about.

- **C.** Rapid elasticity, because resources scaled up and down almost instantaneously and automatically, where owned equipment takes weeks to arrive and days to configure — **correct.** The chapter contrasts servers that take several weeks to be delivered and days or weeks to configure with a cloud environment where computing resources can be scaled up or down almost instantaneously and often automatically, and it uses exactly this holiday-surge case to make the point.
- **D.** Measured service, because the company was billed for the extra hour rather than for the whole month — Measured service explains the shape of the bill afterwards. The capability being described is the change in capacity itself, which happened before any invoice existed.

23. **3.4** A hypothetical language app normally spends a small predictable amount each month on a translation service it reaches through an interface key. One month the bill is several hundred times larger. Investigation shows the key was used by someone else to request an enormous volume of work. What is the chapter's lesson?

- **A.** The cloud is inherently unsafe for a company of this size, because once a customer depends on a provider that provider can charge whatever it likes, and a small firm has no leverage to argue about a bill — The chapter does not conclude that the cloud is unsafe. Its own case shows the charges followed real usage, which is what a metered service does, and it responds with practices rather than avoidance.
- **B.** Measured service means the meter runs for whoever holds the credential, so cost monitoring, regular review of what is provisioned, budget alerts and reading the contract are the defences — **correct.** This is the chapter's own cautionary case, in which a startup faced a bill hundreds of times its usual monthly spend after a credential was apparently stolen and used to request billions of characters of translation, and its five prevention strategies are cost monitoring tools, regular review of resource allocations, budget alerts, educating teams on cost awareness and reading the fine print.
- **C.** The provider is at fault for not detecting the anomaly, so the remedy is contractual rather than operational — Anomaly detection would help, and budget alerts are among the chapter's recommendations, but the customer holds the credential and the chapter puts the vigilance on the customer.
- **D.** This is a billing dispute rather than a security incident, since nothing was stolen and no data was disclosed — A credential was taken and used, which is exactly a security failure, and the chapter's own earlier interface example shows how easily such a key can be left where it can be read.

24. **3.4** A hypothetical insurer wants to run software its own developers wrote, without anyone on staff installing or patching operating systems or database engines, and without buying licences for either. Which service model does the chapter describe for this, and what does the insurer still control?

- **A.** Infrastructure as a service, which supplies processing, storage and networking and leaves the insurer to choose and license everything above them — Infrastructure as a service gives the most control and also the most responsibility: the chapter notes that managing software licences remains the customer's job and setup costs are relatively high.

- **B.** Software as a service, because the insurer is running an application and does not want to manage anything beneath it — Software as a service means using an application the provider supplies, with limited ability to configure it. The insurer wants to run software of its own, which this model does not allow.
- **C.** Platform as a service, which supplies the operating system, web server and database management system while the insurer controls only its own applications — **correct**. The chapter defines platform as a service as customers running their own applications, typically designed using tools the provider supplies, with control over the applications but limited or no control over the underlying infrastructure, and says the customer does not have to worry about purchasing licences for the web servers' operating systems or for database management systems.
- **D.** A private cloud, because running software the organization wrote itself requires infrastructure the organization owns — Writing your own software does not require owning hardware. The chapter's platform model exists precisely so that custom applications can run on rented infrastructure.

25. **3.4** A hypothetical broadcaster rents raw computing capacity and runs its own operating systems on it, converting video into many formats and hosting its own site. Elsewhere the same broadcaster's staff use a rented email service and a rented customer-records application. How does the chapter classify these?

- **A.** Both are software as a service, because in both cases the broadcaster is a customer rather than an owner of the equipment — Being a customer is common to all three models. What separates them is what the provider manages, and the broadcaster is managing operating systems in the first case and nothing at all in the second.
- **B.** Neither is a cloud service, because the broadcaster is managing operating systems in one case and configuring an application in the other — All three models are cloud services. Managing an operating system on rented hardware is what the chapter says infrastructure as a service involves.
- **C.** The first is platform as a service, because the provider supplies the machines on which the broadcaster's software runs — Platform as a service supplies the operating system and database engine as well, so the customer does not install them. Here the broadcaster is choosing and running its own.
- **D.** The first is infrastructure as a service and the second is software as a service, and the difference is how far up the stack the provider's responsibility reaches — **correct**. The chapter gives infrastructure as a service as processing, storage and networking only, with the most customer control and the customer's own licences, and software as a service as using only applications supplied over a cloud infrastructure, with no knowledge or control of what lies beneath.

26. **3.4** A hypothetical regional hospital moves its public information site and its appointment reminders to a rented public service, while keeping patient records on machines in its own building. A board member calls this indecisive. What does the chapter say?

- **A.** It is indecisive, because splitting an infrastructure across two models multiplies cost and complexity for no benefit — The chapter acknowledges that combining approaches adds complexity, and still describes it as the common outcome, because the alternative is treating very different data as though it had the same requirements.
- **B.** It is a hybrid approach, which the chapter presents as what organizations often choose because no single provider meets every need and some data is not worth moving — **correct.** The chapter says organizations often have to partner with different providers and combine public and private clouds, that there is not one solution that fits all, and that they often choose a hybrid approach with certain applications in-house while moving others to the public cloud.
- **C.** It is a private cloud, since keeping any data in-house means the whole arrangement is private by definition — A private cloud is an internal cloud providing self-service access to pooled internal resources. Owning some servers does not make a rented public service private.
- **D.** It is a temporary state that ends the moment the remaining systems are migrated, because the chapter treats a complete move to a public provider as the destination every organization is travelling towards — The chapter never treats full migration as the goal. Its evaluation criteria exist so that each application and data set is decided on its own merits.

27. **3.4** A hypothetical payments company reads a provider's service-level agreement promising very high uptime with refunds if it is missed, and concludes it no longer needs its own continuity plan. What does the chapter say about that conclusion?

- **A.** It is sound, because a contractual uptime commitment transfers the risk of downtime to the provider — A commitment is not the same as a guarantee. The chapter describes such agreements mainly as a vehicle for resolving conflicts in case of problems.
- **B.** It is sound for availability but not for security, since an agreement of this kind covers performance and says nothing about how the provider protects the data it is holding on the pharmacy's behalf — The distinction between availability and security is real, but the flaw in the reasoning is present within availability itself, before security is even considered.
- **C.** It is unsound, because such an agreement promises a service level and pays a refund or a discount rather than guaranteeing availability, and money back does not replace the business lost while the service was down — **correct.** The chapter says service-level agreements do not guarantee the availability of resources but only promise certain service levels and provide refunds or discounts if the promises are not met, and that refunds normally cover only the fees paid and can never offset the opportunity costs of lost business — which is why organizations should plan ahead and replicate their infrastructure in different locations.
- **D.** It is unsound, because the chapter concludes that in-house infrastructure achieves better uptime than any provider — The chapter says the opposite in its fair counterpoint: organizations often realize that even when agreements are missed, a provider can still offer better uptime than a poorly managed in-house infrastructure.

28. **3.4** A hypothetical genomics institute holds many terabytes with one provider and receives a better offer from another. Its analysts assume moving is a weekend's work. Which of the chapter's criteria have they overlooked?

- **A.** Openness, because providers use different infrastructures and ways of storing data, and charges for moving data out make a transfer of this size slow and expensive — **correct**. The chapter says most providers use different infrastructures and different ways to store data, making migration extremely difficult and leading to lock-in, and adds that bandwidth and data transmission costs are a further limitation because providers often charge for moving data into or out of their infrastructure.
- **B.** Viability, because the question is whether the new provider will still exist in several years — Viability is a real criterion and worth asking about, but it concerns whether the provider survives, not why the move itself is difficult.
- **C.** Scalability, because the new provider may not be able to meet the institute's storage needs as the sequencing work grows, and a second migration in three years would cost more than this one — Scalability concerns whether a provider can meet current and future demands. It says nothing about the cost and time of the migration itself.
- **D.** Diversity of offerings, because using two providers is harder to manage than using one — Diversity of offerings is about how many providers an organization must manage. The institute is planning to move rather than to add a second relationship.

29. **3.4** A hypothetical publisher compares a monthly rented server bill against what it believes the same server costs in-house, and concludes renting is more expensive. What does the chapter say about that comparison?

- **A.** It is reliable, because the internal cost of a server is straightforward to work out from its purchase price divided over its expected life, which is the figure the finance system already carries for it — Purchase price and expected life leave out most of the cost. The chapter lists licences, electricity, the data center itself and the staff as parts of the figure organizations usually cannot produce.
- **B.** It is unreliable in one direction only, because rented costs are understated by charges that appear later — The chapter's difficulty is not hidden charges on the rented side, where costs are transparent and easy to track, but the unknown figure on the in-house side.
- **C.** The comparison is hard because the rented figure is exact while many organizations do not know what an in-house server truly costs once licences, electricity, the facility and the staff are counted — **correct**. The chapter says it is easy to calculate the monthly cost of a server in a public cloud while many organizations do not know exactly what it costs to run a comparable server in-house, including the server, the licence fees, the electricity, the data center and the staff, and it acknowledges considerable disagreement about which is ultimately cheaper.
- **D.** The comparison is unnecessary, because the chapter concludes that public cloud infrastructure is cheaper for essentially all workloads — The chapter reaches no such conclusion, and it cites one company that moved back out of a public cloud after deciding owning suited its own workload better.

30. **3.4** A hypothetical engineering consultancy needs a simulation that would take a very long time on any single machine it can afford. It arranges for the work to be split into small pieces and completed by hundreds of ordinary networked computers. How does the chapter distinguish this from cloud computing?

- **A.** There is no distinction; the chapter treats grid computing as an early name for cloud computing — The chapter draws the comparison explicitly in order to separate the two, rather than treating one as a former name for the other.
- **B.** Grid computing uses only machines the organization already owns and has paid for, whereas cloud computing means renting machines that belong to somebody else and are housed in their building — Ownership is not the line the chapter draws, and a grid can be assembled from machines in several organizations. The distinguishing feature is what the resources are pointed at.
- **C.** Grid computing is a way of storing data across many machines, whereas cloud computing is a way of processing it — Grid computing is defined as combining computing power to solve problems that previously needed a supercomputer, which is processing rather than storage.
- **D.** Both use distributed resources, but in a grid the resources are typically applied to a single large problem, whereas cloud resources serve many separate customers and purposes — **correct.** The chapter says that similar to cloud computing, grid computing makes use of distributed resources, but that in contrast to cloud computing the resources in a grid are typically applied to a single large problem, with large tasks broken into small chunks completed by individual computers.

31. **3.4** A hypothetical online seller of camping equipment sells to customers worldwide, but visitors far from its servers wait noticeably longer for pages and video to load. Which of the chapter's trends addresses this, and what does it actually do?

- **A.** A content delivery network, which keeps copies of the content on servers in various geographical locations so the nearest one answers, cutting the delay caused by distance — **correct.** The chapter says the larger the geographical distance between a user and the web server hosting the content, the longer transmission takes, and that content delivery networks reduce this latency by providing servers in various geographical locations holding copies, with the closest one delivering the content — a process normally unnoticed by the user.
- **B.** A private cloud, which would give the seller control over performance by keeping all content on machines it owns — A private cloud changes who owns and controls the infrastructure. It does nothing about the distance between one location and a user on another continent.
- **C.** A wider connection at the seller's own data center, since the delay is caused by the capacity of that single connection rather than by distance, and doubling it would halve the wait for every visitor — Capacity at the origin matters, but the chapter identifies distance itself as the cause of this particular delay, which is why the answer is to move copies closer rather than to widen one pipe.
- **D.** Grid computing, which would combine many small computers to serve the pages faster than one large one could — Grid computing applies many machines to one large computation. Serving a page to a distant visitor is not a computation that can be divided up.

32. **3.5** A hypothetical animal shelter keeps its adoption records in a folder of spreadsheet files, one per month, each edited by whoever is on shift. Two staff now report different totals for the same period. In terms of the supplement on databases, what is the underlying problem?

- **A.** The files are too large for a spreadsheet, so the totals are being truncated before they are displayed — Nothing in the situation suggests a size limit has been reached, and a truncated file would produce obviously missing rows rather than two plausible but different totals.
- **B.** There is no single stored set of records that everyone reads and writes, so the same fact exists in several places at once and nothing decides which copy is authoritative — **correct.** This is the case for storing records once, in one place, and asking questions of them: a database gives every record a single home and a defined shape, so a total is a question asked of the stored rows rather than a copy someone made and then edited.
- **C.** The staff need training in spreadsheet functions, because the totals would agree if every branch used the same formula, and the disagreement is arithmetic rather than anything to do with where the records live — A shared formula would make two copies agree only until the next edit. The disagreement arises from having two sets of records, not two ways of adding them up.
- **D.** Spreadsheets cannot hold dates, so records from the same period are being counted in different months — Spreadsheets handle dates perfectly well. The problem is that the same adoption may exist in more than one file, or in only one of them.

33. **3.5** In the supplement on databases, a hypothetical bicycle-hire service asks two questions of the same stored table: which bicycles are currently out, and how many hires each depot recorded last month. What is the difference between those two questions in query terms?

- **A.** The first chooses which rows to return, and the second groups rows together and returns a calculated value for each group — **correct.** Choosing rows and choosing columns are separate decisions, and summarising is a third: the first question filters to a subset of rows, while the second collapses many rows into one result per depot, which is the difference between filtering and aggregating.
- **B.** The first is a query and the second is a report, so only the first can be expressed as a query at all — Both are queries. Producing a count per group is ordinary query work, and calling the result a report describes what is done with the answer rather than how it is obtained.
- **C.** The first requires the whole table to be read and the second does not, which is why the second finishes sooner in every case — How much of the table is read is a performance detail that depends on the stored data, and it is not what separates the two questions.
- **D.** There is no difference, because both questions return all the rows and the difference is only in how they are displayed afterwards — The second question does not return the underlying rows at all. It returns one row for each depot, which is a different result rather than a different presentation.

34. **3.5** A hypothetical museum runs its collection catalogue on a rented platform, so nobody there installs or patches a database engine. Its curator concludes the museum no longer has to think about its data at all. What does the supplement on databases say?

- **A.** The curator is right, because on a rented platform the provider is responsible for the data as much as for the software beneath it, which is the whole reason an institution pays somebody else to run it — The platform model removes the work of running and updating the software, and the chapter says exactly that. It does not transfer authorship of the data model or responsibility for what the records mean.
- **B.** The curator is wrong only about backups, since everything else about the data is handled by the provider — Backups matter, and so does everything about the design: a provider that keeps the engine running has no view about whether two records describe the same object.
- **C.** The provider manages the platform, but the museum still owns what the data means: which records exist, how they relate to each other, and what must be unique — **correct.** This is where the chapter's service models meet the course's own database work: renting the platform removes the licence and the maintenance, and leaves the design of the tables, the keys and the questions exactly where it was.
- **D.** The curator is wrong because a rented platform cannot host a catalogue, which needs a database engine the museum installs itself — A rented platform is a normal place to run a catalogue, and the chapter names the database management system as one of the things such a platform provides.

35. **3.5** A hypothetical community orchestra keeps one stored table of members, each with the town they live in, and a second stored table of instrument loans, each carrying a member identifier rather than a name. The treasurer asks how many loans went to members from each town. Using the supplement on databases, why can no single row answer that?

- **A.** Because the answer has to be assembled at the moment of asking: the town lives in one table and the loan in another, so the rows are matched on the value they share and then collapsed into one line per town — **correct.** Two things are happening at once, and the supplement separates them: a join matches facts that were deliberately kept apart by the value they share, and a grouping collapses many matched rows into one result per town. Neither result exists in any stored row.
- **B.** Because the loans table is missing a column, and the question becomes answerable only once the town is copied into every loan record as well — Copying the town into every loan is exactly what storing records once is meant to avoid. It creates a second copy of a fact that can then disagree with the first when a member moves.
- **C.** Because a stored table can be read only in the order its rows were written, so a question about towns cannot be asked until the rows have been sorted into towns first and stored again in that order — Rows are not read in the order they were written, and nothing has to be re-stored to ask a new question. That is the difference the supplement draws between a set of files and a database.
- **D.** Because counting is arithmetic rather than retrieval, so a total of this kind has to be worked out in a spreadsheet after the rows themselves have been exported out of the database — Counting per group is ordinary query work. Exporting first would leave the answer sitting in a file that is out of date the moment the next loan is recorded.

36. **3.6** A hypothetical seed cooperative wants three things: a public catalogue for customers, a private area where members read notices and enrol in training, and a shared area where two partner laboratories exchange live test results. Using the supplement on internet uses, how should these be built?

- **A.** Three separate public sites, each with its own address, since separating them by address is what keeps the audiences apart — Three addresses do not restrict anyone. Without authentication, anything published on a public site is public no matter how many addresses it is spread across.
- **B.** An intranet for all three audiences, since the cooperative writes the content in every case and can simply issue accounts to the buying public alongside its members and its two laboratories — Issuing accounts to every customer turns a public catalogue into a barrier to sales, and the chapter separates these uses by focus and audience rather than by who controls the content.
- **C.** One public site with three sections, relying on visitors not to follow links they were not given — An unlinked page is still reachable by anyone who has the address, and the chapter treats authentication and firewalls, not obscurity, as what makes content private.
- **D.** A public internet site, an intranet restricted to authorized members, and an extranet restricted to the two partner laboratories, separated by who is authorized rather than by technology — **correct.** This is the chapter's own trio mapped onto the three uses the course objective names: general public content for anyone with a connection, corporate content for authorized employees or members, and content between business partners restricted to authorized partners, all using the same underlying technologies.

37. **3.6** A hypothetical parts distributor and its largest supplier currently exchange orders by email, and staff on both sides rekey the details into their own systems. Using the supplement on internet uses, what does the chapter say a shared partner arrangement offers instead?

- **A.** Faster email, since the same messages would travel over a dedicated connection rather than the public internet — The delay is not in the email. It is in the rekeying and in the two separate sets of records that rekeying creates.
- **B.** A single point of data entry from which both organizations work, improving the timeliness and accuracy of communication, automating transactions and reducing errors and cycle times — **correct.** The chapter's list of extranet benefits includes dramatically improving the timeliness and accuracy of communications, automating business transactions to reduce processing costs and cycle times, reducing errors by providing a single point of data entry from which data can be updated without reentry, and giving management real-time data.
- **C.** A private cloud shared between the two firms, which is how two organizations come to read the same records — A shared private cloud is not what the chapter proposes here, and an extranet works precisely because it uses the public internet infrastructure with the traffic secured.

- **D.** Nothing that email does not already do, since the benefit of a partner arrangement is presentational rather than operational and the same figures are being sent either way, just through a different window — The chapter's claim is operational: fewer versions of a document, fewer errors, shorter cycles and real-time visibility for management, none of which follows from sending an email.

38. 3.6 A hypothetical printing firm needs to show one delivery schedule to a haulage contractor it works with. The quickest route is to issue the contractor a staff login to the firm's intranet. Using the supplement on internet uses, what is wrong with that?

- **A.** Nothing is wrong, provided the contractor is asked not to open anything other than the delivery schedule while it is logged in — An instruction is not a restriction. The supplement's point is that access is granted by the credential rather than by the request that accompanies it.
- **B.** An employee login carries everything else an employee can reach, so an outside firm would be granted the whole internal site in order to see one schedule — **correct.** This is one of the three ways of getting the audience wrong: partners handed employee access receive everything else an employee can reach along with the one thing they were meant to see. The extranet exists precisely so that authorized partners get a restricted view rather than a staff view.
- **C.** Intranets are technically incapable of showing a page to anyone outside the organization, so the login would simply fail when the contractor tried to use it — There is no such technical barrier. An intranet uses the same web technologies as any other site, and it is authentication rather than the technology that decides who gets in.
- **D.** The schedule should be published on the public site instead, because anything a partner is allowed to see has already stopped being confidential and needs no protection — A schedule shared with one contractor is not public. Publishing it would hand a competitor the firm's delivery pattern, which is the first of the three ways of getting the audience wrong.

39. 3.6 A hypothetical garden supplier sells only through its own website. Its owner argues that because the business owns no shop, its infrastructure matters less than a high-street rival's. Using the supplement on internet uses, what does the chapter's material say?

- **A.** The owner is right, because a rented site transfers the whole question of availability to the provider that operates the servers — Renting moves who operates the machines, not who loses the sale. The supplement is explicit that the availability of the store is the availability of the site.
- **B.** The owner is right for most of the year and wrong only in December, since a seasonal spike is the single moment at which a website has to be treated as infrastructure — The December spike is real and it is not the whole argument. A page that fails to load on an ordinary Monday costs the orders that would have been placed that Monday.
- **C.** It matters more: the site is the shop, so a firm trading electronically strives for the highest availability of its hardware, storage and networking — **correct.** The chapter says organizations depending on electronic trading strive for the highest availability of their hardware, storage and networking components, and describes the five-nines target as leaving just over five minutes of downtime in a year — which for this firm is the door being locked with customers outside.

- **D.** Availability targets belong to organizations large enough to run a data center of their own, so a supplier of this size has no reason to think about downtime at all and should spend the money on stock instead — The chapter attaches availability to the dependence rather than to the size of the organization, and a firm that trades only online is more dependent than a shop with a till, not less.

What to do next. Take one service you use every day and trace it all the way down. Where does the data sit? Whose machines are they, and in whose building? What has to be working for the thing to load — the wire, the address, the server, the power, the cooling? Then ask the two questions this module keeps returning to: what would this cost to own, and what would it cost the business on the day it is not available. Being able to answer those two about something ordinary is what it means to have a working knowledge of an infrastructure.